



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИМЕНИ АКАДЕМИКА С. П. КОРОЛЁВА»

Институт двигателей и энергетических установок

Кафедра автоматических систем энергетических установок имени академика РАН Владимира
Павловича Шорина

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

«РАЗРАБОКА 6-ОСЕВОГО МАНИПУЛЯТОРА С АДАПТИВНОЙ СИСТЕМОЙ УПРАВЛЕНИЯ»

по направлению подготовки 15.03.04

«Автоматизация технологических процессов и производств»

(уровень бакалавриата)

профиль «Мехатронные и робототехнические комплексы»

Студент _____ А.А. Пешков

Руководитель ВКР,

профессор кафедры АСЭУ,

д.т.н. _____ С.П. Мурзин

Консультант ВКР,

Старший преподаватель _____ М.В. Блохин

Нормоконтролер,

Старший преподаватель _____ М.В. Блохин

Институт двигателей и энергетических установок

Кафедра автоматических систем энергетических установок
имени академика РАН В.П. Шорина

«УТВЕРЖДАЮ»

Заведующий кафедрой

 Шахматов Е.В.
« 7 » _____ 2025 г.

ЗАДАНИЕ НА ВЫПУСКНУЮ РАБОТУ БАКЛАВРА

Студенту группы 2414-150304D Пешкову Андрею Александровичу

1 ТЕМА ВЫПУСКНОЙ РАБОТЫ: «Разработка 6-осевого манипулятора с адаптивной системой управления». Утверждена приказом по университету от « » _____ 2025 г. № _____

2 ИСХОДНЫЕ ДАННЫЕ К ВЫПУСКНОЙ РАБОТЕ.

2.1. Состав объекта: 6-осевой робот-манипулятор, лидары, контроллер, устройство для задания движения, шаговые двигатели с обратной связью, редуктора, драйверы шаговых двигателей, преобразователь напряжения, блок питания.

2.2. Назначение и выполняемые функции: Робот-манипулятор, как платформа для последующих разработок систем, подразумевающих как перемещение объектов в условиях динамически меняющегося окружения, так и прямое взаимодействие с человеком в процессе разгрузки и загрузки мобильных роботов и БЛА на автономных станциях доставки заказов. Лидары предназначены для создания трехмерной карты окружающей среды; контроллер обеспечивает управление движением по шести осям и недопущение столкновения с окружающими объектами; устройство для задания движения обеспечивает передачу желаемых координат TCP от оператора к роботу; шаговые двигатели с обратной связью отвечают за точное позиционирование звеньев робота; редуктора отвечают за повышение крутящего момента соединений и точности их позиционирования; драйверы производят управление двигателями; блок питания для запитывания робота манипулятора.

2.3. Технические требования: Питание от сети 380В $\pm 10\%$, 50 Гц $\pm 0,5\%$. Напряжения цепей управления: 24В, 12В, 5В; грузоподъемность: не ниже 8кг, погрешность позиционирования не более: 1мм, скорость перемещения: не менее 500 мм/с.

2.4. Условия эксплуатации: Температура окружающей среды от 0 до +40 °С., влажность до 80%, класс защиты IP44.

2.5. Прочие требования: Интеграция с симуляционной средой Gazebo Sim ROS2 Jazzy.

3 СОДЕРЖАНИЕ РАСЧЕТНО-ПОЯСНИТЕЛЬНОЙ ЗАПИСКИ

3.1 Основная часть

- 1) Литературно-патентный поиск по теме 6-осевых роботов-манипуляторов или систем предотвращения столкновений;
- 2) Разработка структурных и функциональных схем системы управления;
- 3) Обоснование выбора компонентов системы;
- 4) Разработка электрической схемы системы управления;
- 5) Разработка модели 6-осевого робота манипулятора
- 6) Импорт робота манипулятора в среды разработки Gazebo Sim и ROS2 Jazzy;
- 7) Разработка алгоритма управления робота-манипулятора;
- 8) Разработка алгоритма предотвращения столкновений;
- 9) Разработка программы управления роботом в ROS2 Jazzy;
- 10) Разработка сборочного чертежа робота-манипулятора.

4 ПЕРЕЧЕНЬ ГРАФИЧЕСКОГО МАТЕРИАЛА

- 1) Функциональная схема системы управления роботом - _____;
- 2) Структурная схема электрической части робота - _____;
- 3) Схема электрическая принципиальная - _____;
- 4) Спецификации и сборочные чертежи звеньев робота-манипулятора - _____;
- 5) Спецификации и сборочный чертеж робота-манипулятора - _____;
- 6) Чертежи оригинальных деталей - _____;
- 7) Алгоритмы и программы работы системы управления и предотвращения столкновений робота - _____;
- 8) Временные диаграммы работы системы управления.
- 9) Презентация в формате pptx.

Срок представления на кафедру законченной работы:

« _____ » _____ 2025 г.

Задание выдал « _____ » _____ 2025 г.

Задание принял « _____ » _____ 2025 г.

Руководитель _____ / С. П. Мурзин/
(личная подпись)

Студент _____ / А. А. Пешков/
(личная подпись)

Консультант _____ / М. В. Блохин/
(личная подпись)

Согласовано
Руководитель цикла
д.т.н., профессор

_____ / С. А. Матюнин/
(личная подпись)

РЕФЕРАТ

Пояснительная записка: 77 с., 30 рисунков, 4 таблицы 16 источников, 29 приложений.

Графическая часть: 5 листов формата А2, 21 лист формата А3, 36 листов формата А4.

ШЕСТИОСЕВОЙ РОБОТ МАНИПУЛЯТОР; ИНТЕЛЛЕКТУАЛЬНАЯ СИСТЕМА УПРАВЛЕНИЯ; ОБНАРУЖЕНИЕ СТОЛКНОВЕНИЙ; МОДЕЛИРОВАНИЕ; СИМУЛЯЦИЯ; НЕЙРОННЫЕ СЕТИ.

Объектом исследования является 6-осевой манипулятор, предназначенный для выполнения различных манипуляционных задач в динамической среде.

Цель работы - разработка 6-осевого манипулятора с интеллектуальной системой управления, способного эффективно взаимодействовать с окружающей средой, обходить препятствия и предотвращать столкновения.

В процессе работы была создана и импортирована в среду симуляции gazebo кинематическая модель 6-осевого манипулятора, а также проведены тесты на работоспособность кинематики робота.

СОДЕРЖАНИЕ

Введение.....	8
1 Литературно-патентный обзор	9
1.1 Общие сведения	9
1.2 Обзор литературы	9
1.3 Патентный обзор.....	11
2 Анализ технического задания	20
2.1 Описание системы	20
2.2 Режимы работы устройства	20
2.3 Аварийный режим работы	21
3 Разработка структурной и функциональной схемы	22
3.1 Структурная схема.....	22
3.2 Функциональная схема	23
4 Выбор и обоснование элементной базы	25
3.1 Шаговые двигатели	25
3.2 Редукторы	27
3.3 Подшипники.....	30
3.4 Ременная передача	31
3.5 Драйверы двигателей	33
3.6 Сетевой концентратор.....	34
3.7 LiDAR.....	35
3.8 Кнопка аварийной остановки	36
3.9 Кнопка со светодиодом	37
3.10 Вентилятор	38
3.11 Контроллер	39
3.12 Преобразователь напряжения 12 В.....	41
3.13 Преобразователь напряжения 9 В.....	42
3.14 Блок питания	43
5 Разработка схемы электрической принципиальной	45
5.1 Подключение микроконтроллера.....	45
5.2 Подключение лазерных сканеров	45
4.3 Подключение Ethernet-коммутатора.....	46
5.4 Подключение драйверов серводвигателей.....	46
5.5 Подключение серводвигателей с энкодерами	46
5.6 Подключение блоков питания.....	46
6 Разработка кинематической модели манипулятора	47
6.1 Определение системы координат.....	47
6.2 Параметры Денавита-Хартенберга	48
7 Проектирование манипулятора	49
7.1 Основание	49

7.2 Первое звено.....	50
7.3 Второе звено.....	51
7.4 Третье звено.....	51
7.5 Четвертое звено.....	52
7.6 Пятое звено.....	53
7.7 Манипулятор целиком	54
8 Расчет точности позиционирования.....	56
8.1 Погрешность от допусков на звенья.....	56
8.2 Угловые погрешности приводов.....	56
8.3 Линейные погрешности приводов	58
8.4 Общая погрешность позиционирования	59
9 Создание симуляционной модели	60
9.1 Создание проекта.....	60
9.2 Экспорт модели манипулятора	60
9.3 Настройка SDF файла описания модели	61
9.4 Создание файла запуска симуляции	62
9.5 Доработка и добавление файлов подгрузки плагинов и конфигурации симуляции.....	62
9.6 Сборка проекта.....	63
10 Проектирование структуры и разработка алгоритмов управления	64
10.1 Основной алгоритм управления (Robot_Model_Publisher).....	64
10.2 Алгоритм планирования траектории	65
10.3 Алгоритм преобразования координат (TF_Listener).....	69
10.4 Алгоритм публикации состояния звеньев манипулятора (Joint_State_Aggregatoe)	70
10.5 Алгоритм фильтрации данных LiDAR (Lidar_Filter).....	70
10.6 Алгоритм вращения данных LiDAR (Lidar_Processor).....	70
10.7 Алгоритм предупреждения касания (Dead_Area_Detector)	71
10.8 Алгоритм детектора касания (Collision_Detector).....	71
11 Разработка программ управления.....	72
12 Тестирование программы управления	73
13 Разработка конструкции шкафа управления	74
Заключение	75
Список использованных источников	76
Приложение А	78
Приложение Б	82
Приложение В.....	89
Приложение Г	98
Приложение Д.....	108
Приложение Е.....	114

Приложение Ж.....	130
Приложение И	138
Приложение К.....	142
Приложение Л.....	154
Приложение М.....	156
Приложение Н	158
Приложение П	163
Приложение Р	166
Приложение С.....	168
Приложение Т	169
Приложение У	170
Приложение Ф	171
Приложение Х	172
Приложение Ц	173
Приложение Ш	174
Приложение Щ	187
Приложение Э.....	194
Приложение Ю	197
Приложение Я.....	199
Приложение АА.....	203
Приложение АБ.....	206
Приложение АВ.....	208
Приложение АГ	213

ВВЕДЕНИЕ

В последние годы наблюдается значительный прогресс в области робототехники, что связано с увеличением применения роботизированных систем в различных отраслях, таких как промышленность, медицина, логистика и сервисные услуги. Одной из ключевых задач, стоящих перед разработчиками, является создание манипуляторов, способных эффективно взаимодействовать с окружающей средой, обходить препятствия и предотвращать столкновения. Современные 6-осевые манипуляторы, обладая высокой степенью подвижности, требуют внедрения интеллектуальных систем управления, которые обеспечивают их автономность и безопасность в работе.

В связи с вышеизложенными требованиями, возникает необходимость в проведении научно-исследовательской работы, направленной на разработку 6-осевого робота-манипулятора с адаптивной системой управления, способного эффективно обходить препятствия и предотвращать столкновения. Предполагаемый научно-технический уровень разработки включает в себя использование современных методов обработки данных и адаптивные алгоритмы управления, что позволит значительно повысить уровень автономности и безопасности манипулятора.

1 Литературно-патентный обзор

1.1 Общие сведения

Планирование движений звеньев робота-манипулятора, работающего с постоянно меняющимся окружением, требует большого объема информации, и алгоритмов высокой. В результате возникает много проблем:

- низкий уровень адаптивности существующих решений;
- потребность в больших вычислительных ресурсах.

С целью исследования технического уровня и тенденций развития техники был проведен патентный поиск.

Были рассмотрены следующие патенты и научные публикации:

1.2 Обзор литературы

1. Планирование пути для мультироботных систем с использованием нечеткой логики, прорабатываемой в симуляции.

Авторы: Вэйцзе Ши, Чжоу Хэ, Вэй Тан, Вэйфэн Лю, Цзыюэ Ма.
Регистрационный номер: 10.1109/LRA.2022.3165184, 2022.04.06.

Статья представляет новый метод обхода препятствий для мобильных роботов, основанный на улучшенном управлении с использованием нечеткой логики. В работе рассматривается проблема эффективного и безопасного передвижения роботов в динамических средах, где необходимо избегать столкновений с различными препятствиями. Авторы предлагают алгоритм, который сочетает в себе нечеткую логику и адаптивные механизмы, позволяя роботу принимать решения на основе неопределенности и изменяющихся условий окружающей среды.

Метод включает в себя использование сенсоров для сбора данных о расстоянии до препятствий и их характеристиках. На основе этих данных система принимает решения о направлении движения и скорости робота. В статье также представлены результаты экспериментов, показывающие, что

предложенный метод значительно улучшает эффективность обхода препятствий по сравнению с традиционными подходами.

Используемые сенсоры:

- ультразвуковые датчики:
 - а) место установки: на передней и боковых частях робота;
 - б) функция: измеряют расстояние до ближайших объектов, позволяя роботу определять наличие препятствий;
- инфракрасные сенсоры:
 - а) место установки: на передней части робота;
 - б) функция: используются для дополнительного обнаружения объектов и улучшения точности определения расстояний;
- IMU (инерциальные измерительные устройства):
 - а) место установки: внутри робота;
 - б) функция: отслеживают ориентацию и движение робота, что важно для корректного управления и планирования маршрута.

Преимущества:

- эффективность: Улучшенный алгоритм на основе нечеткой логики позволяет роботу более точно и быстро реагировать на изменения в окружающей среде;
- адаптивность: Метод способен адаптироваться к различным условиям и типам препятствий, что делает его универсальным для различных приложений;
- снижение вероятности столкновений: Использование нескольких типов сенсоров повышает надежность системы обнаружения препятствий;
- простота реализации: Нечеткая логика позволяет легко настраивать параметры управления в зависимости от конкретных условий.

Недостатки:

- сложность настройки: хотя нечеткая логика проста в реализации, настройка правил и параметров может быть трудоемкой задачей;

- зависимость от качества сенсоров: Эффективность метода зависит от точности и надежности используемых сенсоров;
- ограниченная производительность в сложных условиях: в условиях высокой динамики или при наличии множества движущихся объектов система может испытывать трудности с принятием решений;
- потенциальные задержки: В зависимости от обработки данных и алгоритмов, могут возникать задержки в реакции робота на изменения в окружающей среде.

1.3 Патентный обзор

1. Способ и система обнаружения столкновения робота-манипулятора с использованием нейронной сети.

Патент US20220398454A1. Патентообладатель: Neuromeka Co Ltd. Документ действует.

Данный патент описывает метод и систему для обнаружения столкновений манипулятора робота с использованием искусственной нейронной сети. Основная цель системы заключается в том, чтобы обеспечить безопасное взаимодействие робота с окружающей средой, предотвращая столкновения с людьми и объектами. Система использует данные от различных сенсоров, установленных на манипуляторе, для обучения нейронной сети, которая может распознавать и реагировать на столкновения в реальном времени.

Манипулятор, аналогичный человеческой руке, может выполнять различные задачи, такие как захват и перемещение объектов. Патент описывает использование датчиков крутящего момента, установленных на каждом суставе, для определения наличия столкновений. Нейронная сеть обучается на основе данных, полученных от этих сенсоров, и может выдавать бинарный сигнал, указывающий на наличие или отсутствие столкновения.

Система также включает в себя актуаторы для управления суставами, энкодеры для измерения углов суставов и контроллер для обработки сигналов. Обучение нейронной сети происходит через нормализацию циклов, что позволяет улучшить точность обнаружения столкновений. В результате, система может эффективно предотвращать столкновения, что делает её полезной в различных областях, таких как автоматизация и робототехника.

Используемые сенсоры:

В системе обнаружения столкновений используются следующие сенсоры:

- датчики крутящего момента:
 - а) место установки: на каждом суставе манипулятора;
 - б) функция: измеряют силу, прикладываемую к суставам, для определения наличия столкновений;
- энкодеры:
 - а) место установки: с одной стороны каждого актуатора;
 - б) функция: измеряют углы суставов для контроля движения.
- датчики давления или резисторы с чувствительностью к силе (FSR):
 - а) место установки: на стороне суставов;
 - б) функция: измеряют контакт и помогают в обнаружении столкновений.

Преимущества:

- безопасность: высокая степень безопасности для людей, работающих рядом с роботами, благодаря быстрому обнаружению столкновений;
- эффективность: нейронная сеть адаптируется к различным условиям и улучшает свои навыки обнаружения столкновений со временем;
- реальное время: обнаружение столкновений происходит в реальном времени, что позволяет быстро реагировать на изменения в окружающей среде;

- универсальность: система может быть применена в различных типах манипуляторов и робототехнических систем.

Недостатки:

- сложность: реализация и обучение нейронной сети могут быть сложными и требовать значительных вычислительных ресурсов;
- зависимость от данных: эффективность системы зависит от качества и объема данных, используемых для обучения нейронной сети;
- потенциальные ошибки: система может допускать ошибки в распознавании столкновений, особенно в нестандартных ситуациях;
- задержка в реакции: возможны задержки в реакции системы в сложных сценариях.

2. Установка для манипулирования частями объекта при их взаимной стыковке.

Патент KR20230173005A. Авторы: Пак Сон Чжу, Бен Квон Мун, Ханну. 2023.12.26. Патент действует.

Патент описывает устройство для обнаружения столкновений и мобильного робота, использующего нейронные сети. Основная цель разработки заключается в повышении безопасности и эффективности передвижения роботов в сложных и динамичных средах. Система использует данные от различных сенсоров для анализа окружающей обстановки и принятия решений о движении. Нейронная сеть обучается на основе данных о столкновениях, что позволяет роботу адаптироваться к изменениям в окружающей среде и избегать потенциальных опасностей.

В патенте описывается, как робот может обрабатывать информацию о препятствиях и принимать соответствующие меры, такие как изменение направления или остановка. Устройство также включает в себя механизмы управления, которые позволяют эффективно реагировать на обнаруженные препятствия. Патент подчеркивает важность интеграции сенсоров и

алгоритмов машинного обучения для достижения высоких показателей безопасности и производительности.

Используемые сенсоры:

- датчики расстояния:
 - а) место установки: на передней и боковых частях робота;
 - б) функция: измеряют расстояние до ближайших объектов, позволяя роботу определять наличие препятствий;
- IMU (инерциальные измерительные устройства):
 - а) место установки: внутри робота;
 - б) функция: отслеживают ориентацию и движение робота, что важно для корректного управления и планирования маршрута;
- камеры:
 - а) место установки: на передней части робота;
 - б) функция: обеспечивают визуальную информацию об окружающей среде, что помогает в распознавании объектов и препятствий.

Преимущества:

- высокая адаптивность: нейронная сеть позволяет роботу адаптироваться к различным условиям и типам препятствий, что делает его универсальным для различных приложений;
- эффективное обнаружение столкновений: Использование нескольких типов сенсоров повышает надежность системы обнаружения препятствий;
- улучшенная безопасность: система обеспечивает высокую степень безопасности для людей и объектов, находящихся вблизи робота;
- интеграция технологий: комбинация сенсоров и алгоритмов машинного обучения позволяет достигать высоких показателей производительности.

Недостатки:

- сложность реализации: Внедрение и настройка нейронной сети могут быть сложными и требовать значительных вычислительных ресурсов;
- зависимость от качества сенсоров: эффективность системы зависит от точности и надежности используемых сенсоров;
- потенциальные ошибки: как и любая система, основанная на машинном обучении, она может допускать ошибки в распознавании столкновений, особенно в нестандартных ситуациях;
- задержка в реакции: возможны задержки в реакции системы в сложных сценариях, что может повлиять на безопасность.

3. Мультирадарная высокоточная система позиционирования для дезинфекционного робота.

Патент CN217467170U. Авторы: Ли Цзичуань, Ляо Цзикун, Суо Сюйдун, Цзэн Чан. 2022.09.20. Документ действует.

Патент описывает многорадарную высокоточную позиционную структуру для дезинфекционного робота. Основная цель разработки заключается в улучшении способности робота к обнаружению препятствий и навигации в сложных условиях.

Робот может выполнять активное избегание препятствий и глобальное позиционирование, что значительно повышает его эффективность в дезинфекции и навигации. Патент подчеркивает, что такая конфигурация позволяет улучшить производительность робота в условиях, где традиционные методы могут быть недостаточно эффективными. В результате, система обеспечивает более безопасное и надежное функционирование дезинфекционного робота в различных средах.

Используемые сенсоры:

- однолинейные лазерные радары:
 - а) место установки: на внешних углах фиксированной базы робота;

б) функция: обеспечивают 360-градусное обнаружение препятствий и помогают в активном избегании столкновений;

– 16-линейный лазерный радар:

а) место установки: на верхней части ультрафиолетовой лампы;

б) функция: используется для глобального позиционирования и трехмерного обнаружения препятствий.

Преимущества:

– 360-градусное обнаружение: два однолинейных лазерных радара обеспечивают полное покрытие, что минимизирует слепые зоны;

– улучшенное позиционирование: 16-линейный лазерный радар позволяет роботу точно определять свое местоположение и избегать препятствий в трехмерном пространстве.

Недостатки:

– сложность конструкции: многочисленные сенсоры могут усложнять конструкцию робота и увеличивать его стоимость;

– зависимость от сенсоров: эффективность системы зависит от точности и надежности используемых лазерных радаров;

– потенциальные проблемы с производительностью: в условиях высокой динамики или при наличии множества движущихся объектов система может испытывать трудности с принятием решений.

4. Система управления роботом.

Патент DE102019122790B4. Патентообладатели: Nvidia Corp. 2021.03.25. Документ действует.

Патент описывает систему управления для манипуляторов, использующих технологии LIDAR и искусственный интеллект для эффективного обхода препятствий и предотвращения столкновений. Основная цель системы заключается в том, чтобы обеспечить высокую точность и безопасность при манипуляции физическими объектами. Система

включает в себя обработку данных, полученных от сенсоров, для определения положения и ориентации объектов в пространстве.

Манипулятор может адаптироваться к различным условиям, используя алгоритмы машинного обучения для улучшения своих навыков захвата и перемещения объектов. Патент также подчеркивает важность интеграции LIDAR-сенсоров, которые обеспечивают трехмерное восприятие окружающей среды, что позволяет роботу эффективно избегать препятствий. В результате, система может выполнять сложные задачи манипуляции с высокой степенью надежности и безопасности.

Используемые сенсоры:

- LIDAR-сенсоры:

- а) место установки: на верхней части манипулятора или на его основании;

- б) функция: обеспечивают трехмерное восприятие окружающей среды, позволяя точно определять расстояние до объектов и их форму;

- камеры:

- а) место установки: на передней части манипулятора;

- б) функция: используются для визуального распознавания объектов и анализа их характеристик;

- датчики силы и момента:

- а) место установки: на суставах манипулятора;

- б) функция: измеряют силу, прикладываемую к объектам, что помогает в безопасном захвате и манипуляции.

Преимущества:

- высокая точность: использование LIDAR-сенсоров позволяет достигать высокой точности в определении расстояний и форм объектов;

- адаптивность: алгоритмы машинного обучения позволяют манипулятору адаптироваться к различным условиям и улучшать свои навыки со временем;
- безопасность: система предотвращает столкновения, обеспечивая безопасность, как для робота, так и для окружающих объектов;
- универсальность: подход может быть применен к различным типам манипуляторов и задачам.

Недостатки:

- сложность реализации: интеграция LIDAR и алгоритмов машинного обучения может быть сложной и требовать значительных вычислительных ресурсов;
- зависимость от качества сенсоров: эффективность системы зависит от точности и надежности используемых сенсоров;
- потенциальные ошибки: как и любая система, основанная на машинном обучении, она может допускать ошибки в распознавании объектов и принятии решений;
- стоимость: использование высокотехнологичных сенсоров, таких как LIDAR, может значительно увеличить стоимость системы.

Рассмотрев все выше представленные научные работы, были выведены основные недостатки существующих решений:

- расположение сенсоров:

Для получения общей картины окружающей обстановки, лидары и камеры располагаются либо на отдельном постаменте, что ограничивает количество мест применения робота, либо в наивысшей точке устройства, что, в малых пространствах, также уменьшает количество контролируемо принимаемых роботом положений.

- низкая степень адаптивности:

Существующие алгоритмы либо работают со статической картой окружения и не способны работать с данными от LIDAR, либо фокусируются

на определение наличия в рабочей зоне препятствия и приостановке алгоритма до устранения преграды.

Приведенные недостатки существующих моделей, делают проблематичным их использование при создании мобильного 6-осевого манипулятора с адаптивной системой управления.

2 Анализ технического задания

2.1 Описание системы

Разрабатываемая система представляет собой 6-осевой робот-манипулятор с адаптивной системой управления, предназначенный для выполнения задач, в условиях динамически меняющегося окружения. Основное назначение манипулятора — обеспечение перемещения объектов массой до 8 кг, а также поддержка взаимодействия с человеком при загрузке и разгрузке мобильных роботов и беспилотных летательных аппаратов на автономных станциях доставки заказов. Система должна быть универсальной платформой для последующих разработок, что подразумевает высокую степень адаптивности и точности в управлении.

2.2 Режимы работы устройства

Для обеспечения функциональности и безопасности системы необходимо реализовать несколько режимов работы, учитывающих как штатные операции, так и возможные нештатные ситуации.

- режим ожидания:

Манипулятор находится в исходном положении, потребляя минимальную мощность и ожидая команды оператора. Этот режим является начальным состоянием системы после включения и успешной инициализации.

- режим задания движения:

Оператор вводит желаемые координаты ТСР через устройство для задания движения. Контроллер планирует траекторию, избегая препятствий на основе данных от LiDAR, а затем манипулятор перемещается в заданную точку.

- аварийный режим:

Активируется при возникновении потенциально опасных ситуаций. Манипулятор останавливается, сохраняя текущее положение, и ожидает пересчета траектории или вмешательства оператора.

2.3 Аварийный режим работы

Аварийный режим активируется в следующих случаях:

- обнаружение препятствия:

Если лидары фиксируют объект на траектории движения, манипулятор останавливается, а система корректирует траекторию.

- столкновение с препятствием:

Если звенья манипулятора касаются препятствий, система останавливает звенья робота, уведомляет об этом оператора и пересчитывает траекторию движения.

3 Разработка структурной и функциональной схемы

В результате анализа технического задания и требований к системе был определён состав проектируемой системы управления 6-осевым манипулятором с адаптивной системой управления, а также составлены структурная (рисунок 1) и функциональная (рисунок 2) схемы.

3.1 Структурная схема

Структурная схема включает в себя следующие элементы:

- центральный процессор (CPU), отвечает за общее управление системой, обработку данных от датчиков, планирование траекторий и коммуникацию с другими компонентами;
- лазерные сканеры (LiDar 360x90), два сканера, обеспечивающие 360-градусное сканирование окружающей среды для создания трёхмерной карты и обнаружения препятствий;
- Ethernet-коммутатор, обеспечивает сетевое соединение между CPU, лазерными сканерами и драйверами серводвигателей для обмена данными;
- драйверы серводвигателей с обратной связью, каждый драйвер управляет одним серводвигателем, обеспечивая точное позиционирование и контроль скорости, а также получая обратную связь от энкодера двигателя;
- серводвигатели с энкодерами, каждый двигатель приводит в движение одну ось манипулятора, а энкодер обеспечивает обратную связь по положению для точного управления;
- механические редукторы, присоединены к каждому серводвигателю для увеличения крутящего момента и повышения точности позиционирования;
- блок питания 48В, питает драйверы серводвигателей;

- преобразователи напряжения:
 - а) 9В, питает Ethernet-коммутатор;
 - б) 12В, питает CPU и лазерные сканеры.

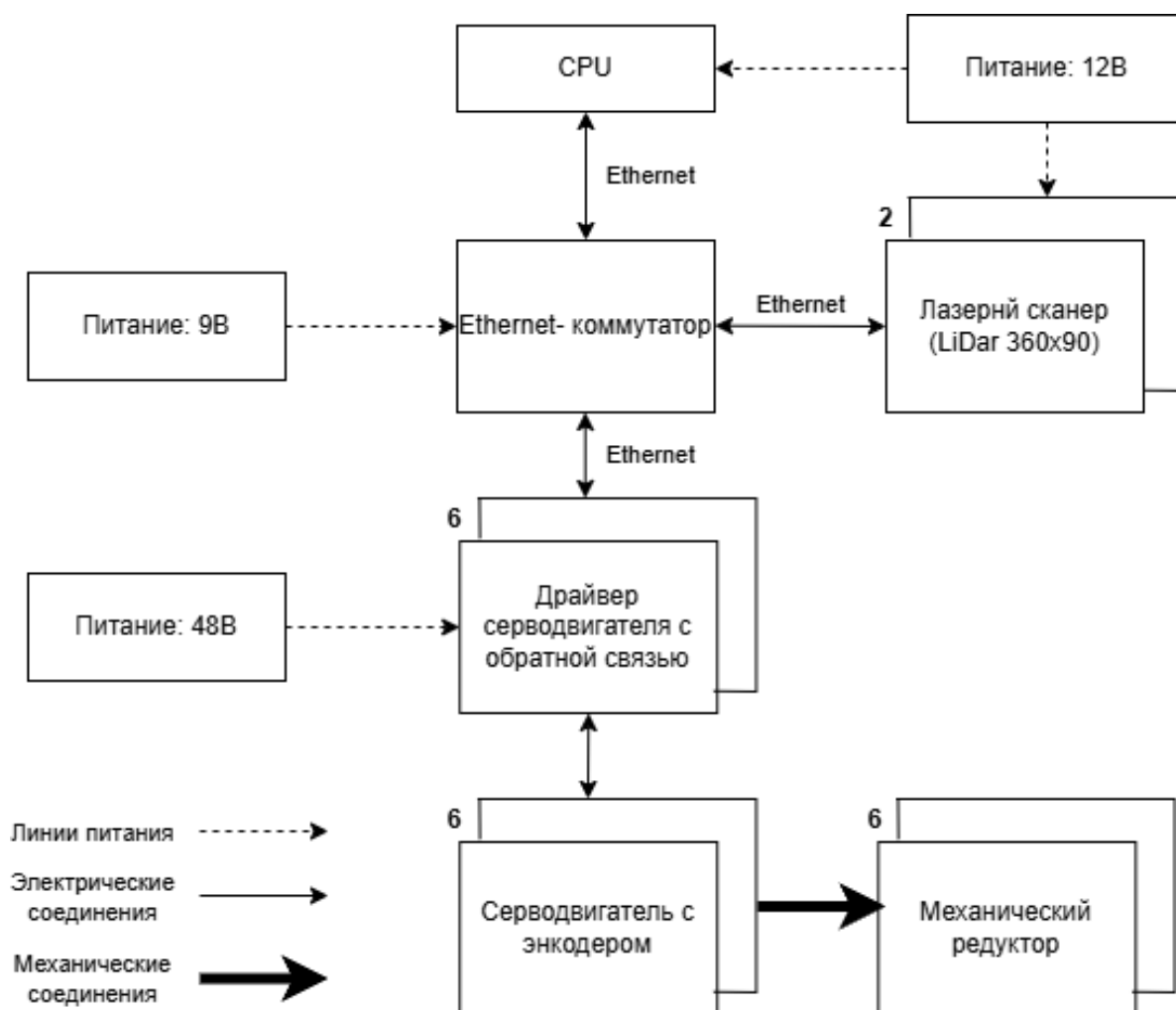


Рисунок 1 – Структурная схема

3.2 Функциональная схема

Функциональная схема состоит из следующих элементов:

- $УУ_0$ – центральное устройства управления;
- $УУ_{дн}$ – управляющее устройство привода;
- $Д_n$ – серводвигатель;
- $СС_э$ – сенсорная система (энкодер);
- $М_n$ – механический элемент (редуктор);
- $МС$ – механические системы (звенья манипулятора);
- $СС_{Ln}$ – сенсорная система (LiDAR);

- u_{on} – управляющий сигнал центрального устройства;
- u_1 – управляющий сигнал драйвера серводвигателя;
- Q_n – вращательное перемещение;
- Q – перемещение.

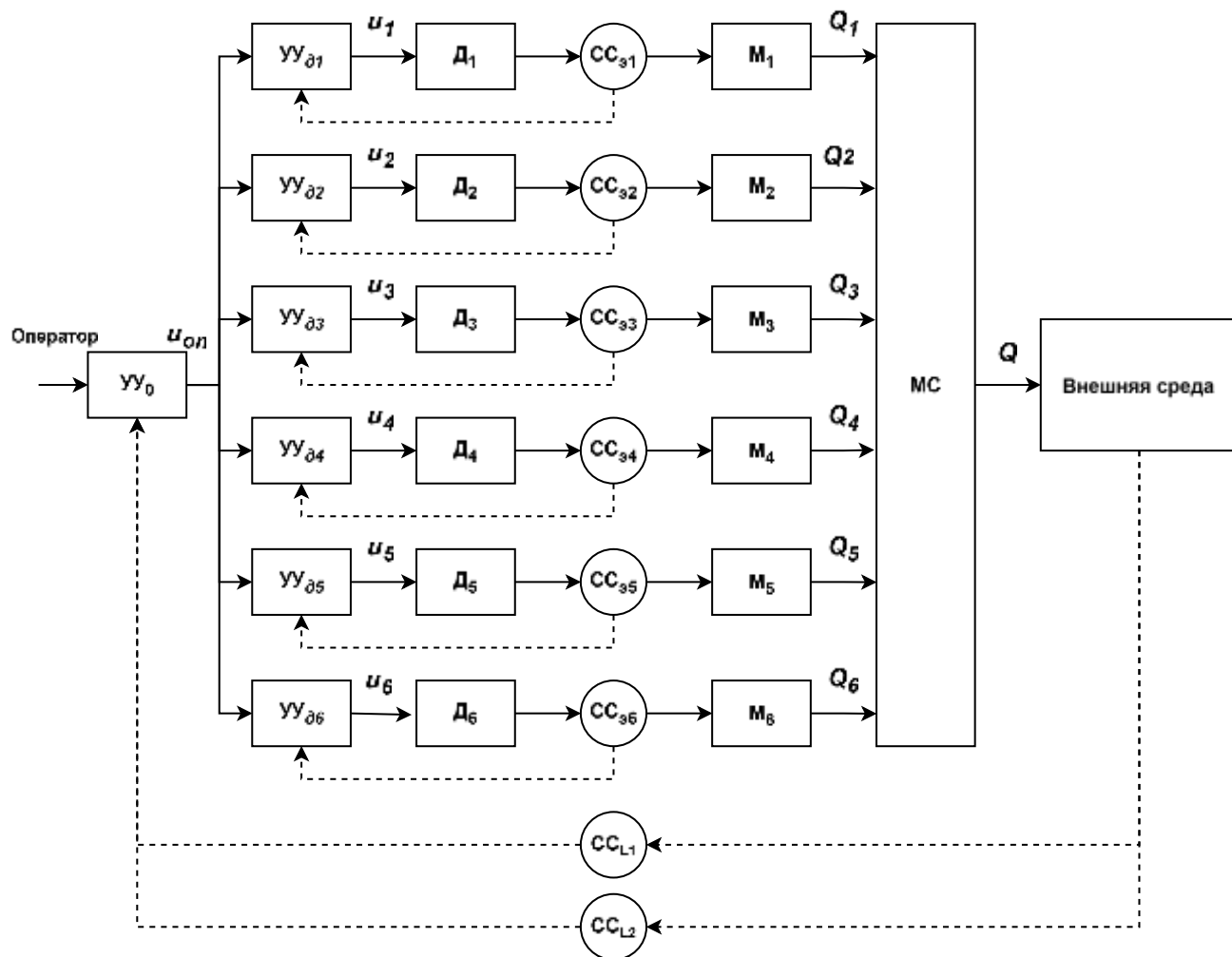


Рисунок 2 – Функциональная схема

4 Выбор и обоснование элементной базы

Для обеспечения требуемых характеристик 6-осевого манипулятора с адаптивной системой управления проведен расчет моментов сил на каждом звене с учетом их массы и расстояния до центра масс. Суммарная длина звеньев составляет 1,5 м (1,8 м – высота манипулятора за вычетом 0,3 м основания), распределенная между осями. Моменты рассчитаны для максимальной нагрузки в положении, когда манипулятор полностью вытянут горизонтально.

Моменты на осях манипулятора:

- ось 1: 1000 Н×м;
- ось 2: 600 Н×м;
- ось 3: 400 Н×м;
- ось 4: 60 Н×м;
- ось 5: 80 Н×м;
- ось 6: 15 Н×м.

3.1 Шаговые двигатели

Шаговые двигатели предназначены для привода осей манипулятора, обеспечивая точное позиционирование и управление движением.

Требования:

- момент: не менее 10 Н×м (оси 1–2), 6 Н×м (ось 3), 4 Н×м (оси 4–6);
- угол шага: не более 1°;
- рабочая температура: от 0 до +45 °С;
- ток обмоток: не более 6,5 А;
- наличие замкнутого контура с энкодером;

В данной работе используются шаговые двигатели Nema 34: 86HB250:

- Технические характеристики 86HB250-156В:
 - а) номинальный момент: 12 Н×м;

- б) ток обмоток: 6 А;
 - в) угол шага: 1.8° ;
 - г) рабочая температура: $-20...+50\text{ }^\circ\text{C}$;
 - д) напряжение: 80 В;
 - е) замкнутый контур: энкодер с разрешением 1000 имп/об;
 - ж) вес: 5,4 кг;
 - з) корпус: IP65.
- Технические характеристики 86HB250-118В:
- а) номинальный момент: 8,5 Н×м;
 - б) ток обмоток: 5,6 А;
 - в) угол шага: $1,8^\circ$;
 - г) рабочая температура: $-20...+50\text{ }^\circ\text{C}$;
 - д) напряжение: 80 В;
 - е) замкнутый контур: энкодер с разрешением 1000 имп/об;
 - ж) вес: 4,2 кг;
 - з) корпус: IP65.
- Технические характеристики 86HB250-82В:
- а) номинальный момент: 4,5 Н×м;
 - б) ток обмоток: 4,2 А;
 - в) угол шага: 1.8° ;
 - г) рабочая температура: $-20...+50\text{ }^\circ\text{C}$;
 - д) напряжение: 80 В;
 - е) замкнутый контур: энкодер с разрешением 1000 имп/об;
 - ж) вес: 2,8 кг;
 - з) корпус: IP65.



Рисунок 3 – Nema 34 86HB250-156B

3.2 Редукторы

Редукторы предназначены для увеличения момента двигателей, обеспечения высокой точности позиционирования и снижения скорости вращения.

Требования:

- передаточное число: не менее 1:100;
- выходной момент: не менее 1000 Н×м (ось 1), 600 Н×м (ось 2), 400 Н×м (ось 3), 60 Н×м (ось 4), 80 Н×м (ось 5), 15 Н×м (ось 6).
- рабочая температура: от 0 до +45 °С;
- КПД: не менее 80%.

В данной работе используются редукторы Hollow Harmonic Drive ZXK58 и 90 Degree Right Angled Planetary Gear Reducer PVF060 (ось 1) Harmonic Joint Reducer: HBK32 (ось 2), HBK25 (оси 3), HBK20 (ось 4), HBK17 (оси 5–6).

- Технические характеристики ZXK58:
 - а) передаточное число: 1:100;
 - б) выходной момент: до 4000 Н×м;
 - в) рабочая температура: -10...+60 °С;

- г) КПД: 85%;
 - д) вес: 4,5 кг;
 - е) максимальная скорость входного вала: 3500 об/мин;
 - ж) точность позиционирования: 0,01°;
 - з) корпус: IP54.
- Технические характеристики PVF060:
- а) передаточное число: 1:100;
 - б) выходной момент: до 1200 Н×м;
 - в) рабочая температура: -10...+60 °С;
 - г) КПД: 85%;
 - д) вес: 4,5 кг;
 - е) максимальная скорость входного вала: 6000 об/мин;
 - ж) точность позиционирования: 0,01°;
 - з) корпус: IP54.



Рисунок 4 – 90 Degree Right Angled Planetary Gear Reducer PVF060

- Технические характеристики HBK32:
- и) передаточное число: 1:100;

- к) выходной момент: до 2000 Н×м;
 - л) рабочая температура: -10...+60 °С;
 - м) КПД: 85%;
 - н) вес: 4,5 кг;
 - о) максимальная скорость входного вала: 3500 об/мин;
 - п) точность позиционирования: 0,01°;
 - р) корпус: IP54.
- Технические характеристики НВК25:
- а) передаточное число: 1:100;
 - б) выходной момент: до 1000 Н×м;
 - в) рабочая температура: -10...+60 °С;
 - г) КПД: 85%;
 - д) вес: 2,8 кг;
 - е) максимальная скорость входного вала: 4000 об/мин;
 - ж) точность позиционирования: 0,01°;
 - з) корпус: IP54.
- Технические характеристики НВК20:
- а) передаточное число: 1:100;
 - б) выходной момент: до 400 Н×м;
 - в) рабочая температура: -10...+60 °С;
 - г) КПД: 85%;
 - д) вес: 1,9 кг;
 - е) максимальная скорость входного вала: 4500 об/мин;
 - ж) точность позиционирования: 0,01°;
 - з) корпус: IP54.
- Технические характеристики НВК17:
- а) передаточное число: 1:100;
 - б) выходной момент: до 200 Н×м;
 - в) рабочая температура: -10...+60 °С;
 - г) КПД: 85%;

- д) вес: 1,2 кг;
- е) максимальная скорость входного вала: 5000 об/мин;
- ж) точность позиционирования: 0,01°;
- з) корпус: IP54.



Рисунок 5 – Harmonic Joint Reducer HBK25

3.3 Подшипники

Подшипники предназначены для восприятия радиальных и осевых нагрузок в сочленениях манипулятора, обеспечивая плавное вращение осей.

Требования:

- диаметр: не менее 90 мм;
- нагрузка: не менее 1 кН (оси 1–3), до 0,08 кН (ось 5);
- рабочая температура: от 0 до +45 °С;
- тип: радиально-упорный.

В данной работе используются подшипники: 872 ГОСТ 23179-78 (ось 1), 860 ГОСТ 23179-78 (оси 2, 3, 5).

- Технические характеристики 872 ГОСТ 23179-78:
 - а) внутренний диаметр: 110 мм;

- б) нагрузка: до 250 кН (радиальная), до 180 кН (осевая);
 - в) рабочая температура: $-30...+120\text{ }^{\circ}\text{C}$;
 - г) скорость вращения: до 3000 об/мин;
 - д) тип: радиально-упорный;
 - е) вес: 2,3 кг;
 - ж) смазка: литиевая.
- Технические характеристики 860 ГОСТ 23179-78:
- а) внутренний диаметр: 100 мм;
 - б) нагрузка: до 200 кН (радиальная), до 150 кН (осевая);
 - в) рабочая температура: $-30...+120\text{ }^{\circ}\text{C}$;
 - г) скорость вращения: до 3500 об/мин;
 - д) тип: радиально-упорный;
 - е) вес: 1,8 кг;
 - ж) смазка: литиевая.



Рисунок 6 – Подшипник 872 ГОСТ 23179-78

3.4 Ременная передача

Ременная передача предназначена для передачи вращательных моментов в третьем звене манипулятора, представленном в приложении Е, обеспечивая плавное вращение пятого сустава.

Требования:

- диаметр делительной окружности: не менее 100 мм;
- номинальная нагрузка на растяжение: не менее 200 Н/мм;
- ширина ремня: не менее 30 мм;
- длина ремня: не менее 4500 мм;

В данной работе используется ременная передача, состоящая из зубчатого шкива BSY-Z50-8M-30-AF-d19 и зубчатого ремня HTD 8M и 8M-30-N.

- Технические характеристики BSY-Z50-8M-30-AF-d19:
 - а) диаметр делительной окружности: 130 мм;
 - б) ширина ремня: 30 мм;
 - в) профиль шкива: HTD 8M;
 - г) типоразмер: 8M-30;
 - д) вес: 0,2 кг;



Рисунок 7 – Шкив зубчатый BSY-Z50-8M-30-AF-d19

- Технические характеристики HTD 8M и 8M-30-N:
 - а) профиль зубцов: HTD 8M;
 - б) ширина: 30 мм;
 - в) длина: до 100 м;
 - г) твердость по Шору: 75 А;
 - д) сила сцепления полимерной базы: 10 Н/мм;
 - е) сила сцепления корда: 700 Н/мм;

ж) номинальная нагрузка на растяжение: 240 Н/мм.



Рисунок 8 – Зубчатый ремень HTD 8M и 8M-30-N

3.5 Драйверы двигателей

Драйверы предназначены для управления шаговыми двигателями, обеспечивая точное позиционирование, регулировку тока и защиту от перегрузок.

Требования:

- рабочее напряжение: 20–70 В;
- ток: до 7 А;
- микрошаг: не более 1/128;
- рабочая температура: от 0 до +45 °С;
- поддержка замкнутого контура.

В данной работе используется драйвер HBS860H (6 шт.).

Технические характеристики:

- рабочее напряжение: 20–80 В;
- ток: до 8 А;
- микрошаг: до 1/256;
- рабочая температура: 0...+50 °С;
- поддержка замкнутого контура: совместимость с энкодерами;
- защита: от перегрузки, перегрева, короткого замыкания;
- интерфейсы: Ethernet;

- вес: 0,6 кг;

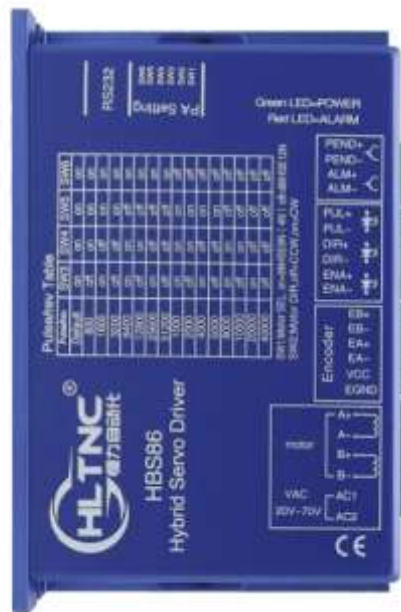


Рисунок 9 – Драйвер HBS860H

3.6 Сетевой концентратор

Сетевой концентратор обеспечивает связь между процессорным модулем и датчиками для передачи данных в реальном времени.

Требования:

- количество портов: не менее 8;
- скорость: не менее 100 Мбит/с;
- питание: 9–12 В, до 1 А;
- рабочая температура: от 0 до +45 °С.

В данной работе используется TP-Link TL-SG1008D.

Технические характеристики:

- количество портов: 8 x 10/100/1000 Мбит/с;
- пропускная способность: 16 Гбит/с;
- питание: 9 В, 0,6 А;
- рабочая температура: 0...+40 °С;
- размеры: 180 x 90 x 25,5 мм;
- вес: 0,4 кг;

- корпус: пластиковый.



Рисунок 10 – TP-Link TL-SG1008D

3.7 LiDAR

LiDAR используется для 3D-сканирования окружающей среды, обеспечивая адаптивное управление манипулятором.

Требования:

- дальность: не менее 20 м;
- угол обзора: $360^\circ \times 80^\circ$;
- точность: не ниже 3 см;
- питание: 12 В;
- рабочая температура: от 0 до $+45^\circ\text{C}$;
- интерфейс: Ethernet.

В данной работе используется Unitree 4D LiDAR L1 (2 шт.).

Технические характеристики:

- дальность: 0,1–30 м;
- угол обзора: $360^\circ \times 90^\circ$;
- точность: 2 см;
- питание: 12 В, 10 Вт;
- рабочая температура: $-10\dots+50^\circ\text{C}$;
- частота сканирования: 10 Гц;
- интерфейс: Ethernet, USB;
- разрешение: $0,1^\circ$ (угловое);
- вес: 0,65 кг;

- корпус: IP65.



Рисунок 11 – Unitree 4D LiDAR L1

3.8 Кнопка аварийной остановки

Кнопка предназначена для немедленного останова манипулятора в аварийной ситуации, обеспечивая безопасность оператора.

Требования:

- напряжение: до 200 В;
- ток: до 8 А;
- тип: нормально-замкнутая;
- рабочая температура: от 0 до +45 °С;
- степень защиты: IP65.

В данной работе используется кнопка EKF XB4 SQXB4BSF-R.

Технические характеристики:

- напряжение: до 250 В;
- ток: до 10 А;
- тип: нормально-замкнутая;
- рабочая температура: -25...+70 °С;

- степень защиты: IP65;
- механическая прочность: до 1 млн циклов;
- диаметр: 22 мм;
- вес: 0,1 кг;
- материал: пластик/металл.



Рисунок 12 – EKF XB4 SQXB4BSF-R

3.9 Кнопка со светодиодом

Кнопка со светодиодом используется для индикации состояния манипулятора и управления его функциями.

Требования:

- напряжение: до 12 В;
- ток: до 300 мА;
- светодиод: любой цвет;
- рабочая температура: от 0 до +45 °С;
- степень защиты: IP54.

В данной работе используется кнопка E-Switch PBH2UOANAGX.

Технические характеристики:

- напряжение: 12 В;
- ток: до 200 мА;
- светодиод: зеленый;
- рабочая температура: -20...+70 °С;

- степень защиты: IP54;
- механическая прочность: до 500 тыс. циклов;
- диаметр: 16 мм;
- вес: 0,05 кг;
- материал: пластик.



Рисунок 13 – E-Switch PBH2UOANAGX

3.10 Вентилятор

Вентиляторы предназначены для охлаждения электроники манипулятора, предотвращая перегрев компонентов.

Требования:

- питание: 12 В, до 0,2 А;
- воздушный поток: не менее 60 CFM;
- рабочая температура: от 0 до +45 °С;
- уровень шума: не более 35 дБ.

В данной работе используются вентиляторы ARCTIC P14 (2 шт.).

Технические характеристики:

- скорость: 200–1700 об/мин;
- питание: 12 В, 0,15 А;
- воздушный поток: 72,8 CFM;
- рабочая температура: -10...+70 °С;
- уровень шума: до 22,5 дБ;

- размеры: 140 x 140 x 27 мм;
- вес: 0,19 кг;
- подшипник: гидродинамический;
- срок службы: 100 000 ч.



Рисунок 14 – ARCTIC P14

3.11 Контроллер

Контроллер предназначен для управления манипулятором, обработки данных с LiDAR и реализации адаптивной системы управления.

Требования:

- процессор: не менее 6 ядер;
- питание: 9–20 В, до 30 Вт;
- интерфейсы: Ethernet;
- рабочая температура: от 0 до +45 °С;
- поддержка: ROS.

В данной работе используется процессорный модуль NVIDIA Jetson Orin NX.

Технические характеристики:

- процессор: 8-ядерный ARM Cortex-A78AE, 2 ГГц;
- GPU: 1024 CUDA-ядра, 32 Tensor-ядра;
- питание: 9–19 В, до 25 Вт;

- интерфейсы: Ethernet (1 Гбит/с), USB 3.0, CAN;
- память: 16 ГБ LPDDR5;
- хранилище: поддержка NVMe SSD;
- рабочая температура: -25...+80 °C;
- размеры: 100 x 87 x 16 мм;
- вес: 0,3 кг;
- поддержка: ROS.



Рисунок 15 – NVIDIA Jetson Orin NX

Расчет нагруженности системы

Общая потребляемая мощность системы рассчитана с учетом всех компонентов:

Двигатели и драйверы:

- ось 1: $80 \text{ В} \times 6 \text{ А} = 480 \text{ Вт}$;
- ось 2: $80 \text{ В} \times 6 \text{ А} = 480 \text{ Вт}$;
- ось 3: $80 \text{ В} \times 5,6 \text{ А} = 448 \text{ Вт}$;
- ось 4: $80 \text{ В} \times 4,2 \text{ А} = 336 \text{ Вт}$;
- ось 5: $80 \text{ В} \times 4,2 \text{ А} = 336 \text{ Вт}$;
- ось 6: $80 \text{ В} \times 4,2 \text{ А} = 336 \text{ Вт}$.

Итого: 2416 Вт.

LiDAR: $2 \times 10 \text{ Вт} = 20 \text{ Вт}$

Вентиляторы: $2 \times 12 \text{ В} \times 0,15 \text{ А} = 3,6 \text{ Вт}$

Процессорный модуль: 25 Вт.

Кнопки и прочее: 5 Вт.

Общая мощность: $2416 + 20 + 3,6 + 25 + 5 = 2465 \text{ Вт}$

3.12 Преобразователь напряжения 12 В

Преобразователь используется для питания LiDAR и вентиляторов, обеспечивая стабильное напряжение.

Требования:

- входное напряжение: 36–60 В;
- выход: 12 В, не менее 40 Вт;
- КПД: не менее 75%;
- рабочая температура: от 0 до +45 °С;
- защита от перегрузки.

В данной работе используется Mean Well SD-50A-12.

Технические характеристики:

- входное напряжение: 36–72 В;
- выход: 12 В, 4,2 А;
- мощность: 50 Вт;
- КПД: 83%;
- рабочая температура: -10...+60 °С;
- защита: от перегрузки, короткого замыкания, перенапряжения;
- размеры: 159 x 97 x 38 мм;
- вес: 0,48 кг;
- корпус: металлический, с пассивным охлаждением.



Рисунок 16 – Mean Well SD-50A-12

3.13 Преобразователь напряжения 9 В

Преобразователь используется для питания сетевого концентратора, обеспечивая стабильное напряжение.

Требования:

- входное напряжение: 36–60 В;
- выход: 9 В, не менее 20 Вт;
- КПД: не менее 75%;
- рабочая температура: от 0 до +45 °С;
- защита от короткого замыкания.

В данной работе используется Mean Well SD-25A-9.

Технические характеристики:

- входное напряжение: 36–72 В;
- выход: 9 В, 2,8 А;
- мощность: 25 Вт;
- КПД: 80%;
- рабочая температура: -10...+60 °С;
- защита: от перегрузки, короткого замыкания, перенапряжения;
- размеры: 99 х 97 х 36 мм;

- вес: 0,3 кг;
- корпус: металлический, с пассивным охлаждением.



Рисунок 17 – Mean Well SD-25A-9

3.14 Блок питания

Блок питания обеспечивает энергоснабжение всех компонентов системы, включая двигатели, драйверы и электронику.

Требования:

- выходное напряжение: 48 В;
- мощность: не менее 2500 Вт;
- вход: 90–264 В AC;
- КПД: не менее 90%;
- рабочая температура: от 0 до +45 °C;
- защита от перегрузки и перегрева.

В данной работе используются два блока питания Mean Well UHP-1500-48.

Технические характеристики:

- выход: 48 В, 31,5 А;
- мощность: 1500 Вт;
- КПД: 95%;
- вход: 90–264 В AC;

- рабочая температура: -30...+70 °С;
- защита: от перегрузки, короткого замыкания, перегрева, перенапряжения;
- размеры: 310 x 140 x 60 мм;
- вес: 3,3 кг;
- охлаждение: встроенный вентилятор;
- срок службы: 50 000 ч.



Рисунок 18 – Mean Well UHP-2500-48

5 Разработка схемы электрической принципиальной

Электрическая принципиальная схема и перечень элементов приведены в приложении А.

5.1 Подключение микроконтроллера

Микроконтроллер АМ питается от напряжения +12 В, поступающего от DC-DC преобразователя UZ3. Порт ETHERNET соединён с Ethernet-коммутатором U1. Порт GPIO01 используется для подключения кнопки SB2 через подтягивающий резистор R1. Порты GPIO07, GPIO12, GPIO13 и GPIO14 подключены к вентиляторам охлаждения M1 и M2.

Порту GPIO01 микроконтроллера, работает на уровне сигнала 3,3 В. Для обеспечения подтяжки к питанию используется резистор R1, подключённый между кнопкой и линией +12 В. Ток на входе порта $I=0,025\text{A}$.

$$R1 = \frac{U}{I} = \frac{12\text{В}}{0,025\text{A}} = 470 \text{ Ом}$$

Ближайший стандартный номинал резистора — 470 Ом.

5.2 Подключение лазерных сканеров

Лазерные сканеры S1, S2 питаются от напряжения +12 В, поступающего от DC-DC преобразователя UZ3. Порты ETHERNET сканеров подключены к Ethernet-коммутатору U1.

4.3 Подключение Ethernet-коммутатора

Ethernet-коммутатор U1 питается от напряжения +9 В, поступающего от DC-DC преобразователя. Порт ETHERNET0 соединён с микроконтроллером АМ, порты ETHERNET1 и ETHERNET2 подключены к лазерным сканерам S1 и S2, а порты ETHERNET3–ETHERNET8 соединены с драйверами серводвигателей DD1–DD6.

5.4 Подключение драйверов серводвигателей

Драйверы серводвигателей DD1–DD6 питаются от напряжения +48 В, поступающего от источников питания UZ1 и UZ2. Порты ETHERNET драйверов соединены с Ethernet-коммутатором U1. Выходы А+, А-, В+, В- подключены к серводвигателям, а выводы EB+, EB-, EA+, EA- и VCC, EGND к энкодерам, встроенным в двигатели.

5.5 Подключение серводвигателей с энкодерами

Серводвигатели M1–M6 подключены к соответствующим драйверам DD1–DD6 через выводы А+, А-, В+, В- для управления. Энкодеры соединены с драйверами через выводы EB+, EB-, EA+, EA- для передачи данных о положении, а выводы VCC и EGND обеспечивают питание энкодеров.

5.6 Подключение блоков питания

Источники питания UZ1 и UZ2 с входом ~400 В, 50 Гц выдают +48 В через выключатель SB1 на драйверы серводвигателей. DC-DC преобразователь UZ3 с входом +48 В выдаёт +12 В для питания микроконтроллера и сканеров. DC-DC преобразователь UZ4 с входом +48 В выдаёт +9 В для питания Ethernet-коммутатора.

6 Разработка кинематической модели манипулятора

Проанализировав техническое задание и требования к системе, была составлена кинематическая модель 6-осевого манипулятора (рисунок 17). А также составлена таблица с параметрами Денавита-Хартенберга (таблица 1) и (таблица 2) для их последующего использования в расчетах матрицы Денавита-Хартенберга.

6.1 Определение системы координат

Для каждого звена манипулятора устанавливается система координат, связанная с его сочленением. Системы координат определяются следующим образом:

- базовая система координат: прикреплена к основанию манипулятора;
- система координат звена 1: связана с первым звеном, вращающимся вокруг вертикальной оси Z_1 ;
- система координат звена 2: прикреплена ко второму звену, вращающемуся вокруг оси Z_2 ;
- система координат звена 3: связана с третьим звеном, вращающемуся вокруг оси Z_3 ;
- система координат звена 4: прикреплена к четвертому звену, вращающемуся вокруг оси Z_4 ;
- система координат звена 5: связана с пятым звеном, вращающемуся вокруг оси Z_5 ;
- система координат звена 6: прикреплена к шестому звену, вращающемуся вокруг оси Z_6 ;
- система координат конечного эффектора: прикреплена к конечному эффектору.

6.2 Параметры Денавита-Хартенберга

По заданным в техническом задании габаритам манипулятора были заданы ограничения на скручивание звеньев представленные в таблице 2.

Таблица 1 - Параметры Денавита-Хартенберга

№ звена	a_i (мм)	α_i (°)	d_i (мм)	Θ_i (°)
1	0	90	393	Θ_1
2	700	0	0	Θ_2
3	0	90	0	Θ_3
4	0	90	700	Θ_4
5	0	-90	0	Θ_5
6	0	0	174	Θ_6

Таблица 2 – Ограничения на скручивание

№ звена	1	2	3	4	5	6
Ограничение (°)	(-172, 172)	(-90, 90)	(-132, 132)	(-172, 172)	(-115, 115)	(-180, 180)

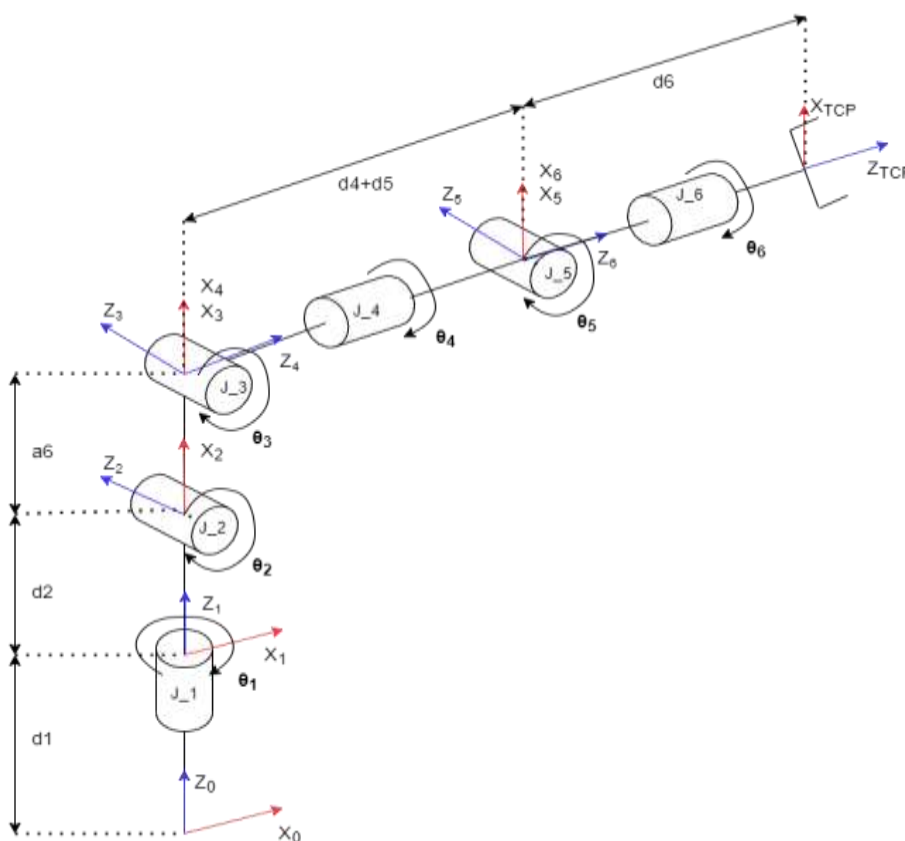


Рисунок 19 – Кинематическая модель 6-осевого манипулятора

7 Проектирование манипулятора

В данной главе приведено описание основных конструктивных узлов манипулятора и расчёты на прочность наиболее нагруженных деталей и узлов.

7.1 Основание

После исследования подобных решений было принято решение конструкцию основания изготавливать из цилиндрической стальной заготовки, так как подобный материал будет обеспечивать достаточную прочность.

Данная конструкция является сильно нагруженной. В связи с этим было проведено моделирование нагрузки на данный элемент конструкции и выполнен расчёт прочности.

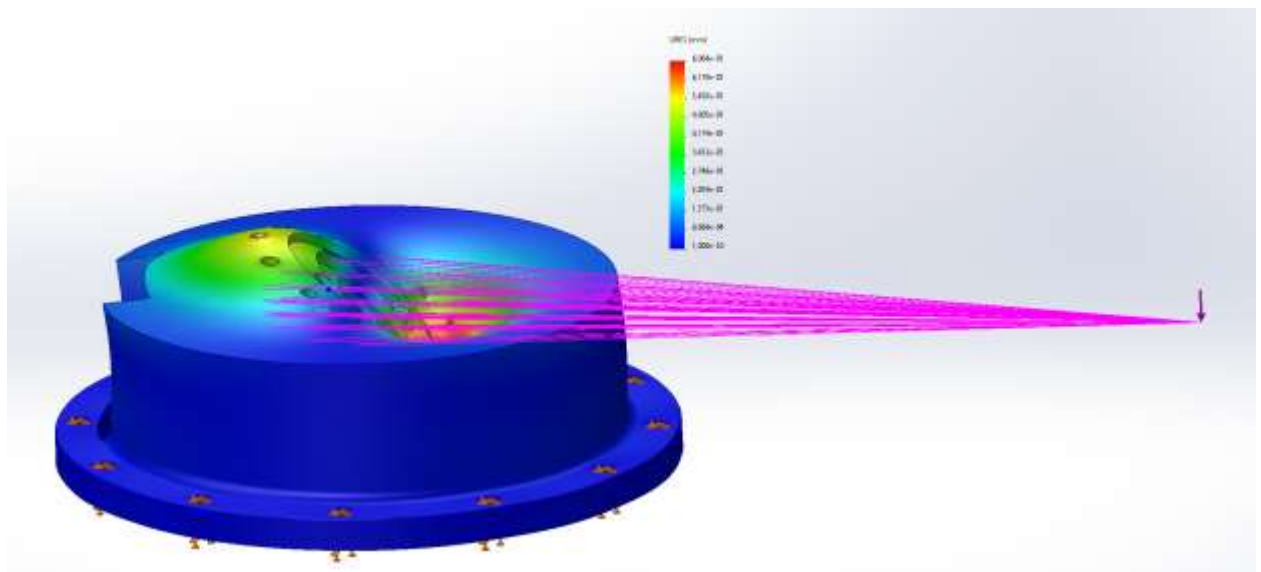


Рисунок 20 – Расчет основания на прочность

Так максимальная деформация при удаленной на 0,75 м вертикальной нагрузке 1200 Н составила 0,2 мм.

Чертежи основания приведены в приложении Б.

7.2 Первое звено

Было принято решение конструкцию первого звена изготавливать из стальной заготовки, так как подобный материал будет обеспечивать достаточную прочность.

Данная конструкция является сильно нагруженной. В связи с этим было проведено моделирование нагрузки на данный элемент конструкции и выполнен расчёт прочности.

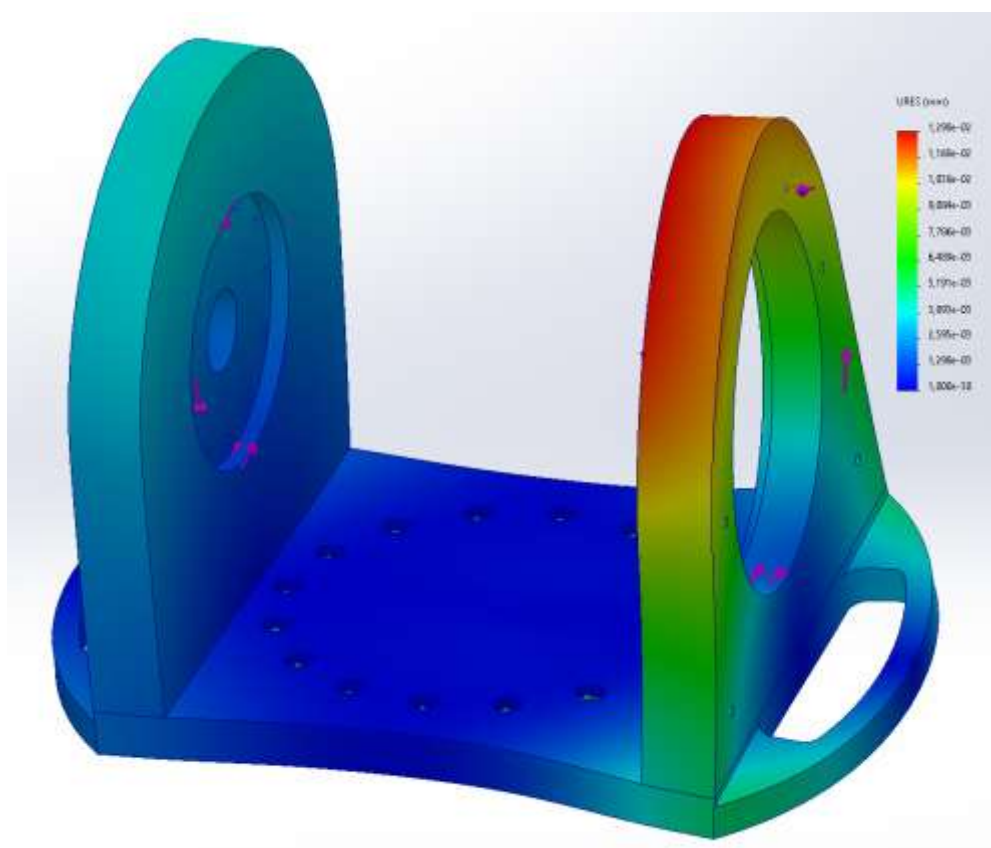


Рисунок 21 – Расчет первого звена на прочность

Так максимальная деформация при приложении вращающего момента в $1200 \text{ Н} \times \text{м}$ составила $0,2 \text{ мм}$.

Чертежи основания приведены в приложении В.

7.3 Второе звено

Конструкция второго звена изготавливается из композита на основе углеволокна, так как подобный материал будет обеспечивать достаточную прочность и легкость.

Данная конструкция является сильно нагруженной. В связи с этим было проведено моделирование нагрузки на данный элемент конструкции и выполнен расчёт прочности.

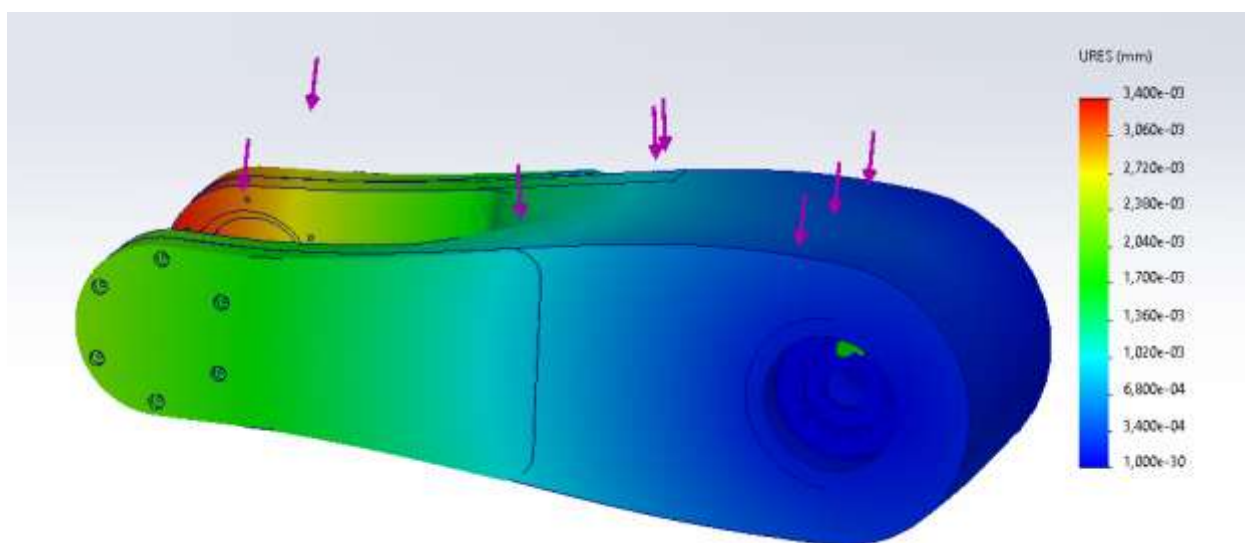


Рисунок 22 – Расчет второго звена на прочность

Так максимальная деформация при приложении силы в 1200 Н составила 0,1 мм.

Чертежи основания приведены в приложении Г.

7.4 Третье звено

Конструкция третьего звена изготавливается из композита на основе углеволокна, так как подобный материал будет обеспечивать достаточную прочность и легкость.

Данная конструкция является сильно нагруженной. В связи с этим было проведено моделирование нагрузки на данный элемент конструкции и выполнен расчёт прочности.

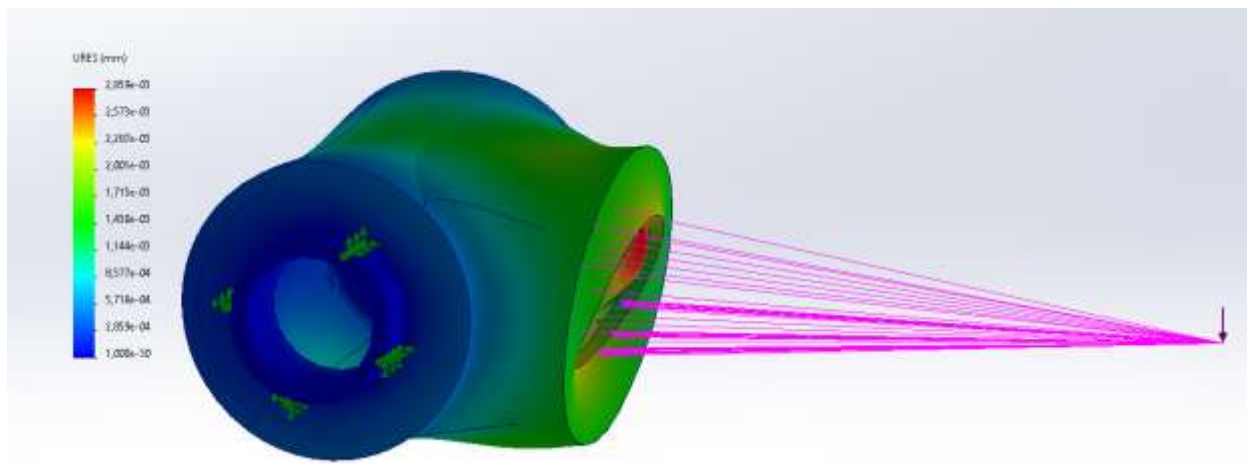


Рисунок 23 – Расчет третьего звена на прочность

Так максимальная деформация при удаленной на 0,5 м вертикальной нагрузке 700 Н составила 0,1 мм.

Чертежи основания приведены в приложении Д.

7.5 Четвертое звено

Конструкция второго звена изготавливается из композита на основе углеволокна, так как подобный материал будет обеспечивать достаточную прочность и легкость.

Данная конструкция является сильно нагруженной. В связи с этим было проведено моделирование нагрузки на данный элемент конструкции и выполнен расчёт прочности.

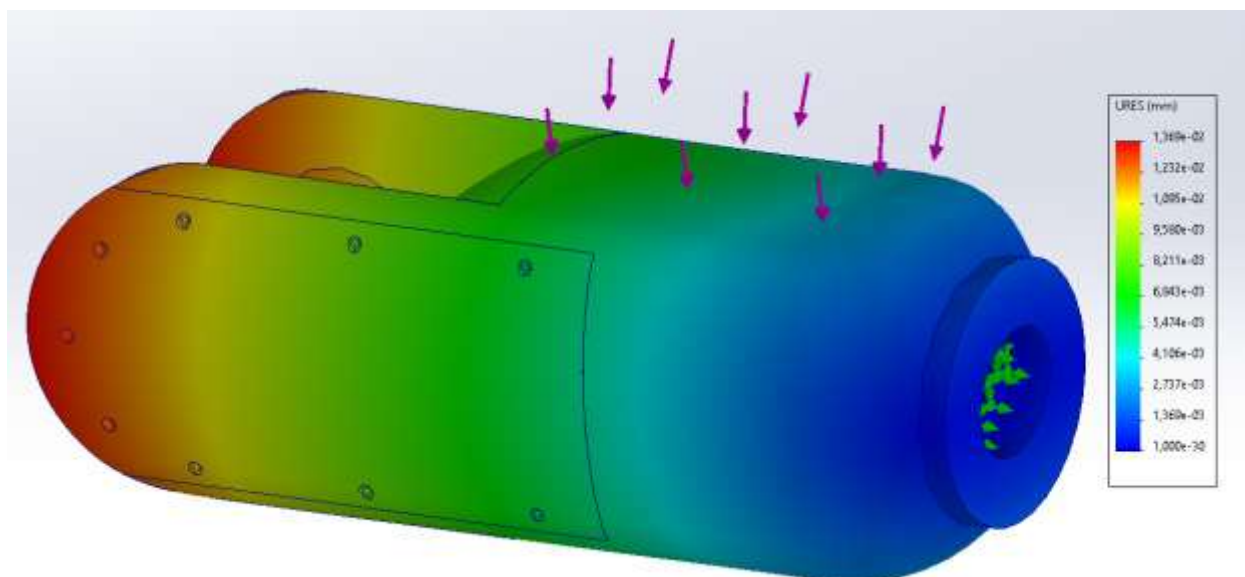


Рисунок 24 – Расчет второго звена на прочность

Так максимальная деформация при приложении силы в 700 Н составила 0,1 мм.

Чертежи основания приведены в приложении Е.

7.6 Пятое звено

Конструкция третьего звена изготавливается из композита на основе углеволокна, так как подобный материал будет обеспечивать достаточную прочность и легкость.

Данная конструкция является сильно нагруженной. В связи с этим было проведено моделирование нагрузки на данный элемент конструкции и выполнен расчёт прочности.

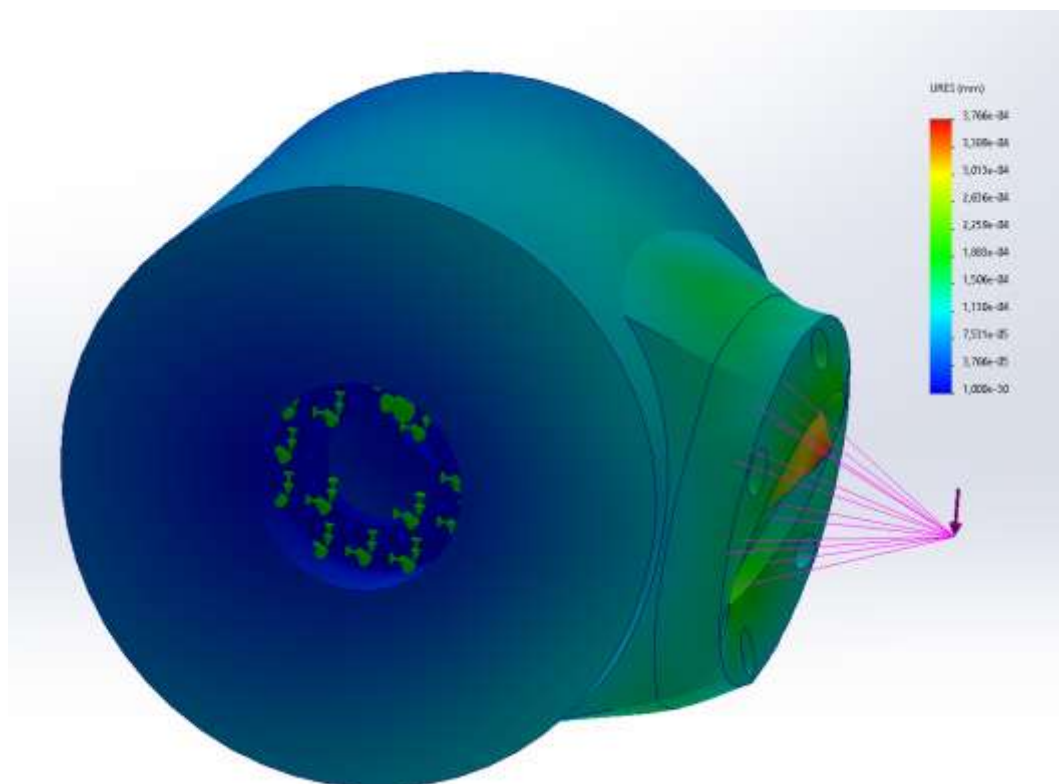


Рисунок 25 – Расчет третьего звена на прочность

Так максимальная деформация при удаленной на 0,2 м вертикальной нагрузке 150 Н составила 0,06 мм.

Чертежи основания приведены в приложении Ж.

7.7 Манипулятор целиком

По итогу проектирования и проверки на прочность звеньев манипулятора была собрана сборочная единица.



Рисунок 26 – Сборочная единица манипулятора

Чертежи сборочной единицы приведены в приложении И.

8 Расчет точности позиционирования

8.1 Погрешность от допусков на звенья

Исходя из допусков на изготовление частей манипулятора равных $\pm IT14/2$ и габаритов звеньев получаем допуски для каждой из частей приведенные в таблице 3.

Таблица 3 – Допуски на размеры звеньев манипулятора

Звено	Допуск
Основание (δ_1)	$\pm 0,07$ мм
Первое звено (δ_2)	$\pm 0,052$ мм
Второе звено (δ_3)	$\pm 0,08$ мм
Третье звено (δ_4)	$\pm 0,04$ мм
Четвертое звено (δ_5)	$\pm 0,07$ мм
Пятое звено (δ_6)	$\pm 0,04$ мм

Для расчета погрешности воспользуемся среднеквадратичной погрешностью и рассчитаем ее по формуле:

$$\delta_{\text{лин}} = \sqrt{\delta_1^2 + \delta_2^2 + \delta_3^2 + \delta_4^2 + \delta_5^2 + \delta_6^2}$$

где $\delta_{\text{лин}}$ – общая погрешность от допусков, мм;

Так среднеквадратичная погрешность составила 0,1486 мм.

8.2 Угловые погрешности приводов

Угловые ошибки состоят из:

- ошибка серводвигателя: шаг выбранного двигателя равен $1,8^\circ$ с ошибкой не более 5% от шага ($\sigma_{\text{мотор}} \approx 0,05196^\circ$);
- ошибка редукторов: люфт выбранных редукторов не превышает 10 угловых секунд.

Погрешность редукторов в градусах:

$$10arcsec = \frac{10}{3600} \approx 0,002778^\circ$$

Рассчитаем угловые ошибки для каждого сустава с учетом редукторов:

– суставы 1 и 5: 2 последовательных редуктора 1:100, итоговое передаточное число 1:10000.

а) Ошибка мотора на выходе:

$$\sigma_{\text{мотор}}/10000 = 0,05196^\circ/10000 = 5,196 \times 10^{-6}^\circ$$

б) Ошибка первого редуктора делится на передаточное число второго:

$$0,002778^\circ/100 = 2,778 \times 10^{-5}^\circ$$

в) Ошибка второго редуктора: $0,002778^\circ$

г) Общее угловое стандартное отклонение ($\sigma_{\theta n}$):

$$\sigma_{\theta n} = \sqrt{(5,196 \times 10^{-6})^2 + (2,778 \times 10^{-5})^2 + 0,002778^2} \approx 0,002778^\circ \approx 4,93 \times 10^{-5} \text{ рад}$$

– суставы 2, 3, 4, 6: 1 редуктор 1:100.

а) Ошибка мотора на выходе:

$$\sigma_{\text{мотор}}/100 = 0,05196^\circ/100 = 5,196 \times 10^{-4}^\circ$$

б) Ошибка редуктора: $0,002778^\circ$

в) Общее угловое стандартное отклонение ($\sigma_{\theta n}$):

$$\sigma_{\theta n} = \sqrt{(5,196 \times 10^{-4})^2 + 0,002778^2} \approx 0,002826^\circ \approx 4,84 \times 10^{-5} \text{ рад}$$

8.3 Линейные погрешности приводов

Для расчета линейных погрешностей по формуле 1 приводов, возьмем длины звеньев из матрицы Денавита-Хартенберга (таблица 1). Получившиеся значения представлены в таблице 4.

$$\sigma_n = L \times \sigma_{\theta n} \quad (1)$$

где σ_n – линейная ошибка, мм;

L – расстояние до рабочей точки, мм;

$\sigma_{\theta n}$ – угловая ошибка, рад.

Таблица 4 – Линейные погрешности приводов

№ Сустава	Расстояние до рабочей точки (мм)	Линейная ошибка (мм)
1	1574	0,077
2	1574	0,076
3	874	0,042
4	824	0,0086
5	174	0,0084

Линейное отклонение не применимо к 6 суставу, так как он и является рабочей точкой и отклонение стремиться к 0.

Общая линейная погрешность ($\sigma_{\theta, \text{общ}}$) приводов вычисляется по формуле:

$$\sigma_{\theta, \text{общ}} = \sqrt{\sigma_1^2 + \sigma_2^2 + \sigma_3^2 + \sigma_4^2 + \sigma_5^2}$$

И равняется 0,15мм.

8.4 Общая погрешность позиционирования

Общая погрешность позиционирования ($\sigma_{\text{общ}}$) вычисляется по формуле:

$$\sigma_{\text{общ}} = \sqrt{\delta_{\text{лин}}^2 + \sigma_{\theta, \text{общ}}^2}$$

Откуда общая погрешность позиционирования равняется 0,2 мм.

9 Создание симуляционной модели

В данной главе приведено описание процесса создания и настройки симуляционной модели 6-осевого манипулятора в программе gazebo sim пакета ROS2 Jazzy.

9.1 Создание проекта

Для запуска и полной работоспособности симуляции пакет ROS требует файл проекта определенной структуры. Поэтому первым этапом создания симуляции является генерация папки проекта. Для этого были проведены следующие действия через терминал:

- подключение пакета ROS2 Jazzy
`source /opt/ros/jazzy/setup.bash`
- создание папок проекта определенной структуры и переход в них
`mkdir ~/ros2_ws/src`
`cd ~/ros2_ws/src`
- создание пакета проекта
`ros2 pkg create --build-type ament_cmake rviz_test_ws`
- скачивание необходимых плагинов
`sudo apt install [название необходимого плагина]`

9.2 Экспорт модели манипулятора

После сборки робота и задания осей запускаем плагин импорта и задаем для каждого из подвижных соединений параметры двигателей и массовые характеристики, проверяем, что все шарниры являются вращательными. А затем производим импорт.

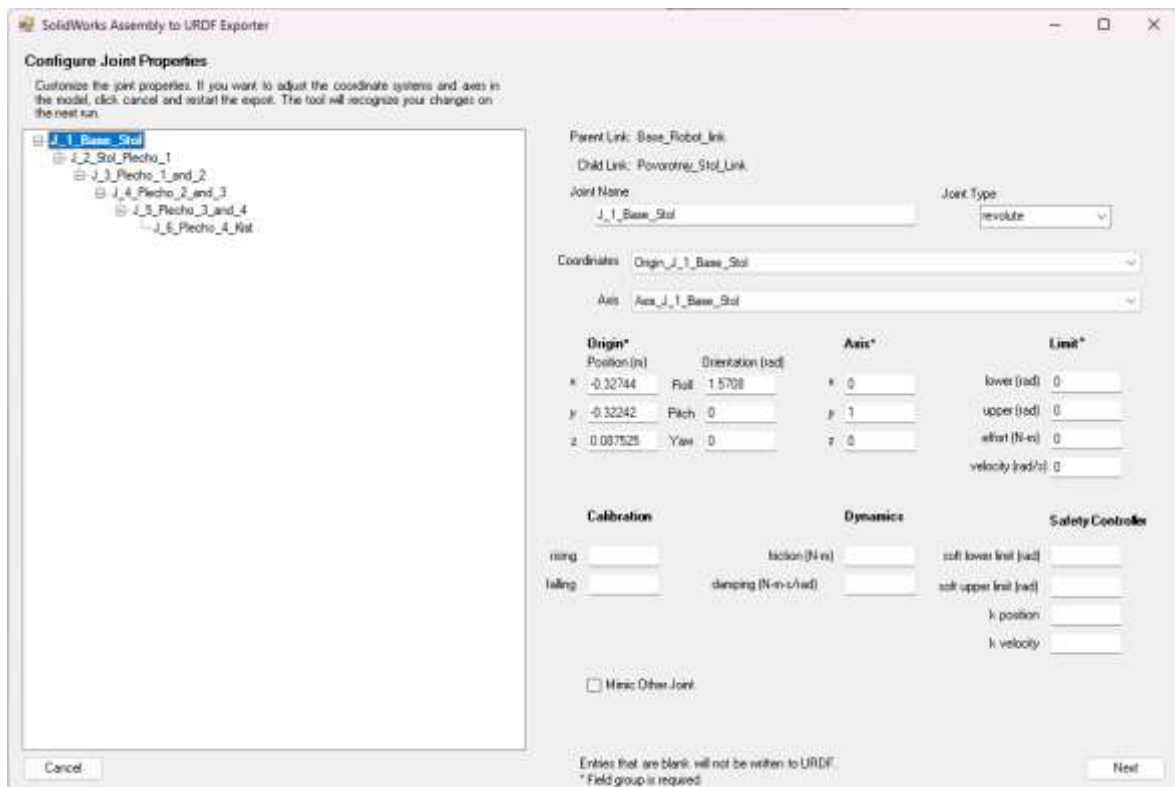


Рисунок 27 – Окно плагина для импорта модели

9.3 Настройка SDF файла описания модели

После создания плагином экспорта, моделей звеньев манипулятора и написания файла описания робота были сделаны следующие доработки:

- конвертация файла описания из формата URDF в SDF;
- изменение пути расположения моделей звеньев;
- корректировка смещения и ориентация системы координат для каждого из звеньев;
- добавление и настройка плагинов:
 - а) задания движения звеньев;
 - б) трансляции состояния звеньев;
 - в) симулирования и трансляции данных LiDAR.
- добавление и настройка сенсоров:
 - а) детектирования столкновений;
 - б) генерации данных LiDAR;

Файл описания со всеми доработками приведен в приложении К.

9.4 Создание файла запуска симуляции

Для запуска симуляции со всеми плагинами и сенсорами, а также их корректной работы был разработан файл запуска, состоящий из следующих пунктов:

- импорт необходимых библиотек;
- настройка пути расположения проекта;
- написание нод для каждого ключевого действия:
 - а) запуск плагина трансляции положения звеньев;
 - б) запуск симуляции Gazebo Sim;
 - в) запуск пакета визуализации Rviz;
 - г) построение модели манипулятора;
 - д) настройка моста связи между Gazebo и ROS.
- запуск всех нод.

Файл запуска приведен в приложении Л.

9.5 Доработка и добавление файлов подгрузки плагинов и конфигурации симуляции

При создании проекта были сгенерированы 2 файла, которые необходимо доработать для дальнейшей работы с симуляцией:

- так в CMakeLists.txt необходимо:
 - а) указать пакеты ROS, которые необходимо добавить в проект;
 - б) указать папки проекта, необходимые для его работоспособности.
- в package.xml необходимо прописать все пакеты ROS, которые будут использованы в проекте.

Файлы подгрузки приведены в приложении М.

Также необходимо интегрировать в программу файл конфигурации визуализации Rviz, который приведен в приложении Н.

9.6 Сборка проекта

После написания, доработки и размещения в нужные папки всех файлов симуляции ее необходимо собрать и запустить. Для этого:

- переходим в основную папку с проектом
`cd ~/ros2_ws`
- настраиваем взаимосвязь ROS с данным проектом
`source install/setup.bash`
- собираем проект
`colcon build`
- запускаем симуляцию
`ros2 launch robot_rviz joint_states.launch.py`

При разработке программы управления в пакете ROS было выявлено, что информация от симуляции транслируется в каналы последовательно, что сильно сказывается на быстродействии и стабильности всей системы. Поэтому для повышения приведенных параметров была разработана структура (рисунок 28), состоящая из нескольких программ, работающих параллельно.

Для работы с 3D данными и облаками точек LiDAR использовалась библиотека Open3D, позволяющая работать с 3D данными, облаками точек LiDAR и SDF файлом описания модели.

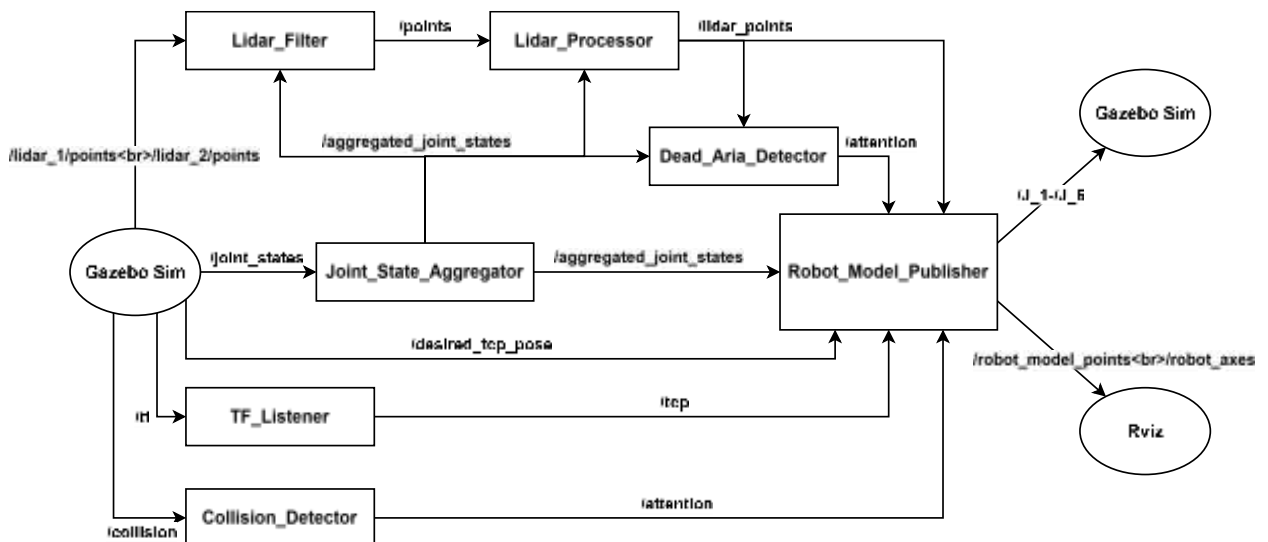


Рисунок 28 – Структура алгоритма управления (Граф ROS)

10.1 Основной алгоритм управления (Robot_Model_Publisher)

Основной алгоритм управления отвечает за объединение данных от всех программ, их анализ и управление звеньями манипулятора. Его основными этапами являются:

- инициализация: Настройка начальных параметров подпрограммы
- получение желаемых координат инструмента: Система принимает целевые координаты положения и ориентации инструмента;

- считывание состояния кнопки разрешения движения;
- расчет обратной задачи кинематики: выполняется проверка достижимости желаемой точки с использованием обратной кинематики. Если точка недостижима, выводится сообщение об ошибке, и процесс останавливается;
- считывание реальных координат инструмента;
- вычисление расстояния между точками: рассчитывается евклидово расстояние и угловое отклонение между текущими и желаемыми координатами для определения, достигнута ли цель с точностью 1 мм и 0,005 радиан;
- получение данных LiDAR;
- планирование пути методом потенциальных полей;
- чтение сообщений в топике прерываний: система проверяет наличие сигналов остановки;
- публикация в топике управления двигателями;
- проверка достижения точки: система определяет, достигнута ли текущая точка траектории;
- проверка завершения траектории: если достигнута последняя точка, система публикует сообщение о завершении задачи, и работа останавливается. В противном случае цикл продолжается до достижения конечной цели.

Основной алгоритм представлен в приложении П.

10.2 Алгоритм планирования траектории

Подпрограмма планирования траектории отвечает за вычисление безопасного и эффективного пути движения манипулятора. Она использует метод потенциальных полей и включает следующие шаги:

- инициализация: настройка начальных параметров подпрограммы;

- получение координат и данных LiDAR: считываются желаемые и реальные координаты инструмента, а также данные LiDAR;
- расчет притягивающей силы: определяется сила, направляющая манипулятор к цели и рассчитывается по формуле:

$$\overline{F_{att}} = k_{att} \times (\overline{P_{target}} - \overline{P_{current}}),$$

$$\overline{F_{att, norm}} = \frac{\overline{F_{att}}}{||\overline{F_{att}}||_2}, \text{ если } ||\overline{F_{att}}||_2 \neq 0,$$

где $\overline{F_{att}}$ - вектор притягивающей силы, Н;

k_{att} - коэффициент усиления притягивающей силы;

$\overline{P_{target}}$ - вектор позиции цели, м;

$\overline{P_{current}}$ - вектор текущей позиции инструмента, м;

$\overline{F_{att, norm}}$ - нормализованный вектор притягивающей силы, Н;

$||\overline{F_{att}}||_2$ - евклидова норма вектора $\overline{F_{att}}$, Н.

- расчет отталкивающей силы: вычисляется сила, отталкивающая манипулятор от препятствий и рассчитывается по формуле:

$$d_i = ||\overline{P_{link, j}} - \overline{P_{obst, i}}||_2,$$

$$F_{rep, i} = \begin{cases} k_{rep} \times \left(\frac{1}{d_i} - \frac{1}{d_{rep}} \right) \times \frac{1}{d_i^2}, & \text{если } d_i < d_{rep}, \\ 0 & \text{иначе} \end{cases}$$

$$\overline{F_{rep, i}} = F_{rep, i} \times \frac{\overline{P_{link, j}} - \overline{P_{obst, i}}}{d_i},$$

$$\overline{F_{rep}} = \sum_i \overline{F_{rep, i}},$$

$$\overline{F_{att, norm}} = \frac{\overline{F_{att}}}{||\overline{F_{att}}||_2}, \text{ если } ||\overline{F_{att}}||_2 \neq 0,$$

где d_i - расстояние между звеном (j) и препятствием (i), м;

$\overline{P_{link, j}}$ - вектор позиции (j)-го звена манипулятора, м;

$\overline{P_{obst, i}}$ - вектор позиции (i)-го препятствия, м;

$F_{rep, i}$ - скалярная величина отталкивающей силы для (i)-го препятствия, Н;

k_{rep} - коэффициент усиления отталкивающей силы;

d_{rep} - радиус действия отталкивающей силы, м;

$\overline{F_{rep, i}}$ - вектор отталкивающей силы от (i)-го препятствия, Н;

$\overline{F_{rep}}$ - суммарный вектор отталкивающих сил, Н;

$\overline{F_{att, norm}}$ - нормализованный вектор притягивающей силы, Н;

$||\overline{F_{att}}||_2$ - евклидова норма вектора F_{att} , Н.

– суммирование и нормализация сил: силы объединяются и нормализуются для определения направления движения и рассчитываются по формуле:

$$\overline{F_{total}} = \overline{F_{att, norm}} + \overline{F_{rep, norm}},$$

$$\overline{F_{total, norm}} = \frac{\overline{F_{total}}}{||\overline{F_{total}}||_2}, \text{ если } ||\overline{F_{total}}||_2 \neq 0,$$

где $\overline{F_{total}}$ - результирующий вектор силы, Н;

$\overline{F_{att, norm}}$ - нормализованный вектор притягивающей силы, Н;

$\overline{F_{rep, norm}}$ - нормализованный вектор отталкивающей силы, Н;

$\overline{F_{total, norm}}$ - нормализованный вектор результирующей силы, Н;

$||\overline{F_{total}}||_2$ - евклидова норма вектора $\overline{F_{total}}$, Н.

– расчет градиента углов: определяются углы поворота суставов на основе результирующей силы, которая рассчитывается по формуле:

$$\Delta\theta = J^+ \times \overline{F_{total, norm}},$$

где $\Delta\theta$ - вектор изменений углов суставов, радианы

$$(\Delta\theta = [\Delta\theta_1, \Delta\theta_2, \dots, \Delta\theta_n]^T);$$

J^+ - псевдообратная матрица Якоби;

$\overline{F_{total, norm}}$ - нормализованный вектор результирующей силы, Н.

– обновление углов: углы суставов корректируются с шагом 0,05 радиан;

– проверка застреваний: если предыдущая и текущая позиции различаются менее чем на 1 мм, увеличивается счетчик застреваний. При превышении 50 застреваний добавляется случайное возмущение углов для выхода из локального минимума;

– расчет расстояния до препятствий: проверяется, находится ли манипулятор ближе 300 мм к препятствиям. Если да, углы корректируются только с учетом отталкивающих сил. Данное действие рассчитывается по формулам:

$$d_{min} = \min_i (||\overline{P_{link, j}} - \overline{P_{obst, i}}||_2),$$

$$\overline{F_{total}} = \overline{F_{rep, norm}},$$

$$\overline{\theta_{new}} = \overline{\theta_{current}} + J^+ \times \overline{F_{total}} \times \Delta t,$$

где d_{min} - минимальное расстояние до препятствий, м;

$\overline{P_{link, j}}$ - вектор позиции (j)-го звена манипулятора, м;

$\overline{P_{obst, i}}$ - вектор позиции (i)-го препятствия, м;

$\overline{F_{total}}$ - результирующий вектор силы, Н;

$\overline{F_{rep, norm}}$ - нормализованный вектор отталкивающей силы, Н;

$\overline{\theta_{new}}$ - обновленный вектор углов, радианы;

$\overline{\theta_{current}}$ - текущий вектор углов, радианы;

J^+ - псевдообратная матрица Якоби;

Δt - шаг обновления углов, радианы ($\Delta t = 0,05$).

- добавление углов в траекторию: обновленные углы записываются в траекторию;
- проверка точности: если отклонение от цели менее 1 мм и 0,005 радиан, подпрограмма возвращает массив углов траектории и завершает работу.

Алгоритм планирования траектории представлен в приложении Р.

10.3 Алгоритм преобразования координат (TF_Listener)

Модуль преобразования координат (TF) обеспечивает трансформацию координат инструмента манипулятора:

- инициализация: настройка системы преобразований;
- чтение TF данных: последовательное получение данных о положении суставов;
- вычисление позиции инструмента: определение положения инструмента на основе TF;
- трансляция позиции: публикация данных о положении инструмента в ROS.

Алгоритм преобразования координат представлен в приложении С.

10.4 Алгоритм публикации состояния звеньев манипулятора (Joint_State_Aggregatoe)

Публикатор состояния обновляет информацию о положении суставов:

- инициализация: подготовка узла ROS;
- получение данных суставов: считывание текущих состояний;
- обновление позиций: обработка данных и публикация единого сообщения в ROS.

Алгоритм публикации состояния звеньев манипулятора представлен в приложении Т.

10.5 Алгоритм фильтрации данных LiDAR (Lidar_Filter)

Обработка данных LiDAR включает:

- инициализация: подготовка системы;
- построение модели манипулятора: создание виртуальной модели по файлу описания из симуляции;
- получение данных: считывание состояния суставов и данных LiDAR;
- фильтрация: объединение и очистка данных LiDAR от шума и точек, принадлежащих звеньям манипулятора;
- публикация: передача обработанных данных в систему;

Алгоритм фильтрации данных LiDAR представлен в приложении У.

10.6 Алгоритм вращения данных LiDAR (Lidar_Processor)

Модуль вращения LiDAR корректирует данные в зависимости от положения манипулятора:

- инициализация: настройка системы;
- получение данных: считывание LiDAR и данных сустава J_1;

- вращение облака точек: корректировка данных с учетом движения манипулятора;
- публикация: передача обработанного облака точек.

Алгоритм вращения данных LiDAR представлен в приложении Ф.

10.7 Алгоритм предупреждения касания (Dead_Area_Detector)

Модуль предупреждения касания предотвращает столкновения:

- инициализация: настройка системы;
- чтение данных: считывание состояний суставов и LiDAR;
- подсчет точек: анализ плотности точек вокруг манипулятора;
- публикация остановки: при превышении порога точек публикуется команда остановки.

Алгоритм предупреждения касания представлен в приложении Х.

10.8 Алгоритм детектора касания (Collision_Detector)

Детектор касания реагирует на физический контакт:

- инициализация: подготовка системы;
- чтение данных: проверка наличия касания;
- публикация остановки: при обнаружении касания публикуется команда остановки.

Алгоритм детектора касания представлен в приложении Ц.

11 Разработка программ управления

Программы были разработаны в среде PyCharm с использованием языка программирования Python 3.10. PyCharm был выбран благодаря его удобным инструментам для разработки и отладки. Python 3.10 обеспечивает высокую производительность и поддержку современных библиотек, необходимых для обработки данных LiDAR и решения сложных математических уравнений.

Листинг программ представлен в приложениях Ш, Щ, Э, Ю, Я, АА, АБ.

По итогу создания и запуска симуляции, а также написания и отладки программ управления был получен 6-осевой манипулятор способный детектировать и огибать препятствия всеми звеньями (рисунок 29). Временные диаграммы для каждого из которых приведены в приложении АВ.



13 Разработка конструкции шкафа управления

Разработка конструкции шкафа управления начинается с определения, элементов, которые будут в нем находиться и его габаритных размеров. к данной системе управления был выбран шкаф WP 6306.910 SYSMATRIX. Размеры данного шкафа управления 600мм x 350мм x 315мм, масса 12,4 кг. В Компас-3D создан сборочный чертеж макетной платы. В шкафу также находятся блоки питания и драйвера шаговых двигателей. В приложении АГ размещен сборочный чертеж шкафа управления, спецификация к нему и чертеж доработки.



Рисунок 30 – Шкаф управления WP 6306.910 SYSMATRIX

ЗАКЛЮЧЕНИЕ

В данной исследовательской работе была поставлена задача разработки 6-осевого манипулятора с интеллектуальной системой управления, способного эффективно взаимодействовать с окружающей средой и предотвращать столкновения. Мы проанализировали современные тенденции в области робототехники и определили ключевые аспекты, которые необходимо учесть для достижения поставленных целей.

Однако следует отметить, что данная работа является лишь начальным этапом в реализации задуманного проекта. Для завершения исследования необходимо выполнить несколько критически важных шагов:

Установка сенсоров восприятия окружающего мира на модель робота и наладка связи с ними за пределами симуляции, что позволит манипулятору получать актуальную информацию о состоянии окружающей среды; Разработка системы расчета обратной задачи кинематики; Создание системы детектирования столкновения.

Кроме того, важным этапом является написание программы для планирования пути движения как ТСР, так и всех звеньев манипулятора в целом. Этот шаг является необходимыми для достижения высокой степени автономности и эффективности работы 6-осевого манипулятора.

Таким образом, дальнейшие исследования и разработки в указанных направлениях позволят значительно продвинуться в создании интеллектуального манипулятора, способного успешно функционировать в сложных и динамичных условиях.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Документация по ROS2 / пер. с англ. [Электронный ресурс]. – URL: <https://docs.ros.org/en/jazzy/index.html> (дата обращения 15.10.2024).
- 2 Как смоделировать роботизированную руку в Gazebo – ROS2 / пер. с англ. [Электронный ресурс]. – URL: <https://automaticaddison.com/how-to-simulate-a-robotic-arm-in-gazebo-ros-2/> (дата обращения 10.11.2024).
- 3 Документация по Gazebo Ionic / пер. с англ. [Электронный ресурс]. – URL: <https://gazebosim.org/docs/latest/sensors/> (дата обращения 20.10.2024).
- 4 СТО СГАУ 02068410-004-2018. Общие требования к учебным текстовым документам [Текст] – Самара: СГАУ, 2018.
- 5 ГОСТ 2.702-2011 Единая система конструкторской документации. Правила выполнения электрических схем [Текст]. - Введ. 2012-01-01. - М.: Издательство стандартов, 2012.
- 6 ГОСТ 2.702-2011 Единая система конструкторской документации. Правила выполнения электрических схем [Текст]. - Введ. 2012-01-01. - М.: Издательство стандартов, 2012.
- 7 Шаговый серводвигатель [Электронный ресурс]. – URL: https://cnc-tehnologi.ru/shagovye-dvigateli-s-enkoderom/shagovyj-servo-dvigatel-86hb250118?utm_source=yandex.direct&utm_medium=cpc&utm_campaign=tovarnykomplekt_71933195&utm_content=gid%7C4845933706%7Cad_id%7C15112743918%7Cphrase_id%7C36982071795&type=search&source=yandex.ru&added=no&block=dynamic_places&pos=1&device=desktop®ion=region_name%7CТольятти%7Cregion_id%7C240&yclid=6812621126734184447 (дата обращения 15.10.2024).
- 8 Гармонические редуктора [Электронный ресурс]. – URL: <https://www.harmonicdrive.net/products/gear-units/hollow-shaft-gear-units/shf-2uh-1w/shf-58-80-2uh-1w> (дата обращения 15.10.2024).

9 Планетарный редуктор [Электронный ресурс]. – URL: https://darxton.ru/catalog_item/reduktor-planetarnyy-plfa60-10-uglovoy/ (дата обращения 15.10.2024).

10 Шкиф зубчатый [Электронный ресурс]. – URL: https://darxton.ru/catalog_item/bsy-z50-8m-30-af-d19-shkiv-zubchatyy-htd-8m/ (дата обращения 15.10.2024).

11 Ремень зубчатый [Электронный ресурс]. – URL: https://darxton.ru/catalog_item/8m-30-n-remen-zubchatyy-razomknuty/ (дата обращения 15.10.2024).

12 Драйвер шагового двигателя [Электронный ресурс]. – URL: <https://cnc-tehnologi.ru/drajvery-shagovykh-servodvigatelej/drajver-shagovogo-servo-dvigatelya-hbs860h?ysclid=mc33sx66rr186439426> (дата обращения 15.10.2024).

13 Комутатор сетевой [Электронный ресурс]. – URL: <https://www.dns-shop.ru/product/2a54c01c110e3330/kommutator-tp-link-ls1008g/> (дата обращения 15.10.2024).

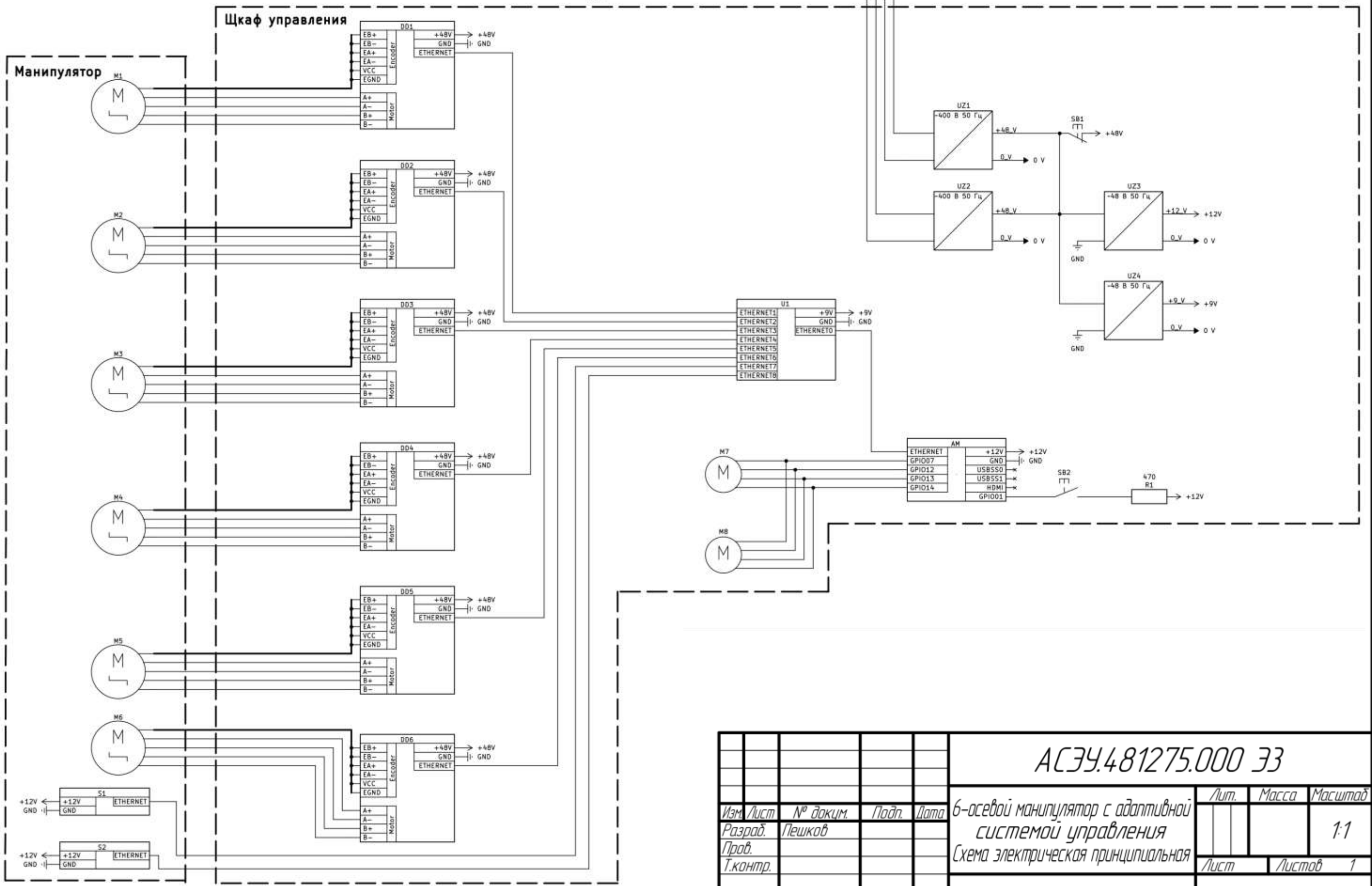
14 LiDAR [Электронный ресурс]. – URL: <https://www.unitree.com/LiDAR> (дата обращения 15.10.2024).

15 Вентилятор охлаждения [Электронный ресурс]. – URL: https://www.onlinetrade.ru/catalogue/ventilyatory_dlya_korpusa-c1322/arctic/ventilyator_dlya_korpusa_arctic_p14_black_black_acfan00123a1612182.html?ysclid=mc33u6coaw514303824&utm_referrer=https%3a%2f%2fya.ru%2f (дата обращения 15.10.2024).

16 Контроллер NVIDIA Jetson Orin NX [Электронный ресурс]. – URL: <https://developer.nvidia.com/embedded/Jetson-modules> (дата обращения 15.10.2024).

ПРИЛОЖЕНИЕ А
Схема электрическая принципиальная

АСУ.481275.000 ЭЗ



					АСЭУ.481275.000 ЭЗ				
					6-осевой манипулятор с адаптивной системой управления Схема электрическая принципиальная		Лит.	Масса	Масштаб
Изм.	Лист	№ докум.	Подп.	Дата					1:1
Разраб.	Пешков								
Пров.									
Т.контр.							Лист	Листов	1
							СНИУ гр.24 14		
Н.контр.									
Утв.									

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Поз. обозначение	Наименование	Кол.	Примечание	Перв. примен.	Справ. №	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата</
------------------	--------------	------	------------	---------------	----------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	----------------

[illegible]

ПРИЛОЖЕНИЕ Б
Сборочная единица основания

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Перв. примен.

Справ. №

Подп. и дата

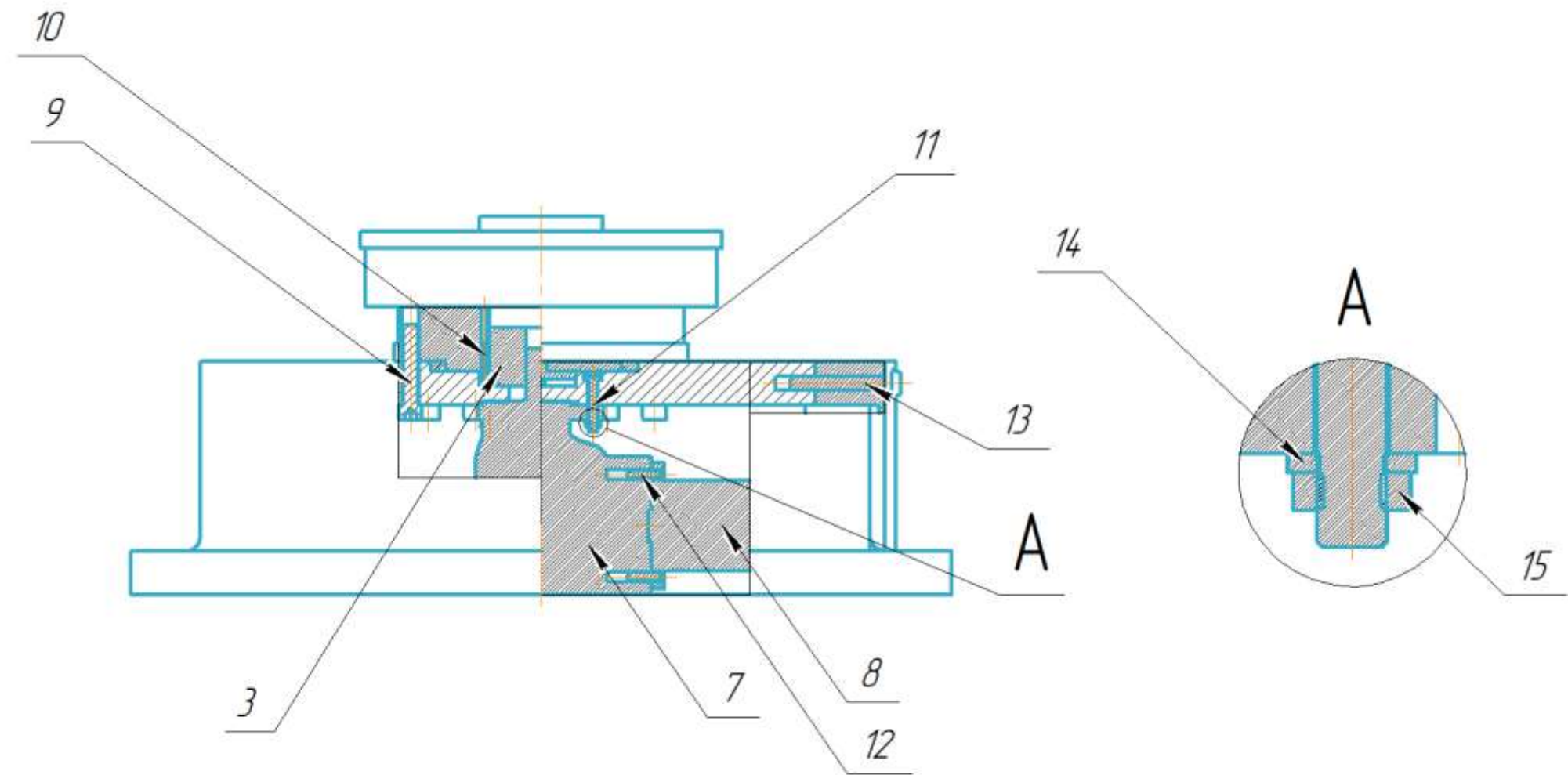
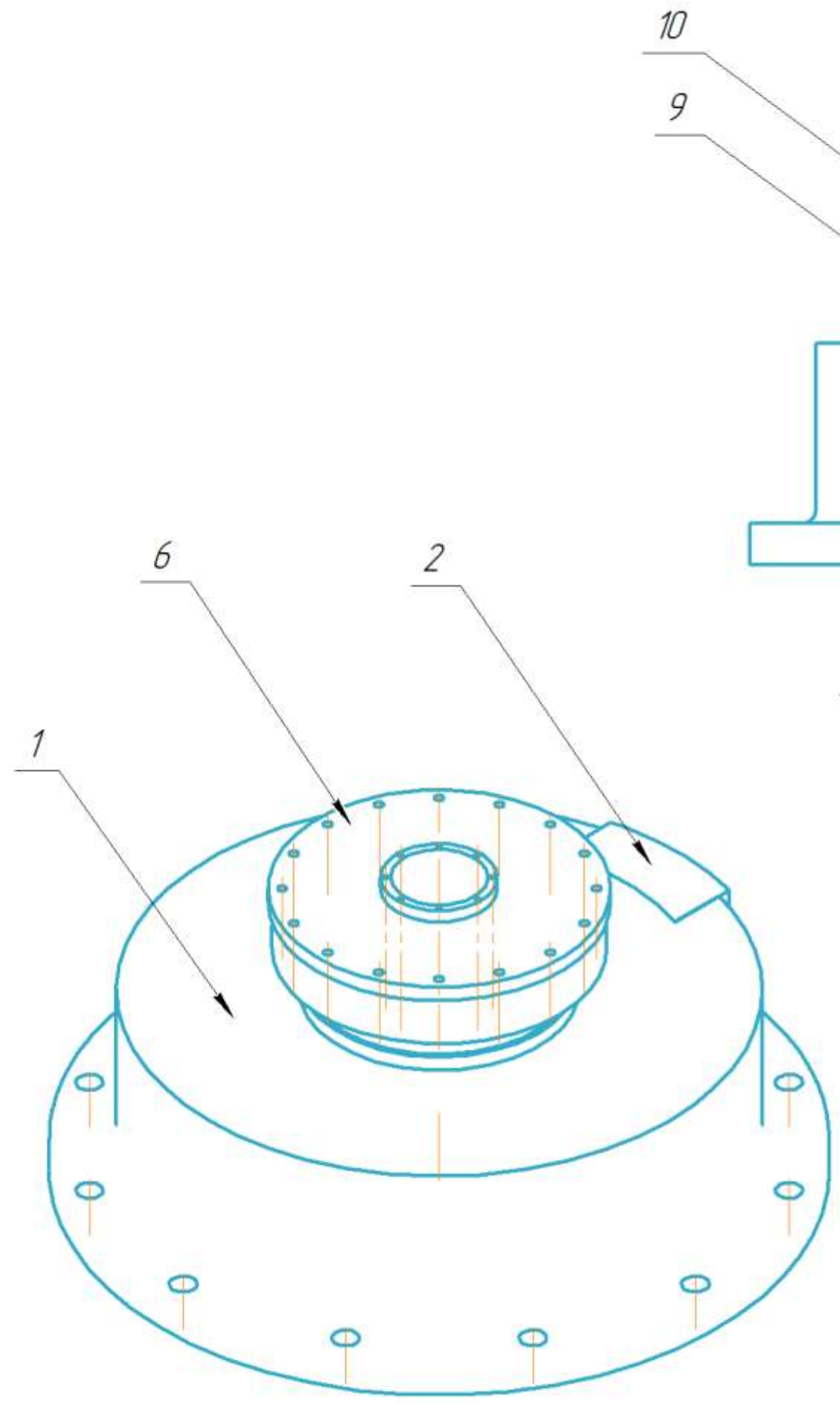
Инв. № дудл.

Взам. инв. №

Подп. и дата

Инв. № подл.

АСЗУ.44244.1001 СБ



- 1 Деталь поз.1 крепится к детали поз.2 болтами М10
- 2 Деталь поз.3 крепится к детали поз.6 болтами М3
- 3 Деталь поз.6 крепится к детали поз.1 болтами М10
- 4 Деталь поз.7 крепится к детали поз.1 болтами М6
- 5 Деталь поз.8 крепится к детали поз.7 болтами М6

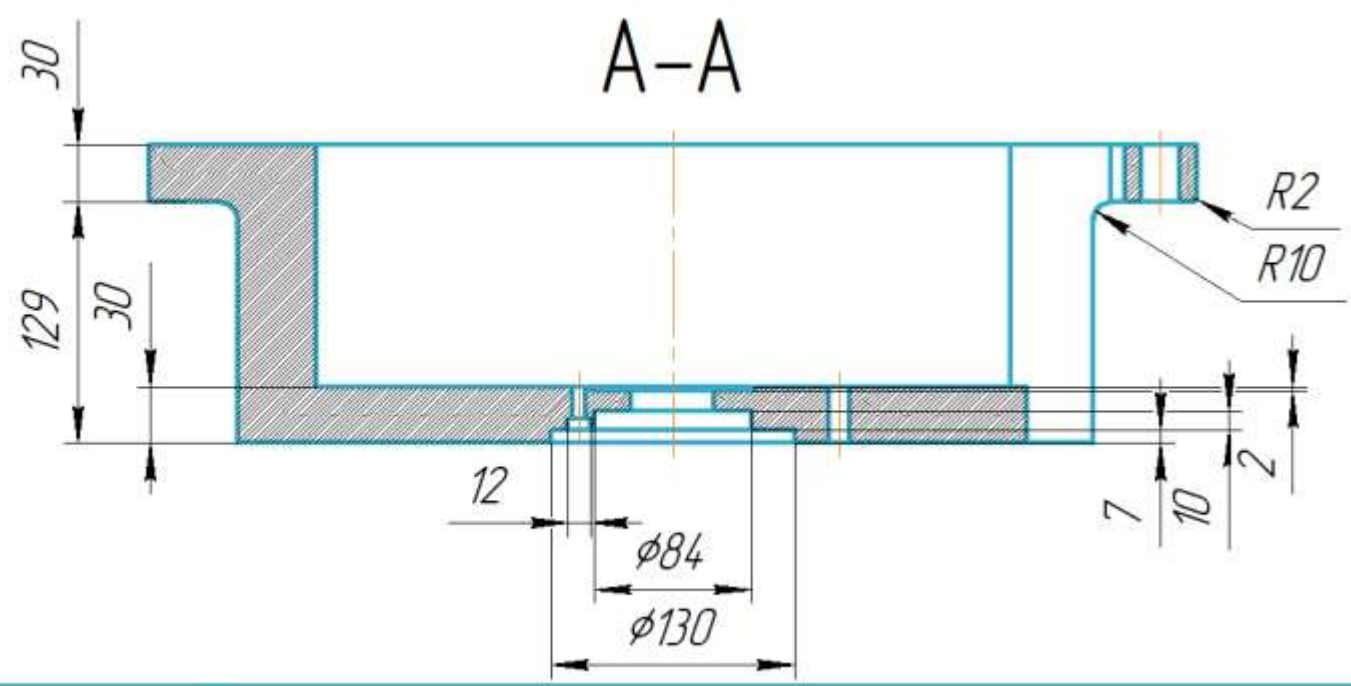
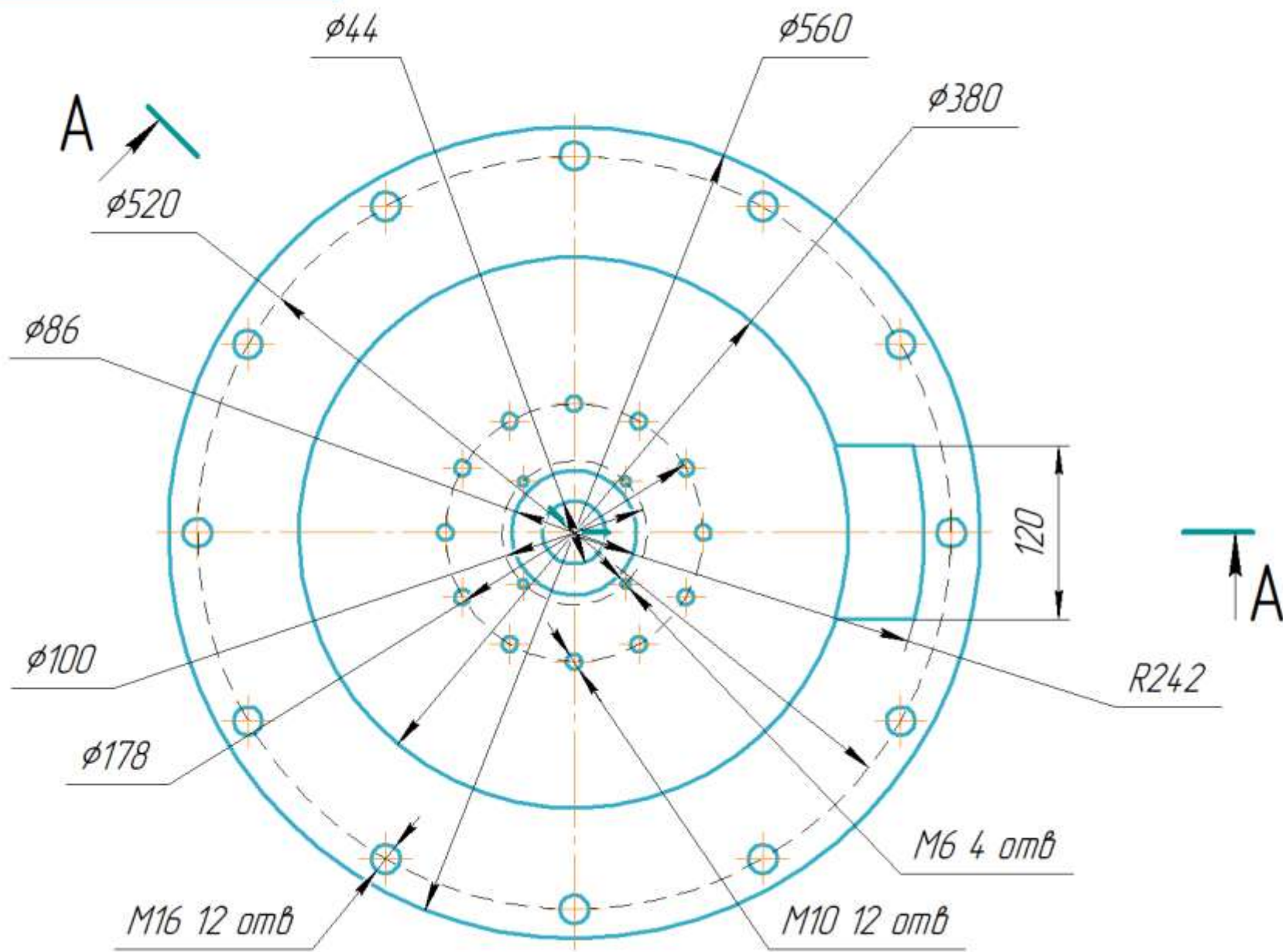
					АСЗУ.44244.1.001 СБ			
					Основание манипулятора Сборочная единица	Лист	Масса	Масштаб
Изм./Лист	№ докум.	Подп.	Дата				120	1:4
Разраб.	Пешков							
Пров.						Лист	Листов	1
Т.контр.						СНИУ гр.24 14		
Н.контр.								
Утв.								

Перв. примен.		Формат	Зона	Поз.	Обозначение	Наименование	Кол.	Примечание
						<u>Документация</u>		
		A3			Основание манипулятора	АСЭУ.44244.1.002 СБ		
						<u>Детали</u>		
		A3	1		АСЭУ.44244.1.007	Основание	1	
		A3	2		АСЭУ.44244.1.008	Основание	1	
						Крышка		
		A4	3		АСЭУ.44244.1.026	Переходный фланец на редуктор ZHK58	1	
						<u>Стандартные изделия</u>		
			6			Редуктор Harmonic Joint Reducer ZHK58	1	
			7			Редуктор планетарный на 90° PVF090	1	
			8			Серводвигатель Nema 34 86HB250-118B	1	
			9			Hexagon socket head cap screw ISO 4762 -M10 x 55	12	
			10			Hexagon socket head cap screw ISO 4762 -M3 x 25	8	
		АСЭУ.44244.1.001						
		Изм.	Лист	№ докум.	Подп.	Дата		
		Разраб.	Пешков					
		Проб.						
		Н.контр.						
		Чтб.						
		Основание манипулятора					Лит.	Лист
							1	2
							СНИУ гр.24 14	

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дучсл.	Подп. и дата
--------------	--------------	--------------	---------------	--------------

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.
Инв. № подл. Подп. и дата Взам. инв. № Инв. № дубл. Подп. и дата Справ. № Перв. примен.

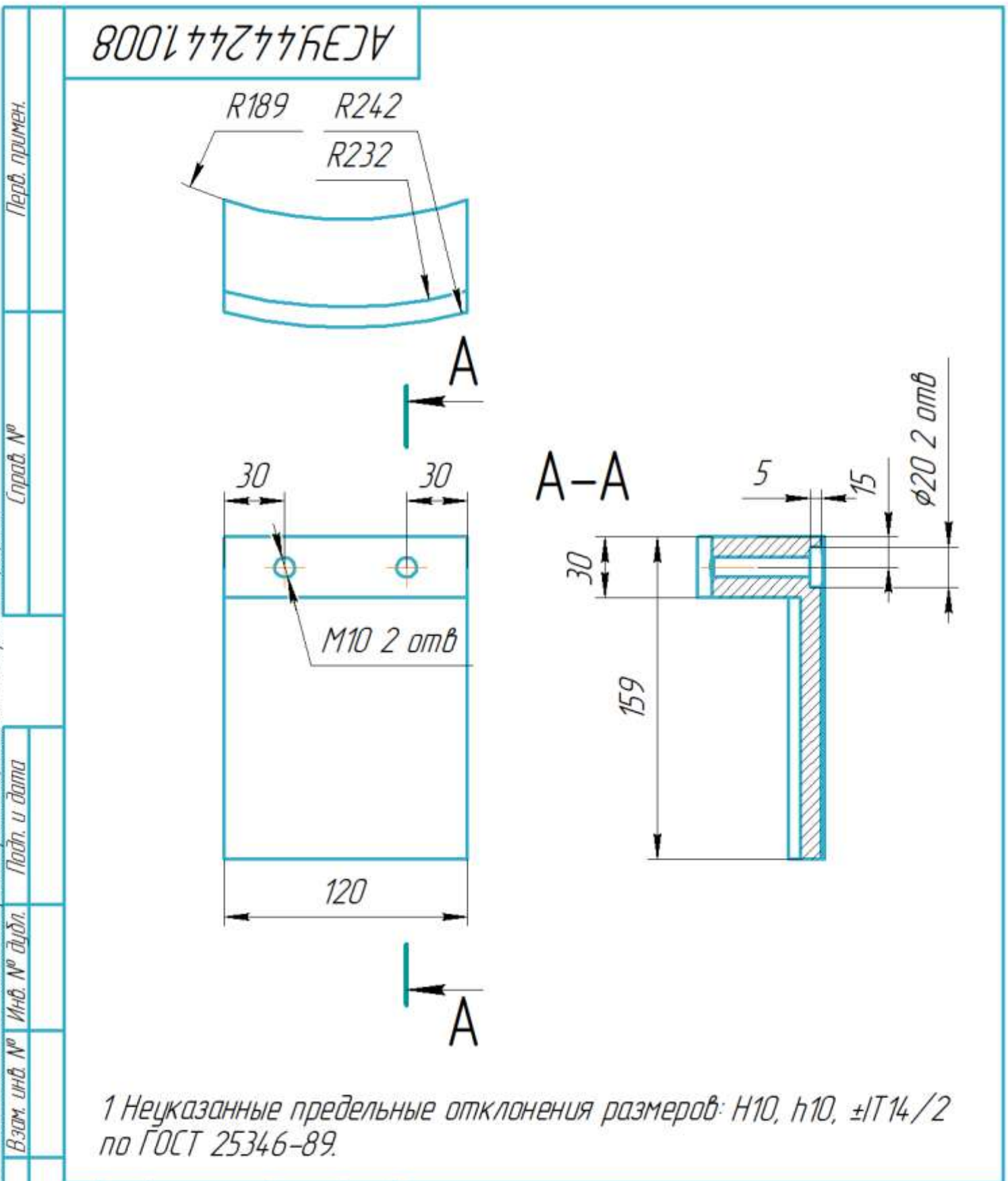
АСЗУ.44244.1007



1 Неуказанные предельные отклонения размеров: H10, h10, $\pm IT14/2$ по ГОСТ 25346-89.

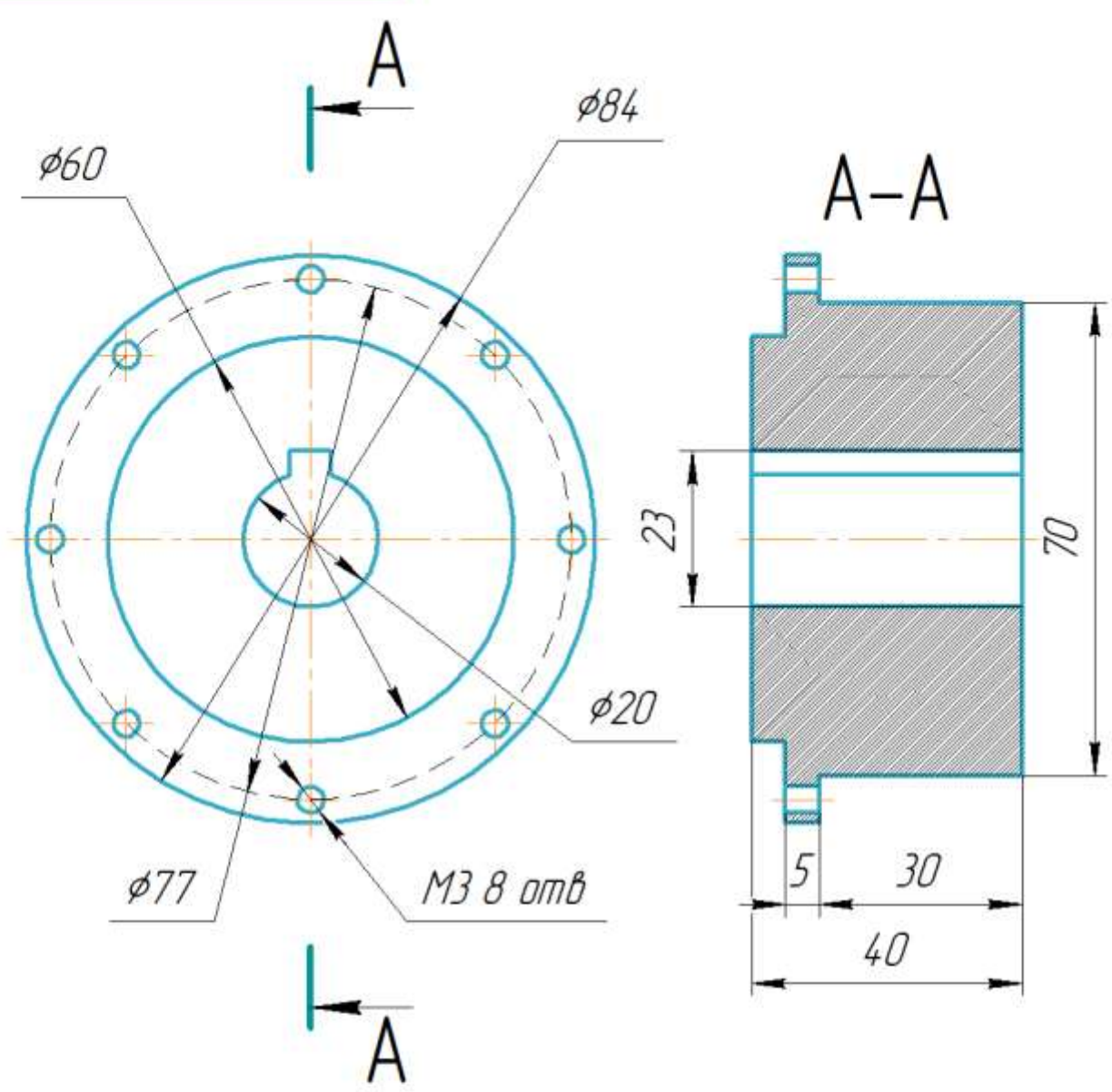
				АСЗУ.44244.1007		
Изм.	Лист	№ докум.	Подп.	Дата	Основание	Лит.
						Масса
Разраб.	Лешков					100
Проб.						1:4
Т.контр.						Лист
Н.контр.						Листов
Утв.						1
					Сталь 08Х18Н10 ГОСТ 5632-2014	СНИУ гр.24 14
					Копировал	Формат А3

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.



АСЗУ.44244.1.008					Лит.			Масса	Масштаб
Изм.	Лист	№ докум.	Подп.	Дата	Основание манипулятора Крышка			1	1:2.5
Разраб.	Пешков								
Пров.									
Т.контр.									
Н.контр.					Сталь 08Х18Н10 ГОСТ 5632-2014			СНИУ гр 24 14	
Утв.									

АСЭУ.44244.1026



1 Неуказанные предельные отклонения размеров: Н10, н10, $\pm IT14/2$ по ГОСТ 25346-89.

АСЭУ.44244.1026

Переходный фланец
не редуктор ZHK58

Сталь 08Х18Н10 ГОСТ 5632-2014

Лист	Масса	Масштаб
1	0.5	1:1
Лист	Листов	1
СНИУ гр.24 14		

Перв. примен.

Справ. №

Подп. и дата

Взам. инв. №

Подп. и дата

Инв. № подл.

Изм.	Лист	№ докум.	Подп.	Дата
Разраб.	Пешков			
Пров.				
Т.контр.				
Н.контр.				
Утв.				

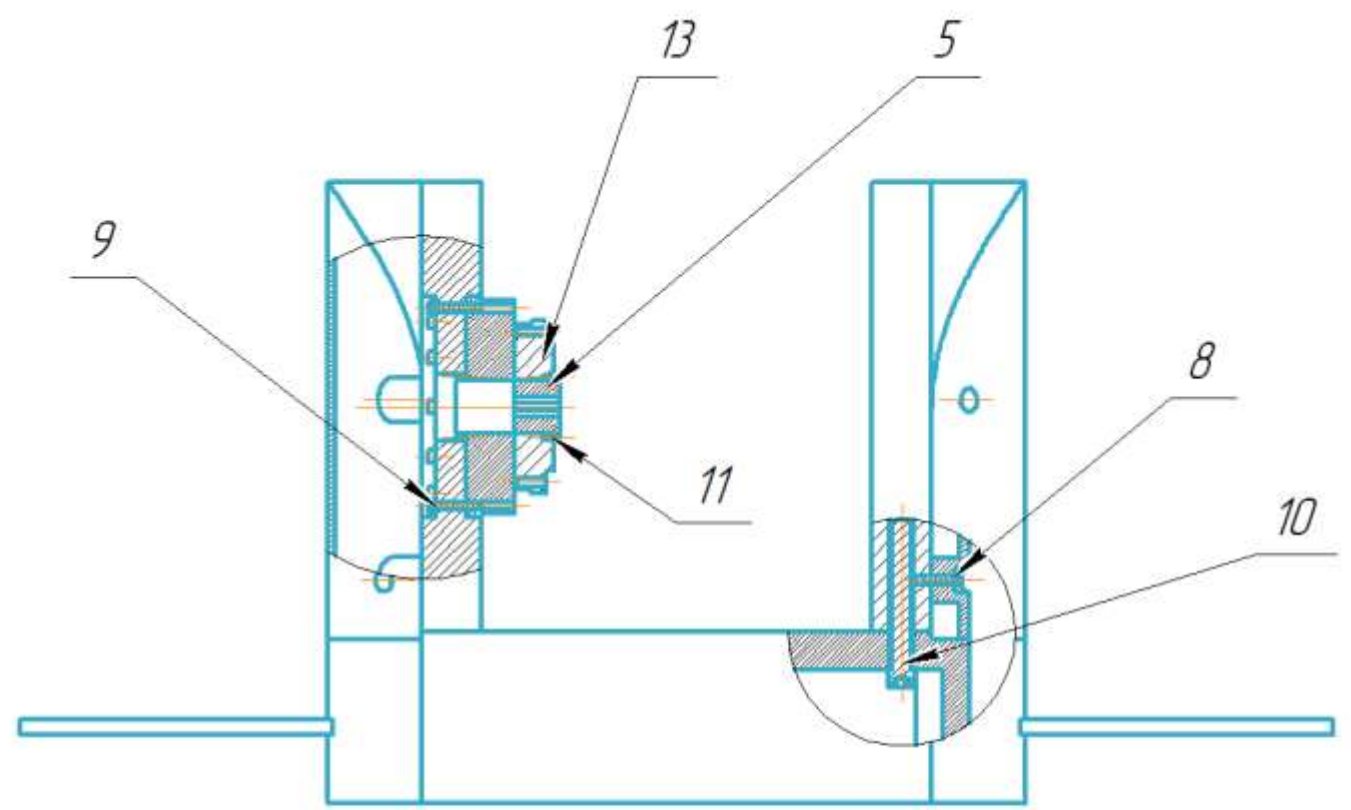
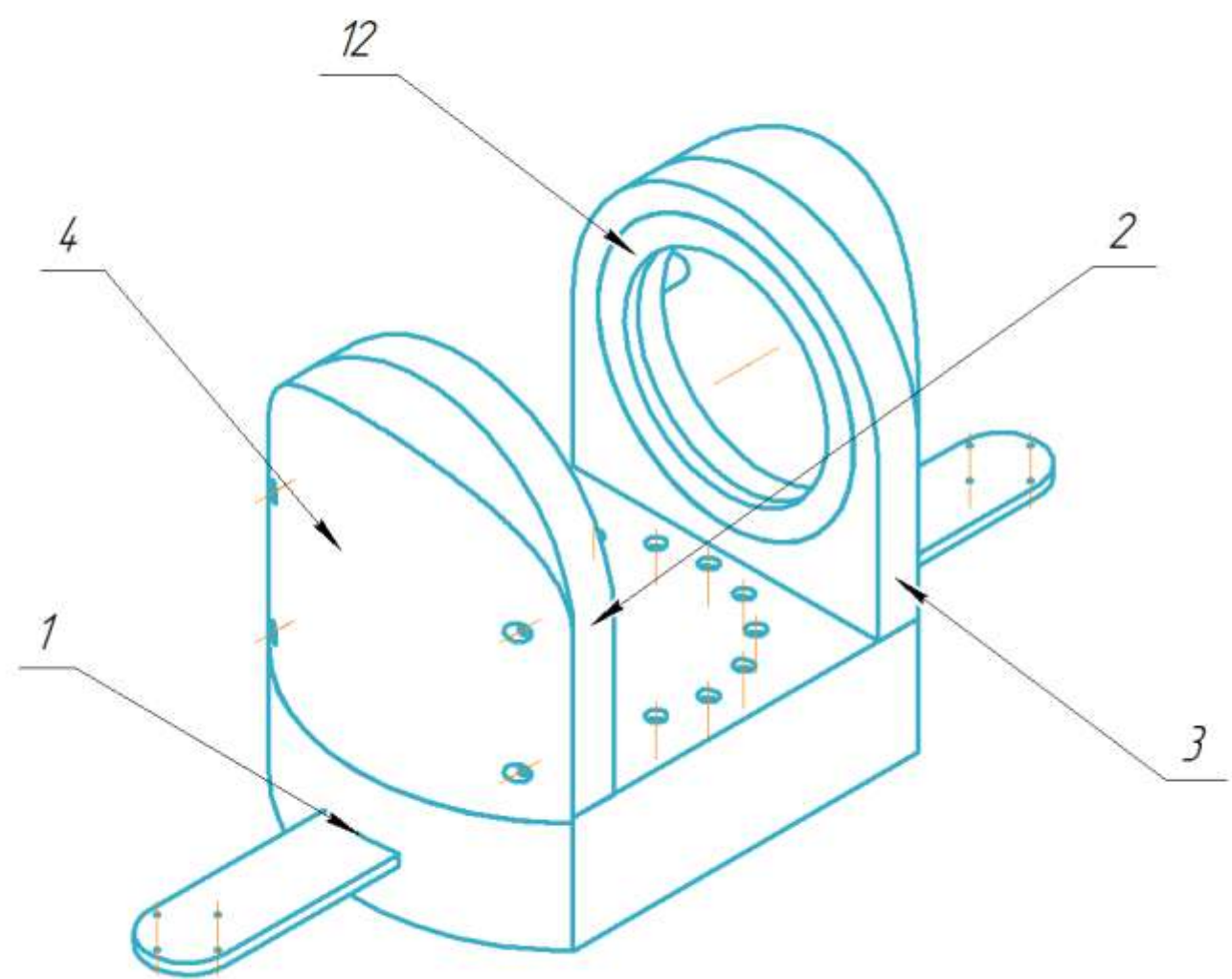
ПРИЛОЖЕНИЕ В
Сборочная единица первого звена

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.
Инв. № подл. Подп. и дата
Взам. инв. № Инв. № дубл. Подп. и дата

Перв. примен.

Справ. №

АСЭУ.44244.1002 СБ



- 1 Деталь поз.1 крепится к детали поз.2 болтами М12
- 2 Детали поз.1 крепятся к детали поз.3 болтами М12
- 3 Детали поз.2 и поз.3 крепятся к деталям поз.4 болтами М5
- 4 Деталь поз.5 крепится к детали поз.13 болтами М2
- 5 Деталь поз.13 крепится к детали поз.2 болтами М5
- 6 Деталь поз.12 запрессовывается в деталь поз.3

					АСЭУ.44244.1.002 СБ			
					Первое звено Сборочная единица	Лит.	Масса	Масштаб
Изм.	Лист	№ докум.	Подп.	Дата			80	1:5
Разраб.		Пешков						
Пров.								
Т.контр.						Лист	Листов	1
Н.контр.						СНИУ гр.24 14		
Утв.								

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Перв. примен.	Формат	Зона	Поз.	Обозначение	Наименование	Кол.	Примечание	
Справ. №					Документация			
	A3			АСЭУ.44244.1.002 СБ	Первое звено			
					Детали			
	A3	1		АСЭУ.44244.1.009	Основание первого звена	1		
	A3	2		АСЭУ.44244.1.010	Первое зано Левое ухо	1		
	A3	3		АСЭУ.44244.1.011	Первое звено Правое ухо	1		
	A3	4		АСЭУ.44244.1.012	Первое зано Крышка	2		
	A4	5		АСЭУ.44244.1.024	Переходный фланец на редуктор HRB32	1		
					Стандартные изделия			
Взам. инв. №		8			Hexagon socket head cap screw ISO 4762 – M5 x 20	8		
		9			Hexagon socket head cap screw ISO 4762 – M5 x 25	12		
		10			Hexagon socket head cap screw	4		
Подп. и дата	АСЭУ.44244.1.002							
	Изм.	Лист	№ докум.	Подп.	Дата			
Инв. № подл.	Разраб.	Пешков				Лит.	Лист	Листов
	Пров.						1	2
Инв. № подл.	Н.контр.					Первое звено		
	Утв.							
						СНИУ гр.2414		

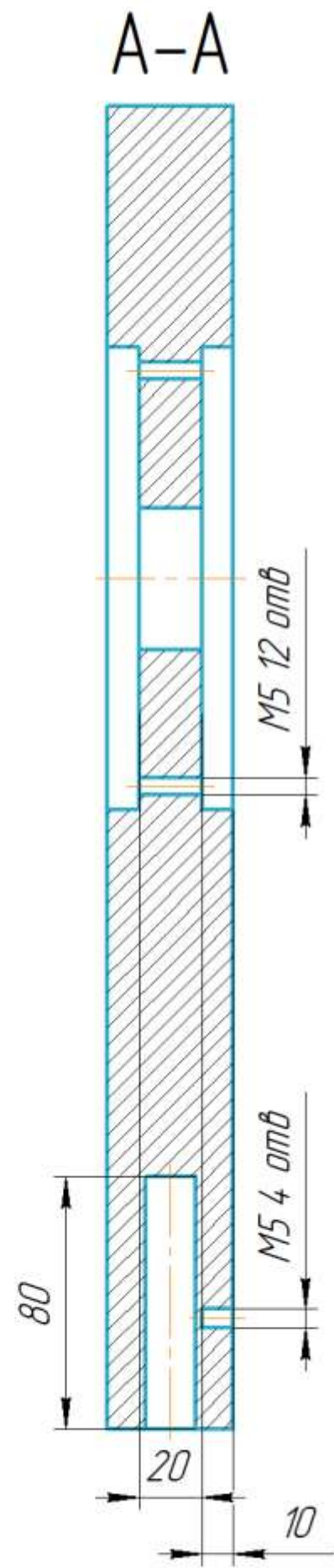
					АСЭУ.442441.002	Лист
Изм.	Лист	№ докум.	Подп.	Дата		2

Формат А3

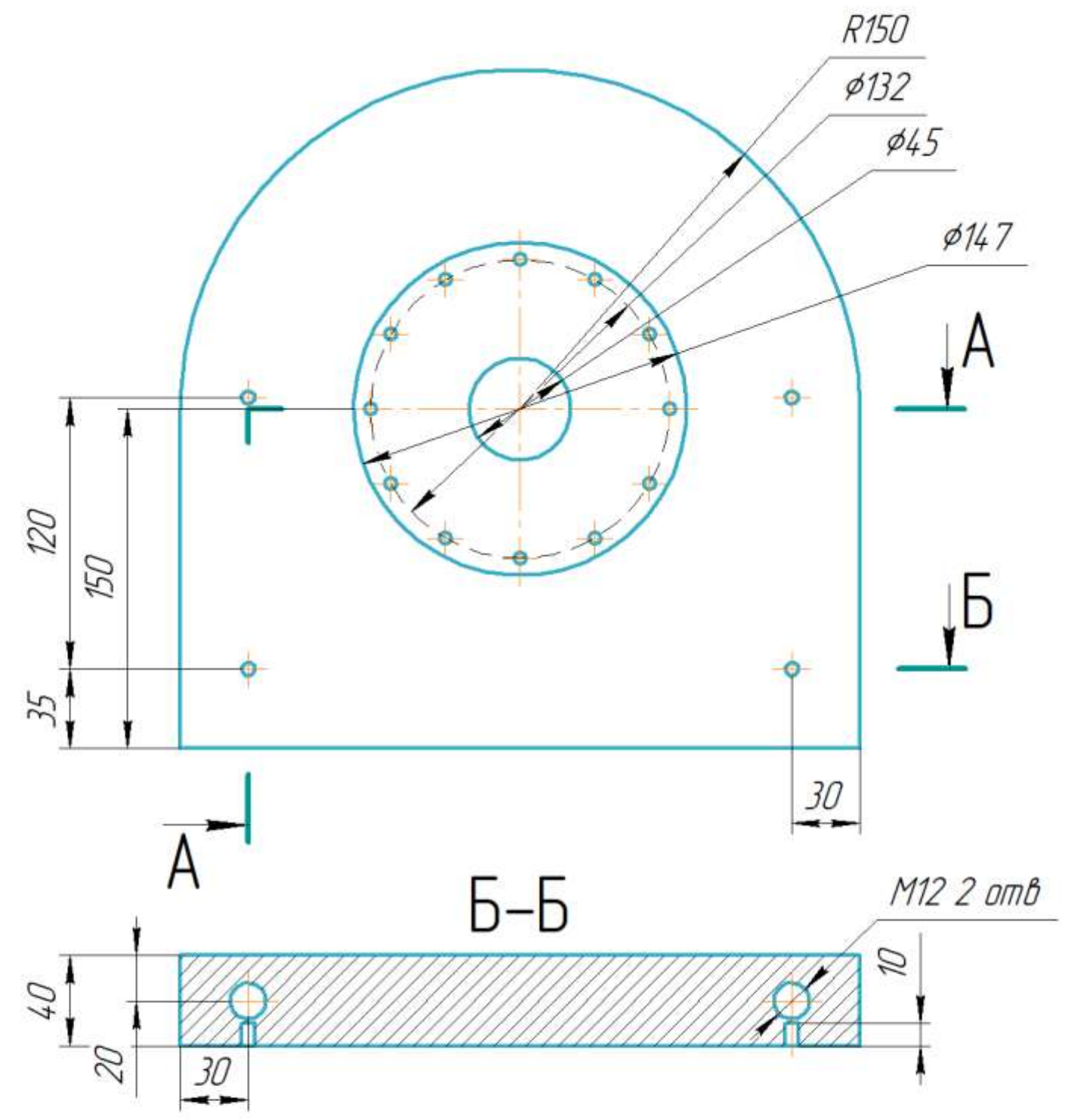
КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Изм. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	Справ. №	Перв. примен.

АСЭУ.44.244.1010



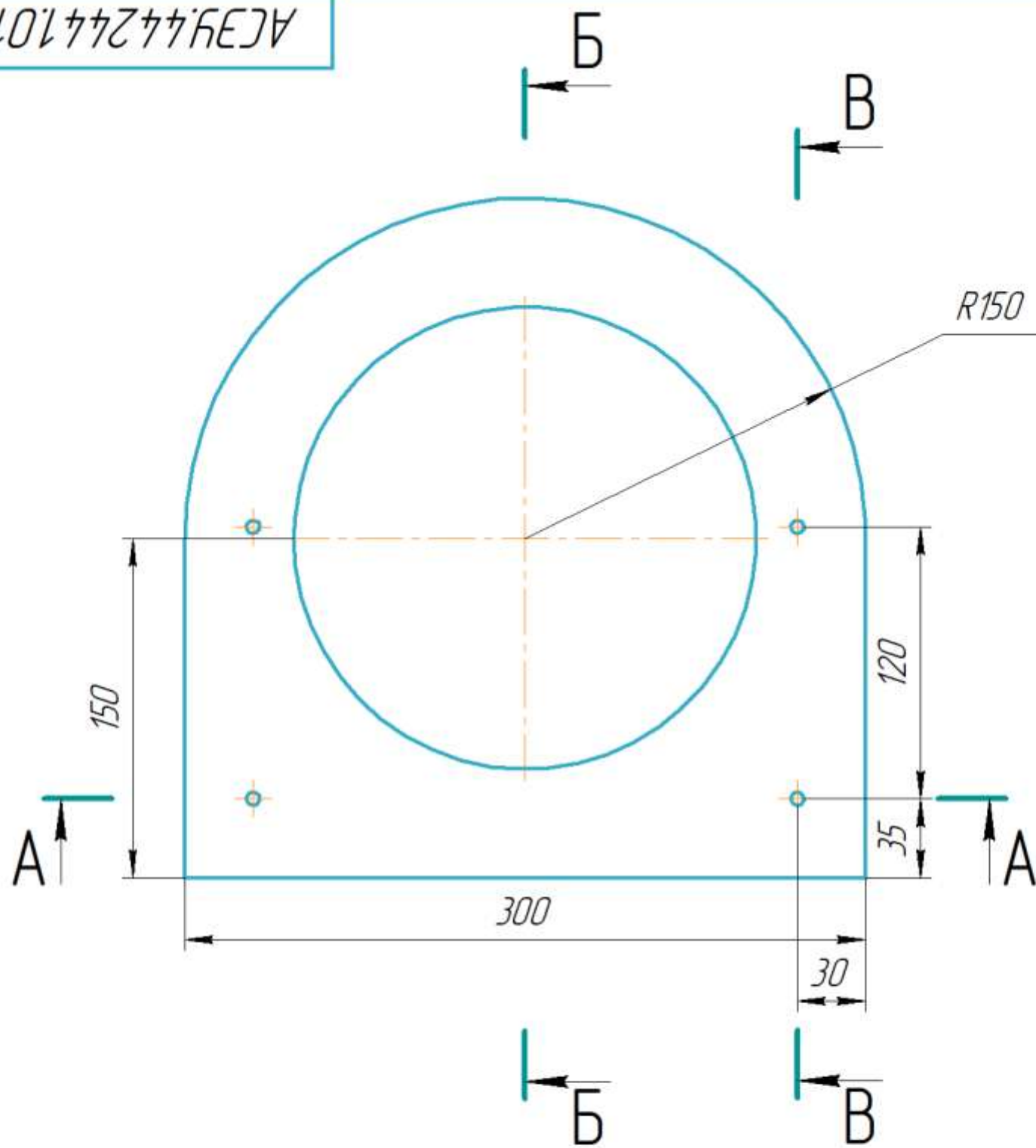
Б



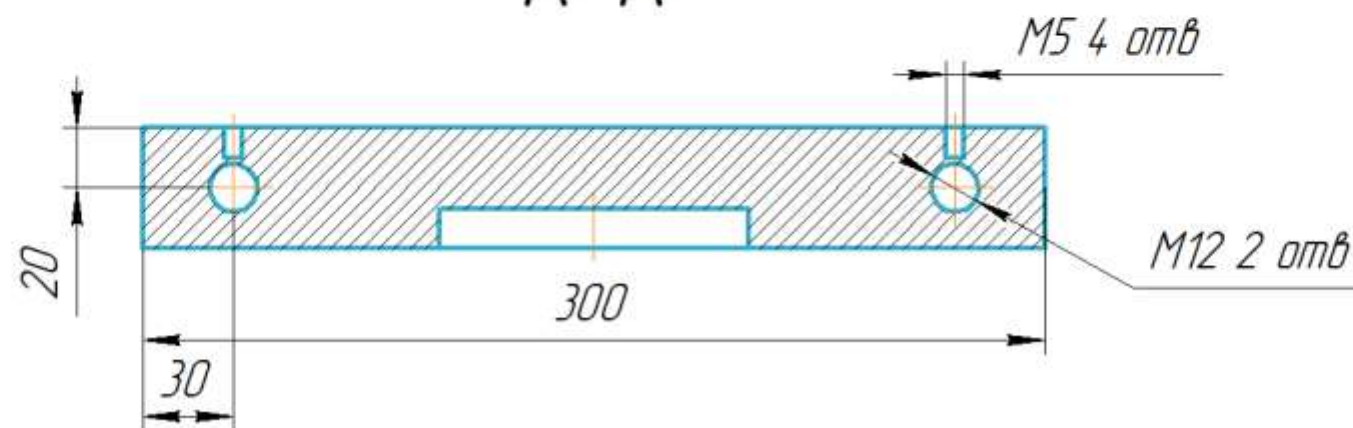
1 Неуказанные предельные отклонения размеров: H10, h10, ±IT14/2 по ГОСТ 25346-89.

АСЭУ.44.244.1010					Первое зано		
Левое ухо					Лист	Масса	Масштаб
Сталь 08Х18Н10 ГОСТ 5632-2014					7	1:2,5	1
СНИУ гр.24 14					Лист	Листов	1

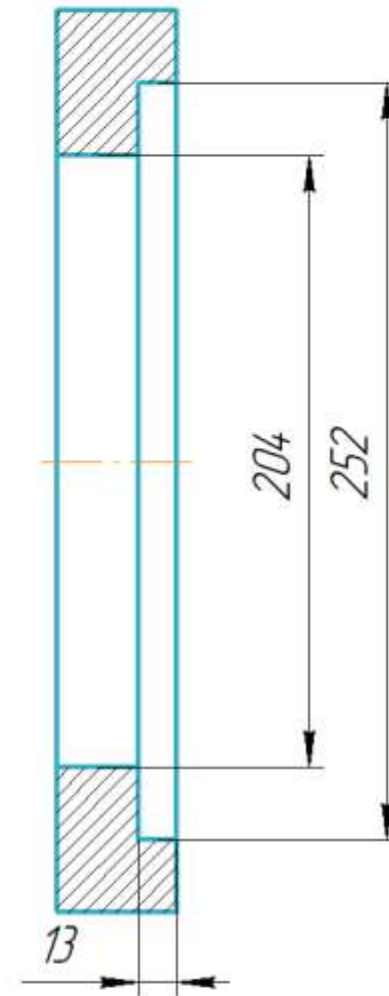
АСЗУ.44244.1011



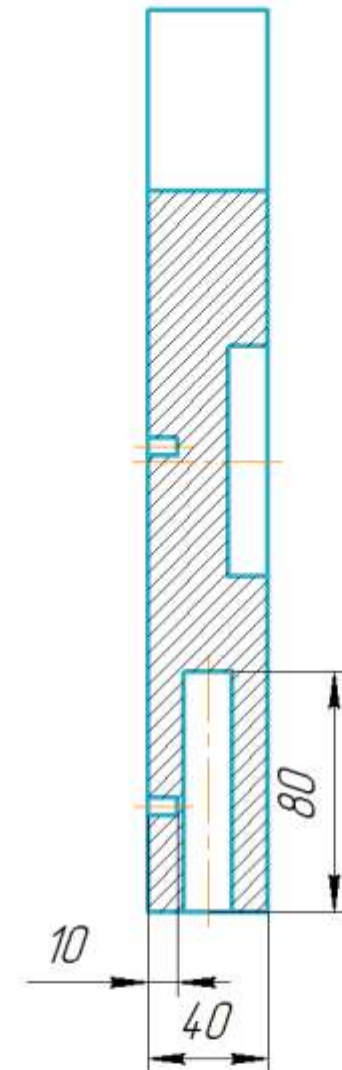
A-A



Б-Б



В-В



1 Неуказанные предельные отклонения размеров: H10, h10, ±IT14/2 по ГОСТ 25346-89.

АСЗУ.44244.1011				Лист	Масса	Масштаб
Первое звено					6	1:2,5
Правое ухо				Лист	Листов	1
Сталь 08Х18Н10 ГОСТ 5632-2014				СНИУ гр.24 14		
Изм. Лист	№ докум.	Подп.	Дата	Копировал		
Разраб.	Пешков			Формат А3		
Проб.						
Т.контр.						
Н.контр.						
Утв.						

1 Неуказанные предельные отклонения размеров: Н10, h10, ±IT14/2 по ГОСТ 25346-89.

- Изготовление изделия выполнять по следующей технологии:
- 2 Изготовить матрицу из Пеноплэкс Фундамент методом фрезерования по чертежу.
 - 3 Нанести слой разделительного воска ПВС-1.
 - 4 Выполнить послойное нанесение углеволокна ВМН-4МС с пропиткой смолой ЭД-20 (7:1 с ПЭПА).
 - 5 Провести вакуумирование при 25 мбар в течение 60 мин.
 - 6 Выполнить термообработку при 120 °С в течение 6 часов.
 - 7 Провести обработку и контроль качества изделия.

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дудл.	Подп. и дата

Изм.	Лист	№ докум.	Подп.	Дата

ПРИЛОЖЕНИЕ Г
Сборочная единица второго звена

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Перв. примен.

Справ. №

Подп. и дата

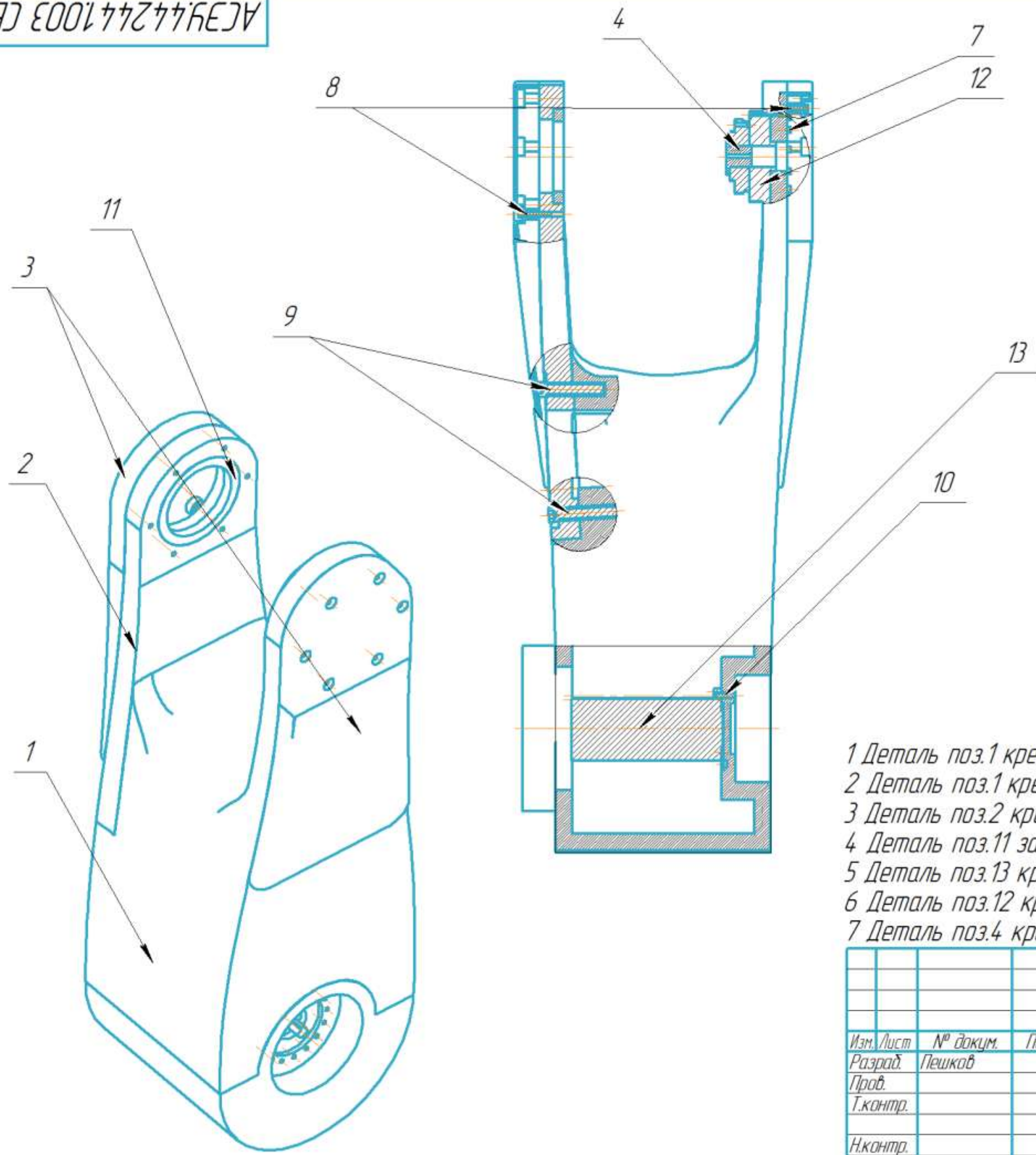
Инв. № дубл.

Взам. инв. №

Подп. и дата

Инв. № подл.

АСЗУ.44.244.1003 СБ



- 1 Деталь поз.1 крепится к детали поз.2 болтами M12
- 2 Деталь поз.1 крепится к детали поз.3 болтами M5
- 3 Деталь поз.2 крепится к детали поз.3 болтами M5
- 4 Деталь поз.11 запресовывается в деталь поз.1
- 5 Деталь поз.13 крепится к детали поз.1 болтами M6
- 6 Деталь поз.12 крепится к детали поз.1 болтами M4
- 7 Деталь поз.4 крепится к детали поз.12 болтами M2

					АСЗУ.44244.1.003 СБ				
Изм. / лист	№ докум.	Подп.	Дата	Второе звено Сборочная единица		Лист.	Масса	Масштаб	
Разраб.	Пешков						50	1:5	
Пров.						Лист		Листов 1	
Т.контр.						СНИУ гр.24 14			
Н.контр.									
Утв.									

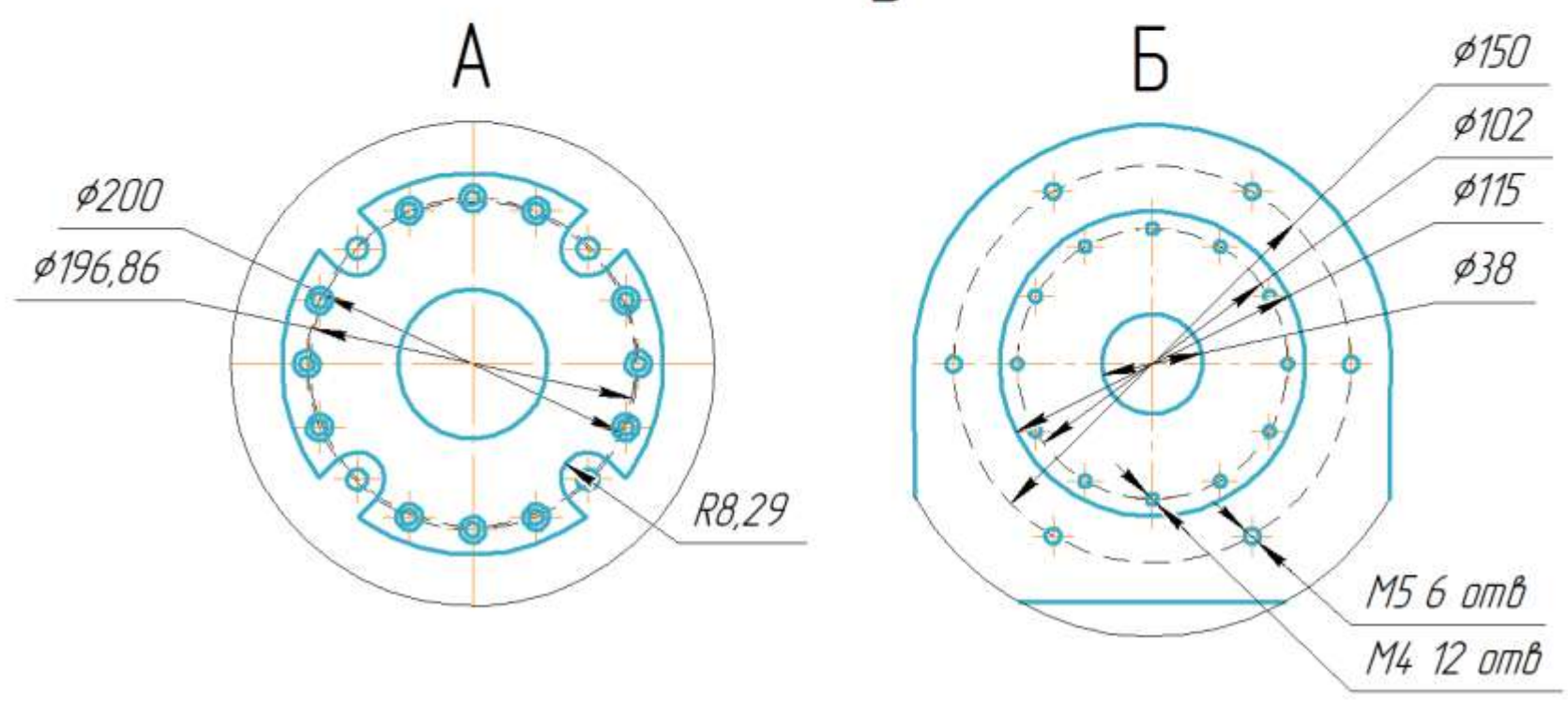
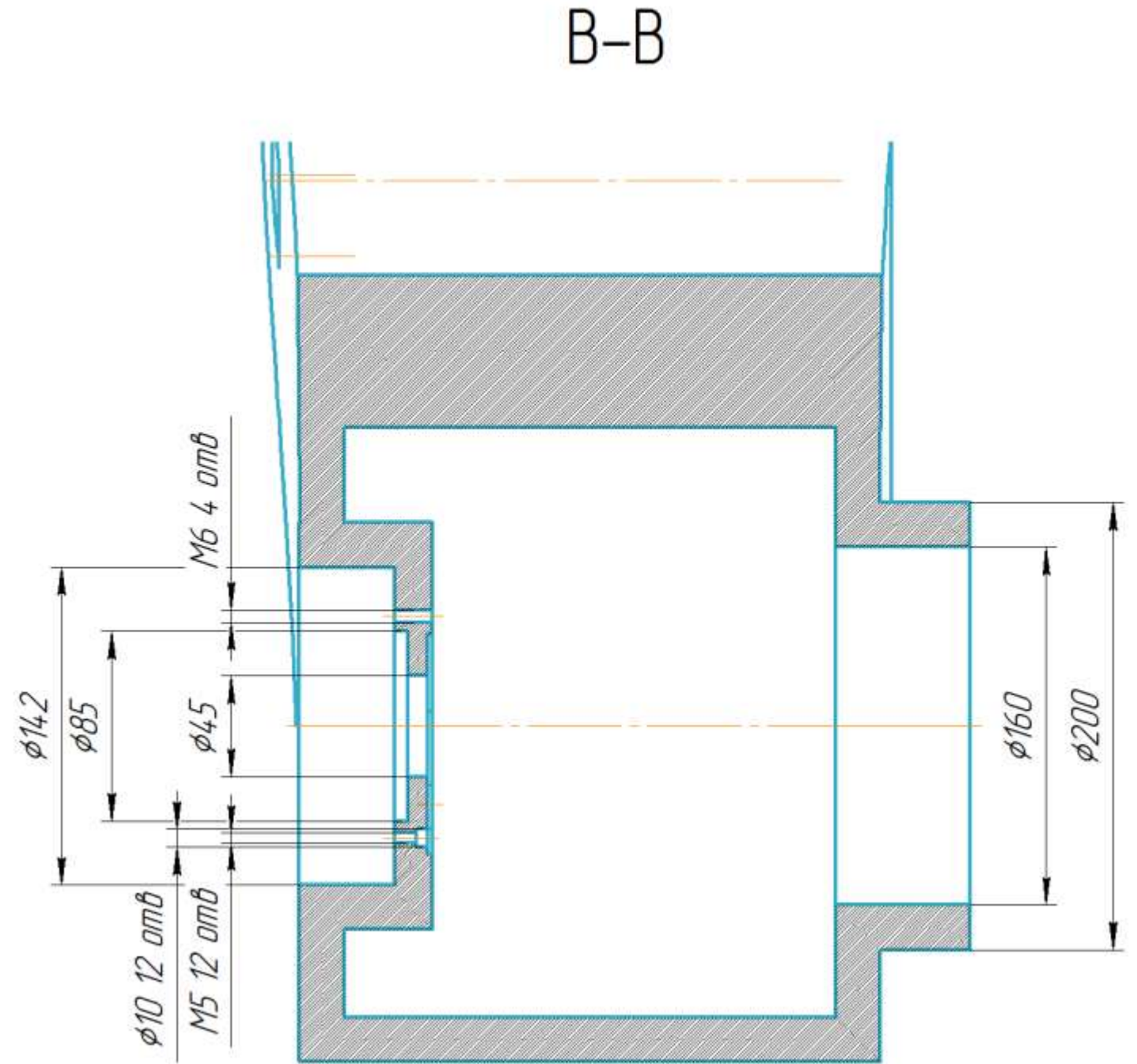
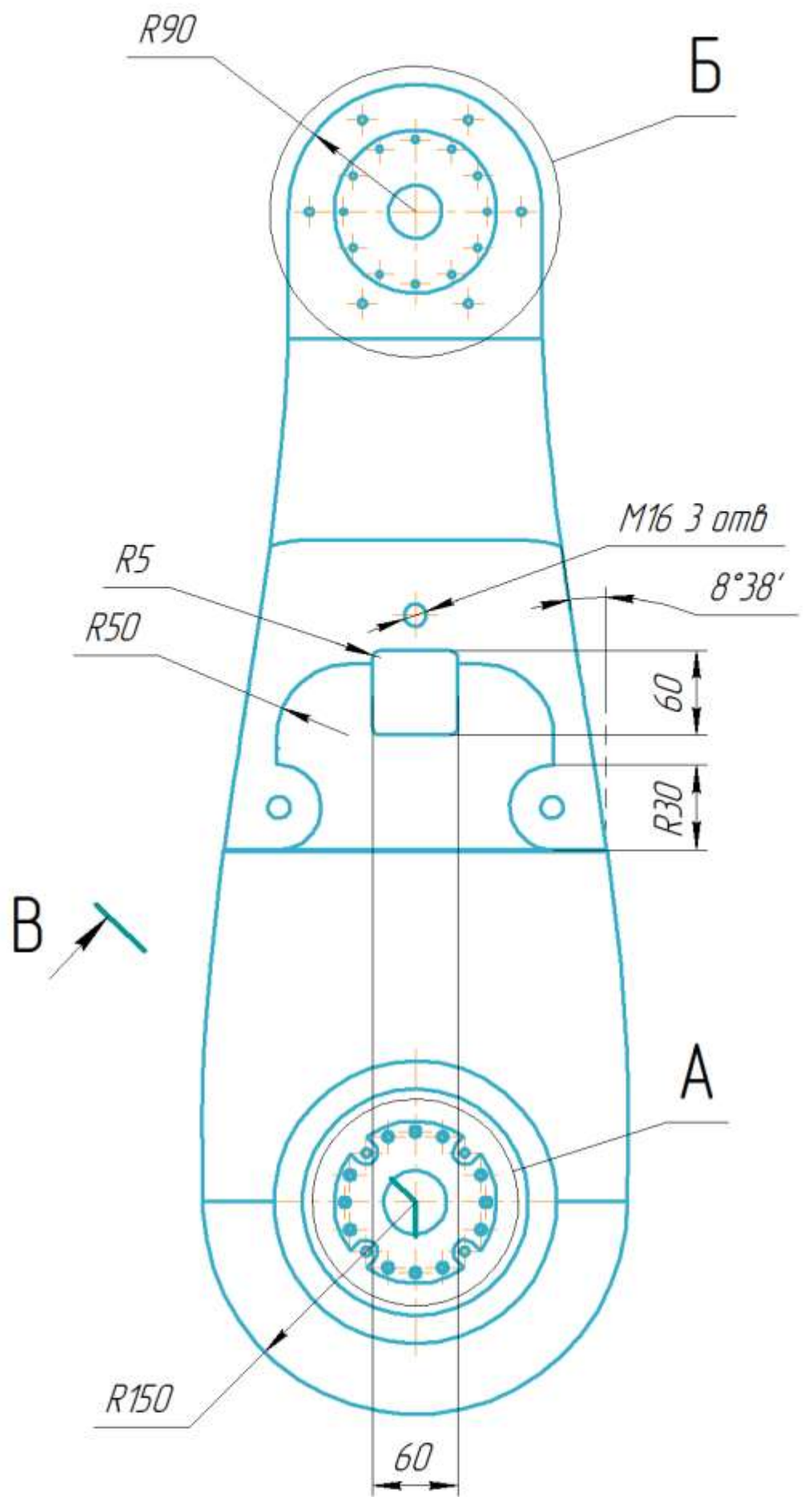
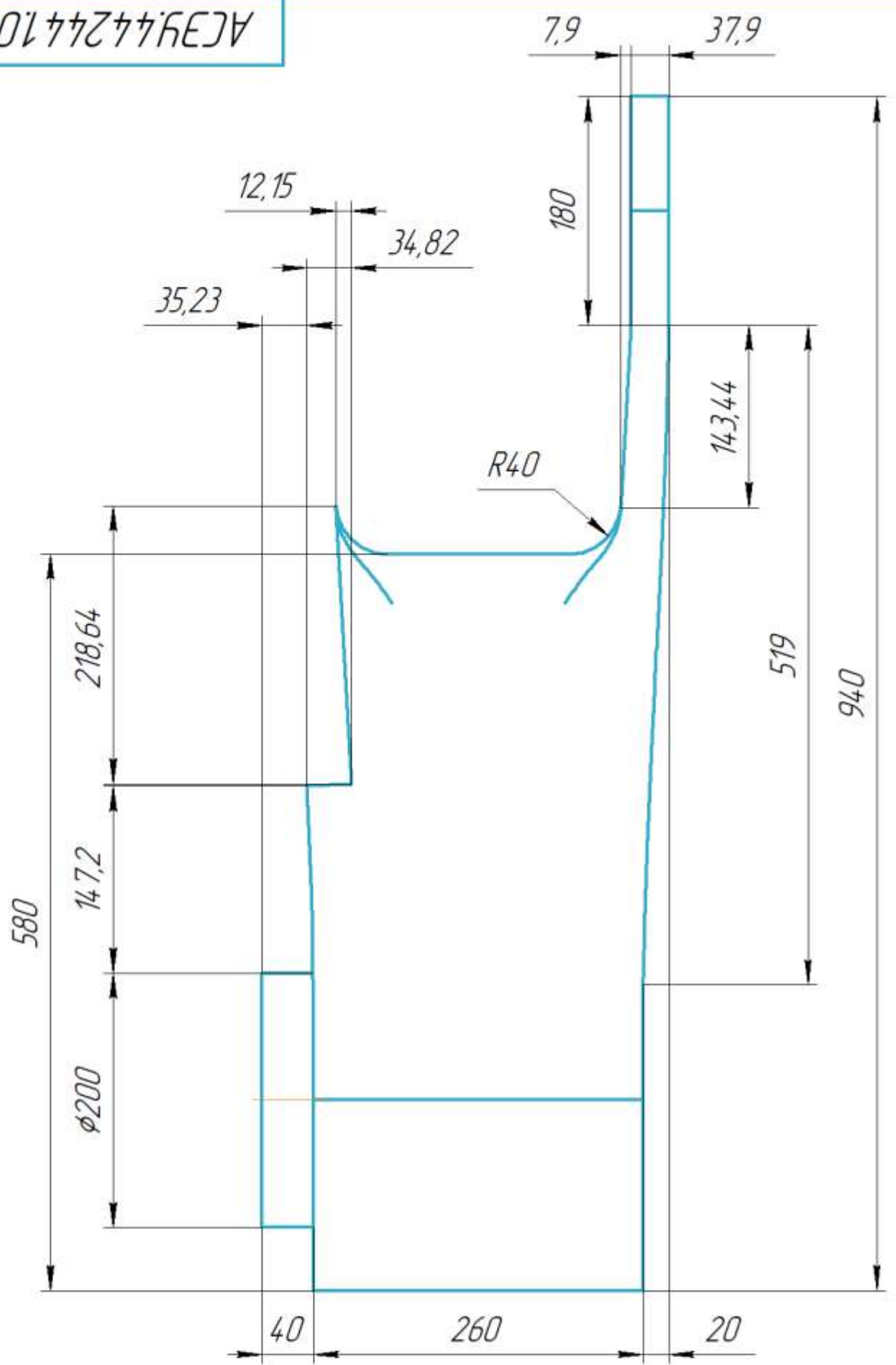
КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Перв. примен.	Формат	Зона	Поз.	Обозначение	Наименование	Кол.	Примечание	
					Документация			
Справ. №	A3			АСЭУ.442441.003 СБ	Второе звено			
					Детали			
	A2	1		АСЭУ.442441.013	Второе звено	1		
					Основная часть			
	A3	2		АСЭУ.442441.014	Второе звено	1		
					Ухо			
	A3	3		АСЭУ.442441.015	Второе звено	2		
					Крышка			
Подп. и дата	A4	4		АСЭУ.442441.025	Переходный фланец на редуктор HRB25	1		
					Стандартные изделия			
		7			Hexagon socket head cap screw ISO 4762 – M4 x 35	12		
		8			Hexagon socket head cap screw ISO 4762 – M5 x 35	12		
		9			Hexagon socket head cap screw ISO 4762 – M12 x 70	3		
		10			Hexagon socket set screw	4		
Инв. № подл.	Изм.	Лист	№ докум.	Подп.	Дата	АСЭУ.442441.003		
	Разраб.	Пешков				Лит.	Лист	Листов
	Проб.						1	2
	Н.контр.					Второе звено		
	Утв.							
						СНИУ гр.24 14		

Формат A4

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.
Инд. № подл. Подп. и дата
Изм. № подл. Подп. и дата
Изм. инд. № Инд. № подл. Подп. и дата
Справ. №
Перв. примен.

АСЗУ.44.244.1013

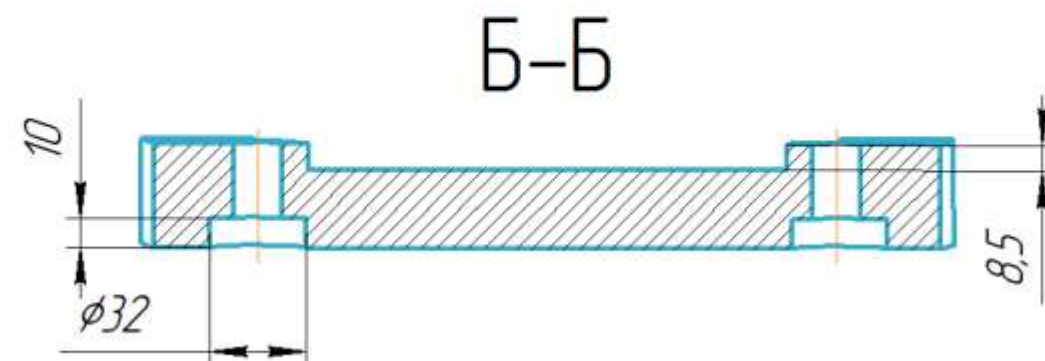
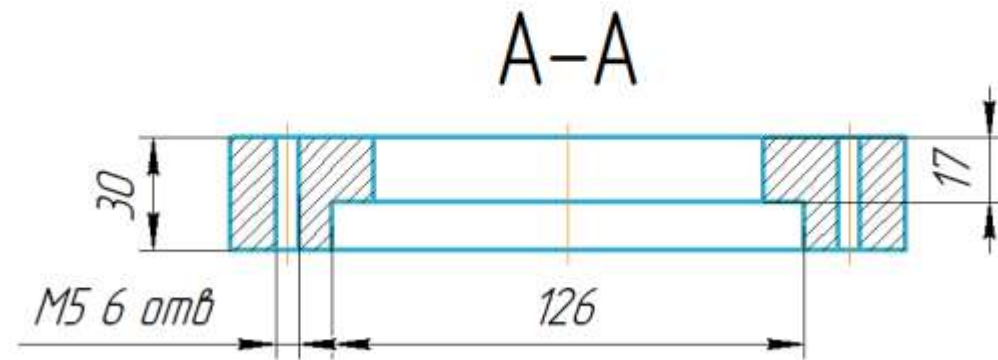
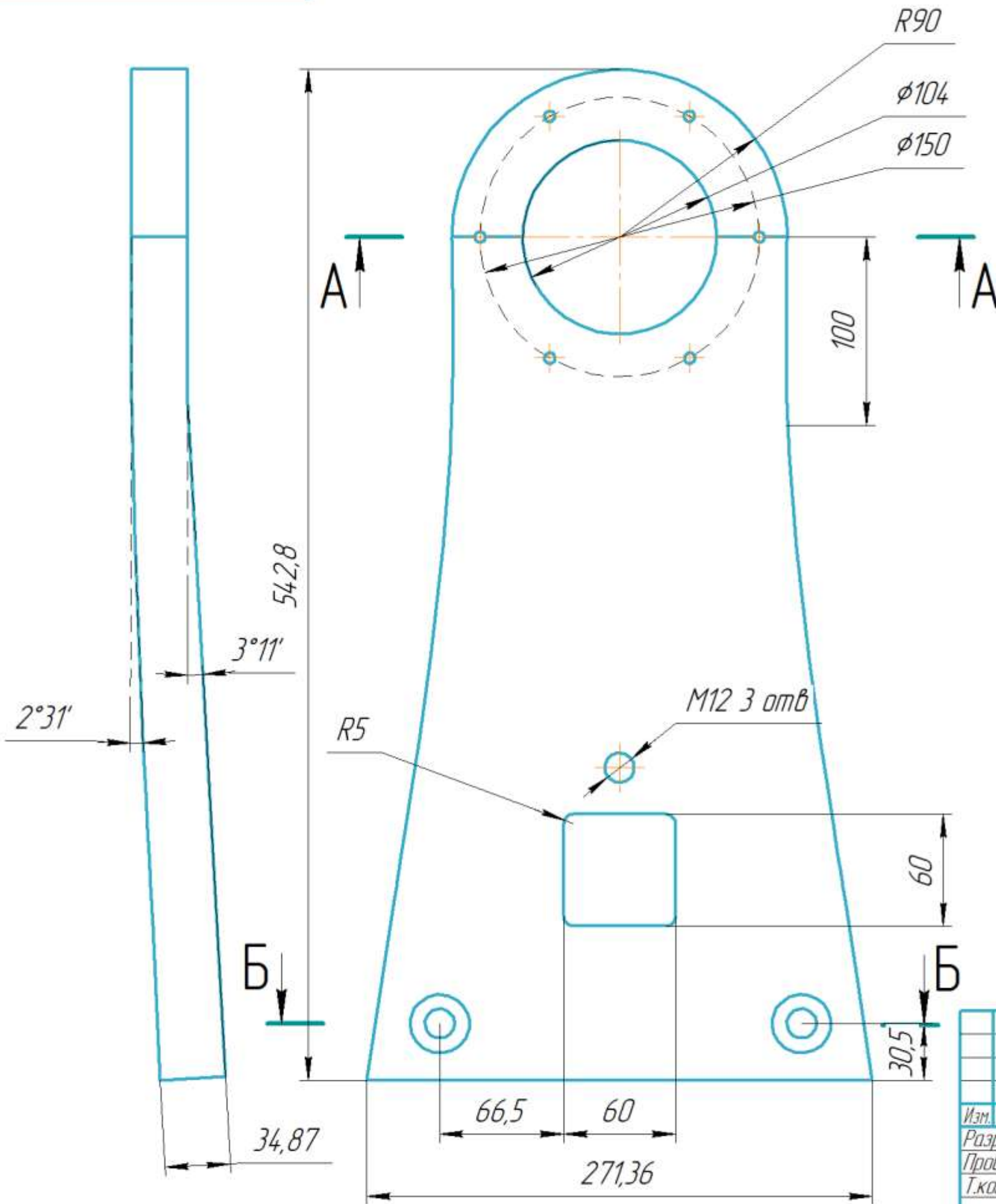


1 Неуказанные предельные отклонения размеров: Н10, h10, ±IT14/2 по ГОСТ 25346-89.

- Изготовление изделия выполнять по следующей технологии:
- 1 Изготовить матрицу из Пеноплэкс Фундамент методом фрезерования по чертежу.
 - 2 Нанести слой разделительного воска ПВС-1.
 - 3 Выполнить послойное нанесение углеволокна ВМН-4МС с пропиткой смолой ЭД-20 (7:1 с ПЭПА).
 - 4 Провести вакуумирование при 25 мбар в течение 60 мин.
 - 5 Выполнить термообработку при 120 °С в течение 6 часов.
 - 6 Провести обработку и контроль качества изделия.

					АСЗУ.44.244.1013		
Изм.	Лист	№ докум.	Подп.	Дата	Второе звено Основная часть		
Разраб.	Лешков						
Пров.					Лит.	Масса	Масштаб
Г.контр.						40	1:4
Н.контр.					Лист	Листов	1
Утв.					СНИУ гр.24 14		
					Копировал		
					Формат А2		

АСЭУ.44244.1014



АСЭУ.44244.1014				Лист 1		
Изм. Лист	№ докум.	Подп.	Дата	Второе звено Ухо		
Разраб.	Пешков			Лист 1		
Пров.				Листов 2		
Т.контр.				СНИУ гр.24 14		
Н.контр.				Композиционный материал на основе углеволокна МН-4МС (ТУ 1916-017-05798606-2015) и эпоксидной смолы ЭД-20 (ГОСТ 10587-84)		
Утв.				Копировал		

1 Неуказанные предельные отклонения размеров: Н10, h10, ±IT14/2 по ГОСТ 25346-89.

Изготовление изделия выполнять по следующей технологии:

2 Изготовить матрицу из Пеноплэкс Фундамент методом фрезерования по чертежу.

3 Нанести слой разделительного воска ПВС-1.

4 Выполнить послойное нанесение углеволокна ВМН-4МС с пропиткой смолой ЭД-20 (7:1 с ПЭПА).

5 Провести вакуумирование при 25 мбар в течение 60 мин.

6 Выполнить термообработку при 120 °С в течение 6 часов.

7 Провести обработку и контроль качества изделия.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Изм.	Лист	№ докум.	Подп.	Дата

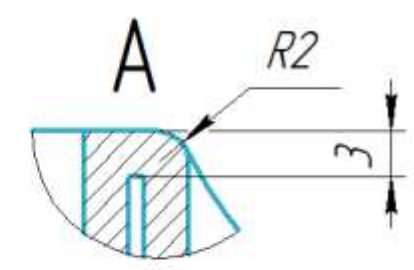
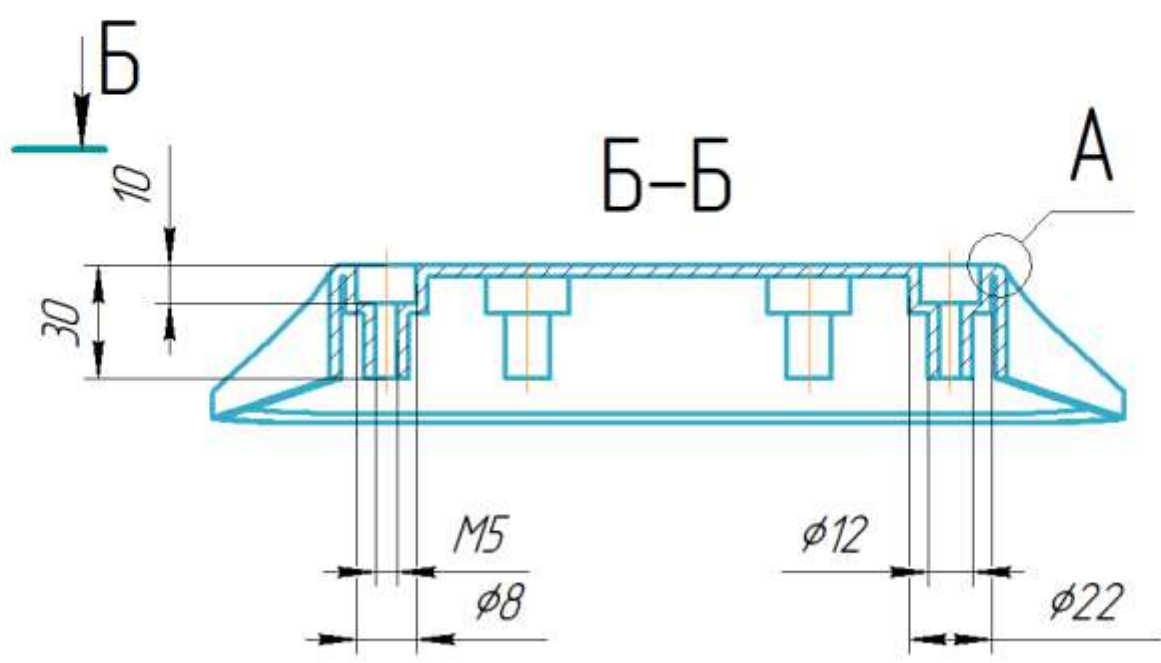
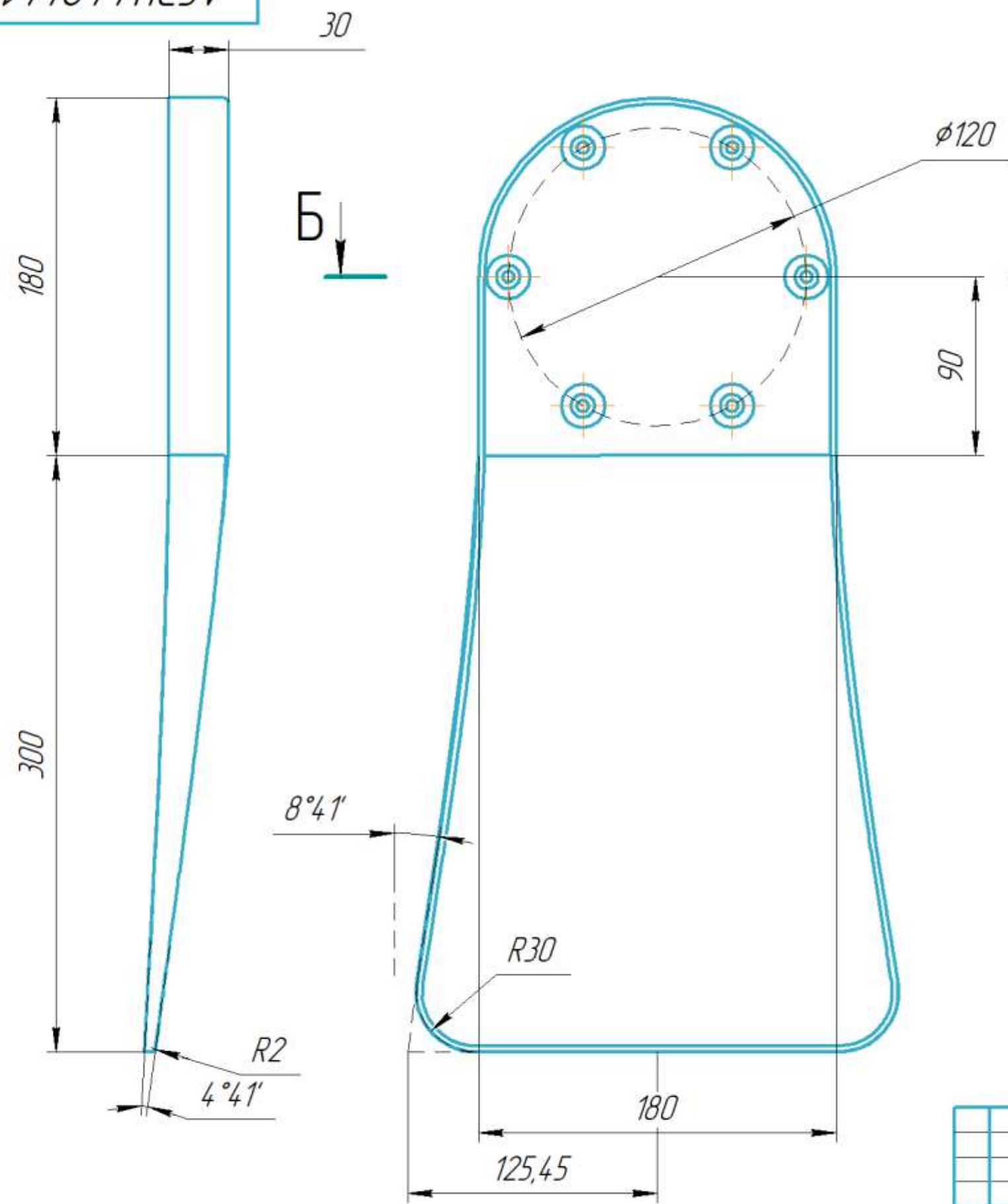
КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Изм. № подл. Подп. и дата Инв. № дудл. Подп. и дата

Справ. №

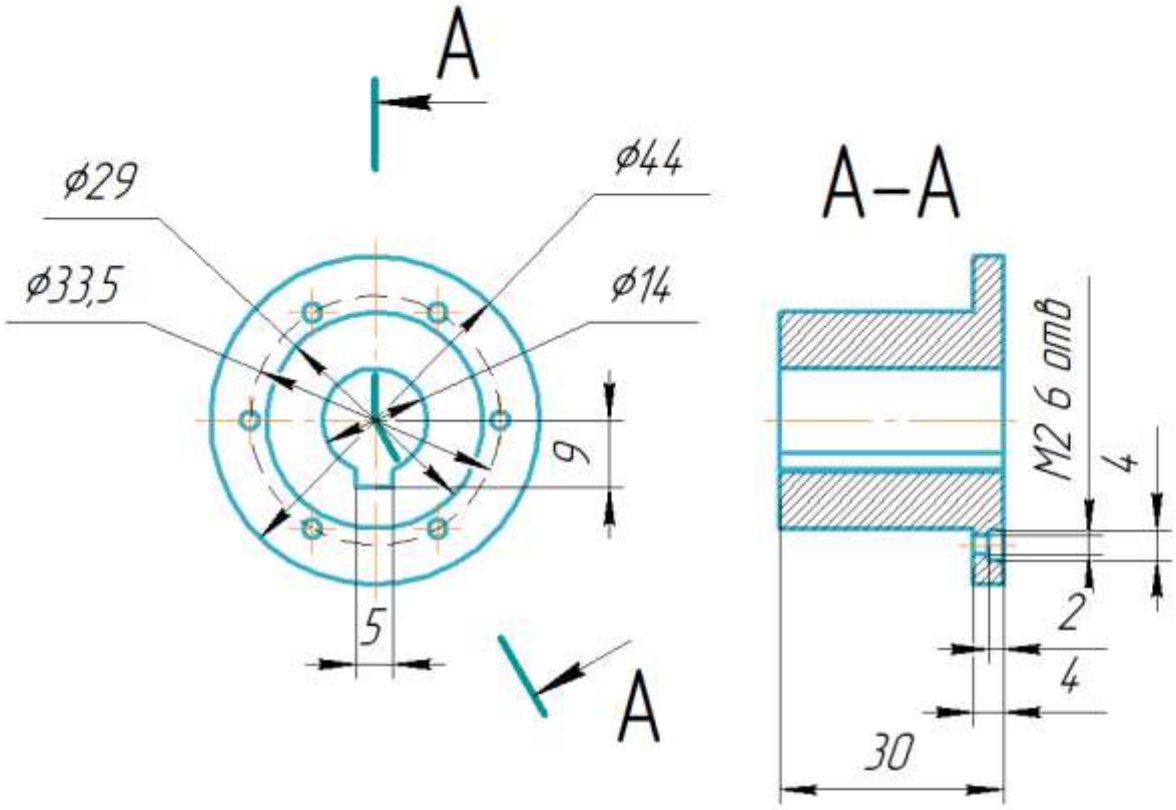
Перв. примен.

АСЭУ.44244.1015



					АСЭУ.44244.1.015				
					Второе звено Крышка	Лист.	Масса	Масштаб	
Изм. Лист	№ докум.	Подп.	Дата				0.5	1:2,5	
Разраб.	Пешков								
Проб.									
Т.контр.						Лист	1	Листов	2
					Композиционный материал на основе углеволокна МН-4МС (ТУ 1916-017-05798606-2015) и эпоксидной смолы ЭД-20 (ГОСТ 10587-84)				
Н.контр.					СНИУ гр.24.14				
Утв.									

АСЭУ.44.244.1025



1 Неуказанные предельные отклонения размеров: H10, h10, ±IT14/2 по ГОСТ 25346-89.

					АСЭУ.44.244.1.025			
					Переходный фланец на редуктор HRB25	Лист	Масса	Масштаб
Изм.	Лист	№ докум.	Подп.	Дата			0.2	1:1
Разраб.		Пешков						
Проб.								
Т.контр.						Лист	Листов	1
Н.контр.					Сталь 08Х18Н10 ГОСТ 5632-2014	СНИУ гр.24 14		
Утв.								

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Перв. примен.

Справ. №

Подп. и дата

Инд. № дудл.

Взам. инв. №

Подп. и дата

Инд. № подл.

Не для коммерческого использования

Копировал

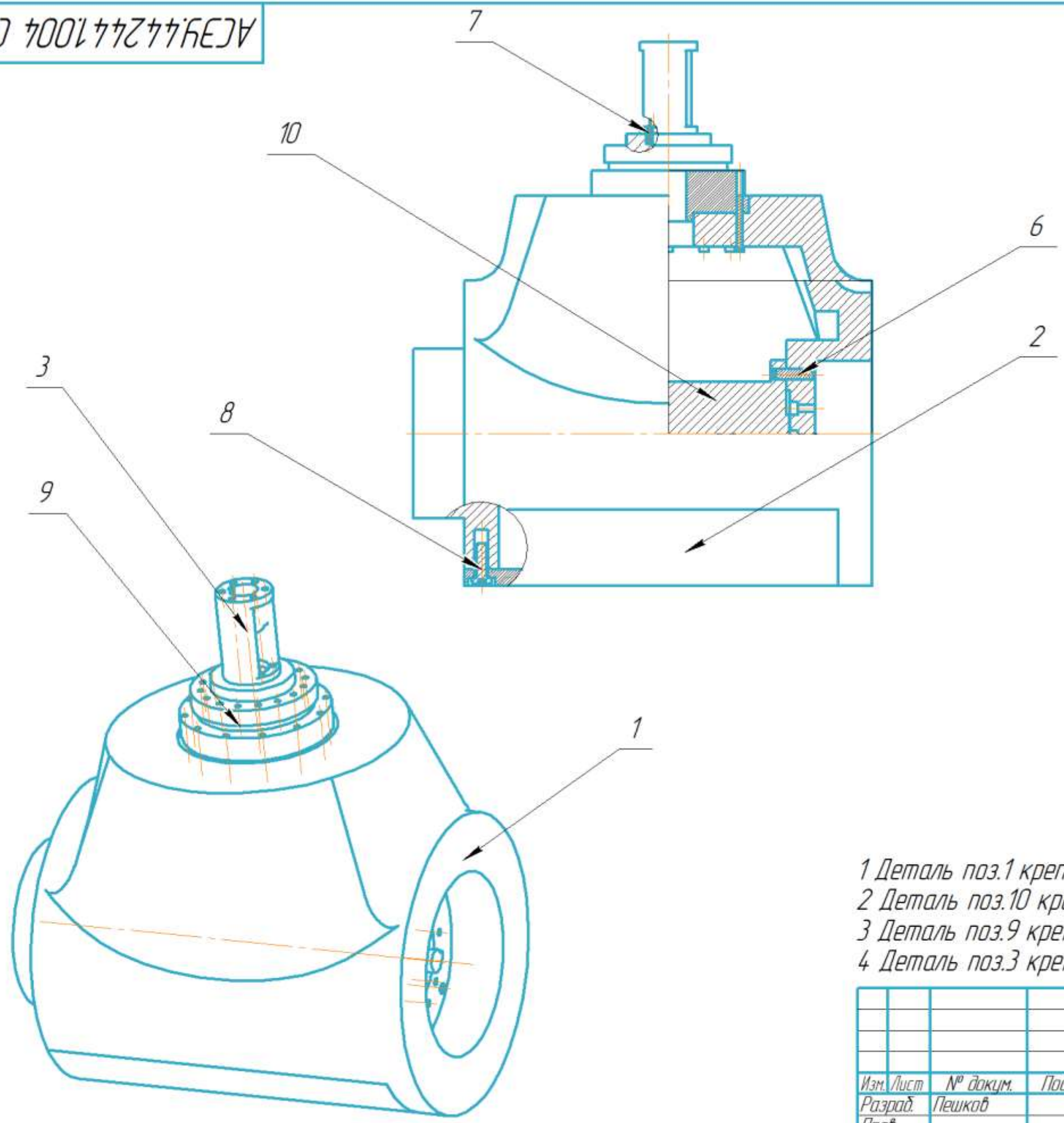
Формат А4

ПРИЛОЖЕНИЕ Д
Сборочная единица третьего звена

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Изм. № подл.	Подп. и дата	Взам. инб. №	Инб. № дубл.	Подп. и дата	Спроб. №	Перв. примен.
--------------	--------------	--------------	--------------	--------------	----------	---------------

АСЗУ.44244.1004 СБ



- 1 Деталь поз.1 крепится к детали поз.2 болтом М6
- 2 Деталь поз.10 крепится к детали поз.1 болтами М6
- 3 Деталь поз.9 крепится к детали поз.1 болтами М3
- 4 Деталь поз.3 крепится к детали поз.9 болтами М2

					АСЗУ.44244.1.004 СБ				
					Третье звено Сборочная единица	Лит.	Масса	Масштаб	
Изм.	Лист	№ докум.	Подп.	Дата			12	1:2.5	
Разраб.		Пешков							
Пров.									
Т.контр.						Лист	Листов	1	
Н.контр.						СНИУ гр.24 14			
Утв.									

Перв. примен.	Формат	Зона	Поз.	Обозначение	Наименование	Кол.	Примечание
Справ. №					Документация		
	A3			АСЭУ.44244.1.004 СБ	Третье звено		
					Детали		
	A3	1		АСЭУ.44244.1.016	Третье звено	1	
					Основная часть		
	A3	2		АСЭУ.44244.1.017	Третье звено	1	
					Крышка		
	A4	3		АСЭУ.44244.1.027	Переходный фланец на редуктор HRB20	1	
Взам. инв. №					Стандартные изделия		
			6		Hexagon socket set screw	4	
					ISO 4026 – M6 x 25		
			7		Pan head screw ISO 7045 – M2 x 8 – Z	6	
			8		Pan head screw ISO 7045 – M6 x 20 – Z	1	
			9		Редуктор Harmonic Joint	1	
					Reducer HBK20		
			10		Серводвигатель	1	
					Nema 34 86HB250-118B		
Инв. № подл.	Изм.	Лист	№ докум.	Подп.	Дата	АСЭУ.44244.1.004	
	Разраб.	Пешков				Лит.	Лист
	Пров.						Листов
	Н.контр.						1
Утв.						Третье звено	
						СНИУ гр.24 14	

Формат А3

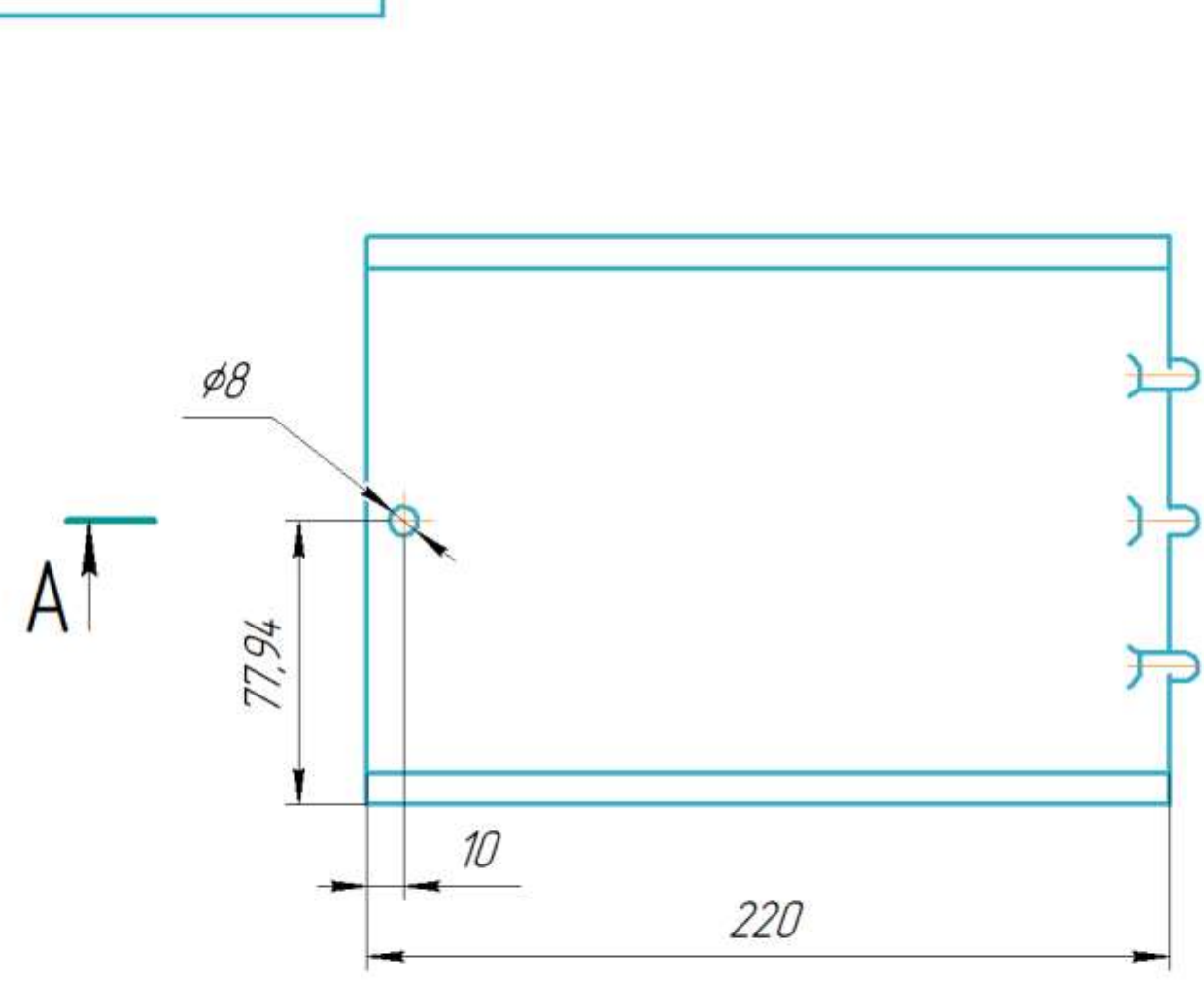
КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Изм. № Подп. и дата
Разраб. Пешков
Пров. Т.контр.
Н.контр. Утв.

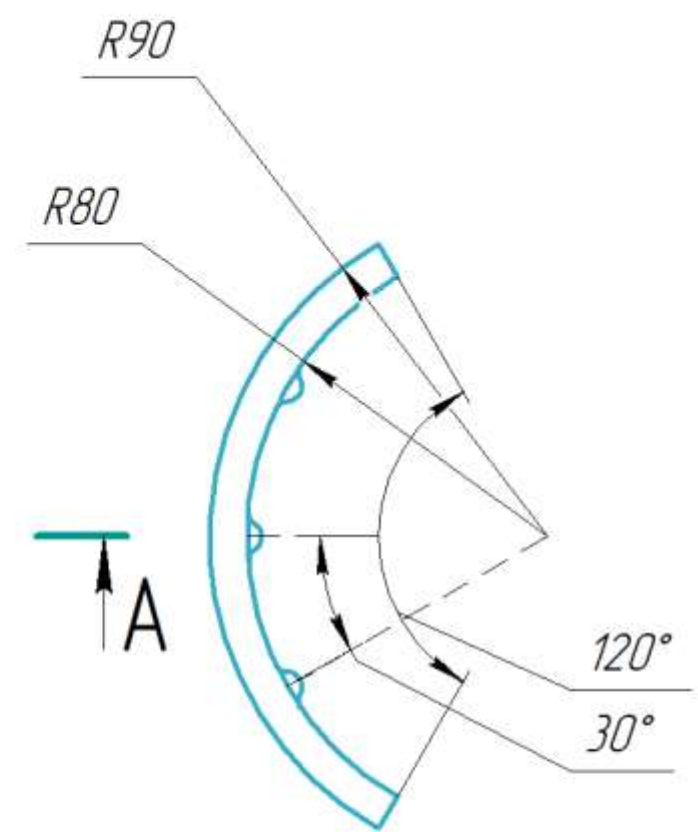
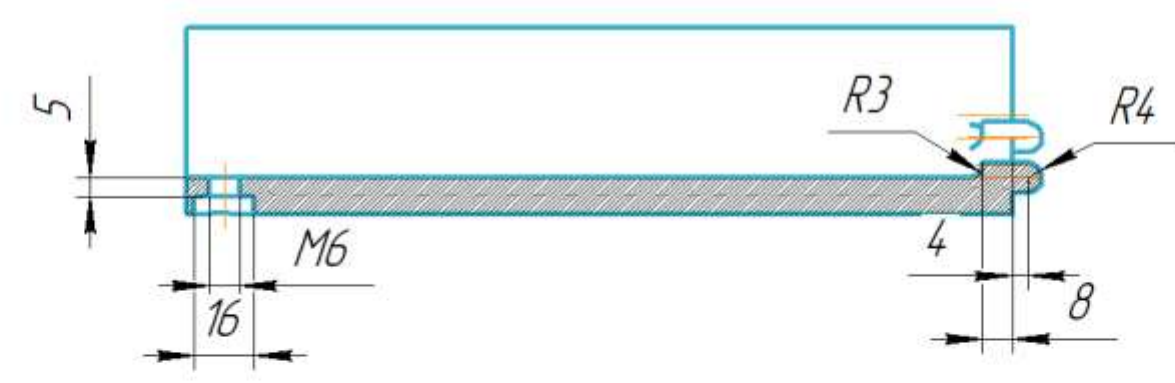
Взам. инв. № Инв. № дцкл.
Подп. и дата

Спроб. №
Перв. примен.

АСЭУ.44244.1017



A-A

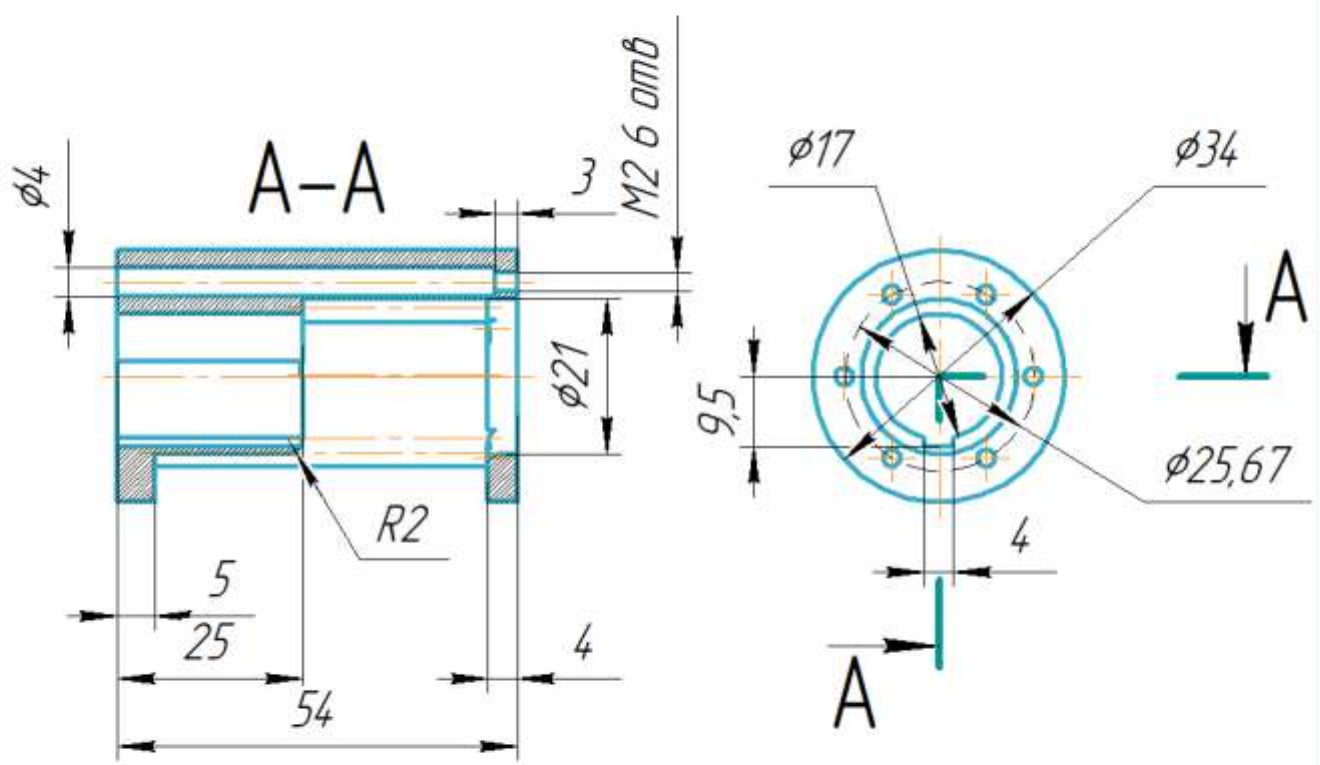


1 Неуказанные предельные отклонения размеров: Н10, h10, ±IT14/2 по ГОСТ 25346-89.

- Изготовление изделия выполнять по следующей технологии:
- 2 Изготовить матрицу из Пеноплэкс Фундамент методом фрезерования по чертежу.
 - 3 Нанести слой разделительного воска ПВС-1.
 - 4 Выполнить послойное нанесение углеволокна ВМН-4МС с пропиткой смолой ЭД-20 (7:1 с ПЭПА).
 - 5 Провести вакуумирование при 25 мбар в течение 60 мин.
 - 6 Выполнить термообработку при 120 °С в течение 6 часов.
 - 7 Провести обработку и контроль качества изделия.

					АСЭУ.44244.1017			
					Третье звено Крышка	Лит.	Масса	Масштаб
Изм.	Лист	№ докум.	Подп.	Дата				
Разраб.	Пешков						0.8	1:2
Пров.								
Т.контр.						Лист	Листов	1
Н.контр.					Композиционный материал на основе углеволокна МН-4МС (ТУ 1916-017-05798606-2015) и эпоксидной смолы ЭД-20 (ГОСТ 10587-84)			
Утв.					СНИУ гр.24 14			

АСЭУ.44244.1027



1 Неуказанные предельные отклонения размеров: Н10, h10, ±IT14/2 по ГОСТ 25346-89.

АСЭУ.44244.1027

Лист	Масса	Масштаб
1	0.5	1:1
Лист	Листов	1
Сталь 08Х18Н10 ГОСТ 5632-2014		СНИУ гр.24 14

Изм.	Лист	№ докум.	Подп.	Дата
Разраб.	Пешков			
Проб.				
Т.контр.				
Н.контр.				
Чтб.				

Переходный фланец
на редуктор HRB20

Сталь 08Х18Н10 ГОСТ 5632-2014

Лист 1
Масса 0.5
Масштаб 1:1
СНИУ гр.24 14

Перв. примен.

Спроб. №

Подп. и дата

Взам. инв. №

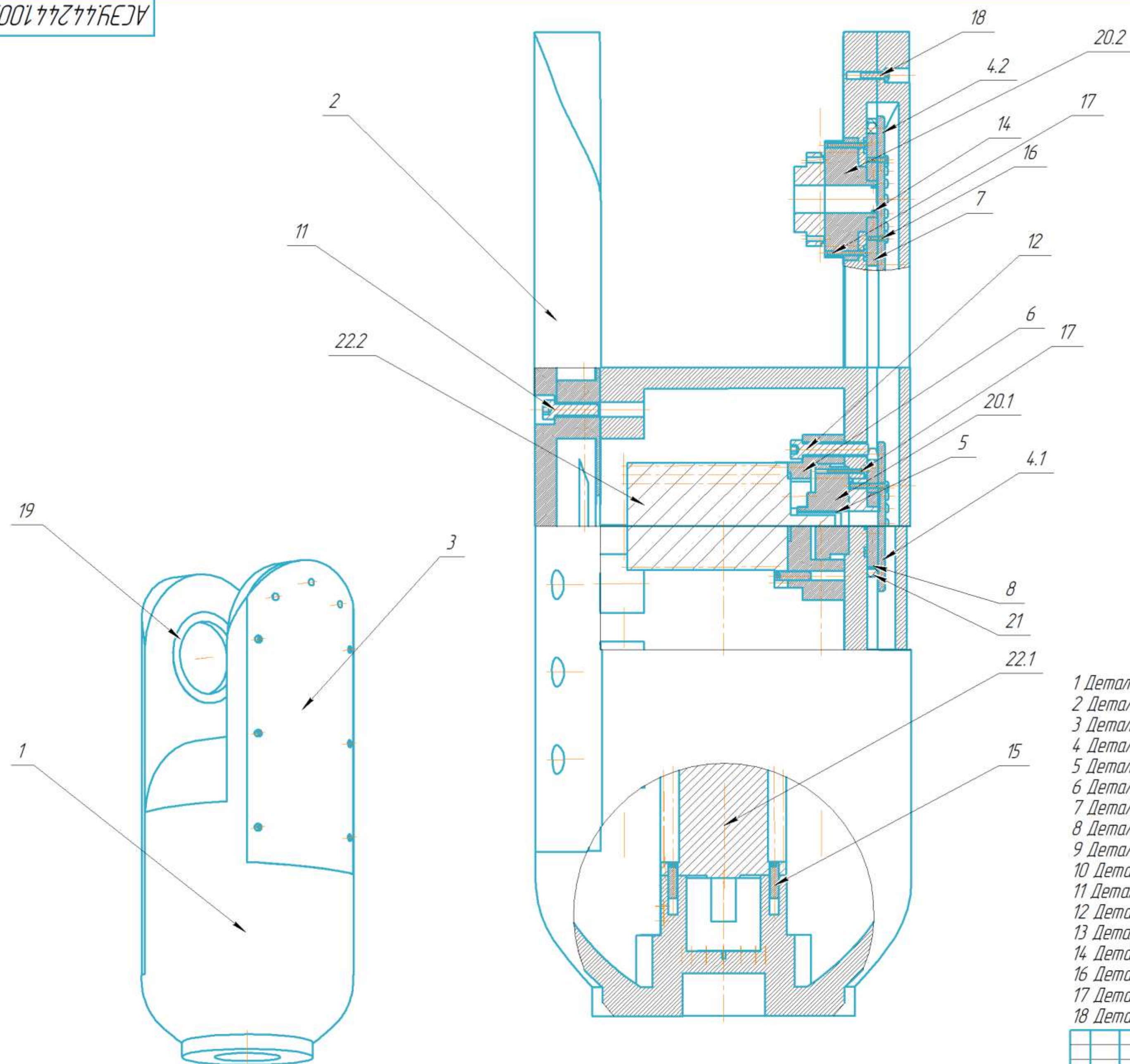
Инв. № дудл.

Подп. и дата

Инв. № подл.

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

ПРИЛОЖЕНИЕ Е
Сборочная единица четвертого звена



- 1 Деталь поз.1 крепится к детали поз.2 болтами М8
- 2 Деталь поз.3 крепится к детали поз.1 болтами М4
- 3 Деталь поз.6 крепится к детали поз.1 болтами М8
- 4 Деталь поз.20.1 крепится к детали поз.6 болтами М3
- 5 Деталь поз.20.2 крепится к детали поз.1 болтами М3
- 6 Деталь поз.4.1 крепится к детали поз.20.1 болтами М3
- 7 Деталь поз.4.2 крепится к детали поз.20.2 болтами М3
- 8 Деталь поз.22.1 крепится к детали поз.1 болтами М6
- 9 Деталь поз.22.2 крепится к детали поз.6 болтами М6
- 10 Деталь поз.19 запрессовывается в деталь поз.2
- 11 Деталь поз.8 крепится к детали поз.20.1 болтами М3
- 12 Деталь поз.4.1 крепится к детали поз.8 болтами М3
- 13 Деталь поз.4.2 крепится к детали поз.7 болтами М3
- 14 Деталь поз.21 натягивается на детали поз.7 и поз.8
- 16 Деталь поз.5 крепится к детали поз.20.1 болтами М2
- 17 Деталь поз.5 крепится к детали поз.20.2 болтами М2
- 18 Деталь поз.7 крепится к детали поз.20.2 болтами М2

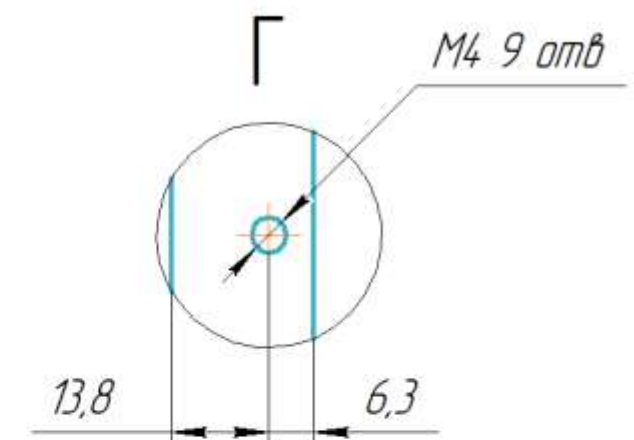
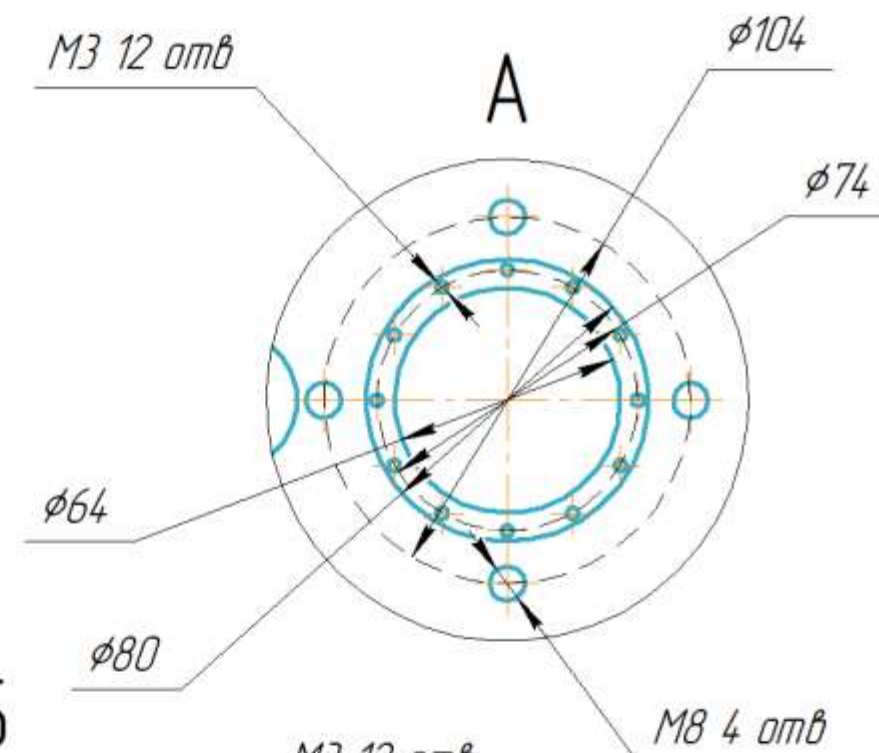
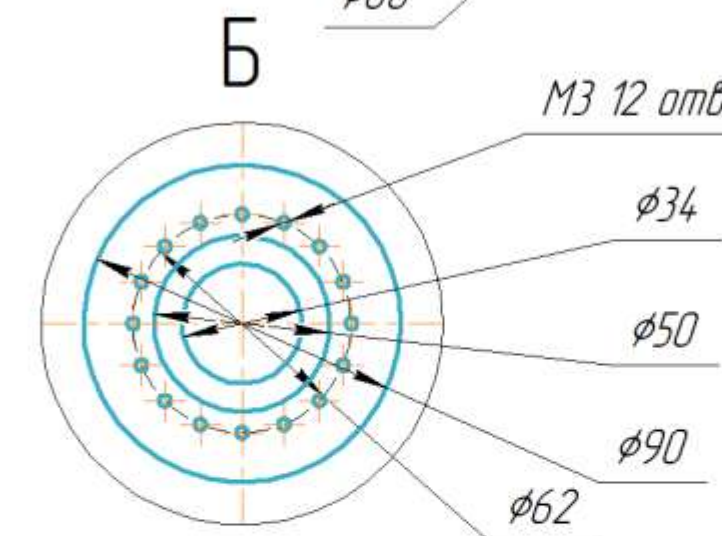
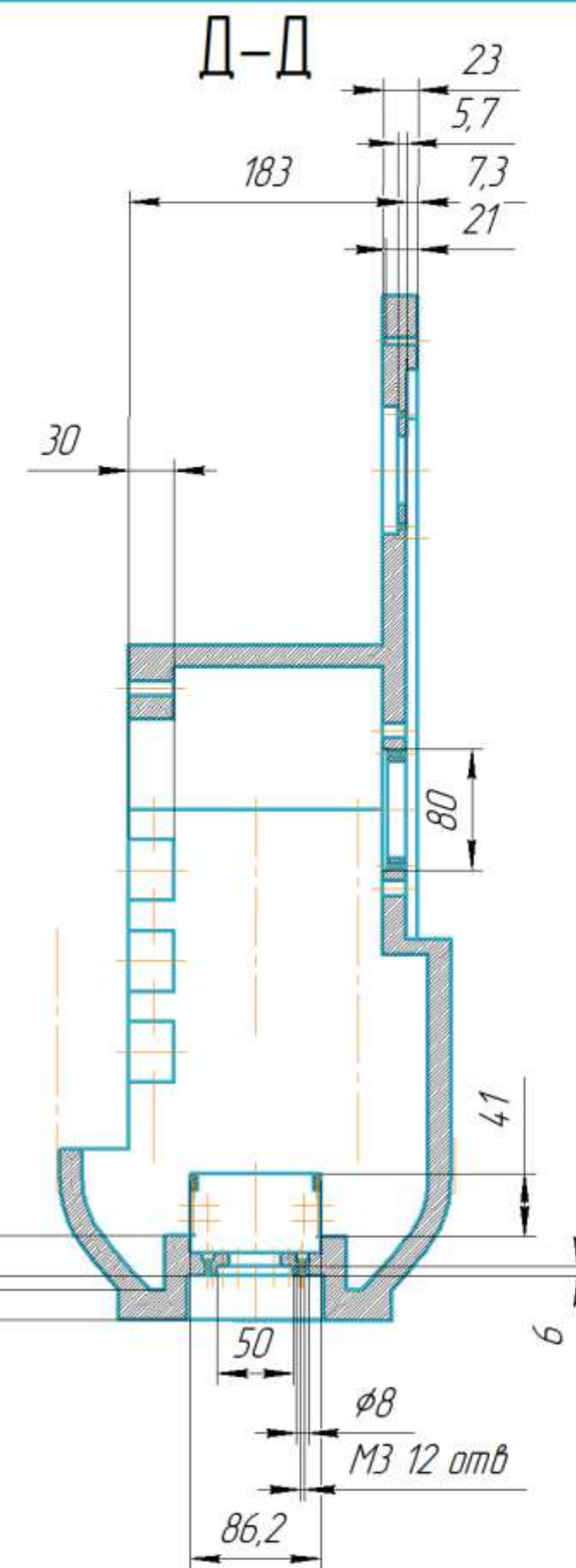
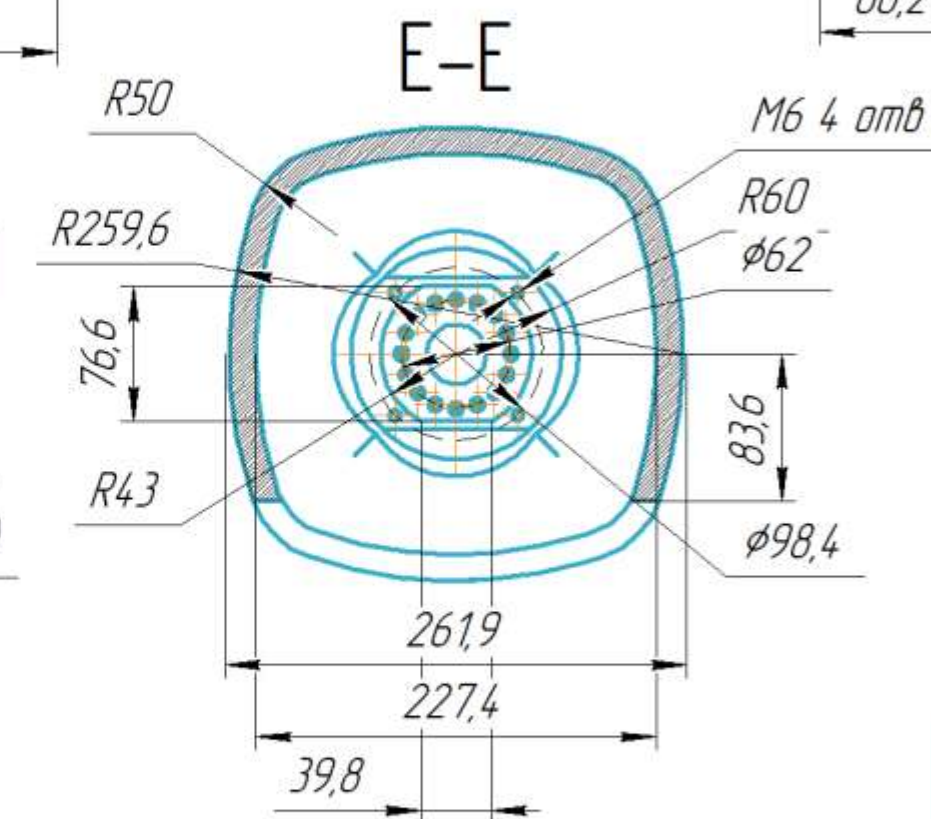
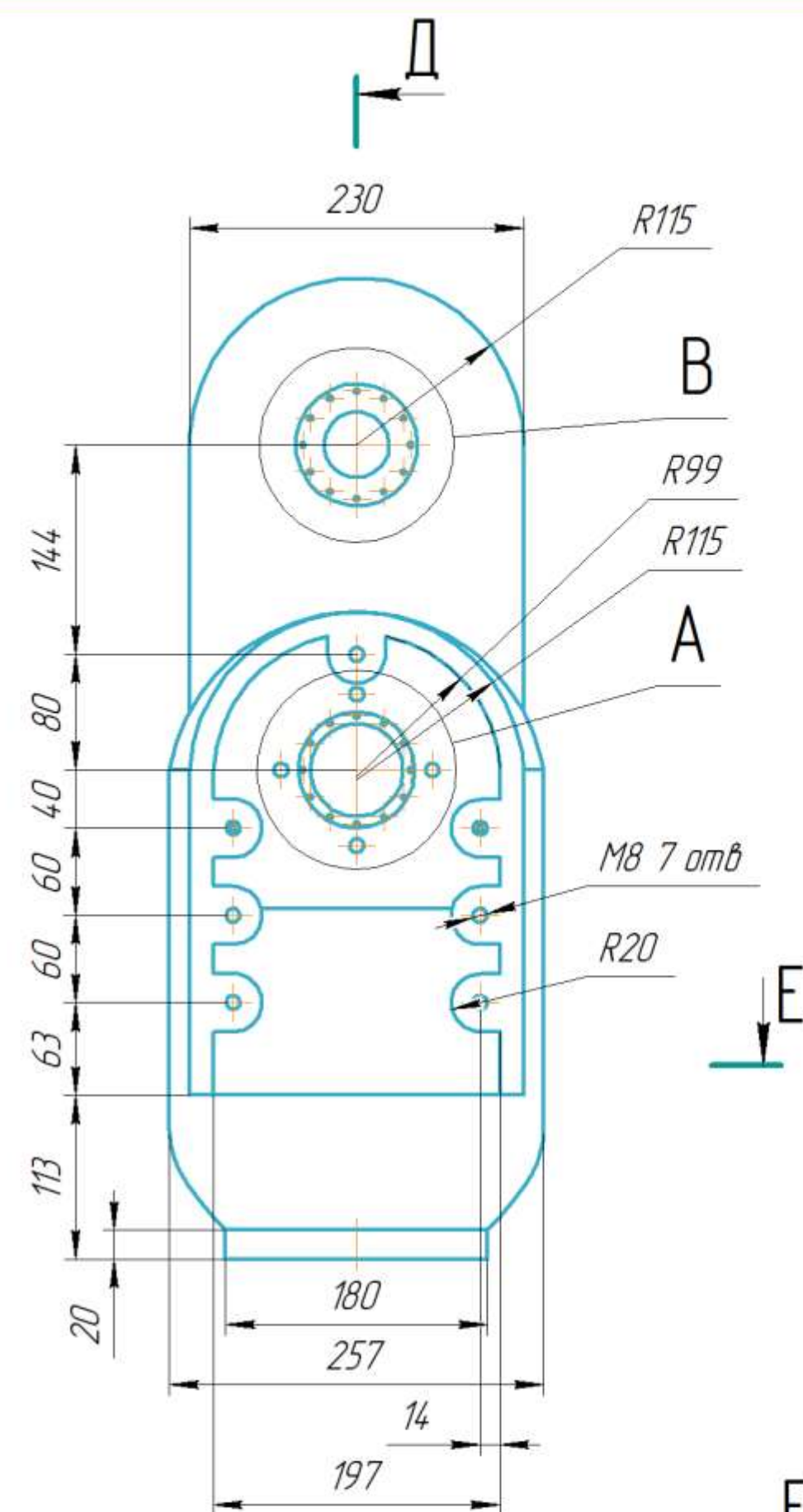
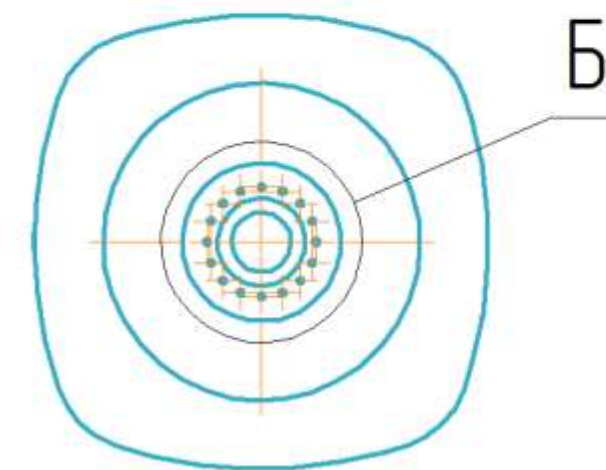
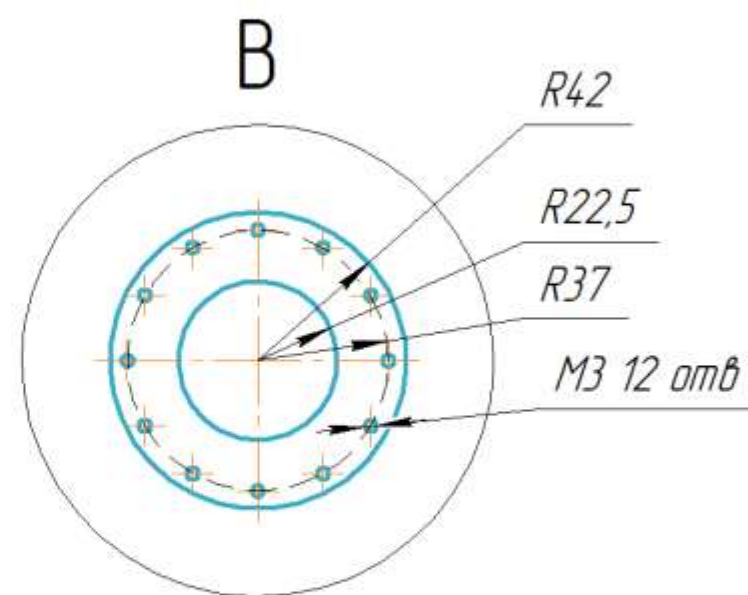
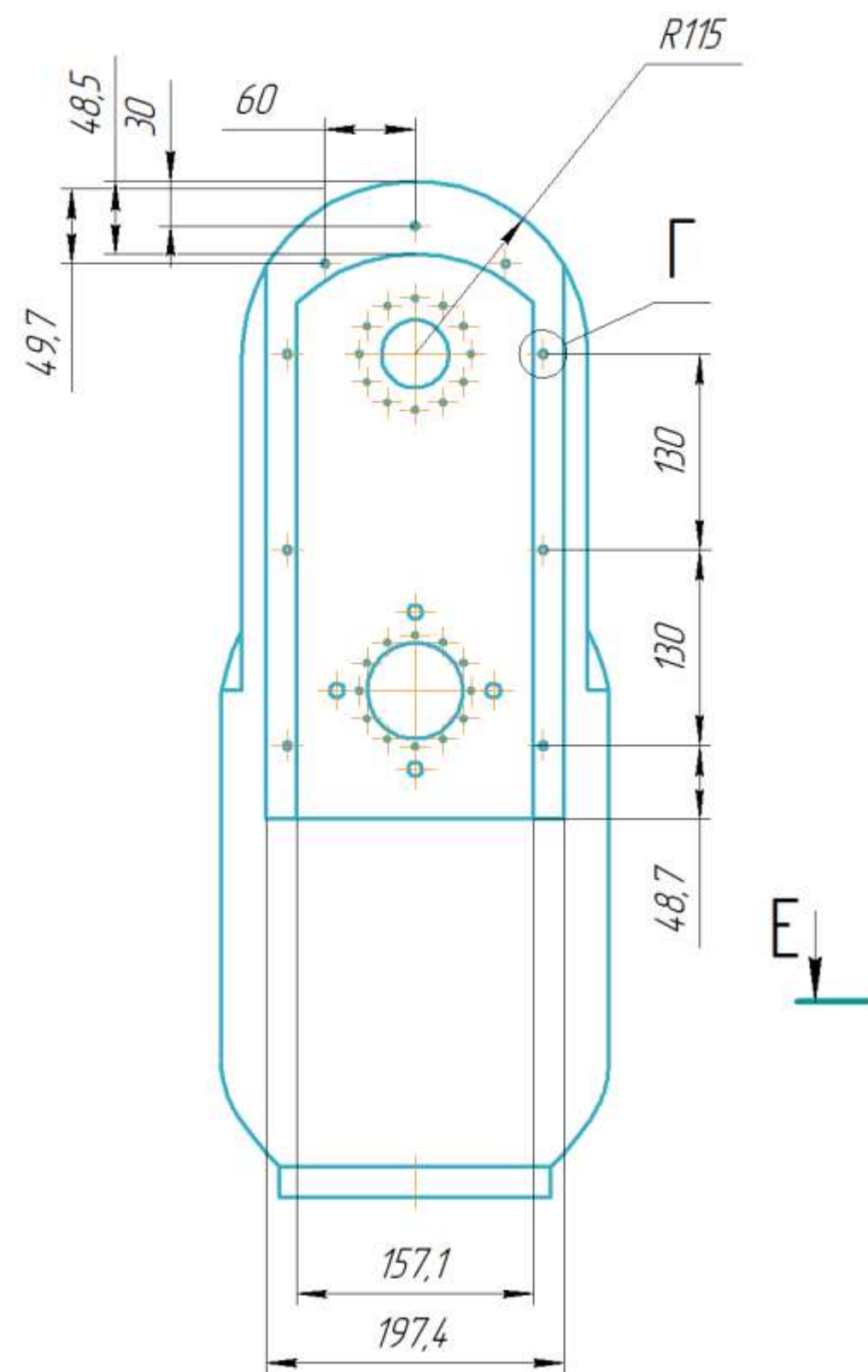
					АСЭУ.442441.005 СБ			
Изм.	Лист	№ докум.	Подп.	Дата	Четвертое звено Сборочная единица	Лист	Масса	Масштаб
Разраб.		Пешков					55	1:2
Проб.								
Т.контр.						Лист	Листов	1
Н.контр.						СНИУ гр.24 14		
Утв.								

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Перв. примен.	Справ. №	Формат	Зона	Поз.	Обозначение	Наименование	Кол.	Примечание			
						Документация					
		A2			АСЭУ.44244.1.005	Четвертое звено					
						Детали					
		A2	1	АСЭУ.44244.1.018	Четвертое звено	1					
					Основная часть						
		M3	2	АСЭУ.44244.1.019	Четвертое звено	1					
					Ухо						
		A3	3	АСЭУ.44244.1.020	Четвертое звено	1					
					Крышка						
		A4	4	АСЭУ.44244.1.023	Защита ремня	2					
		A4	5	АСЭУ.44244.1.028	Переходный фланец	2					
					на редуктор HRB17						
		A4	6	АСЭУ.44244.1.029	Проставка	1					
		A4	7	АСЭУ.751818.000	Зубчатое колесо	1					
					iso 24Z 08A						
					Доработка						
		A4	8	АСЭУ.751818.001	Зубчатое колесо	1					
					iso 26Z 08A						
					Доработка						
						Стандартные изделия					
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	АСЭУ.44244.1.005			Лит.	Лист	Листов	
					Изм.	Лист	№ докум.				
					Разраб.	Пешков	Подп.				Дата
					Проб.						
					Н.контр.						
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	Четвертое звено			Лит.	Лист	Листов	
					Изм.	Лист	№ докум.				
					Разраб.	Пешков	Подп.				Дата
					Проб.						
					Н.контр.						
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	СНИУ гр.24 14			Лит.	Лист	Листов	
					Изм.	Лист	№ докум.				
					Разраб.	Пешков	Подп.				Дата
					Проб.						
					Н.контр.						

					АСЭУ.44244.1.005	Лист
Изм.	Лист	№ докум.	Подп.	Дата		2

8101772777E3V



				АСЭУ.44244.1018			
				Четвертое звено Основная часть			
Изм.	Лист	№ докум.	Подп.	Дата	Лит.	Масса	Масштаб
Разраб.		Пешков				40	1:4
Проб.							
Т.контр.					Лист 1	Листов 2	
					СНИУ гр.24 14		
Н.контр.							
Утв.							
				Композиционный материал на основе углеволокна МН-4МС (ТУ 1916-017-05798606-2015) и эпоксидной смолы ЭД-20 (ГОСТ 10587-84)			

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Не для коммерческого использования

1 Неуказанные предельные отклонения размеров: H10, h10, ±IT14/2 по ГОСТ 25346-89.

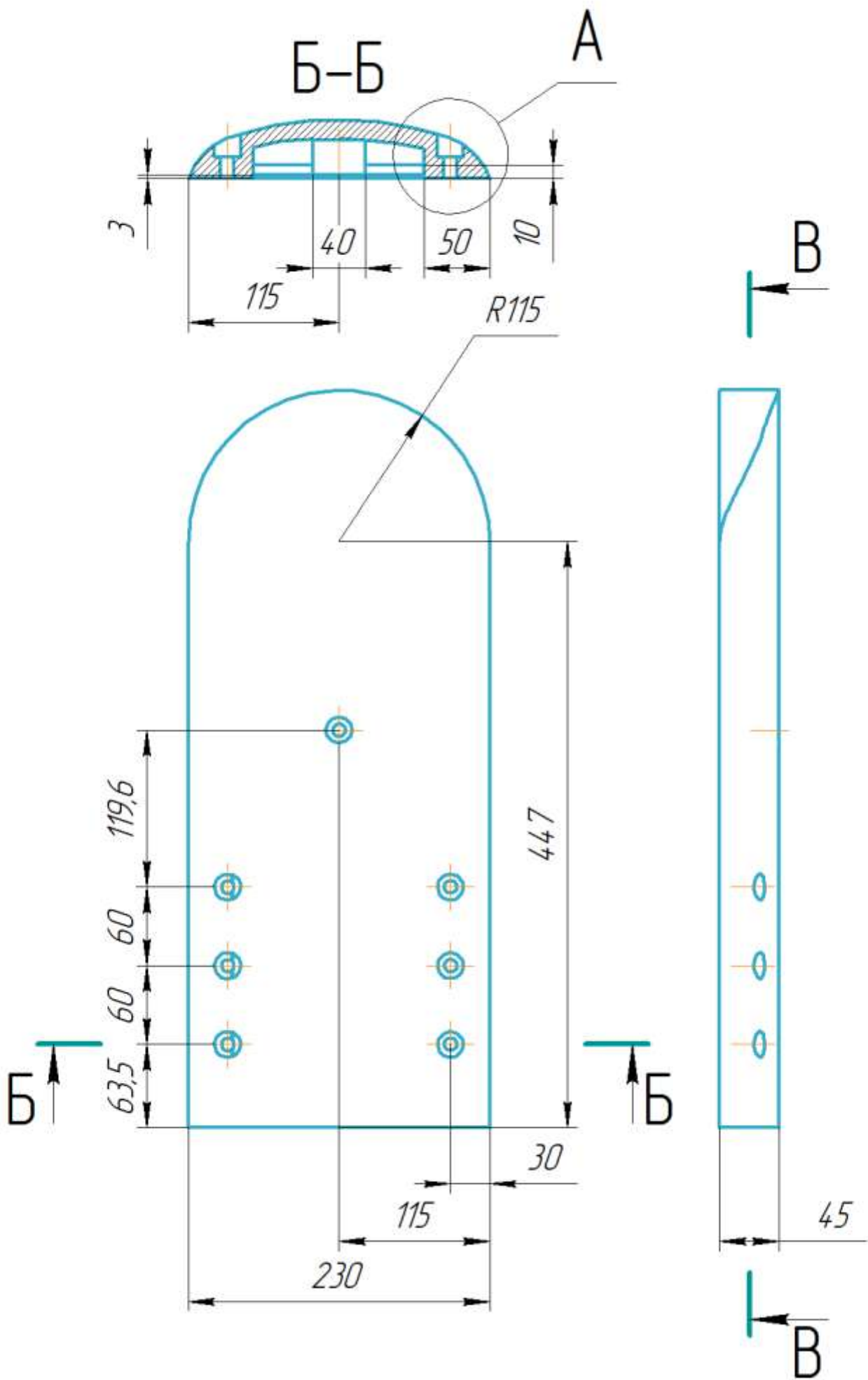
Изготовление изделия выполнять по следующей технологии:

- 2 Изготовить матрицу из Пеноплэкс Фундамент методом фрезерования по чертежу.
- 3 Нанести слой разделительного воска ПВС-1.
- 4 Выполнить послойное нанесение углеволокна ВМН-4МС с пропиткой смолой ЭД-20 (7:1 с ПЭПА).
- 5 Провести вакуумирование при 25 мбар в течение 60 мин.
- 6 Выполнить термообработку при 120 °С в течение 6 часов.
- 7 Провести обработку и контроль качества изделия.

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

АСЭУ.44244.1019



АСЭУ.44244.1019				Лит.			Масса	Масштаб
Четвертое звено							5	1:4
Ухо				Лист 1			Листов 2	
Композиционный материал на основе углеволокна МН-4МС (ТУ 1916-017-05798606-2015) и эпоксидной смолы ЭД-20 (ГОСТ 10587-84)				СНИУ гр.24 14				
Копировал				Формат А3				

АСЭУ.44244.1019

1 Неуказанные предельные отклонения размеров: Н10, h10, ±IT14/2 по ГОСТ 25346-89.

Изготовление изделия выполнять по следующей технологии:

2 Изготовить матрицу из Пеноплэкс Фундамент методом фрезерования по чертежу.

3 Нанести слой разделительного воска ПВС-1.

4 Выполнить послойное нанесение углеволокна ВМН-4МС с пропиткой смолой ЭД-20 (7:1 с ПЭПА).

5 Провести вакуумирование при 25 мбар в течение 60 мин.

6 Выполнить термообработку при 120 °С в течение 6 часов.

7 Провести обработку и контроль качества изделия.

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Изм.	Лист	№ докум.	Подп.	Дата

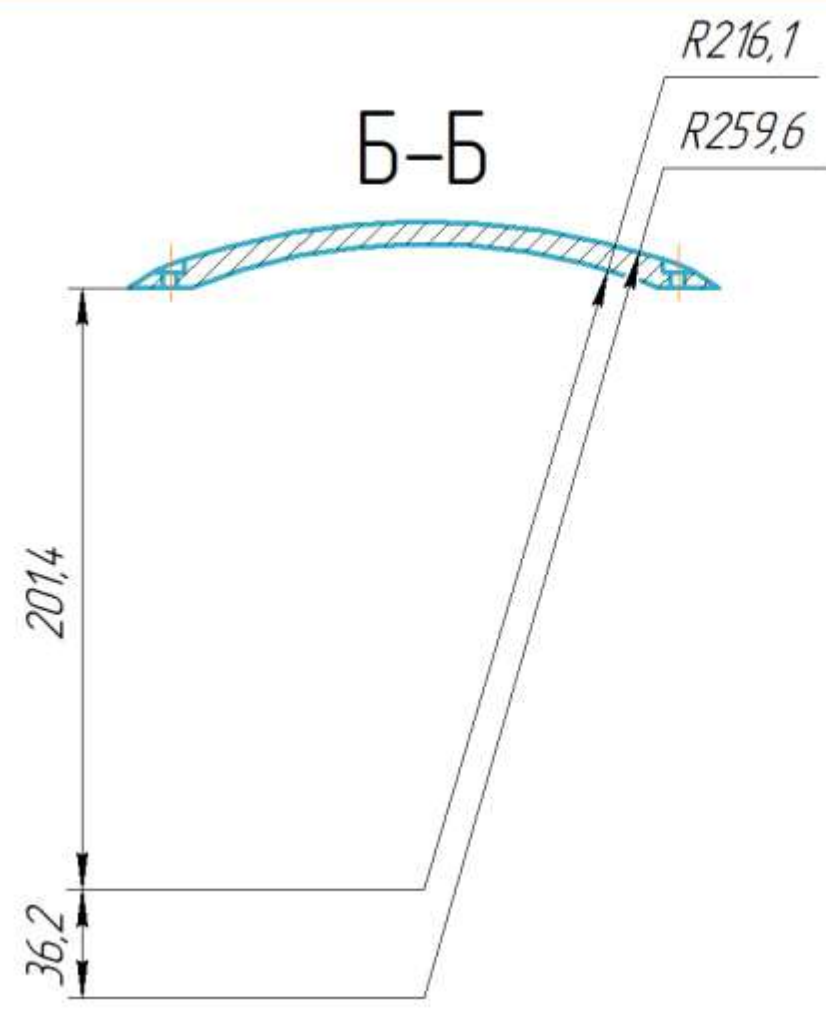
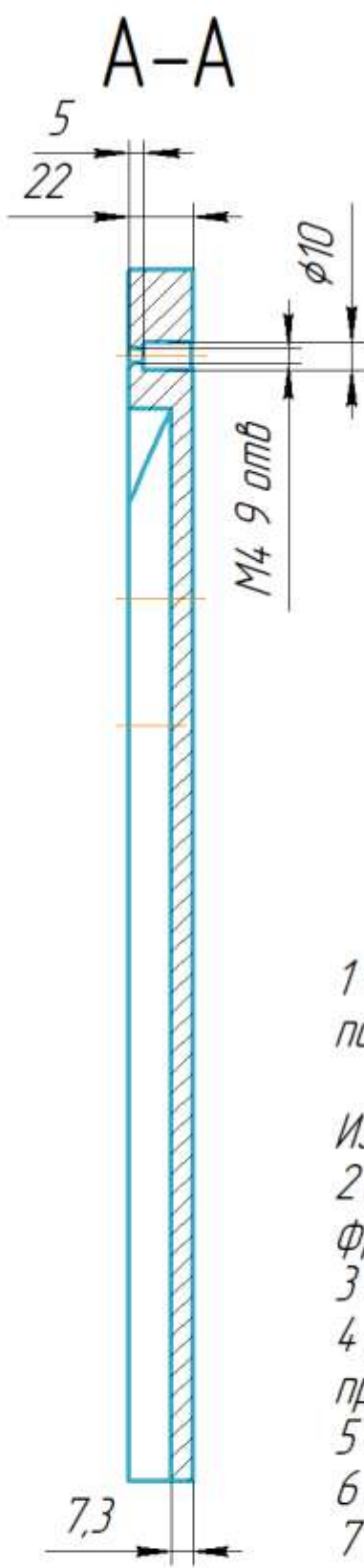
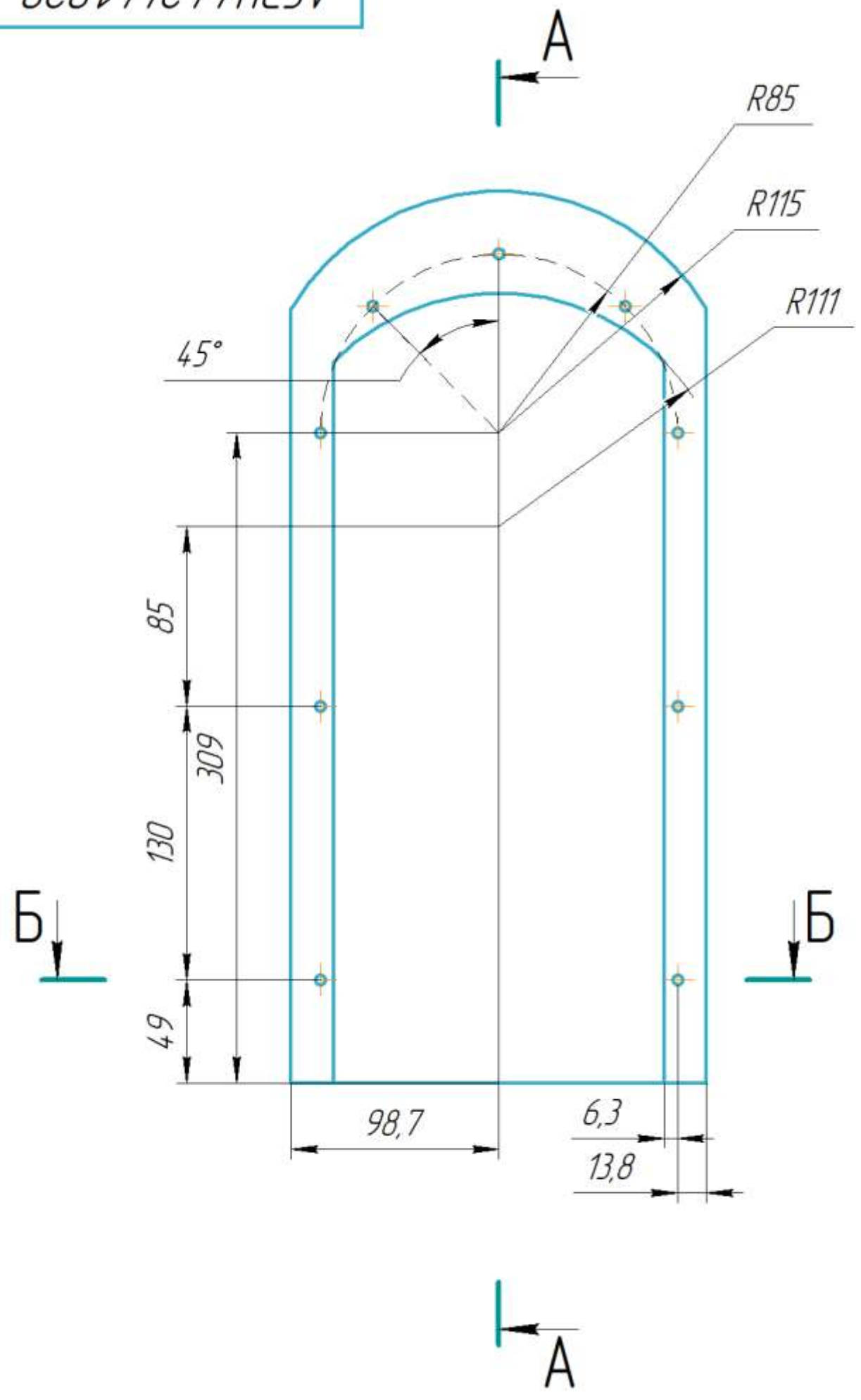
АСЭУ.44244.1019

Лист
2

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Инд. № подл.	Подп. и дата	Взам. инд. №	Инв. № дудл.	Подп. и дата	Справ. №	Перв. примен.

АСЭУ.44244.1020



1 Неуказанные предельные отклонения размеров: Н10, н10, ±IT14/2 по ГОСТ 25346-89.

- Изготовление изделия выполнять по следующей технологии:
- 1 Изготовить матрицу из Пеноплэкс Фундамент методом фрезерования по чертежу.
 - 2 Нанести слой разделительного воска ПВС-1.
 - 3 Выполнить послойное нанесение углеволокна ВМН-4МС с пропиткой смолой ЭД-20 (7:1 с ПЭПА).
 - 4 Провести вакуумирование при 25 мбар в течение 60 мин.
 - 5 Выполнить термообработку при 120 °С в течение 6 часов.
 - 6 Провести обработку и контроль качества изделия.

АСЭУ.44244.1020					Четвертое звено Крышка		
Изм. Лист	№ докум.	Подп.	Дата	Лист	Масса	Масштаб	
Разраб.	Пешков				0.8	1:2,5	
Пров.				Лист	Листов	1	
Т.контр.				СНИУ гр.24 14			
Н.контр.				Композиционный материал на основе углеволокна МН-4МС (ТУ 1916-017-05798606-2015) и эпоксидной смолы ЭД-20 (ГОСТ 10587-84)			
Утв.				Копировал			

Перв. примен.

Спроб. №

Подп. и дата

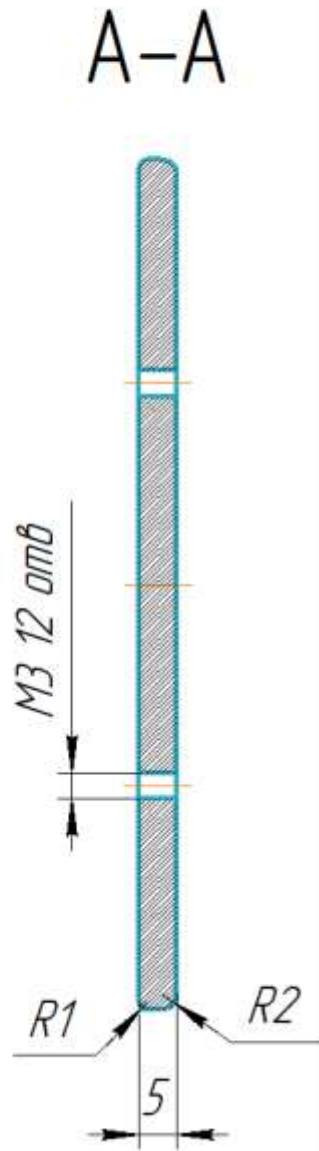
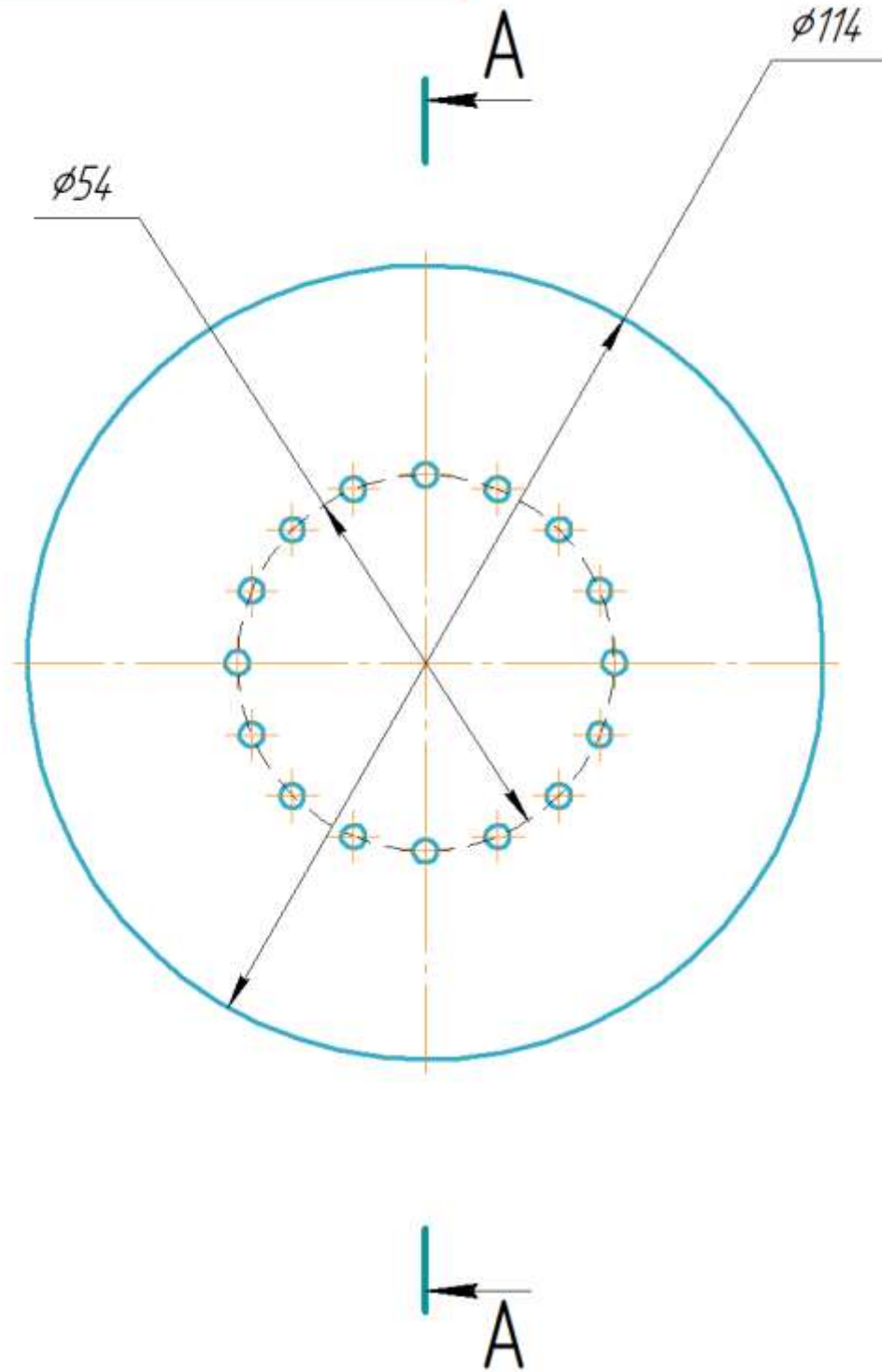
Инв. № дудл.

Взам. инв. №

Подп. и дата

Инв. № подл.

АСЭУ.44.244.1023



АСЭУ.44.244.1023

Изм.	Лист	№ докум.	Подп.	Дата
Разраб.	Пешков			
Проб.				
Т.контр.				
Н.контр.				
Утв.				

Защита ремня

Композиционный материал на основе углеволокна
МН-4МС (ТУ 1916-017-05798606-2015)
и эпоксидной смолы ЭД-20 (ГОСТ 10587-84)

Лит.	Масса	Масштаб
	0.3	1:1
Лист 1	Листов 2	

СНИУ зр.24 14

1 Неуказанные предельные отклонения размеров: H10, h10, ±IT14/2 по ГОСТ 25346-89.

Изготовление изделия выполнять по следующей технологии:

2 Изготовить матрицу из Пеноплэкс Фундамент методом фрезерования по чертежу.

3 Нанести слой разделительного воска ПВС-1.

4 Выполнить послойное нанесение углеволокна ВМН-4МС с пропиткой смолой ЭД-20 (7:1 с ПЭПА).

5 Провести вакуумирование при 25 мбар в течение 60 мин.

6 Выполнить термообработку при 120 °С в течение 6 часов.

7 Провести обработку и контроль качества изделия.

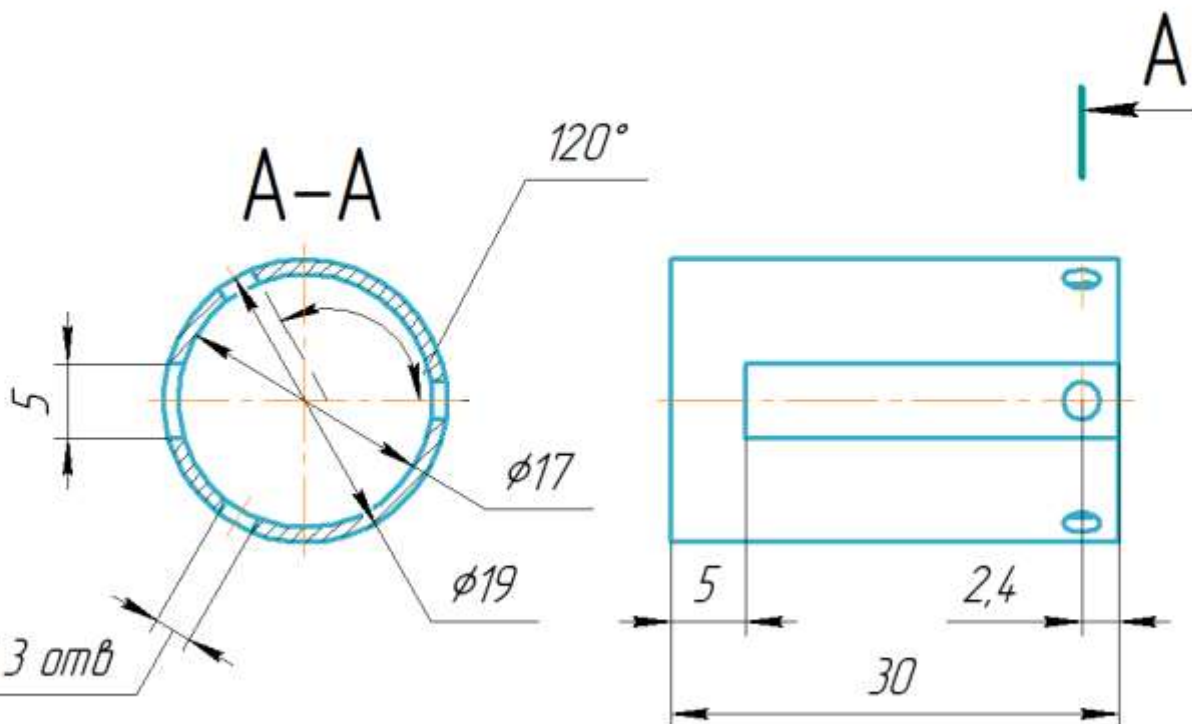
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Изм.	Лист	№ докум.	Подп.	Дата

АСЭУ.44244.1028

Перв. примен.

Справ. №



M2,5 3 отв

1 Неуказанные предельные отклонения размеров: H10, h10, ±IT14/2 по ГОСТ 25346-89.

АСЭУ.44244.1028

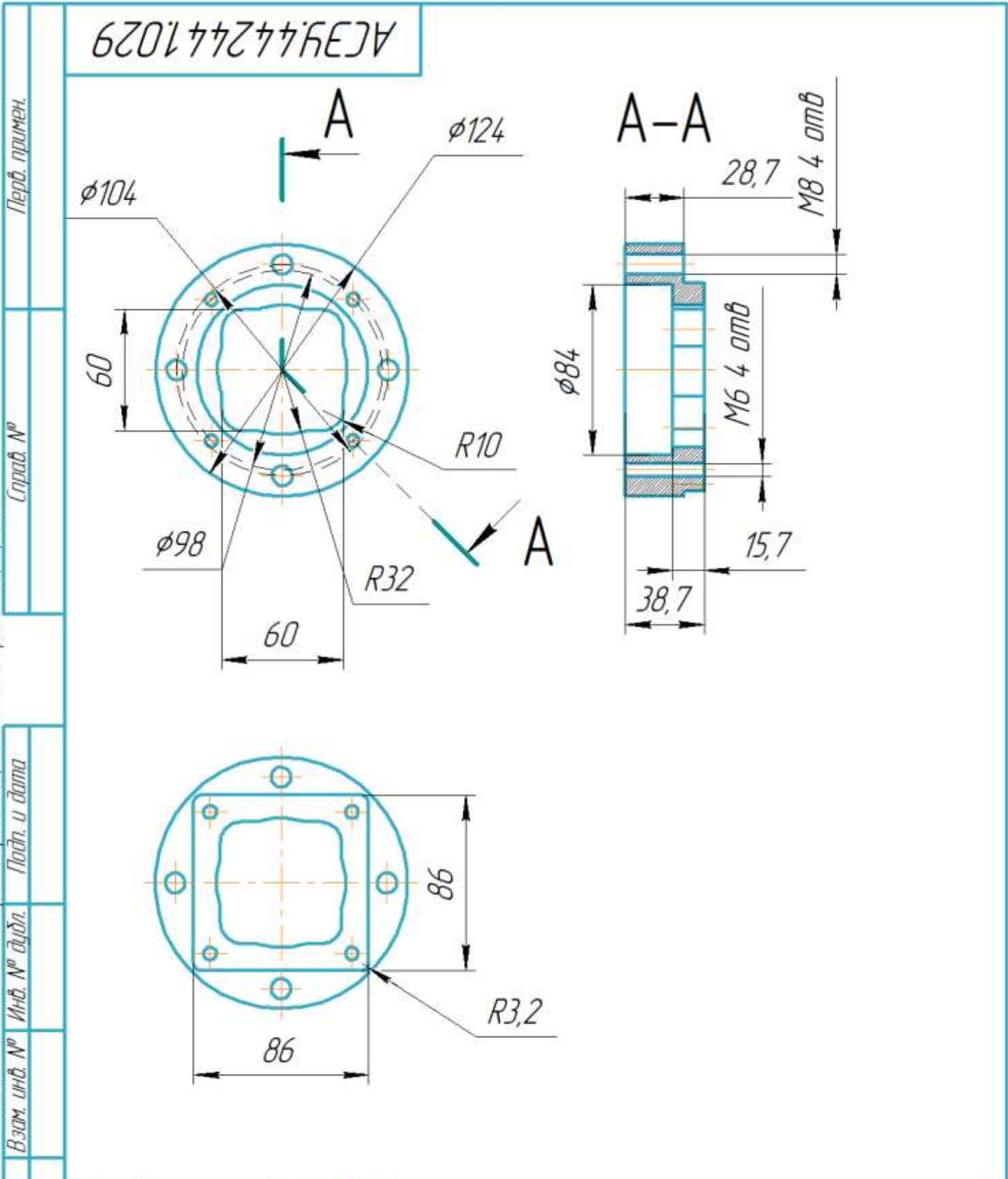
Изм.	Лист	№ докум.	Подп.	Дата
Разраб.	Пешков			
Проб.				
Т.контр.				
Н.контр.				
Утв.				

Переходный фланец
на редуктор HRB17

Сталь 08Х18Н10 ГОСТ 5632-2014

Лист	Масса	Масштаб
	0.1	2:1
Лист	Листов	1

СНИУ гр.24 14



Подп. и дата						АСЭУ.44244.1029		
						Лист	Масса	Масштаб
	Изм. Лист	№ докум.	Подп.	Дата			0.3	1:2.5
Инв. № подл.	Разраб.	Пешков				Проставка		
	Проб.							
	Т.контр.					Лист 1	Листов 2	
						Композиционный материал на основе углеволокна МН-4МС (ТУ 1916-017-05798606-2015) и эпоксидной смолы ЭД-20 (ГОСТ 10587-84)		
	Н.контр.							
Утв.					СНИУ зр.24 14			

1 Неуказанные предельные отклонения размеров: H10, h10, ±IT14/2 по ГОСТ 25346-89.

Изготовление изделия выполнять по следующей технологии:

2 Изготовить матрицу из Пеноплэкс Фундамент методом фрезерования по чертежу.

3 Нанести слой разделительного воска ПВС-1.

4 Выполнить послойное нанесение углеволокна ВМН-4МС с пропиткой смолой ЭД-20 (7:1 с ПЭПА).

5 Провести вакуумирование при 25 мбар в течение 60 мин.

6 Выполнить термообработку при 120 °С в течение 6 часов.

7 Провести обработку и контроль качества изделия.

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Изм.	Лист	№ докум.	Подп.	Дата

АСЭУ.44244.1029

Формат A4

АСЗУ.751818.000

Перв. примен.

Спроб. №

КОМПАС-3D v23 учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Подп. и дата

Инв. № дубл.

Взам. инв. №

Подп. и дата

Инв. № подл.

M3 12 отв

φ54

A

A

A-A

M2 3 отв

2,8

1 Неуказанные предельные отклонения размеров: H10, h10, ±IT14/2 по ГОСТ 25346-89.

АСЗУ.751818.000

Зубчатое колесо
iso 24Z 08A
Доработка

Лист	Масса	Масштаб
		1:1
Лист	Листов	1

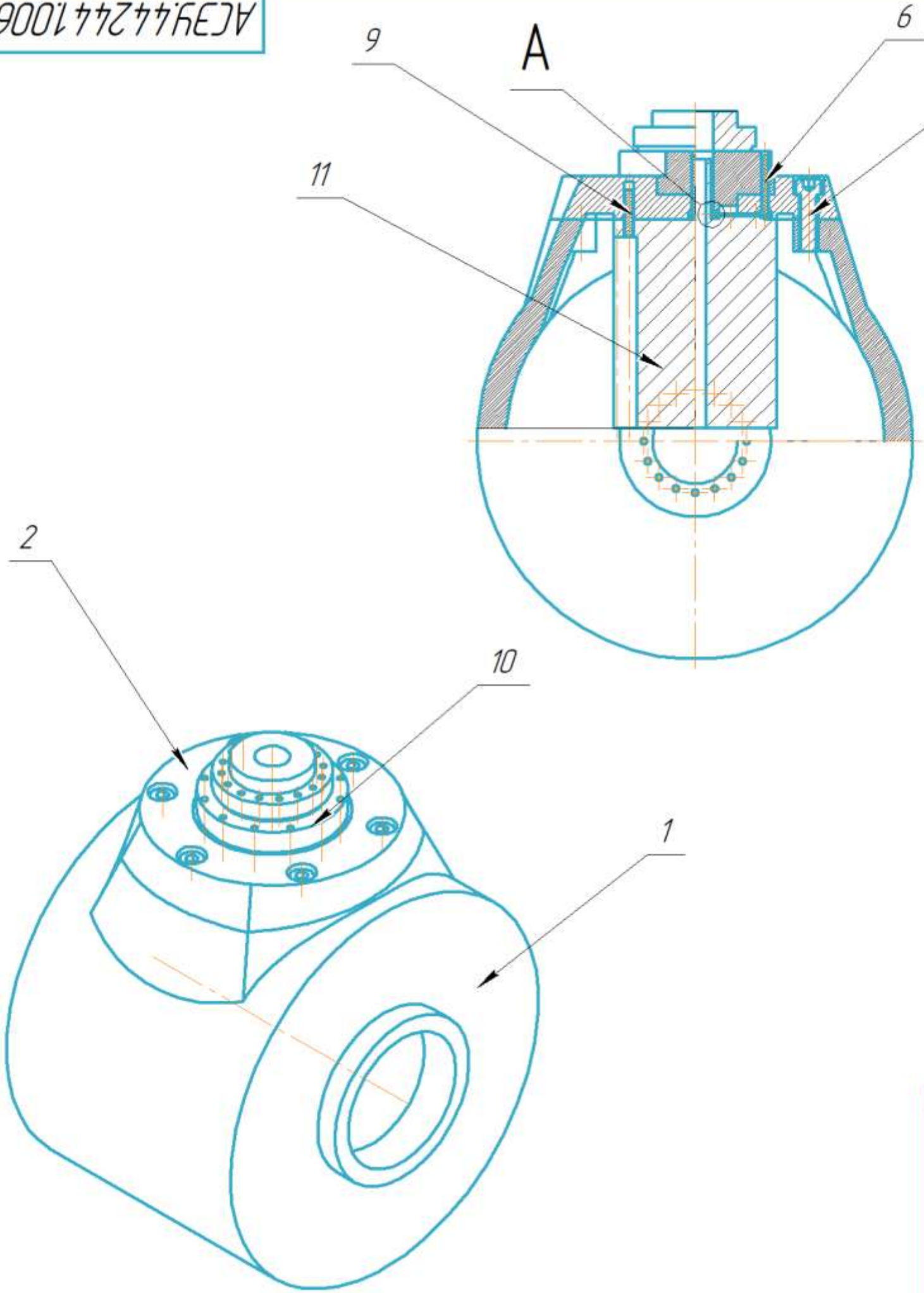
СНИУ гр.24 14

ПРИЛОЖЕНИЕ Ж
Сборочная единица пятого звена

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Перв. примен.	Справ. №	Подп. и дата	Инв. №	Взам. инв. №	Подп. и дата	Инв. №	Подп. и дата

АСЗУ.44244.1006 СБ



- 1 Деталь поз.1 крепится к детали поз.2 болтами М8
- 2 Деталь поз.2 крепится к детали поз.10 болтами М3
- 3 Деталь поз.11 крепится к детали поз.2 болтами М6
- 4 Деталь поз.3 крепится к детали поз.10 болтами М2

АСЗУ.44244.1006 СБ					Лит.			Масса	Масштаб
Пятое звено								12	1:2.5
Сборочная единица					Лист			Листов	1
СНИУ гр.24 14									
Копировал					Формат А3				

Не для коммерческого использования Копировал Формат А4

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Перв. примен.

Справ. №

Подп. и дата

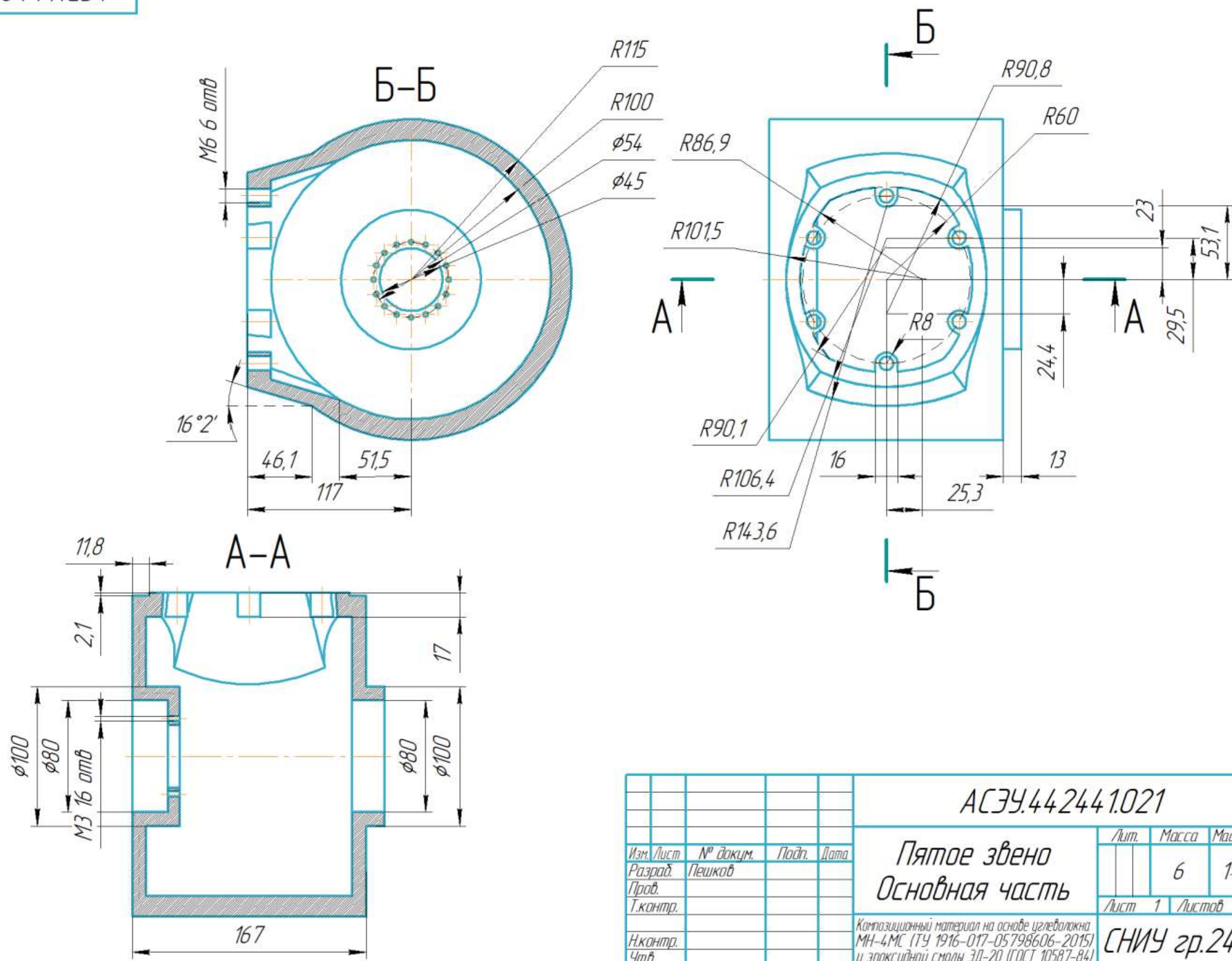
Инв. № дудл.

Взам. инв. №

Подп. и дата

Инв. № подл.

АСЭУ.44244.1021



АСЭУ.44244.1021					
Пятое звено				Лист	Масса
Основная часть				6	1:2,5
Композиционный материал на основе углеволокна				Лист 1	Листов 2
МН-4МС (ТУ 1916-017-05798606-2015)				СНИУ гр.24 14	
и эпоксидной смолы ЭД-20 (ГОСТ 10587-84)				Формат А3	
Копировал					

АСЭУ.44244.1021

1 Неуказанные предельные отклонения размеров: Н10, н10, ±IT14/2 по ГОСТ 25346-89.

- Изготовление изделия выполнять по следующей технологии:
- 2 Изготовить матрицу из Пеноплэкс Фундамент методом фрезерования по чертежу.
 - 3 Нанести слой разделительного воска ПВС-1.
 - 4 Выполнить послойное нанесение углеволокна ВМН-4МС с пропиткой смолой ЭД-20 (7:1 с ПЭПА).
 - 5 Провести вакуумирование при 25 мбар в течение 60 мин.
 - 6 Выполнить термообработку при 120 °С в течение 6 часов.
 - 7 Провести обработку и контроль качества изделия.

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дудл.	Подп. и дата

Изм.	Лист	№ докум.	Подп.	Дата

АСЭУ.44244.1021

Лист
2

КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Перв. примен.

Справ. №

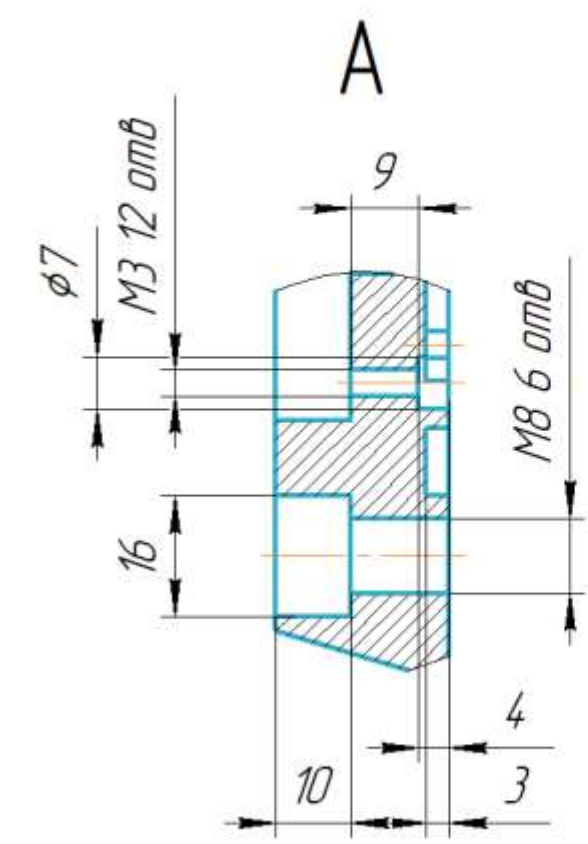
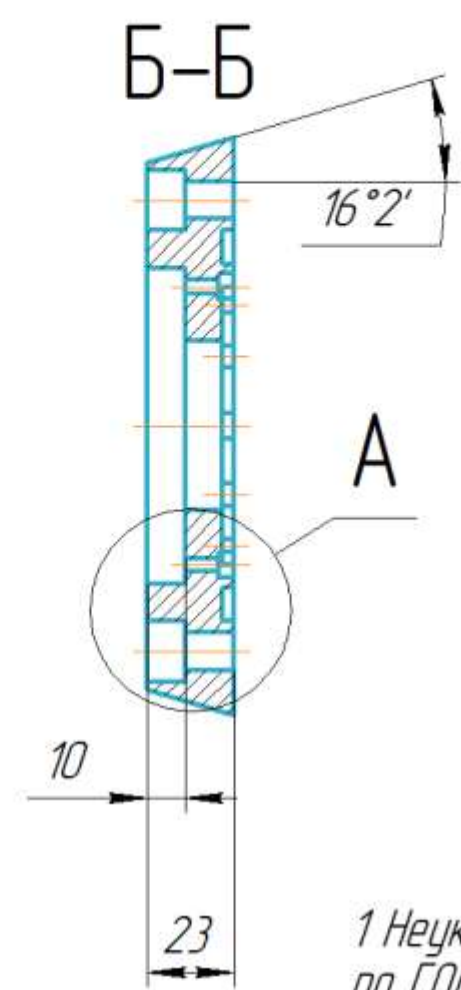
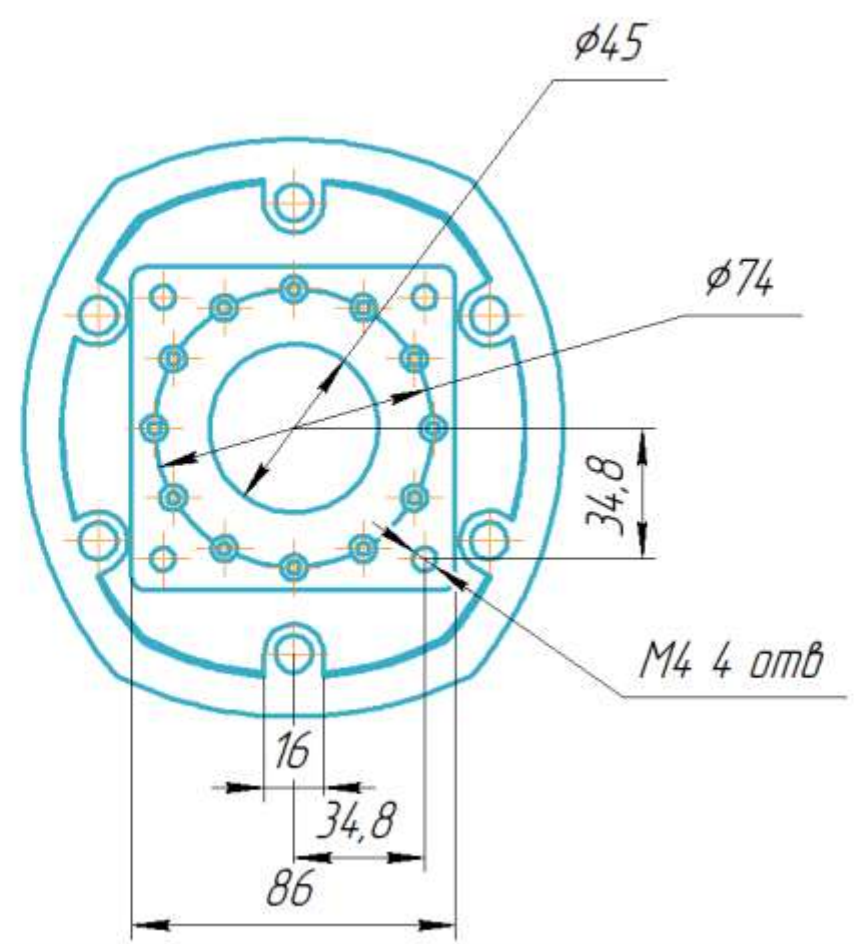
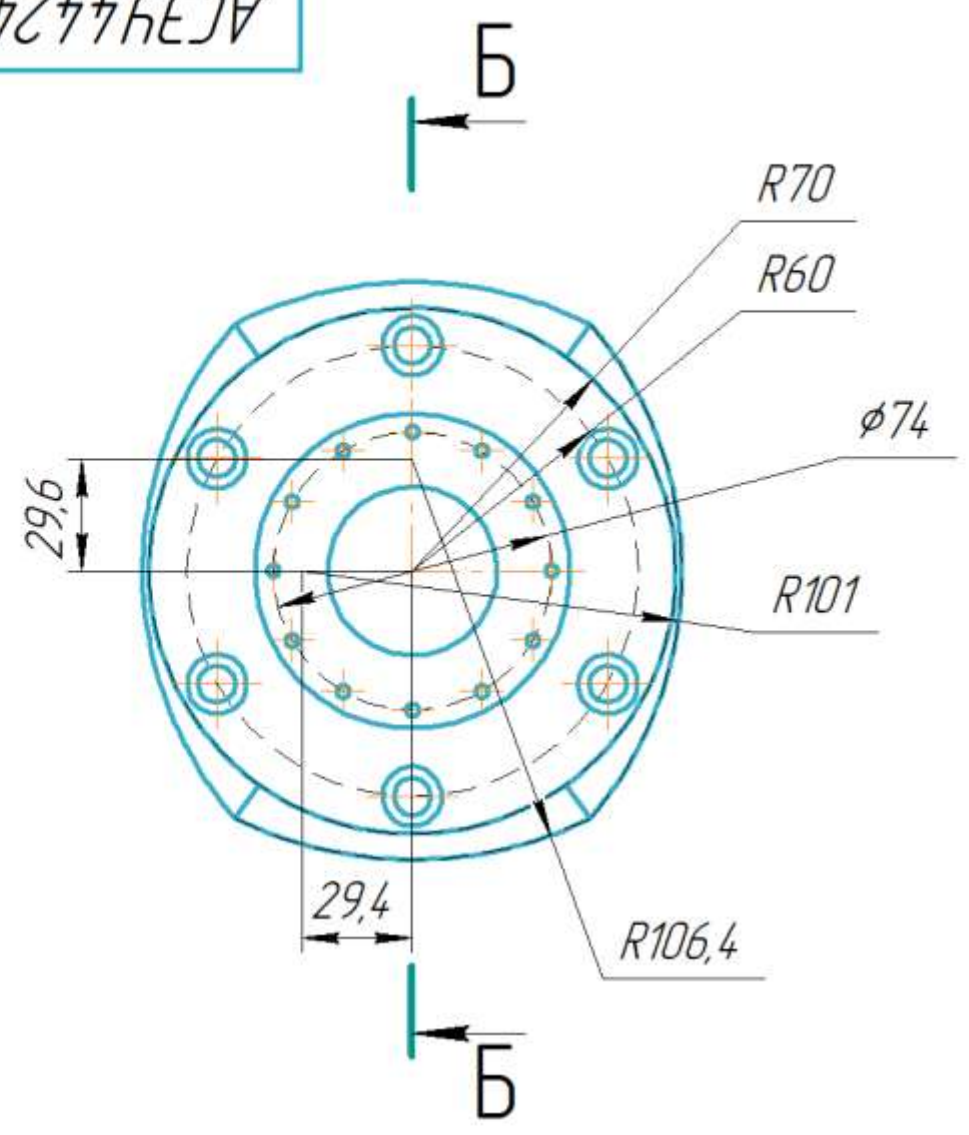
Подп. и дата

Взам. инв. №

Подп. и дата

Инв. № подл.

АСЭУ.44244.1022



1 Неуказанные предельные отклонения размеров: H10, h10, ±IT14/2 по ГОСТ 25346-89.

- Изготовление изделия выполнять по следующей технологии:
- 2 Изготовить матрицу из Пеноплэкс Фундамент методом фрезерования по чертежу.
 - 3 Нанести слой разделительного воска ПВС-1.
 - 4 Выполнить послойное нанесение углеволокна ВМН-4МС с пропиткой смолой ЭД-20 (7:1 с ПЭПА).
 - 5 Провести вакуумирование при 25 мбар в течение 60 мин.
 - 6 Выполнить термообработку при 120 °С в течение 6 часов.
 - 7 Провести обработку и контроль качества изделия.

					АСЭУ.44244.1022			
					Пятое звено Крышка	Лист	Масса	Масштаб
Изм.	Лист	№ докум.	Подп.	Дата			2	1:2
Разраб.	Пешков							
Проб.								
Т.контр.						Лист	Листов	1
Н.контр.					Композиционный материал на основе углеволокна МН-4МС (ТУ 1916-017-05798606-2015) и эпоксидной смолы ЭД-20 (ГОСТ 10587-84)			СНИУ гр.24 14
Утв.								

АСЭУ.44244.1028

Перв. примен.

Справ. №

КОМПАС-3D v23 учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

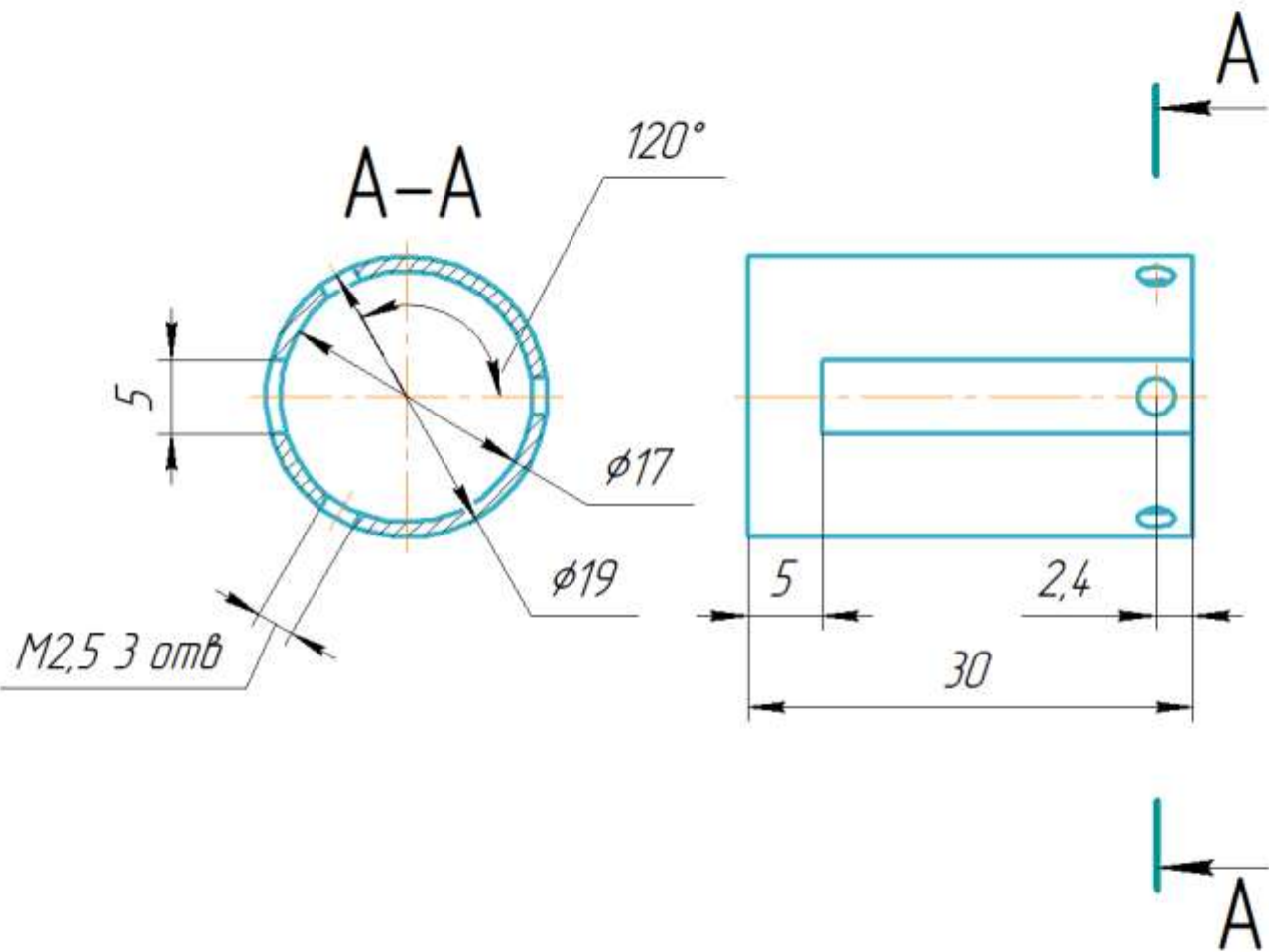
Подп. и дата

Инв. № дудл.

Взам. инв. №

Подп. и дата

Инв. № подл.



1 Неуказанные предельные отклонения размеров: H10, h10, $\pm IT14/2$ по ГОСТ 25346-89.

АСЭУ.44244.1028

Изм.	Лист	№ докум.	Подп.	Дата
Разраб.	Пешков			
Пров.				
Т.контр.				
Н.контр.				
Утв.				

Переходный фланец
на редуктор HRB17

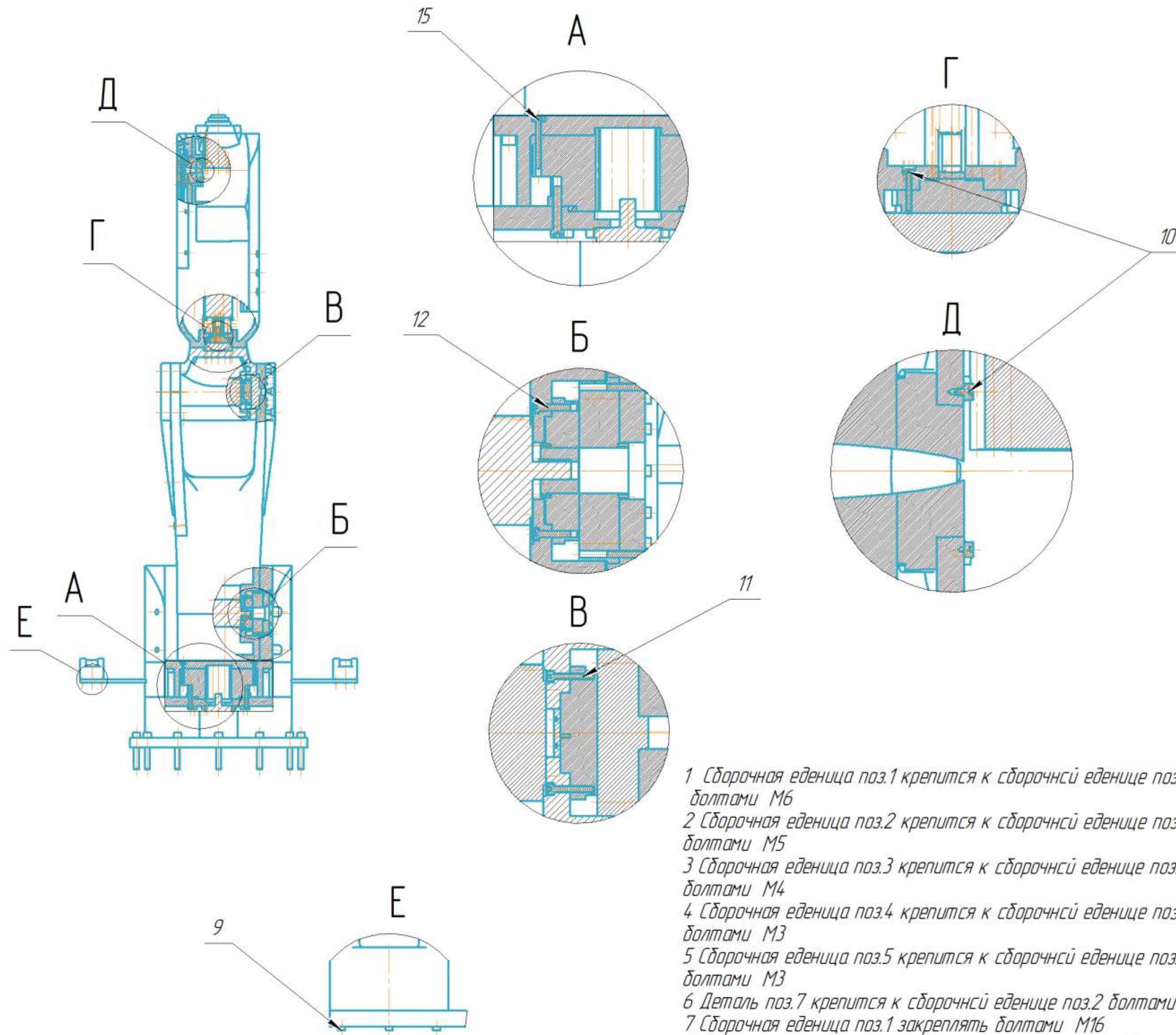
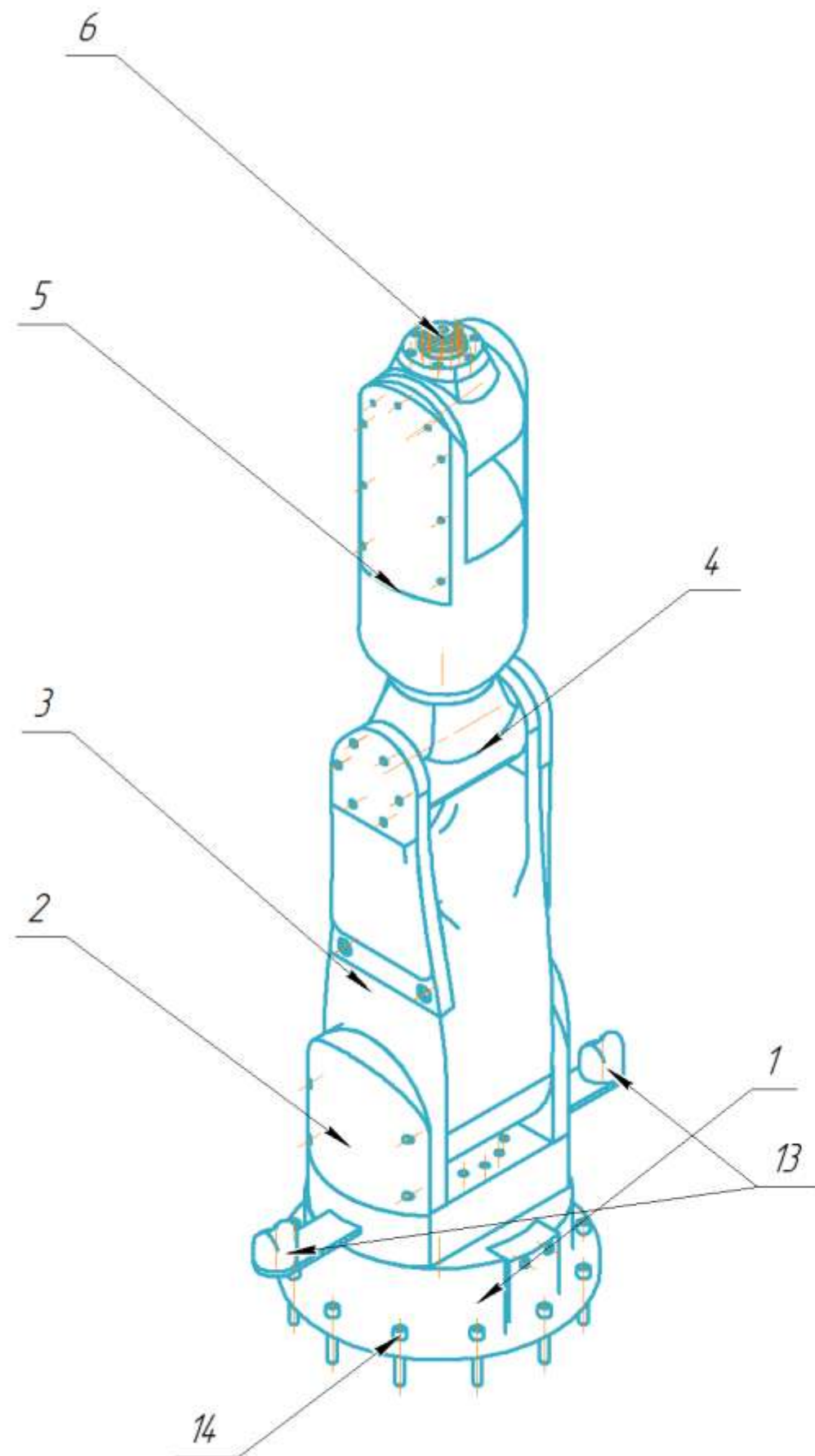
Сталь 08Х18Н10 ГОСТ 5632-2014

Лист	Масса	Масштаб
1	0.1	2:1
Лист	Листов	1

СНИУ гр.24 14

ПРИЛОЖЕНИЕ И
Сборочная единица 6-осевого манипулятора

КМПАС-3D v23 Учебная версия © 2024. ООО "АСКОН-Системы проектирования", Россия. Все права защищены.
Инд. № подл. Подп. и дата
Изд. № инд. Подп. и дата
Справ. № Подп. и дата
Перв. примен.



- 1 Сборочная единица поз.1 крепится к сборочной единице поз.2 болтами М6
- 2 Сборочная единица поз.2 крепится к сборочной единице поз.3 болтами М5
- 3 Сборочная единица поз.3 крепится к сборочной единице поз.4 болтами М4
- 4 Сборочная единица поз.4 крепится к сборочной единице поз.5 болтами М3
- 5 Сборочная единица поз.5 крепится к сборочной единице поз.6 болтами М3
- 6 Деталь поз.7 крепится к сборочной единице поз.2 болтами М2.5
- 7 Сборочная единица поз.1 закрепляется болтами М16
- 8 Дополнительные элементы крепятся к сборочной единице поз.6 болтами М4

				АСЗУ.44.244.1.000 СБ				
				6-осевой манипулятор с адаптивной системой управления Сборочный чертеж	Лит.	Масса	Масштаб	
Изм.	Лист	№ докум.	Подп.		Дата			
Разраб.	Пешков						320	1:10
Пров.								
Т.контр.						Лист	Листов	1
Н.контр.					СНИУ гр.24 14			
Чтб.								

Формат		Зона	Поз.	Обозначение	Наименование	Кол.	Примечание
					Документация		
				АСЭУ.442441.000 СБ	6-осевой манипулятор с адаптивной системой управления		
				АСЭУ.468333.000 ПЗЗ	Перечень элементов		
				АСЭУ.468333.000 ЭЗ	Схема электрическая принципиальная		
				АСЭУ.422410.000 СБ	Шкаф управления		
					Сборочные единицы		
А3	1	АСЭУ.442441.001 СБ		Основание манипулятора	1		
А3	2	АСЭУ.442441.002 СБ		Первое звено	1		
А3	3	АСЭУ.442441.003 СБ		Второе звено	1		
А3	4	АСЭУ.442441.004 СБ		Третье звено	1		
А2	5	АСЭУ.442441.005 СБ		Четвертое звено	1		
А3	6	АСЭУ.442441.006 СБ		Пятое звено	1		
					Стандартные изделия		
				9	Hexagon socket head cap screw ISO 4762 -M2,5 x 16	8	
				10	Hexagon socket head cap screw	24	
				АСЭУ.422410.000			
Изм.	Лист	№ докум.		Подп.	Дата		
Разраб.	Лист	Лист					
Проб.	Лист	Лист					
Н.контр.	Лист	Лист					
Утв.	Лист	Лист					
6-осевой манипулятор с адаптивной системой управления						Лит.	Лист
						1	2
						СНИУ гр.24 14	

Не для коммерческого использования

Формат А4

ПРИЛОЖЕНИЕ К

Файл описания модели манипулятора

```

<sdf version='1.11'>
  <model name='Robot_ai'>
    <link name='base_link'>
      <inertial>
        <pose>0 0 0 0 1.5707926536662187</pose>
        <mass>1000000</mass>
        <inertia>
          <ixx>1.0051518366849099</ixx>
          <ixy>-1.9314511799309001e-09</ixy>
          <ixz>6.9200494914216999e-12</ixz>
          <iyy>1.00515183641807</iyy>
          <iyz>2.9046668445456899e-10</iyz>
          <izz>1.9574702350761899</izz>
        </inertia>
      </inertial>
      <collision name='Base_Robot_link_collision'>
        <pose>0.32744256676702599 0.32241999999999998 0.08 0 0 0</pose>
        <geometry>
          <mesh>
            <scale>1 1 1</scale>
            <uri>file://$(find robot_rviz)/meshes/Base_Robot_link.STL</uri>
          </mesh>
        </geometry>
      </collision>
      <visual name='Base_Robot_link_visual'>
        <pose>0.32744256676702599 0.32241999999999998 0.08 0 0 0</pose>
        <geometry>
          <mesh>
            <scale>1 1 1</scale>
            <uri>file://$(find robot_rviz)/meshes/Base_Robot_link.STL</uri>
          </mesh>
        </geometry>
        <material>
          <diffuse>0.372549027 0.372549027 0.372549027 1</diffuse>
          <ambient>0.372549027 0.372549027 0.372549027 1</ambient>
        </material>
      </visual>
    </link>
    <!-- ***** LIDAR_1 ***** -->
    ->
    <link name="lidar_link_1">
      <pose>0 0.4 0.1 0 0 0</pose>
      <inertial>
        <inertia>
          <ixx>0.001</ixx>
          <ixy>0.000</ixy>
          <ixz>0.000</ixz>

```

```

    <iyy>0.001</iyy>
    <iyz>0.000</iyz>
    <izz>0.001</izz>
  </inertia>
  <mass>0.114</mass>
</inertial>

<sensor name='gpu_lidar' type='gpu_lidar'>
  <topic>lidar_1</topic>
  <update_rate>1000</update_rate>
  <ray>
    <scan>
      <horizontal>
        <samples>640</samples>
        <resolution>1</resolution>
        <min_angle>-3.14</min_angle>
        <max_angle>3.14</max_angle>
      </horizontal>
      <vertical>
        <samples>90</samples>
        <resolution>1</resolution>
        <min_angle>0</min_angle>
        <max_angle>1.57</max_angle>
      </vertical>
    </scan>
    <range>
      <min>0.08</min>
      <max>5.0</max>
      <resolution>0.01</resolution>
    </range>
  </ray>
  <plugin
    filename="gz-sim-sensors-system"
    name="gz::sim::systems::Sensors">
    <render_engine>ogre2</render_engine>
  </plugin>
  <visualize>true</visualize>
</sensor>
</link>

<joint name="lidar_joint_1" type="fixed">
  <parent>Povorotniy_Stol_Link</parent>
  <child>lidar_link_1</child>
  <pose>0 0 0 0 0 0</pose>
</joint>

<!-- ***** -->
<!-- ***** LIDAR_2 ***** -->
<link name="lidar_link_2">
  <pose>0 -0.4 0.1 0 0 0</pose>
  <inertial>
    <inertia>

```



```

    <ixx>0.001</ixx>
    <ixy>0.000</ixy>
    <ixz>0.000</ixz>
    <iyy>0.001</iyy>
    <iyz>0.000</iyz>
    <izz>0.001</izz>
  </inertia>
  <mass>0.114</mass>
</inertial>

<sensor name='gpu_lidar' type='gpu_lidar'>
  <topic>lidar_2</topic>
  <update_rate>1000</update_rate>
  <ray>
    <scan>
      <horizontal>
        <samples>640</samples>
        <resolution>1</resolution>
        <min_angle>-3.14</min_angle>
        <max_angle>3.14</max_angle>
      </horizontal>
      <vertical>
        <samples>90</samples>
        <resolution>1</resolution>
        <min_angle>0</min_angle>
        <max_angle>1.57</max_angle>
      </vertical>
    </scan>
    <range>
      <min>0.08</min>
      <max>5.0</max>
      <resolution>0.01</resolution>
    </range>
  </ray>
  <visualize>true</visualize>
</sensor>
</link>

<joint name="lidar_joint_2" type="fixed">
  <parent>Povorotniy_Stol_Link</parent>
  <child>lidar_link_2</child>
  <pose>0 0 0 0 0 0</pose>
</joint>

<!-- ***** -->

<joint name='J_1_Base_Stol' type='revolute'>
  <pose relative_to='base_link'>0 0 0.17505 0 0 0</pose>
  <parent>base_link</parent>
  <child>Povorotniy_Stol_Link</child>
  <axis>
    <xyz>0 0 1</xyz>

```

```

    <limit>
      <lower>-3.15</lower>
      <upper>3.15</upper>
      <effort>100000</effort>
    </limit>
  </axis>
</joint>
<link name='Povorotniy_Stol_Link'>
  <pose relative_to='J_1_Base_Stol'>0 0 0 1.5707926536662187 0
1.5707926536662187</pose>
  <inertial>
    <pose>-0.017372112782607401 0.13462645768171699 -1.9290275617422901e-06 0 0
0</pose>
    <mass>0.1</mass>
    <inertia>
      <ixx>0.95727352660759302</ixx>
      <ixy>-0.00059376031008356303</ixy>
      <ixz>2.2567449258745202e-06</ixz>
      <iyy>1.1107061866796299</iyy>
      <iyz>8.3581519863609197e-07</iyz>
      <izz>0.86041639916040102</izz>
    </inertia>
  </inertial>
  <collision name='Povorotniy_Stol_Link_collision'>
    <pose>0 0 0 0 0 0</pose>
    <geometry>
      <mesh>
        <scale>1 1 1</scale>
        <uri>file://$(find robot_rviz)/meshes/Povorotniy_Stol_Link.STL</uri>
      </mesh>
    </geometry>
  </collision>
  <visual name='Povorotniy_Stol_Link_visual'>
    <pose>0 0 0 0 0 0</pose>
    <geometry>
      <mesh>
        <scale>1 1 1</scale>
        <uri>file://$(find robot_rviz)/meshes/Povorotniy_Stol_Link.STL</uri>
      </mesh>
    </geometry>
    <material>
      <diffuse>0.990196109 1 1 1</diffuse>
      <ambient>0.990196109 1 1 1</ambient>
    </material>
  </visual>
  <sensor name='my_contact' type='contact'>
    <contact>
      <collision>Povorotniy_Stol_Link_collision</collision>
    </contact>
  </sensor>
</link>
<joint name='J_2_Stol_Plecho_1' type='revolute'>

```

```

    <pose relative_to='Povorotniy_Stol_Link'>0 0.218 0 -1.5707926536057681
1.5707926536662187 3.1415926535746808</pose>
    <parent>Povorotniy_Stol_Link</parent>
    <child>Plecho_1_Link</child>
    <axis>
      <xyz>0 1 0</xyz>
      <limit>
        <lower>-1.58</lower>
        <upper>1.58</upper>
        <effort>100000</effort>
      </limit>
    </axis>
  </joint>
  <link name='Plecho_1_Link'>
    <pose relative_to='J_2_Stol_Plecho_1'>0 0 0 0 0 0</pose>
    <inertial>
      <pose>-3.21432714611714e-07 -0.0010789309414756 -0.24887111912731899 0 0
0</pose>
      <mass>0.1</mass>
      <inertia>
        <ixx>2.04471987918707</ixx>
        <ixy>2.44459223105891e-06</ixy>
        <ixz>8.5349528778858098e-06</ixz>
        <iyy>2.09174637289405</iyy>
        <iyz>-0.232809642467245</iyz>
        <izz>0.67967639590933104</izz>
      </inertia>
    </inertial>
    <collision name='Plecho_1_Link_collision'>
      <pose>0 0 0 0 0 0</pose>
      <geometry>
        <mesh>
          <scale>1 1 1</scale>
          <uri>file://$(find robot_rviz)/meshes/Plecho_1_Link.STL</uri>
        </mesh>
      </geometry>
    </collision>
    <visual name='Plecho_1_Link_visual'>
      <pose>0 0 0 0 0 0</pose>
      <geometry>
        <mesh>
          <scale>1 1 1</scale>
          <uri>file://$(find robot_rviz)/meshes/Plecho_1_Link.STL</uri>
        </mesh>
      </geometry>
      <material>
        <diffuse>0.372549027 0.372549027 0.372549027 1</diffuse>
        <ambient>0.372549027 0.372549027 0.372549027 1</ambient>
      </material>
    </visual>
    <sensor name='my_contact' type='contact'>
      <contact>

```

```

    <collision>Plecho_1_Link_collision</collision>
  </contact>
</sensor>
</link>
<joint name='J_3_Plecho_1_and_2' type='revolute'>
  <pose relative_to='Plecho_1_Link'>0 0 -0.6999999999999996 7.3464102064341006e-06
1.5707926536057681 3.1415926535897931</pose>
  <parent>Plecho_1_Link</parent>
  <child>Plecho_2_Link</child>
  <axis>
    <xyz>0 1 0</xyz>
    <limit>
      <lower>-2.4</lower>
      <upper>2.4</upper>
      <effort>100000</effort>
    </limit>
  </axis>
</joint>
<link name='Plecho_2_Link'>
  <pose relative_to='J_3_Plecho_1_and_2'>0 0 0 0 0</pose>
  <inertial>
    <pose>0.036300648833915899 0.00085212005862833596 9.7723589664866495e-08 0 0
0</pose>
    <mass>0.1</mass>
    <inertia>
      <ixx>0.0544520712435956</ixx>
      <ixy>0.000123072795450505</ixy>
      <ixz>-9.2736408211455702e-08</ixz>
      <iyy>0.049137662162096203</iyy>
      <iyz>1.2815118988579001e-06</iyz>
      <izz>0.066163106229742405</izz>
    </inertia>
  </inertial>
  <collision name='Plecho_2_Link_collision'>
    <pose>0 0 0 0 0</pose>
    <geometry>
      <mesh>
        <scale>1 1 1</scale>
        <uri>file://$(find robot_rviz)/meshes/Plecho_2_Link.STL</uri>
      </mesh>
    </geometry>
  </collision>
  <visual name='Plecho_2_Link_visual'>
    <pose>0 0 0 0 0</pose>
    <geometry>
      <mesh>
        <scale>1 1 1</scale>
        <uri>file://$(find robot_rviz)/meshes/Plecho_2_Link.STL</uri>
      </mesh>
    </geometry>
    <material>
      <diffuse>0.372549027 0.372549027 0.372549027 1</diffuse>

```



```

    <ambient>0.372549027 0.372549027 0.372549027 1</ambient>
  </material>
</visual>
<sensor name='my_contact' type='contact'>
  <contact>
    <collision>Plecho_2_Link_collision</collision>
  </contact>
</sensor>
</link>
<joint name='J_4_Plecho_2_and_3' type='revolute'>
  <pose relative_to='Plecho_2_Link'>0.14000000000000001 0 0 3.1415853071795872 -
4.2351647362715017e-22 -1.5707926536057681</pose>
  <parent>Plecho_2_Link</parent>
  <child>Plecho_3_Link</child>
  <axis>
    <xyz>0 1 0</xyz>
    <limit>
      <lower>-3.15</lower>
      <upper>3.15</upper>
      <effort>100000</effort>
    </limit>
  </axis>
</joint>
<link name='Plecho_3_Link'>
  <pose relative_to='J_4_Plecho_2_and_3'>0 0 0 0 0</pose>
  <inertial>
    <pose>-0.00192346932769394 -0.30284801847342202 0.000208037577046183 0 0
0</pose>
    <mass>0.1</mass>
    <inertia>
      <ixx>0.62318459258838299</ixx>
      <ixy>-0.050656819251739302</ixy>
      <ixz>-1.37120879057179e-06</ixz>
      <iyy>0.163273915359157</iyy>
      <iyz>-6.8252137827531502e-05</iyz>
      <izz>0.57836980186750697</izz>
    </inertia>
  </inertial>
  <collision name='Plecho_3_Link_collision'>
    <pose>0 0 0 0 0</pose>
    <geometry>
      <mesh>
        <scale>1 1 1</scale>
        <uri>file://$(find robot_rviz)/meshes/Plecho_3_Link.STL</uri>
      </mesh>
    </geometry>
  </collision>
  <visual name='Plecho_3_Link_visual'>
    <pose>0 0 0 0 0</pose>
    <geometry>
      <mesh>
        <scale>1 1 1</scale>

```

```

    <uri>file://$(find robot_rviz)/meshes/Plecho_3_Link.STL</uri>
  </mesh>
</geometry>
<material>
  <diffuse>0.372549027 0.372549027 0.372549027 1</diffuse>
  <ambient>0.372549027 0.372549027 0.372549027 1</ambient>
</material>
</visual>
<sensor name='my_contact' type='contact'>
  <contact>
    <collision>Plecho_3_Link_collision</collision>
  </contact>
</sensor>
</link>
<joint name='J_5_Plecho_3_and_4' type='revolute'>
  <pose relative_to='Plecho_3_Link'>0 -0.56000000000000005 0 -1.5707926536057681
1.5707926536662187 0</pose>
  <parent>Plecho_3_Link</parent>
  <child>Plecho_4_Link</child>
  <axis>
    <xyz>0 1 0</xyz>
    <limit>
      <lower>-2.1</lower>
      <upper>2.1</upper>
      <effort>100000</effort>
    </limit>
  </axis>
</joint>
<link name='Plecho_4_Link'>
  <pose relative_to='J_5_Plecho_3_and_4'>0 0 0 0 0 0</pose>
  <inertial>
    <pose>2.38756184822719e-07 0.00097455957917691504 -0.0075070461604227496 0 0
0</pose>
    <mass>0.1</mass>
    <inertia>
      <ixx>0.13945876602702101</ixx>
      <ixy>7.9926499638887303e-07</ixy>
      <ixz>-4.7306132561770001e-07</ixz>
      <iyy>0.167961345710681</iyy>
      <iyz>0.00013834773941723101</iyz>
      <izz>0.15490975912657801</izz>
    </inertia>
  </inertial>
  <collision name='Plecho_4_Link_collision'>
    <pose>0 0 0 0 0 0</pose>
    <geometry>
      <mesh>
        <scale>1 1 1</scale>
        <uri>file://$(find robot_rviz)/meshes/Plecho_4_Link.STL</uri>
      </mesh>
    </geometry>
  </collision>

```

```

<visual name='Plecho_4_Link_visual'>
  <pose>0 0 0 0 0 0</pose>
  <geometry>
    <mesh>
      <scale>1 1 1</scale>
      <uri>file://$(find robot_rviz)/meshes/Plecho_4_Link.STL</uri>
    </mesh>
  </geometry>
  <material>
    <diffuse>0.990196109 1 1 1</diffuse>
    <ambient>0.990196109 1 1 1</ambient>
  </material>
</visual>
<sensor name='my_contact' type='contact'>
  <contact>
    <collision>Plecho_4_Link_collision</collision>
  </contact>
</sensor>
</link>
<joint name='J_6_Plecho_4_Kist' type='revolute'>
  <pose relative_to='Plecho_4_Link'>0 0 -0.17399999999999999 0 3.1415853071795872
3.1415853071795872</pose>
  <parent>Plecho_4_Link</parent>
  <child>Kist_Link</child>
  <axis>
    <xyz>0 0 1</xyz>
    <limit>
      <lower>-3.15</lower>
      <upper>3.15</upper>
      <effort>100000</effort>
    </limit>
  </axis>
</joint>
<link name='Kist_Link'>
  <pose relative_to='J_6_Plecho_4_Kist'>0 0 0 -1.57 0 0</pose>
  <inertial>
    <pose>4.77395900588817e-15 0.027968264206145 -1.6986412276764901e-14 0 0
0</pose>
    <mass>0.1</mass>
    <inertia>
      <ixx>0.00065860025104538204</ixx>
      <ixy>1.01132086694409e-18</ixy>
      <ixz>3.2526065174565099e-19</ixz>
      <iyy>0.000944359788990718</iyy>
      <iyz>-1.30235326814037e-18</iyz>
      <izz>0.00065860025104538204</izz>
    </inertia>
  </inertial>
  <collision name='Kist_Link_collision'>
    <pose>0 0 0 0 0 0</pose>
    <geometry>
      <mesh>

```

```

    <scale>1 1 1</scale>
    <uri>file://$(find robot_rviz)/meshes/Kist_Link.STL</uri>
  </mesh>
</geometry>
</collision>
<visual name='Kist_Link_visual'>
  <pose>0 0 0 0 0 0</pose>
  <geometry>
    <mesh>
      <scale>1 1 1</scale>
      <uri>file://$(find robot_rviz)/meshes/Kist_Link.STL</uri>
    </mesh>
  </geometry>
  <material>
    <diffuse>0.990196109 1 1 1</diffuse>
    <ambient>0.990196109 1 1 1</ambient>
  </material>
</visual>
<sensor name='my_contact' type='contact'>
  <contact>
    <collision>Kist_Link_collision</collision>
  </contact>
</sensor>
</link>
<plugin
  filename="gz-sim-joint-state-publisher-system"
  name="gz::sim::systems::JointStatePublisher">
  <joint_name>J_1_Base_Stol</joint_name>
</plugin>
<plugin
  filename="gz-sim-joint-state-publisher-system"
  name="gz::sim::systems::JointStatePublisher">
  <joint_name>J_2_Stol_Plecho_1</joint_name>
</plugin>
<plugin
  filename="gz-sim-joint-state-publisher-system"
  name="gz::sim::systems::JointStatePublisher">
  <joint_name>J_3_Plecho_1_and_2</joint_name>
</plugin>
<plugin
  filename="gz-sim-joint-state-publisher-system"
  name="gz::sim::systems::JointStatePublisher">
  <joint_name>J_4_Plecho_2_and_3</joint_name>
</plugin>
<plugin
  filename="gz-sim-joint-state-publisher-system"
  name="gz::sim::systems::JointStatePublisher">
  <joint_name>J_5_Plecho_3_and_4</joint_name>
</plugin>
<plugin
  filename="gz-sim-joint-state-publisher-system"
  name="gz::sim::systems::JointStatePublisher">

```

```

    <joint_name>J_6_Plecho_4_Kist</joint_name>
</plugin>
<plugin
  filename="gz-sim-joint-state-publisher-system"
  name="gz::sim::systems::JointStatePublisher">
  <ros>
    <namespace>/robot</namespace>
  </ros>
  <joint_name>lidar_joint_1</joint_name>
  <update_rate>30</update_rate>
</plugin>
<plugin
  filename="gz-sim-joint-state-publisher-system"
  name="gz::sim::systems::JointStatePublisher">
  <ros>
    <namespace>/robot</namespace>
  </ros>
  <joint_name>lidar_joint_2</joint_name>
  <update_rate>30</update_rate>
</plugin>
<plugin
  filename="gz-sim-joint-position-controller-system"
  name="gz::sim::systems::JointPositionController">
  <joint_name>J_1_Base_Stol</joint_name>
  <topic>J_1</topic>
  <use_velocity_commands>1</use_velocity_commands>
  <cmd_max>2</cmd_max>
  <cmd_min>-2</cmd_min>
</plugin>
<plugin
  filename="gz-sim-joint-position-controller-system"
  name="gz::sim::systems::JointPositionController">
  <joint_name>J_2_Stol_Plecho_1</joint_name>
  <topic>J_2</topic>
  <use_velocity_commands>1</use_velocity_commands>
  <cmd_max>2</cmd_max>
  <cmd_min>-2</cmd_min>
</plugin>
<plugin
  filename="gz-sim-joint-position-controller-system"
  name="gz::sim::systems::JointPositionController">
  <joint_name>J_3_Plecho_1_and_2</joint_name>
  <topic>J_3</topic>
  <use_velocity_commands>1</use_velocity_commands>
  <cmd_max>2</cmd_max>
  <cmd_min>-2</cmd_min>
</plugin>
<plugin
  filename="gz-sim-joint-position-controller-system"
  name="gz::sim::systems::JointPositionController">
  <joint_name>J_4_Plecho_2_and_3</joint_name>
  <topic>J_4</topic>

```



```

    <use_velocity_commands>1</use_velocity_commands>
    <cmd_max>2</cmd_max>
    <cmd_min>-2</cmd_min>
</plugin>
<plugin
  filename="gz-sim-joint-position-controller-system"
  name="gz::sim::systems::JointPositionController">
  <joint_name>J_5_Plecho_3_and_4</joint_name>
  <topic>J_5</topic>
  <use_velocity_commands>1</use_velocity_commands>
  <cmd_max>2</cmd_max>
  <cmd_min>-2</cmd_min>
</plugin>
<plugin
  filename="gz-sim-joint-position-controller-system"
  name="gz::sim::systems::JointPositionController">
  <joint_name>J_6_Plecho_4_Kist</joint_name>
  <topic>J_6</topic>
  <use_velocity_commands>1</use_velocity_commands>
  <cmd_max>2</cmd_max>
  <cmd_min>-2</cmd_min>
</plugin>
<plugin filename="gz-sim-pose-publisher-system" name="gz::sim::systems::PosePublisher">
  <publish_link_pose>true</publish_link_pose>
  <link_name>Kist_Link</link_name>
  <topic>/kist_link_gz_position</topic>
  <update_rate>10</update_rate>
</plugin>
</model>
</sdf>

```

ПРИЛОЖЕНИЕ Л

Файл запуска симуляции

```
import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node
import xacro

def generate_launch_description():

    # Пути пакетов
    pkg_ros_gz_sim = get_package_share_directory('ros_gz_sim')
    pkg_ros_gz_sim_demos = get_package_share_directory('robot_rviz')

    # Поиск файла описания модели
    robot_description_file = os.path.join(pkg_ros_gz_sim_demos, 'urdf', 'Robot_ai.sdf')
    robot_description_config = xacro.process_file(
        robot_description_file
    )
    robot_description = {'robot_description': robot_description_config.toxml()}

    # Запуск плагина трансляции положения звеньев
    robot_state_publisher = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        name='robot_state_publisher',
        output='both',
        parameters=[robot_description],
    )

    # Запуск Gazebo Sim
    gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(pkg_ros_gz_sim, 'launch', 'gz_sim.launch.py')
        ),
        launch_arguments={'gz_args': '-r empty.sdf'}.items(),
    )

    # Запуск RViz
    rviz = Node(
        package='rviz2',
        executable='rviz2',
        arguments=['-d', os.path.join(pkg_ros_gz_sim_demos, 'rviz', 'rviz_basic_settings.rviz')],
    )
```

```

# Построение модели
spawn = Node(
    package='ros_gz_sim',
    executable='create',
    parameters=[{'name': 'robot_rviz',
                  'topic': 'robot_description'}],
    output='screen',
)

# Настройка моста связи между Gz и ROS
bridge = Node(
    package='ros_gz_bridge',
    executable='parameter_bridge',
    arguments=[
        # Clock (IGN -> ROS2)
        '/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock',
        # Joint states (IGN -> ROS2)
        '/world/empty/model/robot_rviz/joint_state@sensor_msgs/msg/JointState[gz.msgs.Model',
        # Lidar_1
        '/lidar_1/scan@sensor_msgs/msg/LaserScan[gz.msgs.LaserScan]',
        '/lidar_1/points@sensor_msgs/msg/PointCloud2@gz.msgs.PointCloudPacked',
        # Lidar_2
        '/lidar_2/points@sensor_msgs/msg/PointCloud2@gz.msgs.PointCloudPacked',
        # Joint commands (ROS2 -> IGN)
        '/J_1@std_msgs/msg/Float64[gz.msgs.Double',
        '/J_2@std_msgs/msg/Float64[gz.msgs.Double',
        '/J_3@std_msgs/msg/Float64[gz.msgs.Double',
        '/J_4@std_msgs/msg/Float64[gz.msgs.Double',
        '/J_5@std_msgs/msg/Float64[gz.msgs.Double',
        '/J_6@std_msgs/msg/Float64[gz.msgs.Double',

    ],
    remappings=[
        ('/world/empty/model/robot_rviz/joint_state', 'joint_states'),
    ],
    output='screen'
)

return LaunchDescription(
    [
        # Запуск нод
        gazebo,
        spawn,
        bridge,
        robot_state_publisher,
        rviz,
    ]
)

```

ПРИЛОЖЕНИЕ М
Файлы подгрузки компонентов симуляции
Файл CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(robot_rviz)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES
"Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# Поиск пакетов
find_package(ament_cmake REQUIRED)
find_package(controller_manager REQUIRED)
find_package(gz_ros2_control REQUIRED)
find_package(rclcpp REQUIRED)
find_package(ros_gz REQUIRED)
find_package(ros_gz_bridge REQUIRED)
find_package(ros_gz_image REQUIRED)
find_package(ros_gz_sim REQUIRED)
find_package(ros2_control REQUIRED)
find_package(ros2_controllers REQUIRED)
find_package(trajjectory_msgs REQUIRED)
find_package(xacro REQUIRED)

if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  set(ament_cmake_copyright_FOUND TRUE)
  set(ament_cmake_cpplint_FOUND TRUE)
  ament_lint_auto_find_test_dependencies()
endif()

install(PROGRAMS
  scripts/robot_rl_node.py
  scripts/target_publisher.py
  DESTINATION lib/${PROJECT_NAME}
)

install(
  DIRECTORY config launch maps meshes models params rviz src urdf worlds
  DESTINATION share/${PROJECT_NAME}
)

ament_package()
```

Файл package.xml

```
<package format="3">
<name>robot_rviz</name>
<version>0.0.0</version>
<description>TODO: Package description</description>
<maintainer email="andrei@todo.todo">andrei</maintainer>
<license>TODO: License declaration</license>
<buildtool_depend>ament_cmake</buildtool_depend>
<exec_depend>image_transport_plugins</exec_depend>
<exec_depend>ros_gz_bridge</exec_depend>
<exec_depend>joint_state_publisher</exec_depend>
<exec_depend>robot_state_publisher</exec_depend>
<exec_depend>robot_pose_publisher</exec_depend>
<exec_depend>rviz</exec_depend>
<exec_depend>xacro</exec_depend>
<test_depend>ament_lint_auto</test_depend>
<test_depend>ament_lint_common</test_depend>
<test_depend>ament_lint_common</test_depend>
<exec_depend>rqt_image_view</exec_depend>
<exec_depend>rqt_plot</exec_depend>
<exec_depend>rqt_topic</exec_depend>
<exec_depend>sdformat_urdf</exec_depend>
<export>
<build_type>ament_cmake</build_type>
</export>
</package>
```


ПРИЛОЖЕНИЕ Н

Файл конфигурации Rviz

Panels:

- Class: rviz_common/Displays
Help Height: 78
Name: Displays
Property Tree Widget:
Expanded:
 - /Global Options1
 - /Status1
 - /RobotModel1
 - /TF1
 - /TF1/Frames1Splitter Ratio: 0.5
Tree Height: 617
- Class: rviz_common/Selection
Name: Selection
- Class: rviz_common/Tool Properties
Expanded:
 - /2D Goal Pose1
 - /Publish Point1Name: Tool Properties
Splitter Ratio: 0.5886790156364441
- Class: rviz_common/Views
Expanded:
 - /Current View1Name: Views
Splitter Ratio: 0.5

Visualization Manager:

Class: ""

Displays:

- Alpha: 0.5

Cell Size: 1

Class: rviz_default_plugins/Grid

Color: 160; 160; 164

Enabled: true

Line Style:

Line Width: 0.029999999329447746

Value: Lines

Name: Grid

Normal Cell Count: 0

Offset:

X: 0

Y: 0

Z: 0

Plane: XY

Plane Cell Count: 10

Reference Frame: <Fixed Frame>

Value: true

- Alpha: 1
 - Class: rviz_default_plugins/RobotModel
 - Collision Enabled: false
 - Description File: ""
 - Description Source: Topic
 - Description Topic:
 - Depth: 5
 - Durability Policy: Volatile
 - History Policy: Keep Last
 - Reliability Policy: Reliable
 - Value: /robot_description
 - Enabled: true
 - Links:
 - All Links Enabled: true
 - Expand Joint Details: false
 - Expand Link Details: false
 - Expand Tree: false
 - Link Tree Style: Links in Alphabetic Order
 - base_footprint:
 - Alpha: 1
 - Show Axes: false
 - Show Trail: false
 - base_link:
 - Alpha: 1
 - Show Axes: false
 - Show Trail: false
 - Value: true
 - drivewhl_l_link:
 - Alpha: 1
 - Show Axes: false
 - Show Trail: false
 - Value: true
 - drivewhl_r_link:
 - Alpha: 1
 - Show Axes: false
 - Show Trail: false
 - Value: true
 - front_caster:
 - Alpha: 1
 - Show Axes: false
 - Show Trail: false
 - Value: true
 - gps_link:
 - Alpha: 1
 - Show Axes: false
 - Show Trail: false
 - imu_link:
 - Alpha: 1
 - Show Axes: false
 - Show Trail: false
 - lidar_link:
 - Alpha: 1

Show Axes: false
 Show Trail: false
 Value: true
 Name: RobotModel
 TF Prefix: ""
 Update Interval: 0
 Value: true
 Visual Enabled: true
 - Class: rviz_default_plugins/TF
 Enabled: true
 Frame Timeout: 15
 Frames:
 All Enabled: false
 base_footprint:
 Value: false
 base_link:
 Value: false
 drivewhl_l_link:
 Value: true
 drivewhl_r_link:
 Value: true
 front_caster:
 Value: false
 gps_link:
 Value: false
 imu_link:
 Value: false
 lidar_link:
 Value: true
 Marker Scale: 1
 Name: TF
 Show Arrows: false
 Show Axes: true
 Show Names: true
 Tree:
 base_footprint:
 base_link:
 drivewhl_l_link:
 {}
 drivewhl_r_link:
 {}
 front_caster:
 {}
 gps_link:
 {}
 imu_link:
 {}
 lidar_link:
 {}
 Update Interval: 0
 Value: true
 Enabled: true

Global Options:

Background Color: 48; 48; 48

Fixed Frame: base_link

Frame Rate: 30

Name: root

Tools:

- Class: rviz_default_plugins/Interact

Hide Inactive Objects: true

- Class: rviz_default_plugins/MoveCamera

- Class: rviz_default_plugins/Select

- Class: rviz_default_plugins/FocusCamera

- Class: rviz_default_plugins/Measure

Line color: 128; 128; 0

- Class: rviz_default_plugins/SetInitialPose

Topic:

Depth: 5

Durability Policy: Volatile

History Policy: Keep Last

Reliability Policy: Reliable

Value: /initialpose

- Class: rviz_default_plugins/SetGoal

Topic:

Depth: 5

Durability Policy: Volatile

History Policy: Keep Last

Reliability Policy: Reliable

Value: /goal_pose

- Class: rviz_default_plugins/PublishPoint

Single click: true

Topic:

Depth: 5

Durability Policy: Volatile

History Policy: Keep Last

Reliability Policy: Reliable

Value: /clicked_point

Transformation:

Current:

Class: rviz_default_plugins/TF

Value: true

Views:

Current:

Class: rviz_default_plugins/Orbit

Distance: 4.434264183044434

Enable Stereo Rendering:

Stereo Eye Separation: 0.05999999865889549

Stereo Focal Distance: 1

Swap Stereo Eyes: false

Value: false

Focal Point:

X: 0.31193429231643677

Y: 0.11948385089635849

Z: -0.4807402193546295

Focal Shape Fixed Size: true
Focal Shape Size: 0.05000000074505806
Invert Z Axis: false
Name: Current View
Near Clip Distance: 0.009999999776482582
Pitch: 0.490397572517395
Target Frame: <Fixed Frame>
Value: Orbit (rviz)
Yaw: 1.0503965616226196
Saved: ~
Window Geometry:
Displays:
 collapsed: false
Height: 846
Hide Left Dock: false
Hide Right Dock: false
QMainWindow State:
000000ff00000000fd0000000040000000000000156000002f4fc0200000008fb0000001200530065
006c0065006300740069006f006e00000001e10000009b0000005c00ffffffb0000001e0054006f0
06f006c002000500072006f007000650072007400690065007302000001ed000001df0000018500
0000a3fb000000120056006900650077007300200054006f006f02000001df00000211000001850
0000122fb000000200054006f006f006c002000500072006f007000650072007400690065007300
3203000002880000011d000002210000017afb000000100044006900730070006c006100790073
010000003d000002f4000000c900ffffffb0000002000730065006c0065006300740069006f006e0
0200062007500660066006500720200000138000000aa0000023a00000294fb000000140057006
90064006500530074006500720065006f02000000e6000000d2000003ee0000030bfb0000000c00
4b0069006e0065006300740200000186000001060000030c00000261000000010000010f000002
f4fc0200000003fb0000001e0054006f006f006c002000500072006f007000650072007400690065
00730100000041000000780000000000000000fb0000000a00560069006500770073010000003d
000002f4000000a400ffffffb0000001200530065006c0065006300740069006f006e010000025a0
00000b200000000000000000000000002000004900000000a9fc0100000001fb0000000a005600690
06500770073030000004e00000080000002e10000019700000003000004420000003efc0100000
002fb0000000800540069006d006501000000000000044200000000000000fb0000000800540
069006d0065010000000000000450000000000000000000023f000002f400000004000000040
0000008000000008fc0000000100000002000000010000000a0054006f006f006c00730100000000
ffffff000000000000000000

Selection:
 collapsed: false
Tool Properties:
 collapsed: false
Views:
 collapsed: false
Width: 1200
X: 246
Y: 77

ПРИЛОЖЕНИЕ П
Алгоритм основной программы

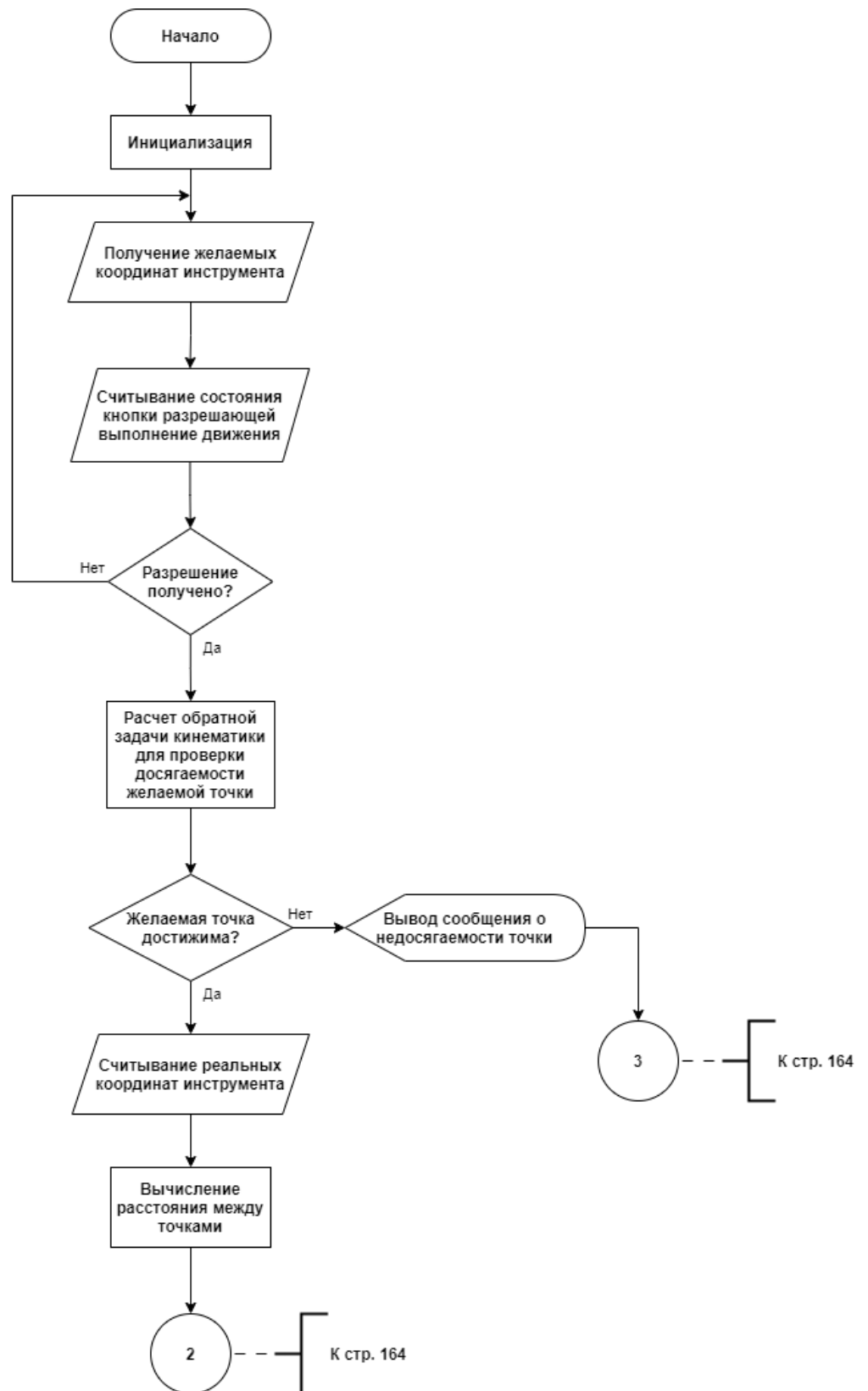


Рисунок П1 – Первый лист основного алгоритма

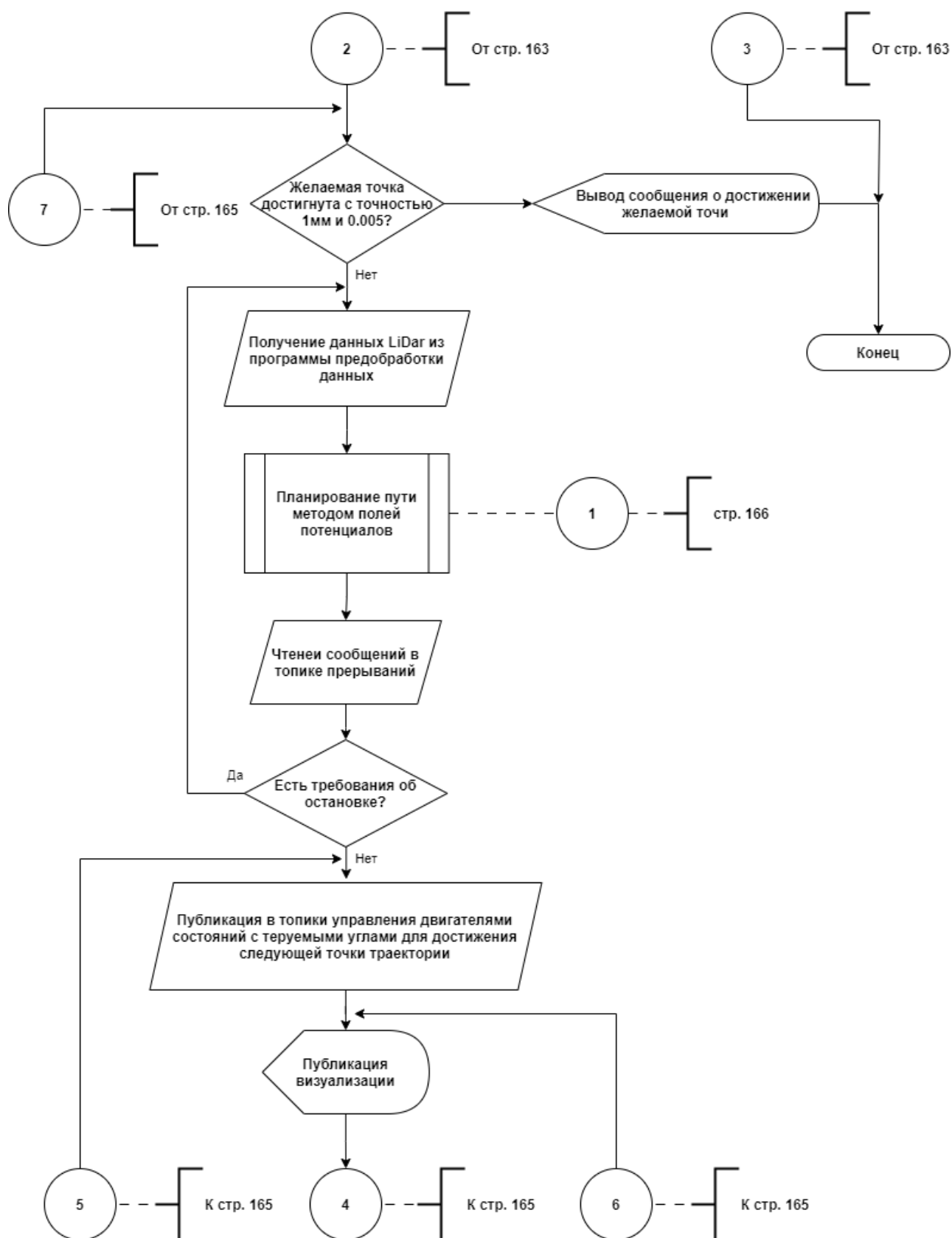


Рисунок П2 – Второй лист основного алгоритма

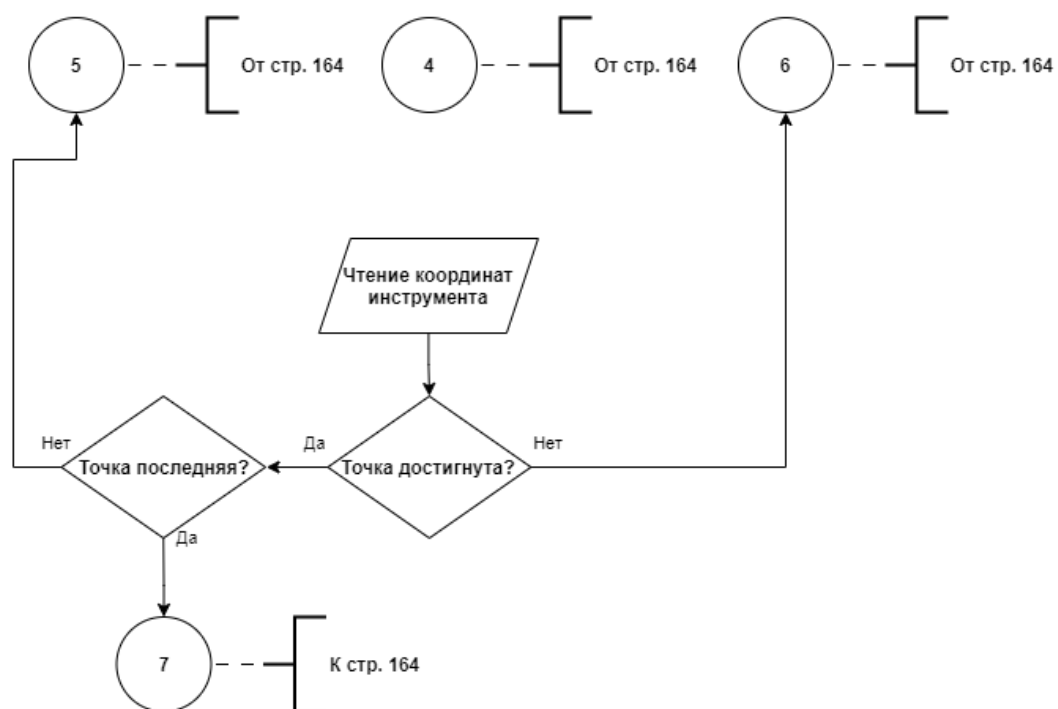


Рисунок ПЗ – Третий лист основного алгоритма

ПРИЛОЖЕНИЕ Р

Алгоритм просчета траектории

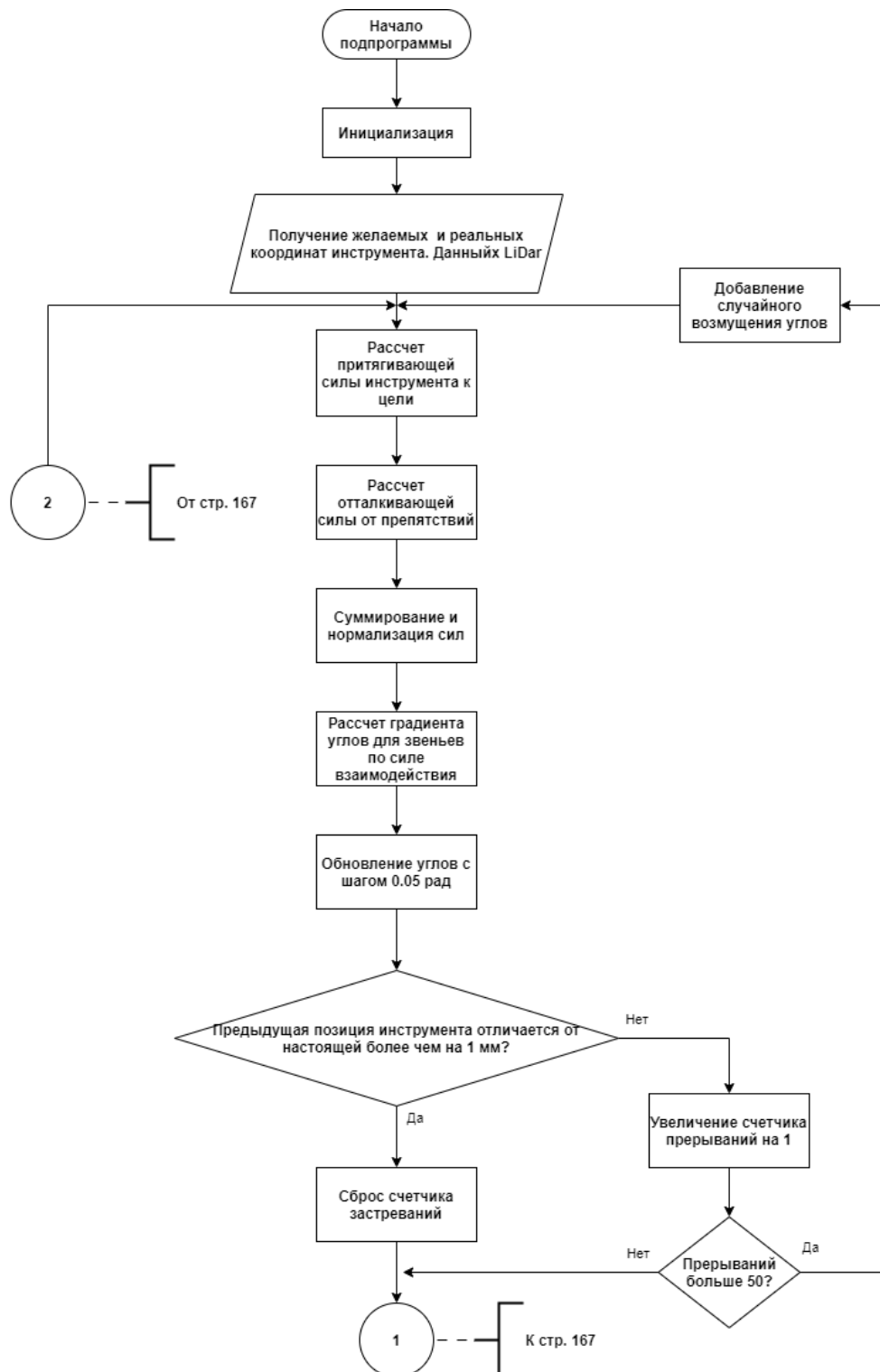


Рисунок Р1 – Первый лист алгоритма
просчета траектории

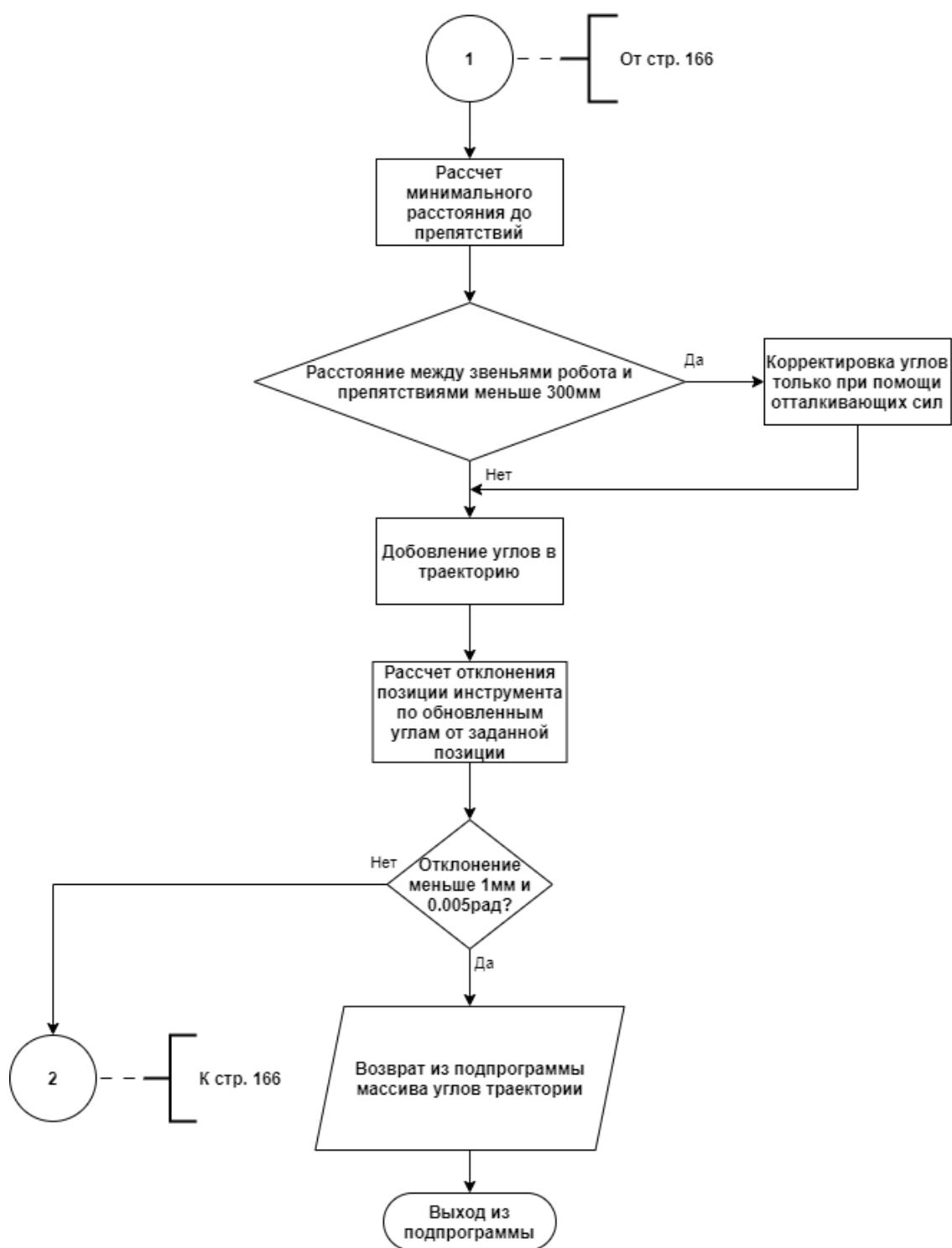


Рисунок Р2 – Второй лист алгоритма
просчета траектории

ПРИЛОЖЕНИЕ С

Алгоритм трансляции положения инструмента



Рисунок С1 – Алгоритм трансляции положения инструмента

ПРИЛОЖЕНИЕ Т
Алгоритм трансляции положения звеньев манипулятора

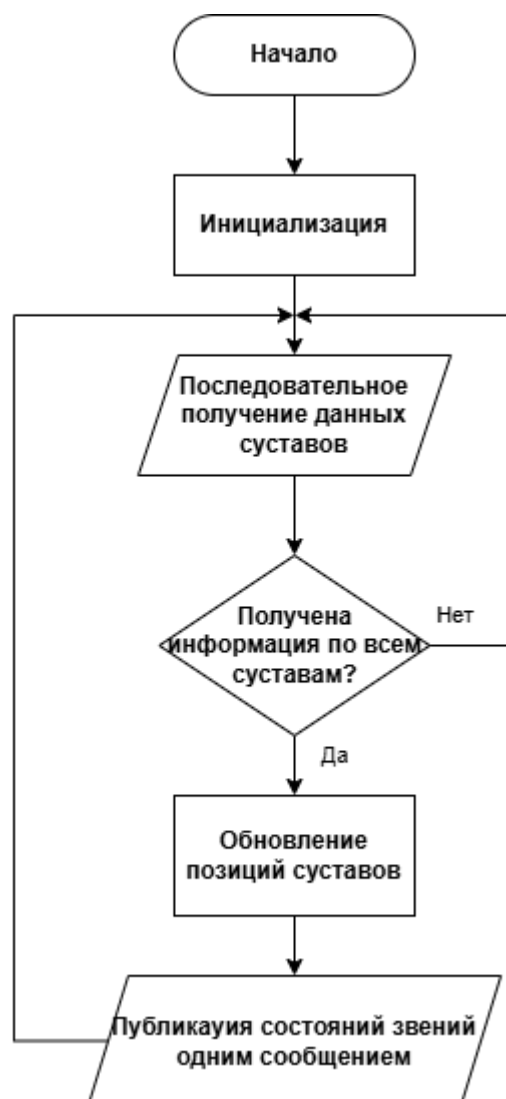


Рисунок Т1 – Алгоритм трансляции положения звеньев манипулятора

ПРИЛОЖЕНИЕ У
Алгоритм фильтрации данных LiDAR



Рисунок У1 – Алгоритм фильтрации данных LiDAR

ПРИЛОЖЕНИЕ Ф
Алгоритм вращения данных LiDAR



Рисунок Ф1 – Алгоритм вращения данных LiDAR

ПРИЛОЖЕНИЕ X

Алгоритм предупреждения касания

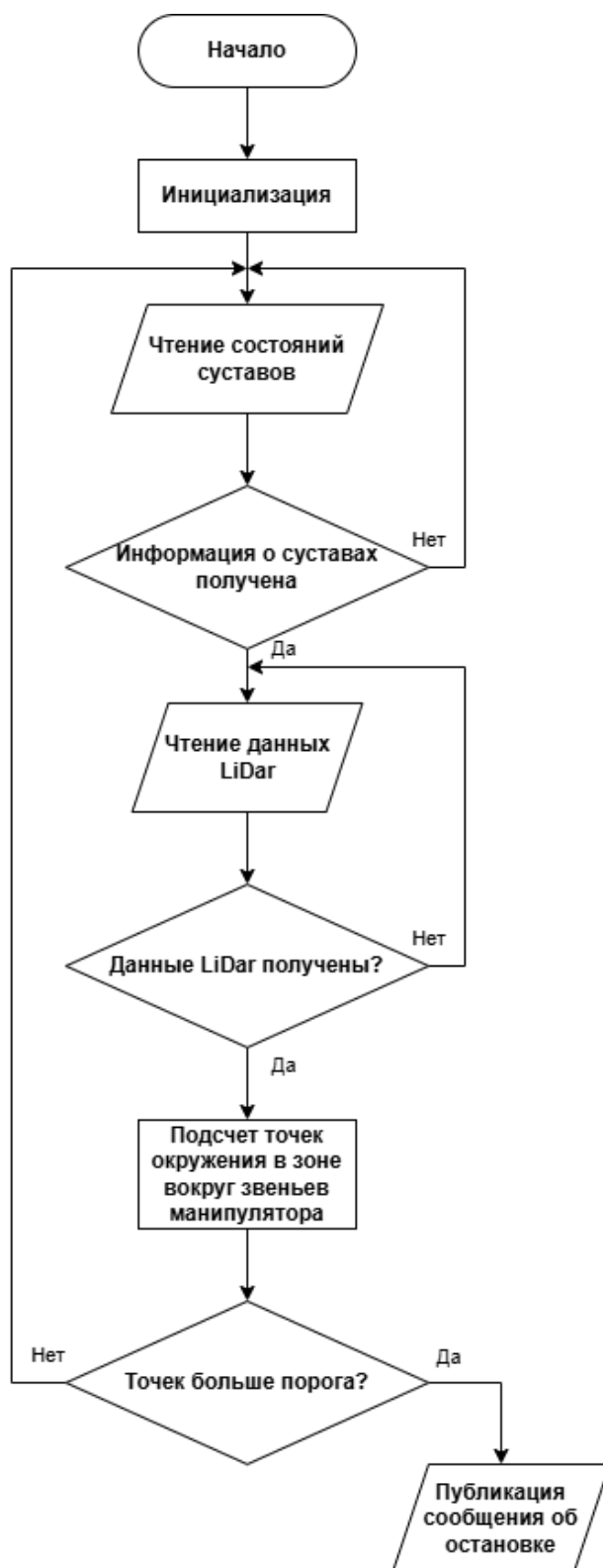


Рисунок X1 – Алгоритм предупреждения касания

ПРИЛОЖЕНИЕ Ц
Алгоритм детектирования касания



Рисунок Ц1 – Алгоритм детектирования касания

ПРИЛОЖЕНИЕ Ш

Листинг основной программы

```
import open3d as o3d
import numpy as np
import xml.etree.ElementTree as ET
from scipy.spatial.transform import Rotation
import os
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState, PointCloud2
from geometry_msgs.msg import PoseStamped, Point
from std_msgs.msg import Float64, Header, Bool
from visualization_msgs.msg import Marker
import sensor_msgs_py.point_cloud2 as pc2
import time

BASE_MESH_DIR = "/home/andrei/rviz_test_ws/src/robot_rviz/meshes"

DH_PARAMS = {
    'theta0': np.deg2rad([0, 90, 90, 0, 0, -180]),
    'd': [393, 0, 0, 700, 0, 174],
    'a': [0, 700, 0, 0, 0, 0],
    'alpha': np.deg2rad([90, 0, 90, 90, -90, 0])
}

def pose_to_transform(pose_str):
    pose = [float(x) for x in pose_str.split()]
    pos = pose[0:3]
    rot = pose[3:6]
    rot_mat = Rotation.from_euler('xyz', rot).as_matrix()
    transform = np.eye(4)
    transform[:3, :3] = rot_mat
    transform[:3, 3] = pos
    return transform

def clean_stl_path(uri):
    return uri.replace("file://", "").replace("$(find robot_rviz)/meshes/", "").strip()

def create_transform_matrix(offset_x, offset_y, offset_z, roll, pitch, yaw):
    transform = np.eye(4)
    rot = Rotation.from_euler('xyz', [roll, pitch, yaw]).as_matrix()
    transform[:3, :3] = rot
    transform[:3, 3] = [offset_x, offset_y, offset_z]
    return transform

class RobotController(Node):
    def __init__(self):
        super().__init__('robot_controller')
```

```

self.links = {}
self.joints = {}
self.robot_model = []
self.combined_cloud = o3d.geometry.PointCloud()
self.link_clouds = {}

self.dh_joints = [
    "J_1_Base_Stol", "J_2_Stol_Plecho_1", "J_3_Plecho_1_and_2",
    "J_4_Plecho_2_and_3", "J_5_Plecho_3_and_4", "J_6_Plecho_4_Kist"
]
self.current_joint_angles = np.zeros(6)
self.real_joint_angles = np.zeros(6)
self.invert_joints = [False, True, False, True, True, False]
self.zero_joints = [False, False, False, False, False, False]
self.joint_limits = np.array([[ -3.0, 3.0], [ -1.57, 1.57], [ -2.3, 2.3], [ -3.0, 3.0], [ -2.0, 2.0], [ -
3.14, 3.14]])

self.target_position = None
self.target_orientation = np.eye(3)
self.real_tcp_positioSTL: self.real_tcp_position = np.zeros(3)
self.real_tcp_orientation = np.eye(3)
self.min_safe_distance = 300.0

self.attractive_gain = 0.5
self.repulsive_gain = 1000.0
self.repulsive_range = 400.0
self.step_size = 0.05

self.declare_parameter('robot_offset_x', -0.32744256676702599)
self.declare_parameter('robot_offset_y', -0.32241999999999998)
self.declare_parameter('robot_offset_z', 0.08)
self.declare_parameter('robot_roll', 0.0)
self.declare_parameter('robot_pitch', 0.0)
self.declare_parameter('robot_yaw', 3.1415926535746808)

self.joint_sub = self.create_subscription(JointState, '/aggregated_joint_states',
self.joint_callback, 10)
self.tcp_sub = self.create_subscription(PoseStamped, '/tcp', self.tcp_callback, 10)
self.lidar_sub = self.create_subscription(PointCloud2, '/lidar_points', self.lidar_callback, 10)
self.attention_sub = self.create_subscription(Bool, '/attention', self.attention_callback, 10)
self.joint_pubs = [self.create_publisher(Float64, f'/J_{i+1}', 10) for i in range(6)]
self.robot_model_pub = self.create_publisher(PointCloud2, '/robot_model_points', 10)
self.trajectory_pub = self.create_publisher(PointCloud2, '/planned_trajectory', 10)
self.all_points_pub = self.create_publisher(PointCloud2, '/all_generated_points', 10)
self.closest_points_pub = self.create_publisher(PointCloud2, '/closest_points', 10)
self.marker_pub = self.create_publisher(Marker, '/robot_axes', 10)

self.closest_points = o3d.geometry.PointCloud()
self.last_lidar_time = 0.0
self.lidar_frequency = 0.0
self.lidar_count = 0
self.lidar_timer = self.create_timer(1.0, self.update_lidar_frequency)

```

```

self.link_offsets = {
    "Base_Robot_link": "0 0 0 0 0",
    "Povorotniy_Stol_Link": "-0.32744256676702599 -0.08 -0.32241999999999998 0 0 0",
    "Plecho_1_Link": "0.32744256676702599 -0.32241999999999998 0.08 0 0 0",
    "Plecho_2_Link": "-0.08 0.32744256676702599 -0.32241999999999998 0 0 0",
    "Plecho_3_Link": "-0.32744256676702599 0.08 0.32241999999999998 0 0 0",
    "Plecho_4_Link": "-0.32744256676702599 0.32241999999999998 0.08 0 0 0",
    "Kist_Link": "-0.32744256676702599 0.08 -0.32744256676702599 0 0 0"
}

self.offset_transforms = {link_name: pose_to_transform(pose) for link_name, pose in
self.link_offsets.items()}

self.axis_offsets = {
    "J_1_Base_Stol": [-0.32744256676702599, -0.32241999999999998, -0.08],
    "J_2_Stol_Plecho_1": [0.32744256676702599, -0.32241999999999998, 0.08],
    "J_3_Plecho_1_and_2": [-0.08, 0.32744256676702599, -0.32744256676702599],
    "J_4_Plecho_2_and_3": [-0.32744256676702599, -0.08, 0.32744256676702599],
    "J_5_Plecho_3_and_4": [-0.32744256676702599, 0.32241999999999998, 0.08],
    "J_6_Plecho_4_Kist": [-0.32744256676702599, -0.32744256676702599, -0.08]
}

self.is_paused = False # Флаг для остановки робота
self.load_sdf()
self.build_robot_model()
self.wait_for_initial_data()
self.timer = self.create_timer(0.02, self.control_loop)
self.get_logger().info("Контроллер инициализирован")

def load_sdf(self):
    tree = ET.parse("/home/andrei/rviz_test_ws/src/robot_rviz/urdf/Robot_ai.sdf")
    root = tree.getroot()
    for link in root.findall("./link"):
        link_name = link.get("name")
        pose_elem = link.find("pose")
        pose = pose_elem.text if pose_elem is not None else "0 0 0 0 0 0"
        visual = link.find("./visual/geometry/mesh/uri")
        collision = link.find("./collision/geometry/mesh/uri")
        stl_path = None
        if visual is not None:
            stl_path = os.path.join(BASE_MESH_DIR, clean_stl_path(visual.text))
        elif collision is not None:
            stl_path = os.path.join(BASE_MESH_DIR, clean_stl_path(collision.text))
        self.links[link_name] = {"pose": pose, "stl_path": stl_path, "transform": np.eye(4)}

    for joint in root.findall("./joint"):
        joint_name = joint.get("name")
        parent = joint.find("parent").text
        child = joint.find("child").text
        pose_elem = joint.find("pose")
        pose = pose_elem.text if pose_elem is not None else "0 0 0 0 0 0"
        axis_elem = joint.find("axis/xyz")
        axis = axis_elem.text if axis_elem is not None else "0 0 1"

```

```

        if "lidar" not in child.lower():
            self.joints[joint_name] = {
                "parent": parent, "child": child, "pose": pose,
                "axis": [float(x) for x in axis.split()]
            }
        else:
            self.joints[joint_name] = {"parent": parent, "child": child, "pose": pose}

def dh_matrix(self, theta, d, a, alpha):
    ct, st = np.cos(theta), np.sin(theta)
    ca, sa = np.cos(alpha), np.sin(alpha)
    return np.array([
        [ct, -st*ca, st*sa, a*ct],
        [st, ct*ca, -ct*sa, a*st],
        [0, sa, ca, d],
        [0, 0, 0, 1]
    ])

def compute_link_transform(self, link_name, visited=None):
    if visited is None:
        visited = set()
    if link_name in visited:
        return np.eye(4)
    visited.add(link_name)

    link = self.links[link_name]
    link_pose = pose_to_transform(link["pose"])
    full_transform = link_pose

    for joint_name, joint in self.joints.items():
        if joint["child"] == link_name:
            parent_transform = self.compute_link_transform(joint["parent"], visited.copy())
            joint_transform = pose_to_transform(joint["pose"])
            if "axis" in joint:
                try:
                    joint_index = self.dh_joints.index(joint_name)
                    angle_deg = self.current_joint_angles[joint_index]
                    angle_rad = np.deg2rad(angle_deg)
                except ValueError:
                    angle_rad = 0.0
                axis = np.array(joint["axis"])
                rot_axis = Rotation.from_rotvec(angle_rad * axis).as_matrix()
                offset = np.array(self.axis_offsets.get(joint_name, [0.0, 0.0, 0.0]))
                T_offset = np.eye(4)
                T_offset[:3, 3] = offset
                T_offset_inv = np.eye(4)
                T_offset_inv[:3, 3] = -offset
                R_axis_4x4 = np.eye(4)
                R_axis_4x4[:3, :3] = rot_axis
                T_rot = T_offset @ R_axis_4x4 @ T_offset_inv
                joint_transform = joint_transform @ T_rot
            full_transform = parent_transform @ joint_transform @ link_pose

```



```

        break
    return full_transform

def build_robot_model(self):
    self.combined_cloud = o3d.geometry.PointCloud()
    self.link_clouds = {}
    axis_geometries = []

    for link_name, link_data in self.links.items():
        stl_path = link_data["stl_path"]
        if stl_path and os.path.exists(stl_path):
            mesh = o3d.io.read_triangle_mesh(stl_path)
            if not mesh.is_empty():
                transform = self.compute_link_transform(link_name)
                if link_name in self.offset_transforms:
                    transform = transform @ self.offset_transforms[link_name]
                mesh.transform(transform)
                cloud = mesh.sample_points_uniformly(number_of_points=1000)
                self.link_clouds[link_name] = cloud
                self.combined_cloud += cloud

    if len(self.combined_cloud.points) > 0:
        self.combined_cloud.paint_uniform_color([0, 0, 1])

    for joint_name, joint in self.joints.items():
        if "axis" in joint:
            joint_transform = self.compute_link_transform(joint["parent"]) @
            pose_to_transform(joint["pose"])
            axis = np.array(joint["axis"])
            offset = self.axis_offsets.get(joint_name, [0.0, 0.0, 0.0])
            points = np.array([[0, 0, 0], axis * 0.1])
            points_homogeneous = np.hstack((points, np.ones((2, 1))))
            points_transformed = (joint_transform @ points_homogeneous.T).T[:, :3]
            lines = o3d.geometry.LineSet()
            lines.points = o3d.utility.Vector3dVector(points_transformed)
            lines.lines = o3d.utility.Vector2iVector([[0, 1]])
            lines.colors = o3d.utility.Vector3dVector([[0, 0, 1]])
            axis_geometries.append(lines)

    self.robot_model = axis_geometries

def forward_kinematics(self, angles):
    T = np.eye(4)
    link_positions = []
    theta_total = DH_PARAMS['theta0'] + angles
    for i in range(6):
        T = T @ self.dh_matrix(theta_total[i], DH_PARAMS['d'][i], DH_PARAMS['a'][i],
        DH_PARAMS['alpha'][i])
        link_positions.append(T[:3, 3].copy())
    P, R = T[:3, 3], T[:3, :3]
    return P, R, np.array(link_positions)

```

```

def check_reachability(self, P_target):
    max_reach = sum(DH_PARAMS['a']) + sum(DH_PARAMS['d'])
    dist = np.linalg.norm(P_target)
    return dist <= max_reach

def inverse_kinematics(self, P_target, R_target):
    def error(q):
        P, R = self.forward_kinematics(q)[:2]
        pos_error = np.linalg.norm(P - P_target)
        rot_error = np.linalg.norm(R - R_target, 'fro')
        return pos_error + 20 * rot_error

    q0 = self.real_joint_angles.copy()
    from scipy.optimize import minimize
    result = minimize(error, q0, bounds=self.joint_limits, method='SLSQP',
                      options={'disp': False, 'maxiter': 1000, 'ftol': 1e-8})
    if result.success:
        P, R = self.forward_kinematics(result.x)[:2]
        self.get_logger().info(f'ИК: решение={result.x}, позиция={P}, ориентация=\n{R},
                               ошибка={result.fun}')
        return result.x
    else:
        self.get_logger().error(f'ИК: ошибка={result.message}')
        raise ValueError("IK failed")

def compute_gradient(self, current_angles, force_direction, eps=1e-6):
    P_current, _, _ = self.forward_kinematics(current_angles)
    delta_angles = np.zeros(6)
    for i in range(6):
        angles_plus = current_angles.copy()
        angles_plus[i] += eps
        P_plus, _, _ = self.forward_kinematics(angles_plus)
        gradient = (P_plus - P_current) / eps
        delta_angles[i] = np.dot(force_direction, gradient)
    return delta_angles / (np.linalg.norm(delta_angles) + 1e-6)

def compute_potential_forces(self, current_angles):
    P_current, _, link_positions = self.forward_kinematics(current_angles)

    attractive_force = self.attractive_gain * (self.target_position - P_current)
    attractive_force_norm = np.linalg.norm(attractive_force)
    if attractive_force_norm > 0:
        attractive_force /= attractive_force_norm

    repulsive_force = np.zeros(3)
    if self.closest_points.has_points():
        kdtree = o3d.geometry.KDTreeFlann(self.closest_points)
        for pos in link_positions:
            [k, idx, dists] = kdtree.search_knn_vector_3d(pos, 1)
            if k > 0:
                dist = np.sqrt(dists[0])
                if dist < self.repulsive_range:

```

```

        obstacle_pos = np.asarray(self.closest_points.points)[idx[0]]
        direction = pos - obstacle_pos
        dist = max(dist, 1e-6)
        force_magnitude = self.repulsive_gain * (1.0 / dist - 1.0 / self.repulsive_range) /
(dist ** 2)
        repulsive_force += force_magnitude * direction / np.linalg.norm(direction)

    total_force = attractive_force + repulsive_force
    total_force_norm = np.linalg.norm(total_force)
    if total_force_norm > 0:
        total_force /= total_force_norm

    delta_angles = self.compute_gradient(current_angles, total_force)
    return self.step_size * delta_angles

def simplify_trajectory(self, trajectory, min_distance=20.0):
    """Упрощает траекторию, удаляя точки, которые находятся ближе заданного
расстояния."""
    if len(trajectory) <= 2:
        return trajectory

    simplified = [trajectory[0]] # Сохраняем первую точку
    last_point = self.forward_kinematics(trajectory[0])[0] # Позиция первой точки

    for angles in trajectory[1:]:
        current_point = self.forward_kinematics(angles)[0]
        distance = np.linalg.norm(current_point - last_point)
        if distance >= min_distance:
            simplified.append(angles)
            last_point = current_point

    return np.array(simplified)

def potential_field_plan(self, start_angles, max_steps=1000, tolerance=10.0,
max_stuck_steps=50, switch_distance=200.0):
    trajectory = [start_angles]
    current_angles = start_angles.copy()
    all_points = [start_angles]
    stuck_counter = 0
    last_position = None

    for step in range(max_steps):
        P_current, _, link_positions = self.forward_kinematics(current_angles)
        dist_to_target = np.linalg.norm(P_current - self.target_position)

        if dist_to_target < tolerance:
            self.get_logger().info(f'Цель достигнута на шаге {step},
расстояние={dist_to_target:.2f}')
            break

        if dist_to_target < switch_distance:

```

```

        self.get_logger().info(f"Шаг {step}: близко к цели ({dist_to_target:.2f} мм),
переключаемся на ИК")
        break

    if last_position is not None and np.linalg.norm(P_current - last_position) < 1.0:
        stuck_counter += 1
        if stuck_counter > max_stuck_steps:
            self.get_logger().info(f"Шаг {step}: застрял, применяем случайное
возмущение")
            perturbation = np.random.uniform(-0.1, 0.1, size=6)
            current_angles += perturbation
            current_angles = np.clip(current_angles, self.joint_limits[:, 0], self.joint_limits[:, 1])
            stuck_counter = 0
            continue
        else:
            stuck_counter = 0

    delta_angles = self.compute_potential_forces(current_angles)
    new_angles = current_angles + delta_angles
    new_angles = np.clip(new_angles, self.joint_limits[:, 0], self.joint_limits[:, 1])

    P_new, _, new_link_positions = self.forward_kinematics(new_angles)
    min_dist = self.get_min_distance_to_obstacles(new_link_positions, self.closest_points)

    if min_dist < self.min_safe_distance:
        self.get_logger().info(f"Шаг {step}: слишком близко к препятствию ({min_dist:.2f}
мм), корректируем")
        repulsive_force = np.zeros(3)
        kdtree = o3d.geometry.KDTreeFlann(self.closest_points)
        for pos in new_link_positions:
            [k, idx, dists] = kdtree.search_knn_vector_3d(pos, 1)
            if k > 0 and np.sqrt(dists[0]) < self.repulsive_range:
                obstacle_pos = np.asarray(self.closest_points.points)[idx[0]]
                direction = pos - obstacle_pos
                dist = max(np.sqrt(dists[0]), 1e-6)
                force_magnitude = self.repulsive_gain * (1.0 / dist - 1.0 / self.repulsive_range) /
(dist ** 2)
                repulsive_force += force_magnitude * direction / np.linalg.norm(direction)

    if np.linalg.norm(repulsive_force) > 0:
        repulsive_force /= np.linalg.norm(repulsive_force)
        delta_angles_rep = self.compute_gradient(current_angles, repulsive_force)
        new_angles = current_angles + self.step_size * delta_angles_rep
        new_angles = np.clip(new_angles, self.joint_limits[:, 0], self.joint_limits[:, 1])

    trajectory.append(new_angles)
    all_points.append(new_angles)
    current_angles = new_angles
    last_position = P_new
    self.get_logger().info(f"Шаг {step}: позиция={P_new}, расстояние до
цели={dist_to_target:.2f}")

```

```

        self.publish_all_points(all_points)
        simplified_trajectory = self.simplify_trajectory(trajecory, min_distance=20.0) #
Упрощаем траекторию
        self.get_logger().info(f"Траектория упрощена: {len(trajecory)} ->
{len(simplified_trajectory)} точек")
        return simplified_trajectory, current_angles

def get_min_distance_to_obstacles(self, link_positions, obstacle_cloud):
    if not obstacle_cloud.has_points():
        return float('inf')

    kdtree = o3d.geometry.KDTreeFlann(obstacle_cloud)
    min_dist = float('inf')

    for pos in link_positions:
        [k, idx, dists] = kdtree.search_knn_vector_3d(pos, 1)
        if k > 0:
            min_dist = min(min_dist, np.sqrt(dists[0]))

    return min_dist

def joint_callback(self, msg):
    for name, position in zip(msg.name, msg.position):
        if name in self.dh_joints:
            idx = self.dh_joints.index(name)
            angle = position
            if self.zero_joints[idx]:
                self.real_joint_angles[idx] = 0.0
            else:
                self.real_joint_angles[idx] = -angle if self.invert_joints[idx] else angle
    self.current_joint_angles = self.real_joint_angles.copy()
    self.build_robot_model()

def tcp_callback(self, msg):
    pose = msg.pose
    self.real_tcp_position = np.array([pose.position.x * 1000, pose.position.y * 1000,
pose.position.z * 1000])
    quat = [pose.orientation.x, pose.orientation.y, pose.orientation.z, pose.orientation.w]
    self.real_tcp_orientation = Rotation.from_quat(quat).as_matrix()

def lidar_callback(self, msg):
    points_list = list(pc2.read_points(msg, field_names=("x", "y", "z"), skip_nans=True))
    if not points_list:
        return
    points = np.array([[p['x'], p['y'], p['z']] for p in points_list]) * 1000
    self.closest_points.points = o3d.utility.Vector3dVector(points)
    self.last_lidar_time = time.time()
    self.lidar_count += 1

def update_lidar_frequency(self):
    current_time = time.time()
    if current_time > self.last_lidar_time + 1.0:

```



```

        self.get_logger().warn(f"LiDAR: данные устарели, последнее обновление
{current_time - self.last_lidar_time:.2f} сек назад")
        self.lidar_frequency = self.lidar_count
        self.lidar_count = 0

def attention_callback(self, msg):
    """Обработка сообщений из топика /attention."""
    if msg.data: # Если обнаружено касание или приближение
        self.get_logger().warn("Обнаружено касание или приближение к препятствию,
остановка робота")
        self.is_paused = True
        # Остановка робота путем повторной отправки текущих углов
        for i, angle in enumerate(self.current_joint_angles):
            cmd = -angle if self.invert_joints[i] else angle
            msg = Float64(data=cmd)
            self.joint_pubs[i].publish(msg)
        # Пересчет траектории из текущего положения
        self.get_logger().info("Пересчет траектории из текущего положения")
        trajectory, final_angles = self.potential_field_plan(self.current_joint_angles)
        self.publish_trajectory(trajectory)
        self.is_paused = False # Возобновляем движение после пересчета
    else:
        self.is_paused = False # Разрешаем движение, если нет препятствий

def wait_for_initial_data(self):
    timeout = 10.0
    start_time = time.time()
    while not np.any(self.real_joint_angles) and (time.time() - start_time < timeout):
        rclpy.spin_once(self, timeout_sec=0.1)
    while not self.closest_points.has_points() and (time.time() - start_time < timeout):
        rclpy.spin_once(self, timeout_sec=0.1)

def publish_robot_model(self):
    robot_transform = create_transform_matrix(
        self.get_parameter('robot_offset_x').value, self.get_parameter('robot_offset_y').value,
        self.get_parameter('robot_offset_z').value, self.get_parameter('robot_roll').value,
        self.get_parameter('robot_pitch').value, self.get_parameter('robot_yaw').value
    )

    if len(self.combined_cloud.points) > 0:
        points = np.asarray(self.combined_cloud.points)
        points_homogeneous = np.hstack((points, np.ones((points.shape[0], 1))))
        transformed_points = (robot_transform @ points_homogeneous.T).T[:, :3]
        color_uint32 = np.uint32(255)
        rgb = np.full((transformed_points.shape[0], 1), color_uint32.view(np.float32))
        points_with_rgb = np.hstack((transformed_points / 1000.0, rgb))
        header = Header(stamp=self.get_clock().now().to_msg(), frame_id="base_link")
        fields = [pc2.PointField(name=n, offset=i*4, datatype=pc2.PointField.FLOAT32,
count=1) for i, n in enumerate(["x", "y", "z", "rgb"])]
        cloud_msg = pc2.create_cloud(header, fields, points_with_rgb)
        self.robot_model_pub.publish(cloud_msg)

```

```

marker = Marker()
marker.header = Header(stamp=self.get_clock().now().to_msg(), frame_id="base_link")
marker.ns = "robot_axes"
marker.id = 0
marker.type = Marker.LINE_LIST
marker.action = Marker.ADD
marker.scale.x = 0.01
marker.color.a = 1.0
marker.color.b = 1.0
for axis_lines in self.robot_model:
    points = np.asarray(axis_lines.points)
    points_homogeneous = np.hstack((points, np.ones((points.shape[0], 1))))
    transformed_points = (robot_transform @ points_homogeneous.T).T[:, :3]
    for i in range(0, len(transformed_points), 2):
        marker.points.append(Point(x=transformed_points[i][0], y=transformed_points[i][1],
z=transformed_points[i][2]))
        marker.points.append(Point(x=transformed_points[i+1][0],
y=transformed_points[i+1][1], z=transformed_points[i+1][2]))
    self.marker_pub.publish(marker)

def publish_trajectory(self, trajectory):
    points = []
    for angles in trajectory:
        P, _ = self.forward_kinematics(angles)[:2]
        points.append(P / 1000.0)
    points = np.array(points, dtype=np.float32)
    header = Header(stamp=self.get_clock().now().to_msg(), frame_id="base_link")
    cloud_msg = pc2.create_cloud_xyz32(header, points)
    self.trajectory_pub.publish(cloud_msg)

def publish_all_points(self, all_points):
    points = []
    for angles in all_points:
        P, _ = self.forward_kinematics(angles)[:2]
        points.append(P / 1000.0)
    points = np.array(points, dtype=np.float32)
    header = Header(stamp=self.get_clock().now().to_msg(), frame_id="base_link")
    cloud_msg = pc2.create_cloud_xyz32(header, points)
    self.all_points_pub.publish(cloud_msg)

def control_loop(self):
    if self.is_paused:
        self.get_logger().debug("Контроллер приостановлен из-за препятствия")
        return

    rclpy.spin_once(self, timeout_sec=0.01)
    self.publish_robot_model()

    if self.target_position is None:
        return

    current_time = time.time()

```

```

if current_time - self.last_lidar_time > 2.0:
    self.get_logger().warn(f"Control: данные LiDAR устарели ({current_time -
self.last_lidar_time:.2f} сек)")

if not self.check_reachability(self.target_position):
    self.get_logger().warn("Control: цель недостижима")
    self.target_position = None
    return

trajectory, final_angles = self.potential_field_plan(self.current_joint_angles)
self.publish_trajectory(trajectory)

P_final, _, _ = self.forward_kinematics(final_angles)
dist_to_target = np.linalg.norm(P_final - self.target_position)

if dist_to_target < 200.0:
    try:
        self.get_logger().info(f"Переход к ИК: расстояние до цели={dist_to_target:.2f} мм")
        goal_angles = self.inverse_kinematics(self.target_position, self.target_orientation)
        trajectory = np.vstack((trajectory, goal_angles))
        self.publish_trajectory(trajectory)
    except ValueError as e:
        self.get_logger().error(f"Ошибка ИК: {e}")
        self.target_position = None
        return

tolerance = 0.05
max_wait = 0.03
for step, angles in enumerate(trajectory):
    if self.is_paused: # Прерываем выполнение траектории, если обнаружено
препятствие
        self.get_logger().info(f"Шаг {step}: выполнение траектории прервано из-за
препятствия")
        break
    for i, angle in enumerate(angles):
        cmd = -angle if self.invert_joints[i] else angle
        msg = Float64(data=cmd)
        self.joint_pubs[i].publish(msg)
    self.current_joint_angles = angles.copy()

start_time = time.time()
while time.time() - start_time < max_wait:
    rclpy.spin_once(self, timeout_sec=0.02)
    if np.all(np.abs(self.current_joint_angles - angles) < tolerance):
        break
    time.sleep(0.02)
else:
    self.get_logger().warn(f"Шаг {step}: тайм-аут, углы не достигнуты")

P, _ = self.forward_kinematics(angles)[:2]
self.get_logger().info(f"Шаг {step}: позиция={P}")

```

```

    if not self.is_paused:
        self.target_position = None
        self.get_logger().info("Цель достигнута")

    def set_target(self, position, orientation):
        self.target_position = np.array(position)
        self.target_orientation = np.array(orientation)
        self.get_logger().info(f"Цель установлена: позиция={position},
ориентация=\n{orientation}")

def main():
    rclpy.init()
    controller = RobotController()
    controller.set_target([1000, 800, 1000], np.eye(3))
    try:
        rclpy.spin(controller)
    except KeyboardInterrupt:
        controller.get_logger().info("Прерывание")
    finally:
        controller.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

ПРИЛОЖЕНИЕ Ц

Листинг программы фильтрации данных LiDAR

```
import open3d as o3d
import numpy as np
import xml.etree.ElementTree as ET
from scipy.spatial.transform import Rotation
import os
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import PointCloud2, JointState
from std_msgs.msg import Header
import sensor_msgs_py.point_cloud2 as pc2

BASE_MESH_DIR = "/home/andrei/rviz_test_ws/src/robot_rviz/meshes"

def pose_to_transform(pose_str):
    pose = [float(x) for x in pose_str.split()]
    pos = pose[0:3]
    rot = pose[3:6]
    rot_mat = Rotation.from_euler('xyz', rot).as_matrix()
    transform = np.eye(4)
    transform[:3, :3] = rot_mat
    transform[:3, 3] = pos
    return transform

def clean_stl_path(uri):
    cleaned = uri.replace("file://", "").replace("$ (find robot_rviz)/meshes/", "")
    return cleaned.strip()

def create_axis_lines(joint_transform, axis, offset, length=0.1):
    offset = np.array(offset, dtype=float)
    if offset.shape != (3,):
        raise ValueError(f'Offset должен быть вектором [x, y, z], получено: {offset}')

    origin = offset
    points = np.array([
        origin,
        origin + axis * length
    ])
    points_homogeneous = np.hstack((points, np.ones((2, 1)))) # Исправлено: hstack вместо
    hstack
    points_transformed = (joint_transform @ points_homogeneous.T).T
    points_transformed = points_transformed[:, :3]
    lines = [[0, 1]]
    colors = [[0, 0, 1]] # Синий цвет
    line_set = o3d.geometry.LineSet()
    line_set.points = o3d.utility.Vector3dVector(points_transformed)
    line_set.lines = o3d.utility.Vector2iVector(lines)
    line_set.colors = o3d.utility.Vector3dVector(colors)
```

```

return line_set

class LidarProcessor(Node):
    def __init__(self):
        super().__init__('lidar_processor')
        self.links = {}
        self.joints = {}
        self.robot_model = []
        self.combined_cloud = o3d.geometry.PointCloud()

        self.lidar1_sub = self.create_subscription(PointCloud2, '/lidar_1/points',
self.lidar_callback(1), 5)
        self.lidar2_sub = self.create_subscription(PointCloud2, '/lidar_2/points',
self.lidar_callback(2), 5)
        self.joint_sub = self.create_subscription(JointState, '/aggregated_joint_states',
self.joint_state_callback, 5)

        self.points_pub = self.create_publisher(PointCloud2, '/points', 5)

        self.lidar1_cloud = o3d.geometry.PointCloud()
        self.lidar2_cloud = o3d.geometry.PointCloud()
        self.lidar_data_received = [False, False]

        self.dh_joints = [
            "J_1_Base_Stol",
            "J_2_Stol_Plecho_1",
            "J_3_Plecho_1_and_2",
            "J_4_Plecho_2_and_3",
            "J_5_Plecho_3_and_4",
            "J_6_Plecho_4_Kist"
        ]
        self.current_joint_angles = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]

        self.invert_joints = [False, True, True, False, True, False]
        self.zero_joints = [True, False, False, False, False, False]

        self.load_sdf()
        self.build_robot_model()

        self.lidar_transforms = {
            "lidar_link_1": self.compute_link_transform("lidar_link_1") @
self.offset_transforms.get("lidar_link_1", np.eye(4)),
            "lidar_link_2": self.compute_link_transform("lidar_link_2") @
self.offset_transforms.get("lidar_link_2", np.eye(4))
        }

        self.lidar_base_correction = "-0.32744256676702599 -0.16 -0.08 0 -1.5707926536057681
1.5707926536057681"
        self.lidar_base_transform = pose_to_transform(self.lidar_base_correction)

        self.lidar_final_transforms = {
            "lidar_link_1": self.lidar_base_transform @ self.lidar_transforms["lidar_link_1"],

```



```

        "lidar_link_2": self.lidar_base_transform @ self.lidar_transforms["lidar_link_2"]
    }

    lidar1_pos = self.lidar_final_transforms["lidar_link_1"][:3, 3]
    lidar2_pos = self.lidar_final_transforms["lidar_link_2"][:3, 3]
    distance = np.linalg.norm(lidar1_pos - lidar2_pos)

    self.timer = self.create_timer(0.000001, self.process_and_publish)

def joint_state_callback(self, msg):
    for name, position in zip(msg.name, msg.position):
        if name in self.dh_joints:
            index = self.dh_joints.index(name)
            angle = np.rad2deg(position)
            if self.zero_joints[index]:
                self.current_joint_angles[index] = 0.0
            else:
                self.current_joint_angles[index] = -angle if self.invert_joints[index] else angle
    self.build_robot_model()

def load_sdf(self):
    tree = ET.parse("/home/andrei/rviz_test_ws/src/robot_rviz/urdf/Robot_ai.sdf")
    root = tree.getroot()

    for link in root.findall("./link"):
        link_name = link.get("name")
        pose_elem = link.find("pose")
        pose = pose_elem.text if pose_elem is not None else "0 0 0 0 0 0"
        visual = link.find("./visual/geometry/mesh/uri")
        collision = link.find("./collision/geometry/mesh/uri")
        stl_path = None
        if visual is not None:
            relative_path = clean_stl_path(visual.text)
            stl_path = os.path.join(BASE_MESH_DIR, relative_path)
        elif collision is not None:
            relative_path = clean_stl_path(collision.text)
            stl_path = os.path.join(BASE_MESH_DIR, relative_path)
        self.links[link_name] = {"pose": pose, "stl_path": stl_path, "transform": np.eye(4)}

    for joint in root.findall("./joint"):
        joint_name = joint.get("name")
        parent = joint.find("parent").text
        child = joint.find("child").text
        pose_elem = joint.find("pose")
        pose = pose_elem.text if pose_elem is not None else "0 0 0 0 0 0"
        axis_elem = joint.find("axis/xyz")
        axis = axis_elem.text if axis_elem is not None else "0 0 1"
        if "lidar" not in child.lower():
            self.joints[joint_name] = {
                "parent": parent,
                "child": child,
                "pose": pose,

```

```

        "axis": [float(x) for x in axis.split()]
    }
    else:
        self.joints[joint_name] = {"parent": parent, "child": child, "pose": pose}

self.link_offsets = {
    "Base_Robot_Link": "0 0 0 0 0 0",
    "Povorotniy_Stol_Link": "-0.32744256676702599 -0.08 -0.32241999999999998 0 0 0",
    "Plecho_1_Link": "0.32744256676702599 -0.32241999999999998 0.08 0 0 0",
    "Plecho_2_Link": "-0.08 0.32744256676702599 -0.32241999999999998 0 0 0",
    "Plecho_3_Link": "-0.32744256676702599 0.08 0.32241999999999998 0 0 0",
    "Plecho_4_Link": "-0.32744256676702599 0.32241999999999998 0.08 0 0 0",
    "Kist_Link": "-0.32744256676702599 0.08 -0.32744256676702599 0 0 0"
}

self.offset_transforms = {link_name: pose_to_transform(pose) for link_name, pose in
self.link_offsets.items()}

self.axis_offsets = {
    "J_1_Base_Stol": [-0.32744256676702599, -0.32241999999999998, -0.08],
    "J_2_Stol_Plecho_1": [0.32744256676702599, -0.32241999999999998, 0.08],
    "J_3_Plecho_1_and_2": [-0.08, 0.32744256676702599, -0.32744256676702599],
    "J_4_Plecho_2_and_3": [-0.32744256676702599, -0.08, 0.32744256676702599],
    "J_5_Plecho_3_and_4": [-0.32744256676702599, 0.32241999999999998, 0.08],
    "J_6_Plecho_4_Kist": [-0.32744256676702599, -0.32744256676702599, -0.08]
}

def compute_link_transform(self, link_name, visited=None):
    if visited is None:
        visited = set()
    if link_name in visited:
        return np.eye(4)
    visited.add(link_name)

    link = self.links[link_name]
    link_pose = pose_to_transform(link["pose"])
    full_transform = link_pose

    for joint_name, joint in self.joints.items():
        if joint["child"] == link_name:
            parent_name = joint["parent"]
            parent_transform = self.compute_link_transform(parent_name, visited.copy())
            joint_transform = pose_to_transform(joint["pose"])

            if "axis" in joint:
                try:
                    joint_index = self.dh_joints.index(joint_name)
                    angle_deg = self.current_joint_angles[joint_index]
                    angle_rad = np.deg2rad(angle_deg)
                except ValueError:
                    angle_rad = 0.0
                axis = np.array(joint["axis"])
                rot_axis = Rotation.from_rotvec(angle_rad * axis).as_matrix()

```

```

        if joint_name in self.axis_offsets:
            offset = np.array(self.axis_offsets[joint_name])
            T_offset = np.eye(4)
            T_offset[:3, 3] = offset
            T_offset_inv = np.eye(4)
            T_offset_inv[:3, 3] = -offset
            R_axis_4x4 = np.eye(4)
            R_axis_4x4[:3, :3] = rot_axis
            T_rot = T_offset @ R_axis_4x4 @ T_offset_inv
            joint_transform = joint_transform @ T_rot
        else:
            joint_transform[:3, :3] = joint_transform[:3, :3] @ rot_axis

    full_transform = parent_transform @ joint_transform @ link_pose
    break
return full_transform

def compute_joint_transform(self, joint):
    parent_name = joint["parent"]
    parent_transform = self.compute_link_transform(parent_name)
    joint_transform = pose_to_transform(joint["pose"])
    return parent_transform @ joint_transform

def build_robot_model(self):
    self.combined_cloud = o3d.geometry.PointCloud()
    self.robot_model = []
    axis_geometries = []

    for link_name, link_data in self.links.items():
        stl_path = link_data["stl_path"]
        if stl_path and os.path.exists(stl_path):
            mesh = o3d.io.read_triangle_mesh(stl_path)
            if not mesh.is_empty():
                transform = self.compute_link_transform(link_name)
                if link_name in self.offset_transforms:
                    transform = transform @ self.offset_transforms[link_name]
                mesh.transform(transform)
                cloud = mesh.sample_points_uniformly(number_of_points=1000)
                self.combined_cloud += cloud

    if len(self.combined_cloud.points) > 0:
        self.combined_cloud.paint_uniform_color([0.5, 0.5, 0.5])

    for joint_name, joint in self.joints.items():
        if "axis" in joint:
            joint_transform = self.compute_joint_transform(joint)
            axis = np.array(joint["axis"])
            offset = self.axis_offsets.get(joint_name, [0.0, 0.0, 0.0])
            axis_lines = create_axis_lines(joint_transform, axis, offset, length=0.1)
            axis_geometries.append(axis_lines)

```

```

self.robot_model = axis_geometries

def lidar_callback(self, lidar_id):
    def callback(msg):
        self.lidar_data_received[lidar_id - 1] = True
        points = []
        for p in pc2.read_points(msg, field_names=("x", "y", "z"), skip_nans=True):
            points.append([p[0], p[1], p[2]])
        cloud = o3d.geometry.PointCloud()
        cloud.points = o3d.utility.Vector3dVector(np.array(points))
        if lidar_id == 1:
            self.lidar1_cloud = cloud
        else:
            self.lidar2_cloud = cloud
    return callback

def process_and_publish(self):
    if not all(self.lidar_data_received):
        return

    lidar1_cloud_transformed = o3d.geometry.PointCloud()
    lidar1_cloud_transformed.points =
o3d.utility.Vector3dVector(np.asarray(self.lidar1_cloud.points))
    lidar1_cloud_transformed.transform(self.lidar_final_transforms["lidar_link_1"])

    lidar2_cloud_transformed = o3d.geometry.PointCloud()
    lidar2_cloud_transformed.points =
o3d.utility.Vector3dVector(np.asarray(self.lidar2_cloud.points))
    lidar2_cloud_transformed.transform(self.lidar_final_transforms["lidar_link_2"])

    lidar_combined_cloud = lidar1_cloud_transformed + lidar2_cloud_transformed

    robot_kdtree = o3d.geometry.KDTreeFlann(self.combined_cloud)
    threshold = 0.01
    filtered_indices = []

    lidar_points = np.asarray(lidar_combined_cloud.points)
    for i, point in enumerate(lidar_points):
        [k, idx, dist] = robot_kdtree.search_knn_vector_3d(point, 1)
        if k > 0 and dist[0] > threshold:
            filtered_indices.append(i)

    filtered_cloud = lidar_combined_cloud.select_by_index(filtered_indices)

    if len(filtered_cloud.points) > 0:
        o3d.io.write_point_cloud("filtered_lidar_cloud.pcd", filtered_cloud)

    points = np.asarray(filtered_cloud.points)
    header = Header()
    header.stamp = rclpy.time.Time().to_msg()

```

```

        header.frame_id = "base_link"
        cloud_msg = pc2.create_cloud_xyz32(header, points)
        self.points_pub.publish(cloud_msg)

def main(args=None):
    rclpy.init(args=args)
    lidar_processor = LidarProcessor()

    try:
        rclpy.spin(lidar_processor)
    except KeyboardInterrupt:
        lidar_processor.get_logger().info("Программа завершена пользователем")
    finally:
        lidar_processor.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

ПРИЛОЖЕНИЕ Э

Листинг программы вращения данных LiDAR

```
import numpy as np
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState, PointCloud2
from std_msgs.msg import Header
import sensor_msgs_py.point_cloud2 as pc2
from scipy.spatial.transform import Rotation

def create_transform_matrix(offset_x, offset_y, offset_z, roll, pitch, yaw):
    """Создание матрицы трансформации 4x4 на основе смещения и углов Эйлера"""
    transform = np.eye(4)
    rot = Rotation.from_euler('xyz', [roll, pitch, yaw]).as_matrix()
    transform[:3, :3] = rot
    transform[:3, 3] = [offset_x, offset_y, offset_z]
    return transform

def create_rotation_matrix_z(angle_rad):
    """Создание матрицы вращения вокруг оси Z на заданный угол в радианах"""
    transform = np.eye(4)
    rot = Rotation.from_euler('z', angle_rad).as_matrix()
    transform[:3, :3] = rot
    return transform

class LidarProcessor(Node):
    def __init__(self):
        super().__init__('lidar_processor')
        self.lidar_cloud_raw = None

        # Подписка на топик лидара и суставов
        self.lidar_sub = self.create_subscription(PointCloud2, '/points', self.lidar_callback, 5)
        self.joint_sub = self.create_subscription(JointState, '/aggregated_joint_states',
        self.joint_state_callback, 5)

        # Публикатор для облака точек лидара
        self.lidar_pub = self.create_publisher(PointCloud2, '/lidar_points', 5)

        # Текущий угол сустава J_1_Base_Stol
        self.j1_angle_deg = 0.0

        # Параметры для корректировки положения лидара
        self.declare_parameter('lidar_offset_x', -0.32744256676702599)
        self.declare_parameter('lidar_offset_y', -0.32241999999999998)
        self.declare_parameter('lidar_offset_z', 0.0)
        self.declare_parameter('lidar_roll', 0.0) # в радианах
        self.declare_parameter('lidar_pitch', 0.0) # в радианах
        self.declare_parameter('lidar_yaw', 3.1415926535746808) # в радианах
```



```

def joint_state_callback(self, msg):
    """Обновление угла сустава J_1_Base_Stol из топика /aggregated_joint_states"""
    for name, position in zip(msg.name, msg.position):
        if name == "J_1_Base_Stol":
            self.j1_angle_deg = np.rad2deg(position)
            break

def lidar_callback(self, msg):
    """Обработка данных с лидара из топика /points и немедленная публикация"""
    points = []
    for p in pc2.read_points(msg, field_names=("x", "y", "z"), skip_nans=True):
        points.append([p[0], p[1], p[2]])
    self.lidar_cloud_raw = np.array(points)

    # Немедленная публикация при получении новых данных
    self.publish_lidar_points()

def publish_lidar_points(self):
    """Публикация облака точек лидара с учетом корректировки и противоположного вращения"""
    if self.lidar_cloud_raw is None or len(self.lidar_cloud_raw) == 0:
        return

    # Получение параметров корректировки для лидара
    lidar_offset_x = self.get_parameter('lidar_offset_x').get_parameter_value().double_value
    lidar_offset_y = self.get_parameter('lidar_offset_y').get_parameter_value().double_value
    lidar_offset_z = self.get_parameter('lidar_offset_z').get_parameter_value().double_value
    lidar_roll = self.get_parameter('lidar_roll').get_parameter_value().double_value
    lidar_pitch = self.get_parameter('lidar_pitch').get_parameter_value().double_value
    lidar_yaw = self.get_parameter('lidar_yaw').get_parameter_value().double_value

    # Создание матрицы трансформации для лидара
    lidar_transform = create_transform_matrix(lidar_offset_x, lidar_offset_y, lidar_offset_z,
                                              lidar_roll, lidar_pitch, lidar_yaw)

    # Создание матрицы противоположного вращения относительно J_1_Base_Stol
    j1_angle_rad = np.deg2rad(self.j1_angle_deg)
    counter_rotation = create_rotation_matrix_z(j1_angle_rad) # Исправлено на
    противоположное вращение

    # Применение трансформаций к облаку точек лидара
    lidar_points = self.lidar_cloud_raw
    lidar_points_homogeneous = np.hstack((lidar_points, np.ones((lidar_points.shape[0], 1))))
    transformed_lidar_points = (lidar_transform @ lidar_points_homogeneous.T).T[:, :3]
    transformed_lidar_points_homogeneous = np.hstack((transformed_lidar_points,
    np.ones((transformed_lidar_points.shape[0], 1))))
    final_lidar_points = (counter_rotation @ transformed_lidar_points_homogeneous.T).T[:, :3]

    # Добавление красного цвета
    rgb = np.zeros((final_lidar_points.shape[0], 1), dtype=np.float32)
    rgb.fill(16711680) # Красный в формате RGB как float32 (255, 0, 0)
    points_with_rgb = np.hstack((final_lidar_points, rgb))

```

```

# Публикация
header = Header()
header.stamp = self.get_clock().now().to_msg()
header.frame_id = "base_link"
fields = [
    pc2.PointField(name="x", offset=0, datatype=pc2.PointField.FLOAT32, count=1),
    pc2.PointField(name="y", offset=4, datatype=pc2.PointField.FLOAT32, count=1),
    pc2.PointField(name="z", offset=8, datatype=pc2.PointField.FLOAT32, count=1),
    pc2.PointField(name="rgb", offset=12, datatype=pc2.PointField.FLOAT32, count=1),
]
lidar_cloud_msg = pc2.create_cloud(header, fields, points_with_rgb)
self.lidar_pub.publish(lidar_cloud_msg)

def main(args=None):
    rclpy.init(args=args)
    lidar_processor = LidarProcessor()

    try:
        rclpy.spin(lidar_processor)
    except KeyboardInterrupt:
        lidar_processor.get_logger().info("Программа завершена пользователем")
    finally:
        lidar_processor.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

ПРИЛОЖЕНИЕ Ю

Листинг программы трансляции состояния звеньев

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState
import numpy as np

class JointStateAggregator(Node):
    def __init__(self):
        super().__init__('joint_state_aggregator')
        # Список суставов в нужном порядке
        self.joint_names = [
            'J_1_Base_Stol',
            'J_2_Stol_Plecho_1',
            'J_3_Plecho_1_and_2',
            'J_4_Plecho_2_and_3',
            'J_5_Plecho_3_and_4',
            'J_6_Plecho_4_Kist'
        ]
        # Словарь для хранения позиций суставов
        self.joint_positions = {name: 0.0 for name in self.joint_names}
        # Подписка на /joint_states
        self.sub = self.create_subscription(
            JointState,
            '/joint_states',
            self.joint_state_callback,
            5
        )
        # Публикация в /aggregated_joint_states
        self.pub = self.create_publisher(JointState, '/aggregated_joint_states', 5)
        self.timer = self.create_timer(0.000001, self.publish_aggregated_state)

    def joint_state_callback(self, msg):
        # Обновляем позиции суставов из входящего сообщения
        for name, pos in zip(msg.name, msg.position):
            if name in self.joint_names:
                self.joint_positions[name] = pos
        self.get_logger().debug(f"Updated joint positions: {self.joint_positions}")

    def publish_aggregated_state(self):
        # Создаём сообщение с текущими позициями всех суставов
        msg = JointState()
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.name = self.joint_names
        msg.position = [self.joint_positions[name] for name in self.joint_names]
        self.pub.publish(msg)
        self.get_logger().debug(f"Published aggregated joint state: {msg.position}")

def main(args=None):
```

```

rclpy.init(args=args)
node = JointStateAggregator()
try:
    rclpy.spin(node)
except KeyboardInterrupt:
    node.get_logger().info("Shutting down")
finally:
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

ПРИЛОЖЕНИЕ Я

Листинг программы трансляции координат инструмента манипулятора

```
import rclpy
from rclpy.node import Node
from tf2_msgs.msg import TFMessage
from geometry_msgs.msg import Transform, PoseStamped
import numpy as np
from math import atan2, asin, pi, sqrt, cos, sin

class TFListener(Node):
    def __init__(self):
        super().__init__('tf_listener')
        # Подписка на /tf
        self.tf_sub = self.create_subscription(
            TFMessage,
            '/tf',
            self.tf_callback,
            10
        )
        # Публикация в /tcp
        self.tcp_pub = self.create_publisher(PoseStamped, '/tcp', 10)
        self.transforms = {}
        # Цепочка преобразований начинается с Povorotniy_Stol_Link
        self.chain = [
            ('base_link', 'Povorotniy_Stol_Link'),
            ('Povorotniy_Stol_Link', 'Plecho_1_Link'),
            ('Plecho_1_Link', 'Plecho_2_Link'),
            ('Plecho_2_Link', 'Plecho_3_Link'),
            ('Plecho_3_Link', 'Plecho_4_Link'),
            ('Plecho_4_Link', 'Kist_Link')
        ]

    def quaternion_to_euler(self, x, y, z, w):
        """Преобразование кватерниона в углы Эйлера (roll, pitch, yaw)."""
        sinr_cosp = 2 * (w * x + y * z)
        cosr_cosp = 1 - 2 * (x * x + y * y)
        roll = atan2(sinr_cosp, cosr_cosp)

        sinp = 2 * (w * y - z * x)
        pitch = asin(np.clip(sinp, -1.0, 1.0))

        siny_cosp = 2 * (w * z + x * y)
        cosy_cosp = 1 - 2 * (y * y + z * z)
        yaw = atan2(siny_cosp, cosy_cosp)

        return roll, pitch, yaw

    def euler_to_quaternion(self, roll, pitch, yaw):
        """Преобразование углов Эйлера в кватернион."""
```

```

cy = cos(yaw * 0.5)
sy = sin(yaw * 0.5)
cp = cos(pitch * 0.5)
sp = sin(pitch * 0.5)
cr = cos(roll * 0.5)
sr = sin(roll * 0.5)

w = cr * cp * cy + sr * sp * sy
x = sr * cp * cy - cr * sp * sy
y = cr * sp * cy + sr * cp * sy
z = cr * cp * sy - sr * sp * cy

return [x, y, z, w]

def quaternion_multiply(self, q1, q2):
    """Умножение двух кватернионов [x, y, z, w]."""
    x1, y1, z1, w1 = q1
    x2, y2, z2, w2 = q2
    w = w1 * w2 - x1 * x2 - y1 * y2 - z1 * z2
    x = w1 * x2 + x1 * w2 + y1 * z2 - z1 * y2
    y = w1 * y2 - x1 * z2 + y1 * w2 + z1 * x2
    z = w1 * z2 + x1 * y2 - y1 * x2 + z1 * w2
    return [x, y, z, w]

def compose_transform(self, trans1, trans2):
    """Композиция двух преобразований."""
    t1 = np.array([trans1.translation.x,
                    trans1.translation.y,
                    trans1.translation.z])
    t2 = np.array([trans2.translation.x,
                    trans2.translation.y,
                    trans2.translation.z])

    q1 = [trans1.rotation.x,
           trans1.rotation.y,
           trans1.rotation.z,
           trans1.rotation.w]
    q2 = [trans2.rotation.x,
           trans2.rotation.y,
           trans2.rotation.z,
           trans2.rotation.w]

    q_total = self.quaternion_multiply(q1, q2)

    q1_conj = [-q1[0], -q1[1], -q1[2], q1[3]]
    t2_quat = [t2[0], t2[1], t2[2], 0.0]
    t2_rot = self.quaternion_multiply(self.quaternion_multiply(q1, t2_quat), q1_conj)[:3]
    t_total = t1 + np.array(t2_rot)

    return t_total, q_total

def tf_callback(self, msg):

```



```

for transform in msg.transforms:
    key = (transform.header.frame_id, transform.child_frame_id)
    self.transforms[key] = transform

all_transforms_available = all(key in self.transforms for key in self.chain)

if all_transforms_available:
    # Инициализация начальными значениями для Povorotniy_Stol_Link
    total_translation = np.array([0.0, 0.0, 0.0])
    total_quaternion = np.array([0.0, 0.0, 0.0, 1.0]) # Единичный кватернион

    for i, (parent, child) in enumerate(self.chain):
        transform = self.transforms[(parent, child)]
        if i == 0:
            # Для первого преобразования берём его как есть
            total_translation = np.array([
                transform.translation.x,
                transform.translation.y,
                transform.translation.z
            ])
            total_quaternion = np.array([
                transform.rotation.x,
                transform.rotation.y,
                transform.rotation.z,
                transform.rotation.w
            ])
        else:
            # Последующие преобразования комбинируем
            t_total, q_total = self.compose_transform(
                self.TransformWrapper(total_translation, total_quaternion),
                transform
            )
            total_translation = t_total
            total_quaternion = q_total

    # Преобразование кватерниона в углы Эйлера для лога
    roll, pitch, yaw = self.quaternion_to_euler(
        total_quaternion[0],
        total_quaternion[1],
        total_quaternion[2],
        total_quaternion[3]
    )

    # Публикация в /tcp с z как вертикальной осью, округление до 0.00001 без
    экспонент
    tcp_msg = PoseStamped()
    tcp_msg.header.stamp = self.get_clock().now().to_msg()
    tcp_msg.header.frame_id = 'Povorotniy_Stol_Link'
    tcp_msg.pose.position.x = float(f'{total_translation[0]:.4f}') # x остаётся x
    tcp_msg.pose.position.y = float(f'{total_translation[1]:.4f}') # y остаётся y
    (горизонталь)
    tcp_msg.pose.position.z = float(f'{total_translation[2]:.4f}') # z как вертикаль

```

```

tcp_msg.pose.orientation.x = float(f'{total_quaternion[0]:.4f}')
tcp_msg.pose.orientation.y = float(f'{total_quaternion[1]:.4f}')
tcp_msg.pose.orientation.z = float(f'{total_quaternion[2]:.4f}')
tcp_msg.pose.orientation.w = float(f'{total_quaternion[3]:.4f}')
self.tcp_pub.publish(tcp_msg)

class TransformWrapper:
    def __init__(self, translation, quaternion):
        self.translation = Transform().translation
        self.rotation = Transform().rotation
        self.translation.x = float(translation[0])
        self.translation.y = float(translation[1])
        self.translation.z = float(translation[2])
        self.rotation.x = float(quaternion[0])
        self.rotation.y = float(quaternion[1])
        self.rotation.z = float(quaternion[2])
        self.rotation.w = float(quaternion[3])

def main(args=None):
    rclpy.init(args=args)
    node = TFListener()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.get_logger().info("Shutting down")
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

ПРИЛОЖЕНИЕ АА

Листинг программы предупреждения касания

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState, PointCloud2
from std_msgs.msg import Bool
import numpy as np
import open3d as o3d
import sensor_msgs_py.point_cloud2 as pc2

class ForbiddenZoneMonitor(Node):
    def __init__(self):
        super().__init__('forbidden_zone_monitor')

        self.min_safe_distance = 300.0 # Радиус запретной зоны (мм), совпадает с
RobotController
        self.point_threshold = 100     # Порог точек для остановки
        self.link_positions = None     # Позиции звеньев робота

        self.joint_sub = self.create_subscription(JointState, '/aggregated_joint_states',
self.joint_callback, 10)
        self.lidar_sub = self.create_subscription(PointCloud2, '/lidar_points', self.lidar_callback, 10)
        self.attention_pub = self.create_publisher(Bool, '/attention', 10)

        self.joint_angles = np.zeros(6)
        self.dh_joints = [
            "J_1_Base_Stol", "J_2_Stol_Plecho_1", "J_3_Plecho_1_and_2",
            "J_4_Plecho_2_and_3", "J_5_Plecho_3_and_4", "J_6_Plecho_4_Kist"
        ]
        self.invert_joints = [False, True, False, True, True, False]
        self.zero_joints = [False, False, False, False, False, False]

        self.get_logger().info("Монитор запретной зоны запущен")

    def dh_matrix(self, theta, d, a, alpha):
        ct, st = np.cos(theta), np.sin(theta)
        ca, sa = np.cos(alpha), np.sin(alpha)
        return np.array([
            [ct, -st*ca, st*sa, a*ct],
            [st, ct*ca, -ct*sa, a*st],
            [0, sa, ca, d],
            [0, 0, 0, 1]
        ])

    def forward_kinematics(self, angles):
        T = np.eye(4)
        link_positions = []
        theta_total = np.deg2rad([0, 90, 90, 0, 0, -180]) + angles
        dh_params = {
```

```

        'd': [393, 0, 0, 700, 0, 174],
        'a': [0, 700, 0, 0, 0, 0],
        'alpha': np.deg2rad([90, 0, 90, 90, -90, 0])
    }
    for i in range(6):
        T = T @ self.dh_matrix(theta_total[i], dh_params['d'][i], dh_params['a'][i],
dh_params['alpha'][i])
        link_positions.append(T[:3, 3].copy())
    return np.array(link_positions)

def joint_callback(self, msg):
    for name, position in zip(msg.name, msg.position):
        if name in self.dh_joints:
            idx = self.dh_joints.index(name)
            angle = position
            if self.zero_joints[idx]:
                self.joint_angles[idx] = 0.0
            else:
                self.joint_angles[idx] = -angle if self.invert_joints[idx] else angle
        self.link_positions = self.forward_kinematics(self.joint_angles)

def lidar_callback(self, msg):
    if self.link_positions is None:
        return

    points_list = list(pc2.read_points(msg, field_names=("x", "y", "z"), skip_nans=True))
    if not points_list:
        return

    points = np.array([[p['x'] * 1000, p['y'] * 1000, p['z'] * 1000] for p in points_list])
    cloud = o3d.geometry.PointCloud()
    cloud.points = o3d.utility.Vector3dVector(points)
    kdtree = o3d.geometry.KDTreeFlann(cloud)

    num_points_in_zone = 0
    for pos in self.link_positions:
        [k, _, dists] = kdtree.search_radius_vector_3d(pos, self.min_safe_distance)
        num_points_in_zone += k

    if num_points_in_zone > self.point_threshold:
        self.get_logger().warn(f"Обнаружено {num_points_in_zone} точек в запретной зоне,
остановка")
        self.attention_pub.publish(Bool(data=True))
    else:
        self.attention_pub.publish(Bool(data=False))

def main():
    rclpy.init()
    monitor = ForbiddenZoneMonitor()
    try:
        rclpy.spin(monitor)
    except KeyboardInterrupt:

```

```
        monitor.get_logger().info("Прерывание")
    finally:
        monitor.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

ПРИЛОЖЕНИЕ АБ

Листинг программы детектирования касания

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Bool
from gz import transport13 as gz_transport
from gz.msgs11 import contacts_pb2

class CollisionMonitor(Node):
    def __init__(self):
        super().__init__('collision_monitor')

        # Список топиков для детектирования касаний
        self.contact_topics = [
            '/world/empty/model/robot_rviz/link/Kist_Link/sensor/my_contact/contact',
            '/world/empty/model/robot_rviz/link/Plecho_1_Link/sensor/my_contact/contact',
            '/world/empty/model/robot_rviz/link/Plecho_2_Link/sensor/my_contact/contact',
            '/world/empty/model/robot_rviz/link/Plecho_3_Link/sensor/my_contact/contact',
            '/world/empty/model/robot_rviz/link/Plecho_4_Link/sensor/my_contact/contact',
            '/world/empty/model/robot_rviz/link/Povorotniy_Stol_Link/sensor/my_contact/contact'
        ]
        self.contact_received = {topic: False for topic in self.contact_topics}

        # Публишер для топика /attention
        self.attention_pub = self.create_publisher(Bool, '/attention', 10)

        # Инициализация gz-transport для подписки на топики контактов
        try:
            self.gz_node = gz_transport.Node()
            for topic in self.contact_topics:
                success = self.gz_node.subscribe(contacts_pb2.Contacts, topic,
self.gz_contact_callback)
                if success:
                    self.get_logger().info(f"Subscribed to Gazebo contact topic: {topic}")
                else:
                    self.get_logger().error(f"Failed to subscribe to Gazebo contact topic: {topic}")
            except Exception as e:
                self.get_logger().error(f"Failed to initialize gz-transport: {e}")

            self.get_logger().info("Монитор столкновений запущен")

        def gz_contact_callback(self, contacts):
            """Обработка сообщений о контактах из Gazebo."""
            for topic in self.contact_topics:
                self.contact_received[topic] = len(contacts.contact) > 0

            # Проверяем, есть ли столкновение в любом из топиков
            collision_detected = any(self.contact_received.values())
```



```

# Публикуем сообщение в /attention
attention_msg = Bool()
attention_msg.data = collision_detected
self.attention_pub.publish(attention_msg)

if collision_detected:
    self.get_logger().warn("Обнаружено столкновение")
else:
    self.get_logger().debug("Столкновений не обнаружено")

def main():
    rclpy.init()
    monitor = CollisionMonitor()
    try:
        rclpy.spin(monitor)
    except KeyboardInterrupt:
        monitor.get_logger().info("Прерывание")
    finally:
        monitor.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

ПРИЛОЖЕНИЕ АВ

Временные диаграммы

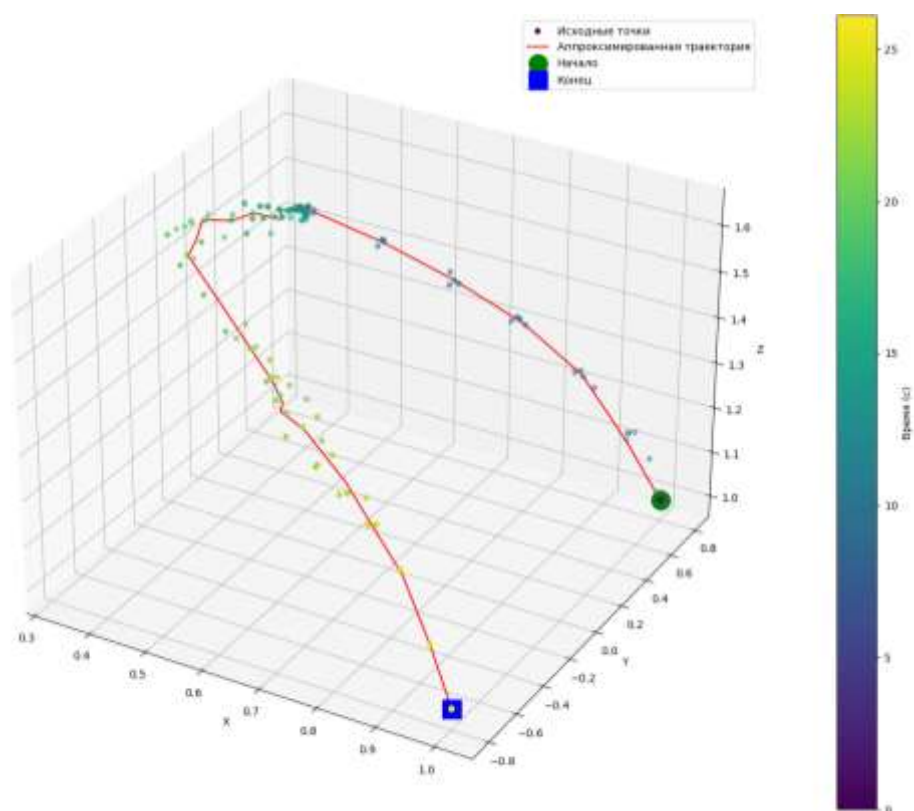


Рисунок Т1 – 3D временная диаграмма для координат инструмента манипулятора

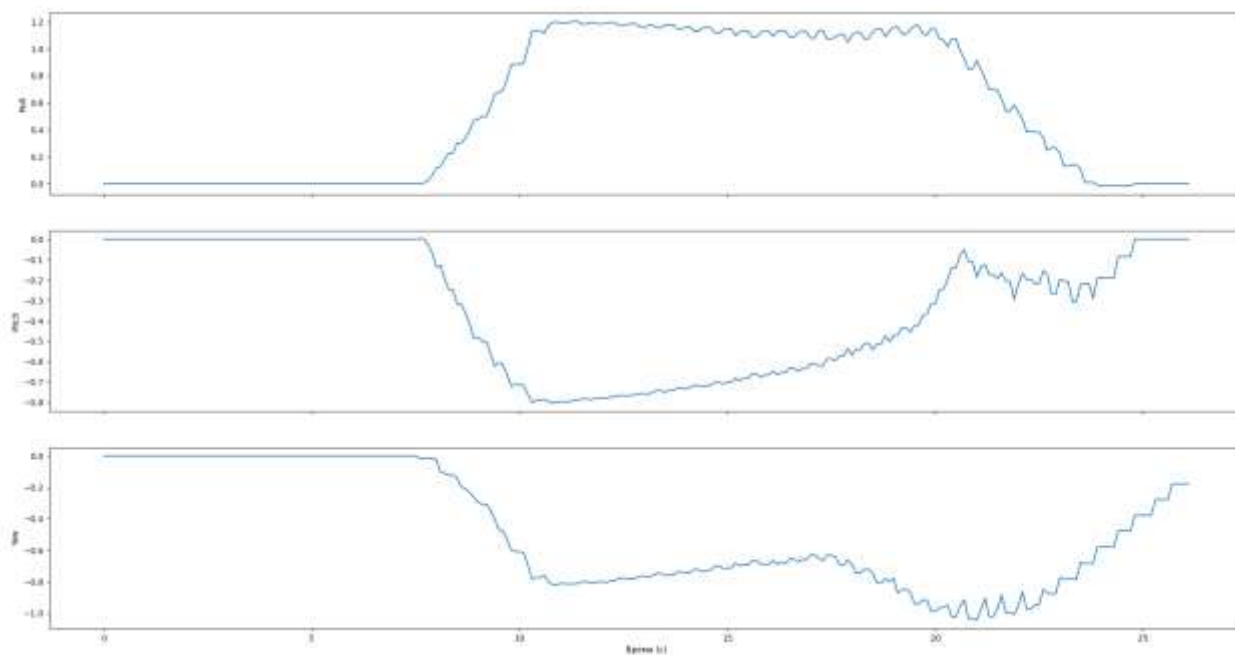


Рисунок Т2 – Временная диаграмма для углов Эйлера инструмента манипулятора

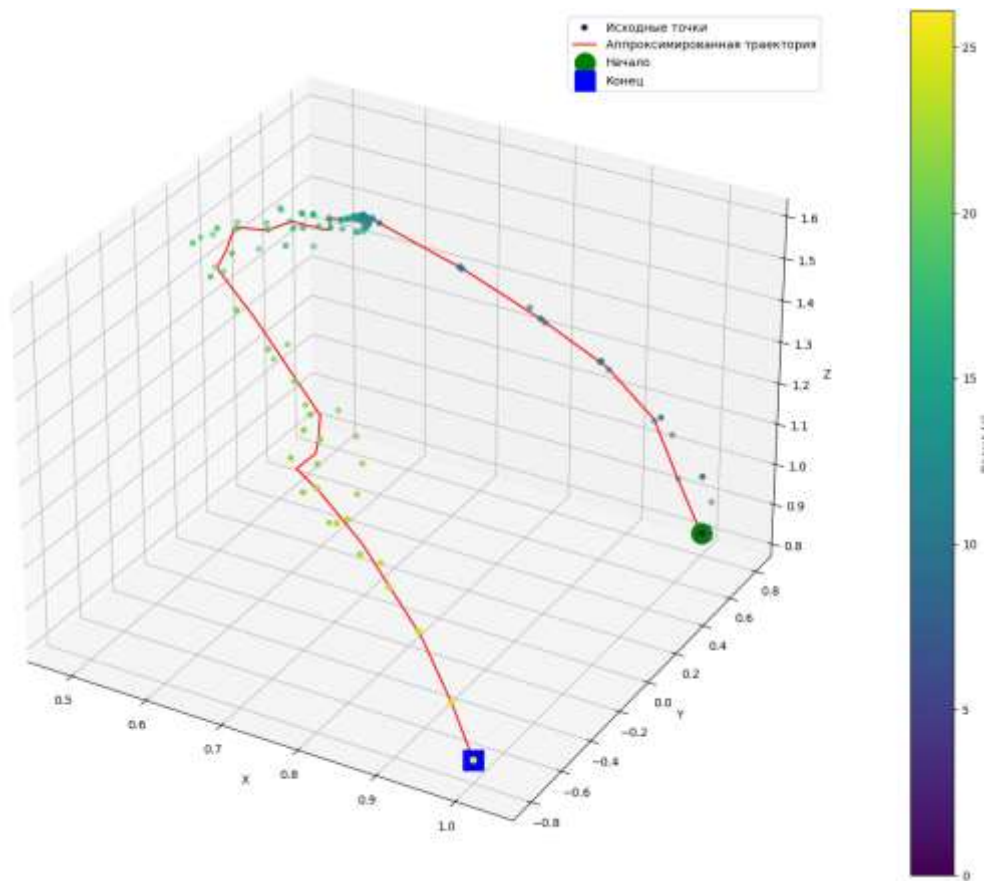


Рисунок Т3 – 3D временная диаграмма для координат пятого сустава манипулятора

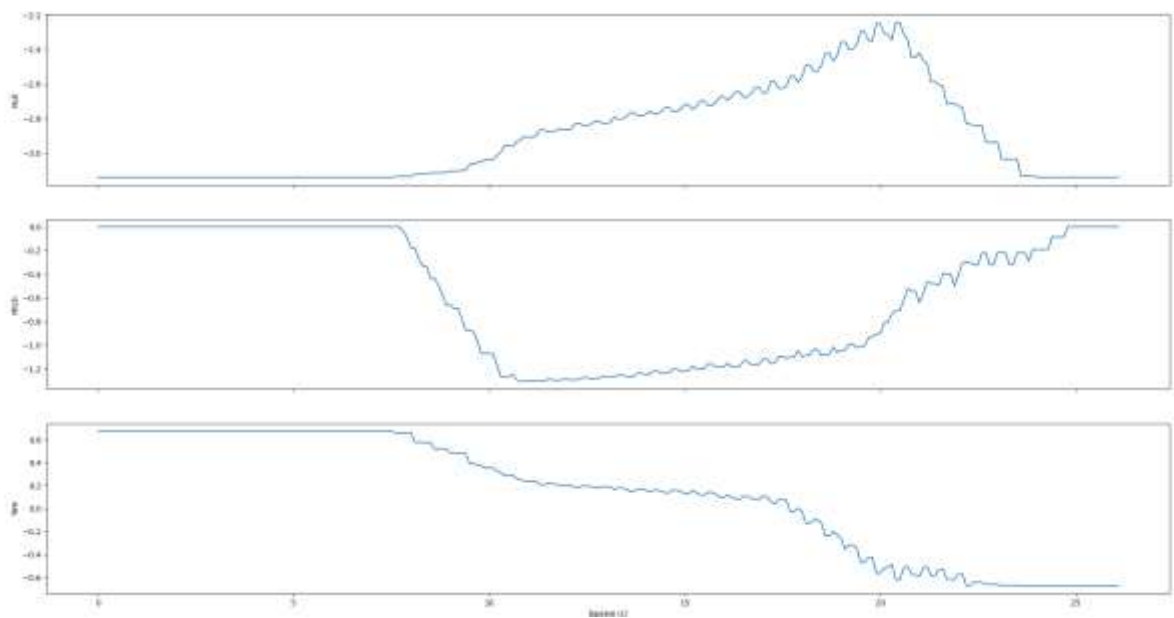


Рисунок Т4 – Временная диаграмма для углов Эйлера пятого сустава манипулятора

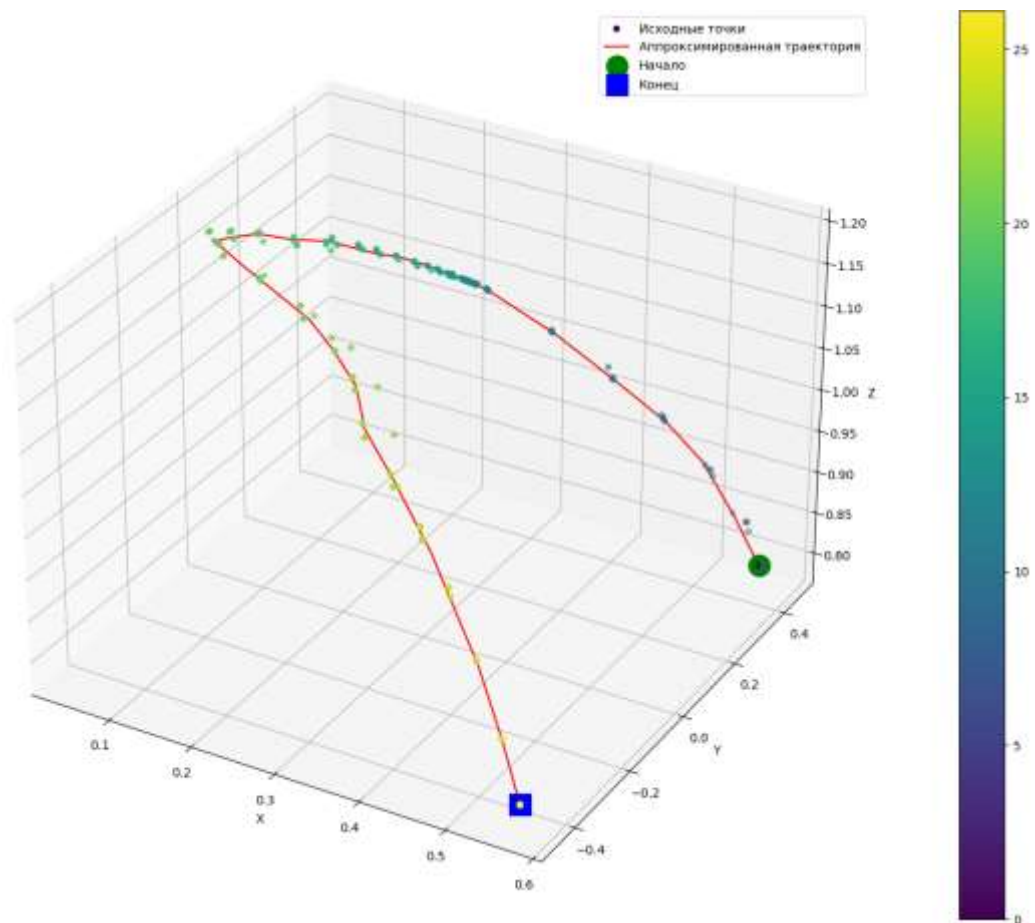


Рисунок Т5 – 3D временная диаграмма для координат четвертого сустава манипулятора

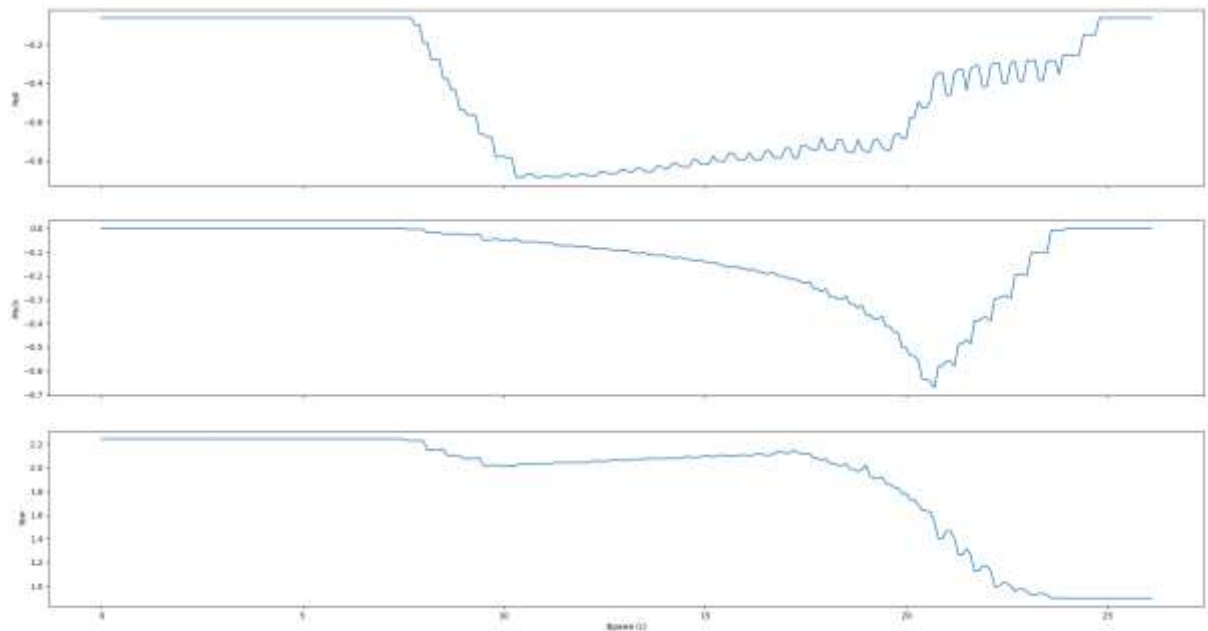


Рисунок Т6 – Временная диаграмма для углов Эйлера четвертого сустава манипулятора

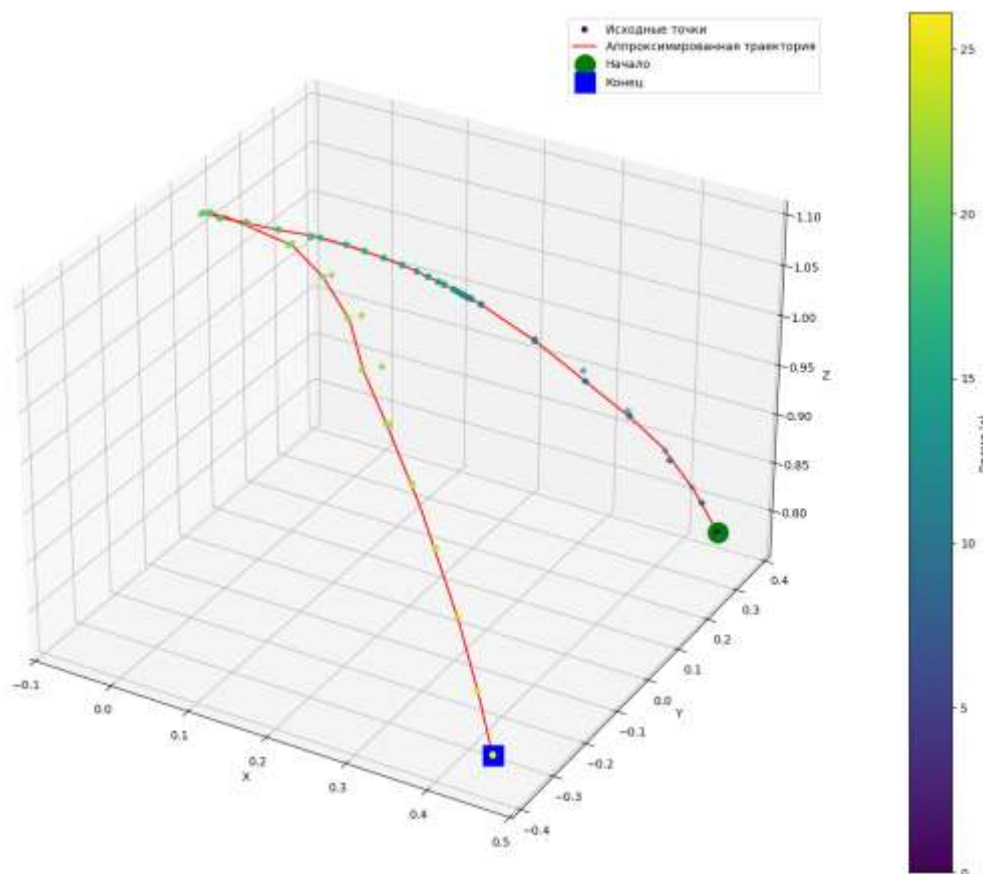


Рисунок Т7 – 3D временная диаграмма для координат третьего сустава манипулятора

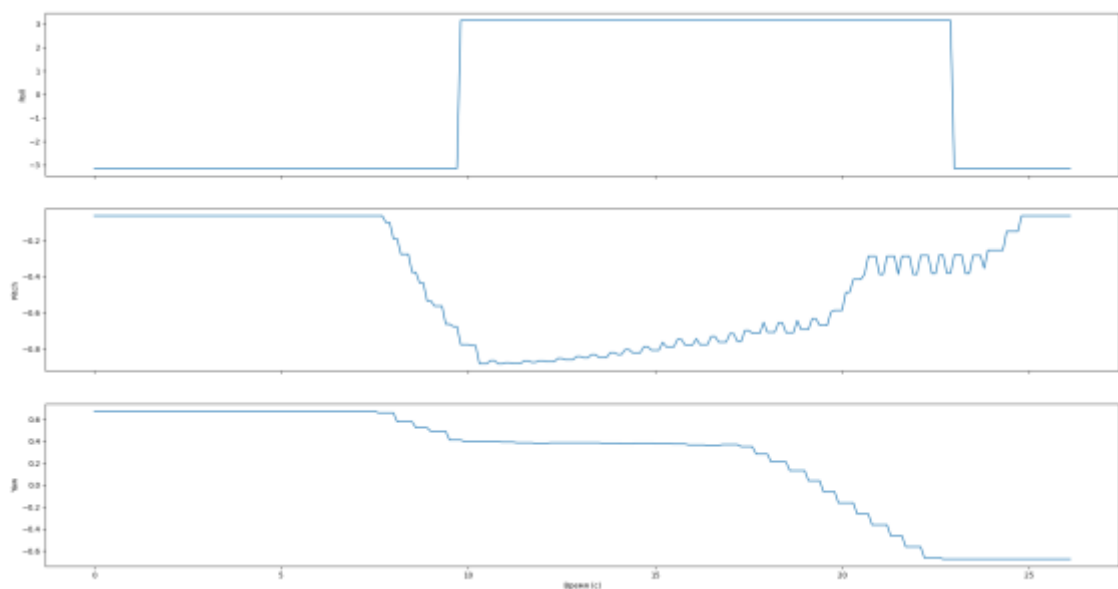


Рисунок Т8 – Временная диаграмма для углов Эйлера третьего сустава манипулятора

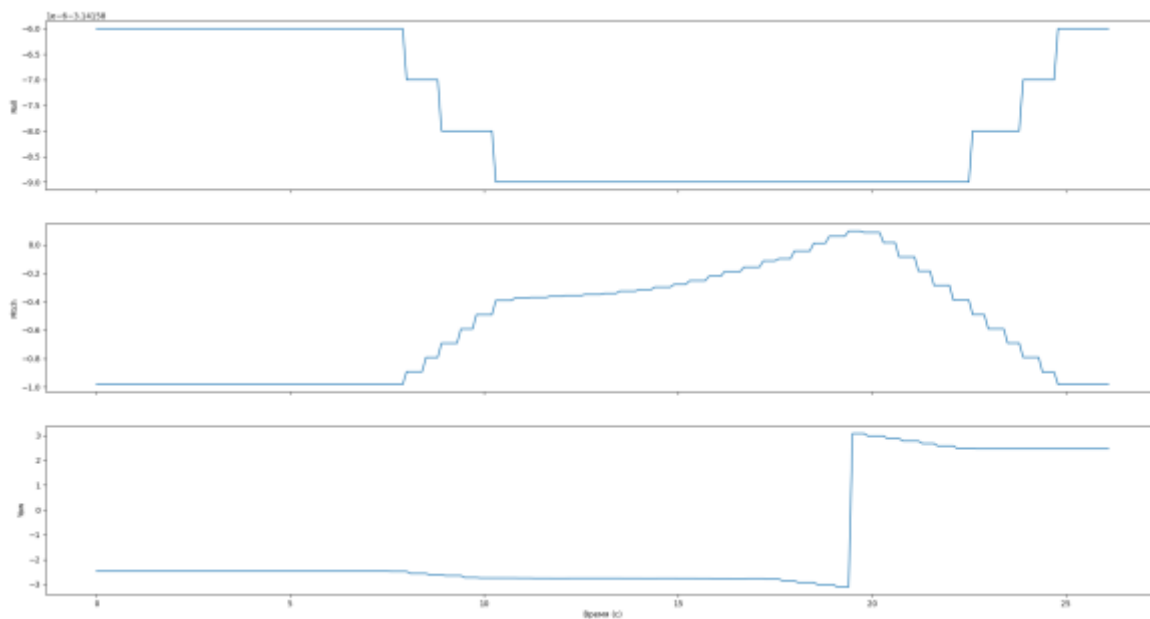


Рисунок Т9 – Временная диаграмма для углов Эйлера второго сустава манипулятора

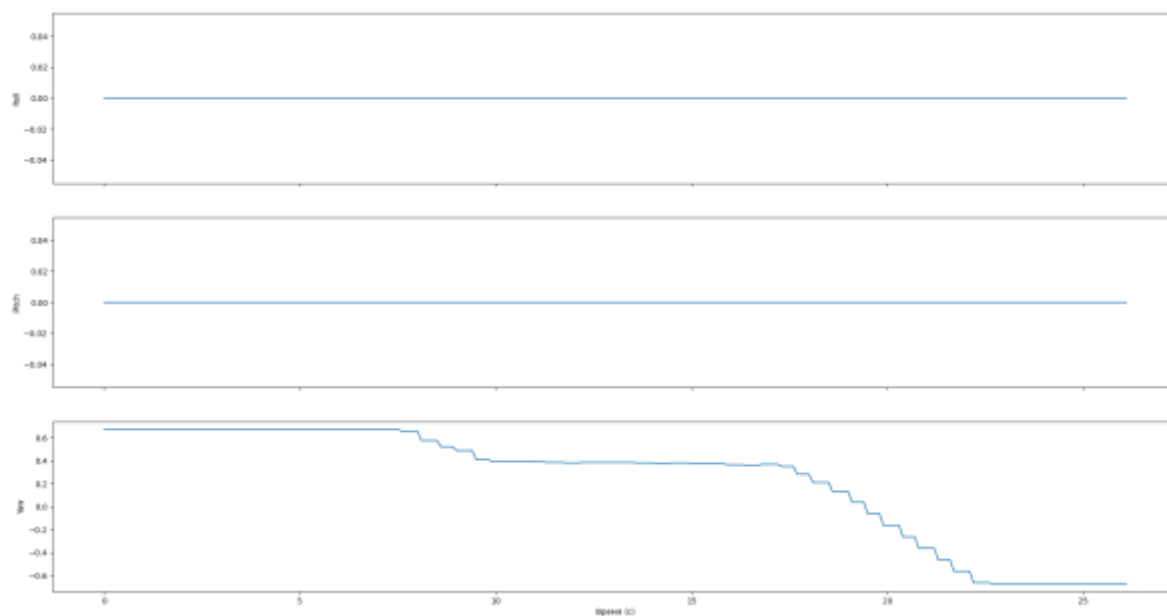
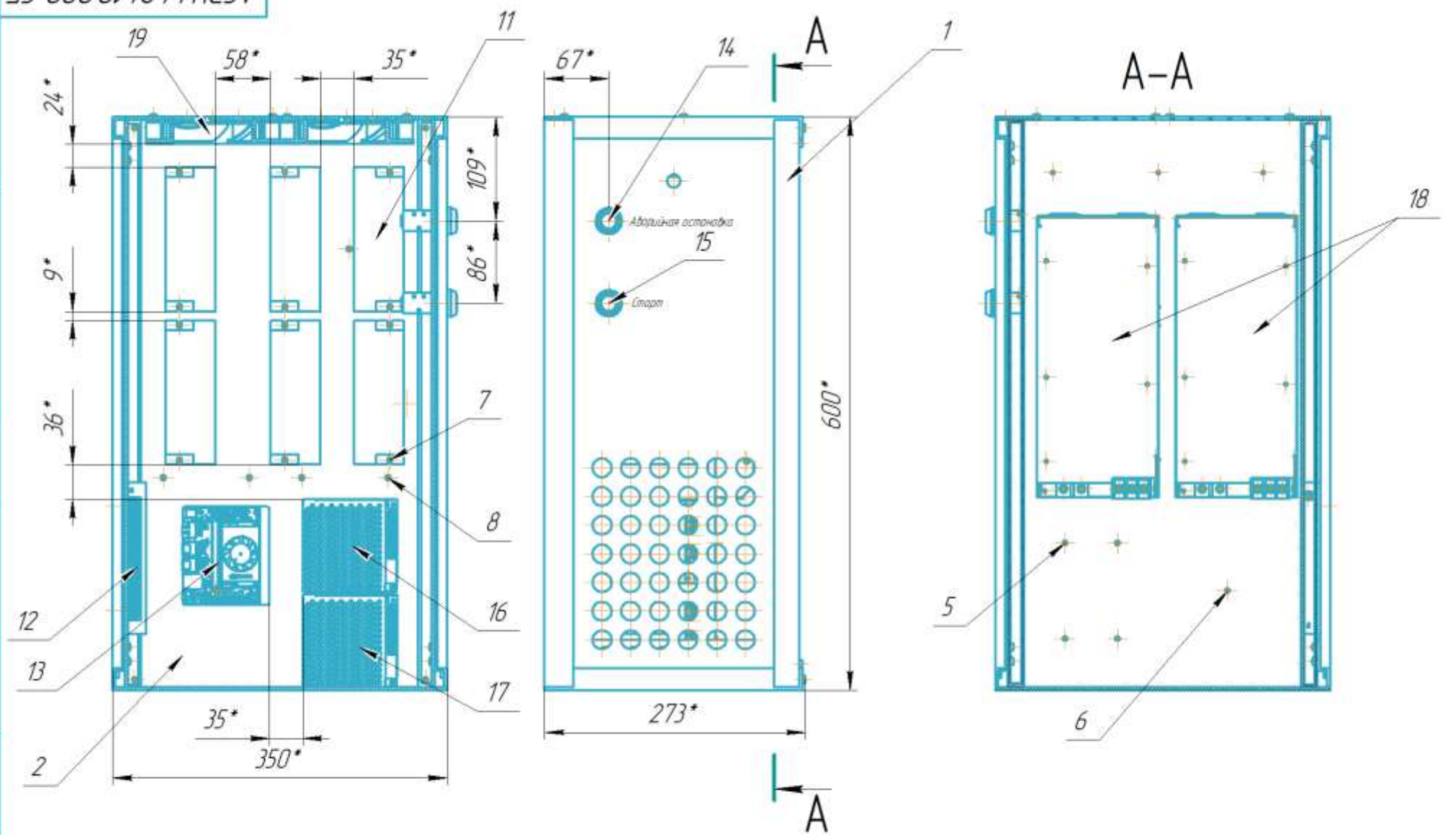


Рисунок Т10 – Временная диаграмма для углов Эйлера первого сустава манипулятора

ПРИЛОЖЕНИЕ АГ
Сборочный чертеж шкафа управления

КОМПАС-3D v23 Учебная версия © 2024 ООО "АКЮН-Системы проектирования" Россия Все права защищены
Инд. № подл. Подп. и дата
Инд. № подл. Подп. и дата
Взам. инд. № Инд. № подл. Подп. и дата
Спроект. №

АСЭУ.442410.000 СБ



					АСЭУ.442410.000 СБ			
Изм.	Лист	№ докум.	Подп.	Дата	Шкаф управления	Лит.	Масса	Масштаб
Разраб.	Пешков							1:4
Проб.								
Т.контр.						Лист 1	Листов 2	
Н.контр.						СНИУ гр.2414		
Утв.								

- 1 *Размеры для справки
- 2 Монтировать деталь поз.2 в деталь поз.1 в пазы при сборке
- 3 Монтировать детали поз.11 на деталь поз.2 винтами М3
- 4 Монтировать детали поз.12 на деталь поз.2 винтами М3
- 5 Монтировать детали поз.13 на деталь поз.2 винтами М2.5
- 6 Монтировать детали поз.16 на деталь поз.2 винтами М2.5
- 7 Монтировать детали поз.17 на деталь поз.2 винтами М2.5
- 8 Монтировать детали поз.18 на деталь поз.2 винтами М4
- 9 Монтировать детали поз.19 на деталь поз.2 винтами М3
- 10 Монтаж оборудования проводить согласно СНиП 3.05.07-85
- 11 Монтаж и подключение элементов производить в соответствии с их инструкциями по эксплуатации и принципиальной электрической схемы
- 12 Обеспечить проток воздуха к вентиляционным отверстиям
- 13 Установить дверь на корпус шкафа

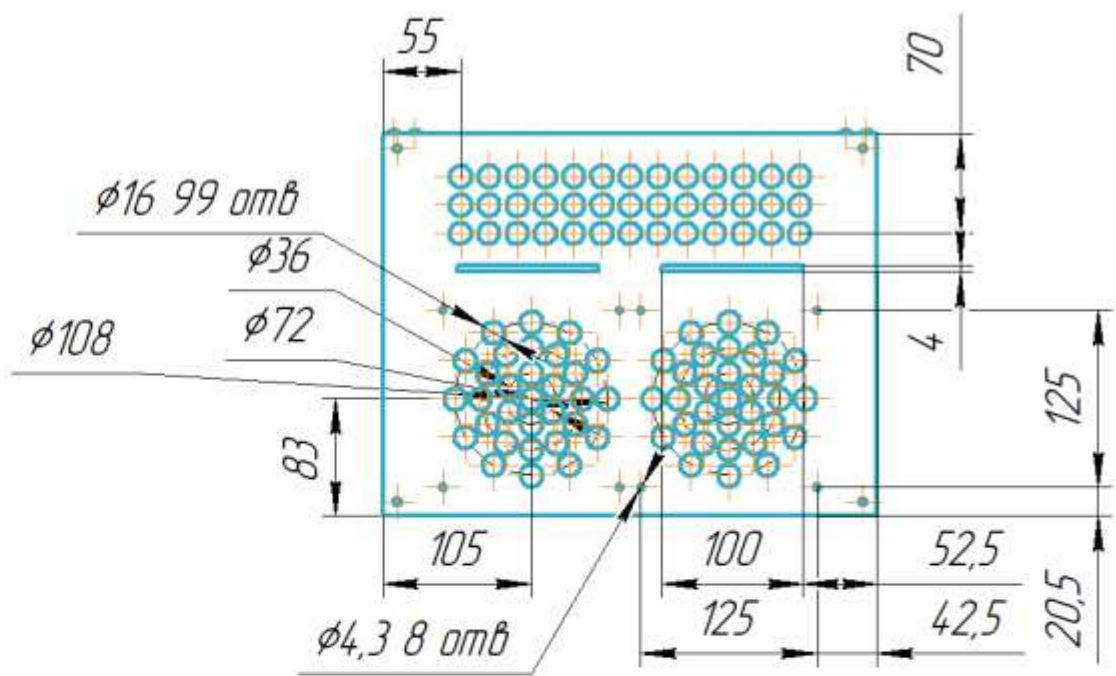
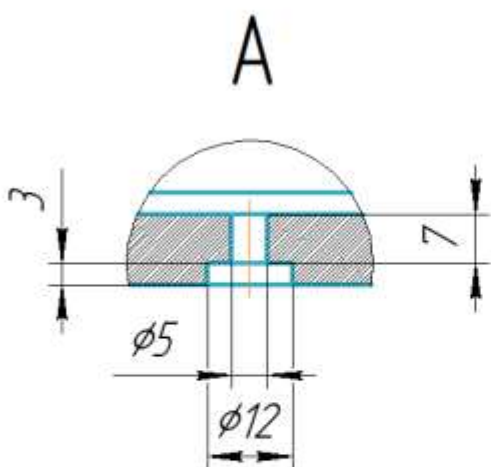
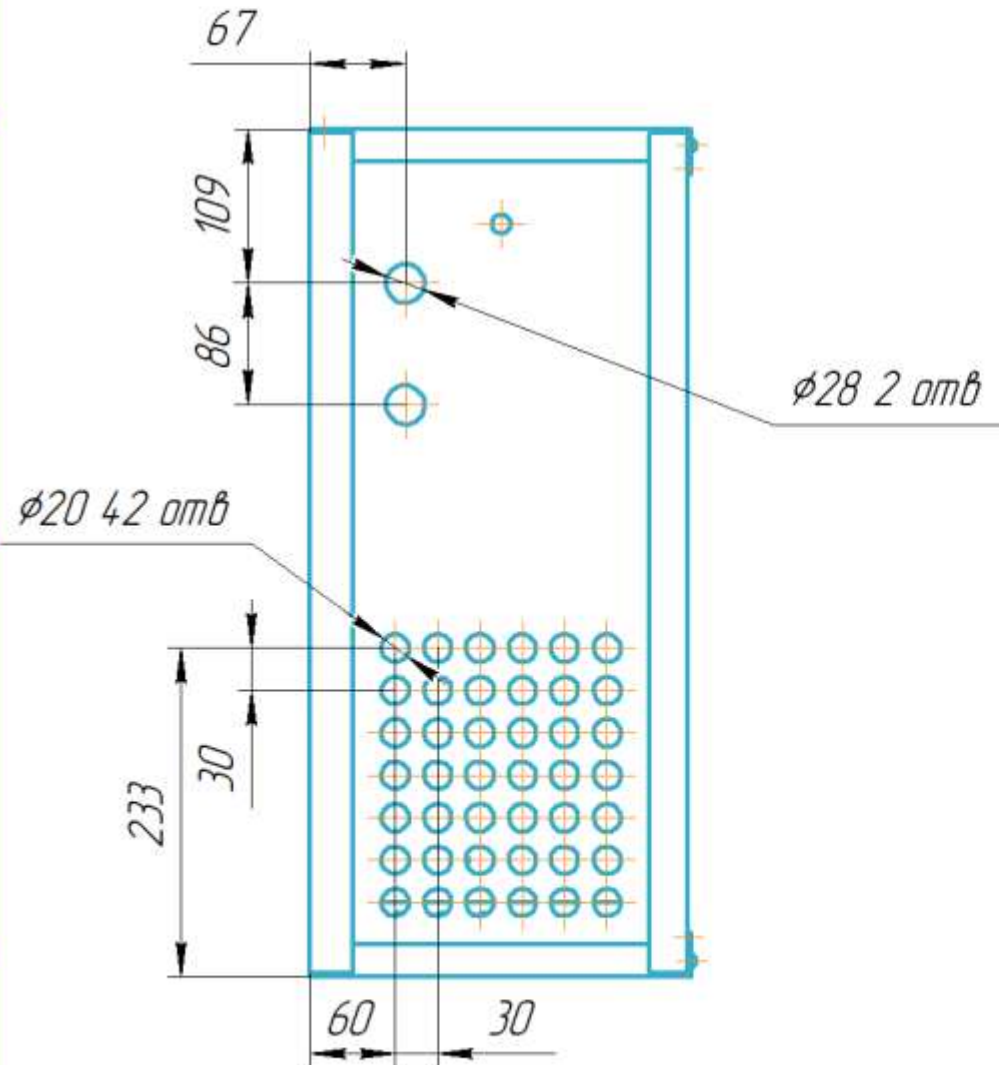
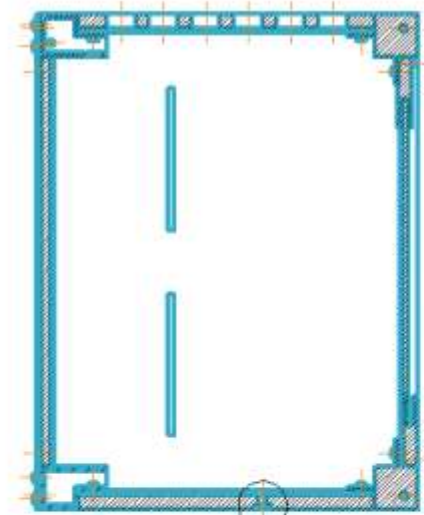
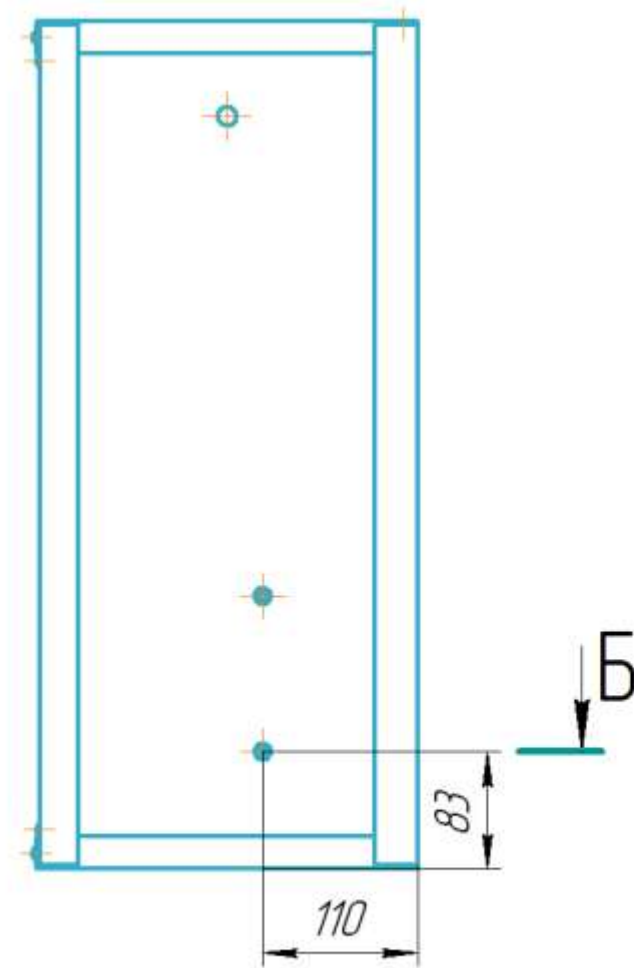
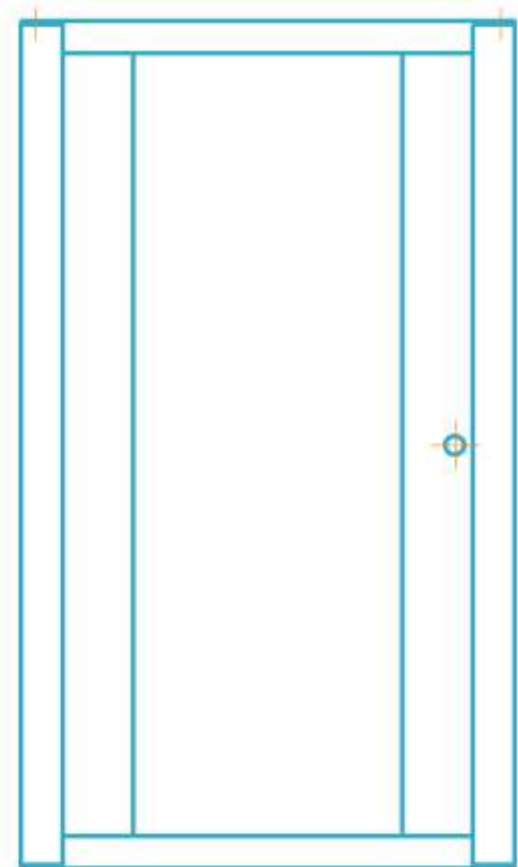
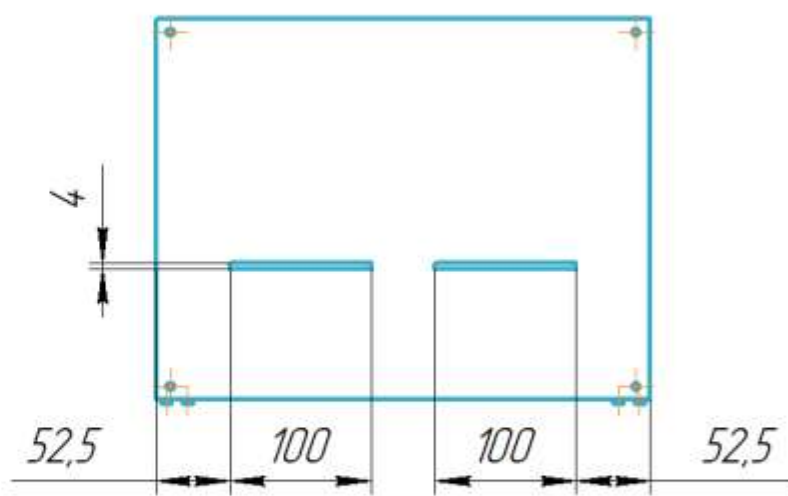
КОМПАС-3D v23 Учебная версия © 2024 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Инв. № подл.		Подп. и дата		Взам. инв. №		Инв. № дудл.		Подп. и дата	

Не для коммерческого использования Копировал Формат A4

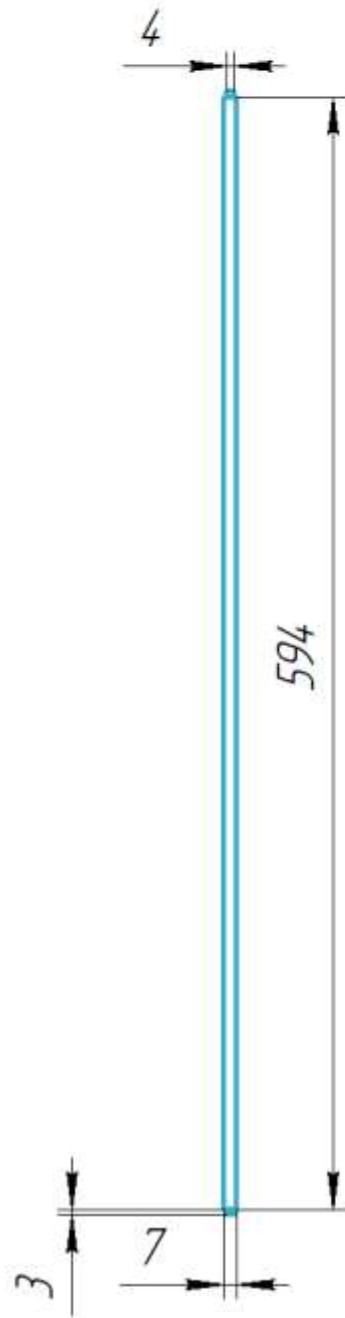
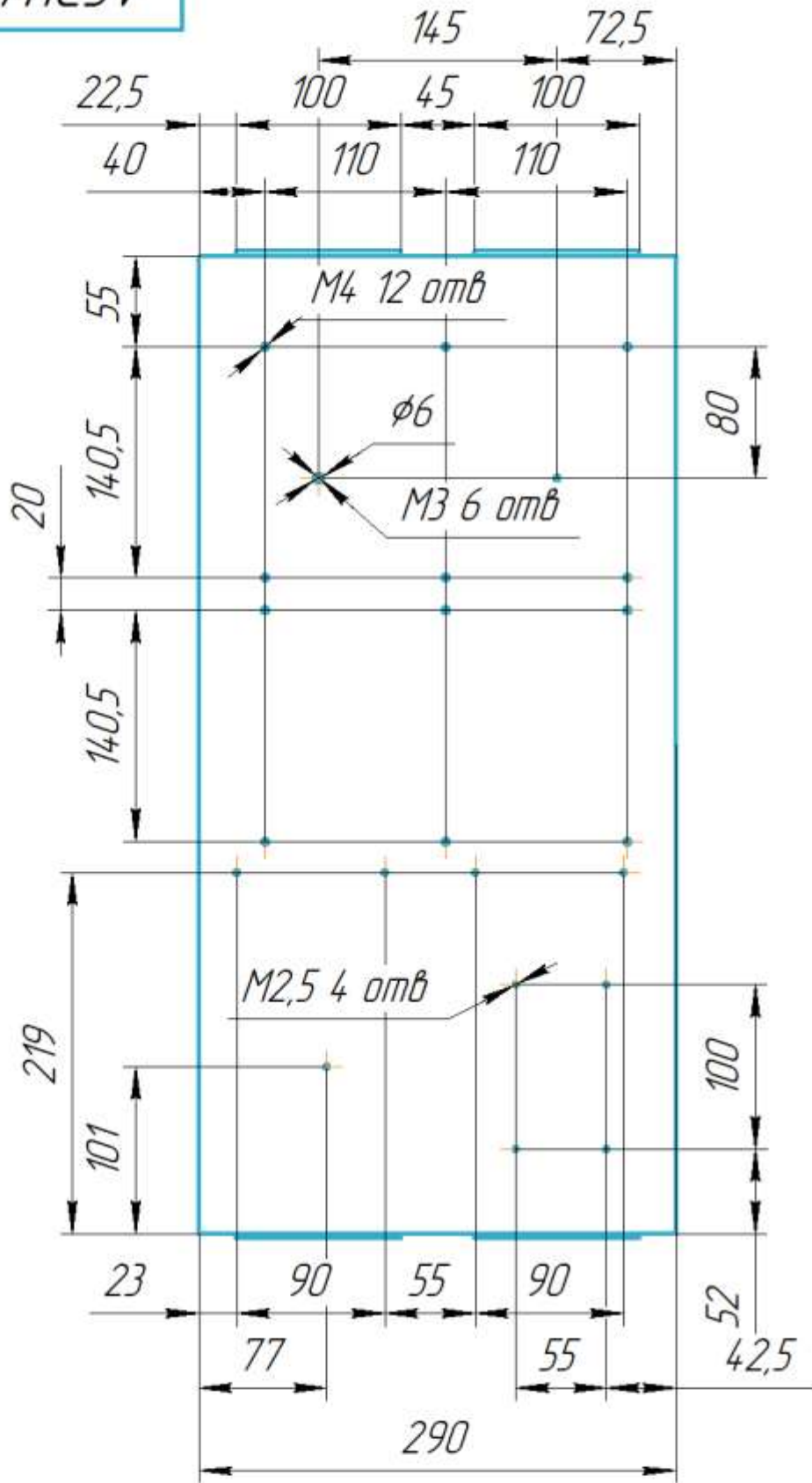
2

АСЭУ.4224.10.001



АСЭУ.4224.10.001					Шкаф Управления RAL 9004RAL 7000 Доработка		
Изм.	Лист	№ докум.	Подп.	Дата	Лит.	Масса	Масштаб
Разраб.	Пешков					0	1:5
Проб.					Лист	Листов	1
Т.контр.					СНИУ гр 24 14		
Н.контр.					Копировал		
Чтб.					Формат А2		

АСЭУ.4.224.10.002



АСЭУ.4.224.10.002				Лист	Масса	Масштаб
Изм. Лист	№ докум.	Подп.	Дата	Монтажная панель	9,5	1:4
Разраб.	Пешков					
Пров.						
Т.контр.				Лист	Листов	1
Н.контр.				Сталь 08Х18Н10 ГОСТ 5632-2014		
Утв.				СНИУ гр.24 14		